

UWB Localization System

1.0

Generated by Doxygen 1.17.0

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Data Structure Index	5
3.1 Data Structures	5
4 File Index	7
4.1 File List	7
5 Topic Documentation	9
5.1 Timing constants	9
5.1.1 Detailed Description	10
5.1.2 Macro Definition Documentation	10
5.1.2.1 ALL_ID	10
5.1.2.2 DEEP_SLEEP	10
5.1.2.3 SYNC_TIMEOUT_MAX	10
5.1.2.4 T_EARLY_WKUP	10
5.1.2.5 T_FINAL_RX_WAIT	10
5.1.2.6 T_FINAL_TX_ASAP_TICKS	10
5.1.2.7 T_FINAL_TX_WAIT_MS	10
5.1.2.8 T_PASSIVE_TX_ASAP_TICKS	11
5.1.2.9 T_PASSIVE_TX_WAIT_MS	11
5.1.2.10 T_RESPONSE_RX_WAIT	11
5.1.2.11 T_SHARE_RX_WAIT	11
5.1.2.12 T_SYNC_RX_ANSWER	11
5.1.2.13 T_SYNC_RX_SET	11
5.1.2.14 T_TWR_RX_WAIT	11
5.1.2.15 UWB_US_TO_TICKS	11
6 Directory Documentation	13
6.1 App/Accelerometer Directory Reference	13
6.2 App Directory Reference	13
6.3 App/Blink Directory Reference	13
6.4 App/Calculations Directory Reference	14
6.5 App/DWM Directory Reference	14
6.6 App/Generic Directory Reference	14
6.7 App/UWB_app Directory Reference	14
7 Data Structure Documentation	15
7.1 BatteryStatus_t Struct Reference	15
7.1.1 Field Documentation	15

7.1.1.1 battery_voltage_V	15
7.1.1.2 charge_complete	15
7.1.1.3 is_charging	15
7.1.1.4 usb_connected	15
7.2 boot_config_t Struct Reference	16
7.2.1 Field Documentation	16
7.2.1.1 charge_only	16
7.2.1.2 hints	16
7.2.1.3 valid	16
7.3 coop_ekf_t Struct Reference	16
7.3.1 Field Documentation	17
7.3.1.1 initialised	17
7.3.1.2 last_tick_ms	17
7.3.1.3 n_peers	17
7.3.1.4 P	17
7.3.1.5 peer_away	17
7.3.1.6 peer_ids	17
7.3.1.7 peer_imu_valid	17
7.3.1.8 peer_seeded	18
7.3.1.9 Q	18
7.3.1.10 self_slot	18
7.3.1.11 stationary_count	18
7.3.1.12 x	18
7.4 dwm_rx_frame_t Struct Reference	18
7.4.1 Field Documentation	19
7.4.1.1 data	19
7.4.1.2 fp_q8	19
7.4.1.3 len	19
7.4.1.4 rssi_q8	19
7.4.1.5 rx_timestamp	19
7.4.1.6 status	19
7.4.1.7 type	19
7.5 dwm_rx_raw_frame_t Struct Reference	19
7.5.1 Field Documentation	20
7.5.1.1 data	20
7.5.1.2 len	20
7.5.1.3 rx_diag	20
7.5.1.4 status	20
7.5.1.5 ts_buf	20
7.5.1.6 type	20
7.6 dwm_tx_frame_t Struct Reference	20
7.6.1 Field Documentation	21

7.6.1.1 data	21
7.6.1.2 len	21
7.6.1.3 tx_timestamp	21
7.7 ekf_node_hint_t Struct Reference	21
7.7.1 Field Documentation	21
7.7.1.1 id	21
7.7.1.2 x	22
7.7.1.3 y	22
7.7.1.4 z	22
7.8 first_order_t Struct Reference	22
7.8.1 Field Documentation	22
7.8.1.1 initiator_id	22
7.8.1.2 responder_id	22
7.8.1.3 result_distance_tick	22
7.8.1.4 twr	23
7.9 imu_fifo_data_t Struct Reference	23
7.9.1 Detailed Description	23
7.9.2 Field Documentation	23
7.9.2.1 count	23
7.9.2.2 samples	23
7.10 imu_sample_t Struct Reference	23
7.10.1 Detailed Description	24
7.10.2 Field Documentation	24
7.10.2.1 accel_raw	24
7.10.2.2 gravity_raw	24
7.10.2.3 timestamp_ticks	24
7.11 measurements_t Struct Reference	24
7.11.1 Field Documentation	25
7.11.1.1 final	25
7.11.1.2 final_antenna_unreliable	25
7.11.1.3 final_rx	25
7.11.1.4 passive	25
7.11.1.5 passive_antenna_unreliable	25
7.11.1.6 passive_count	25
7.11.1.7 passive_device_id	25
7.11.1.8 passive_rx	26
7.11.1.9 poll	26
7.11.1.10 poll_antenna_unreliable	26
7.11.1.11 poll_rx	26
7.11.1.12 resp	26
7.11.1.13 resp_tx	26
7.12 msg_final_t Struct Reference	26

7.12.1 Field Documentation	27
7.12.1.1 entries	27
7.12.1.2 entry_antenna_unreliable	27
7.12.1.3 entry_count	27
7.12.1.4 entry_id	27
7.12.1.5 entry_pwr_diff_q8	27
7.12.1.6 final_tx_ts	27
7.12.1.7 IMU_vel_horiz	27
7.12.1.8 IMU_vel_vert	27
7.12.1.9 poll_tx_ts	28
7.12.1.10 resp_antenna_unreliable	28
7.12.1.11 resp_pwr_diff_q8	28
7.12.1.12 resp_rx_ts	28
7.12.1.13 seq_num	28
7.13 msg_passive_t Struct Reference	28
7.13.1 Field Documentation	29
7.13.1.1 entries	29
7.13.1.2 entry_antenna_unreliable	29
7.13.1.3 entry_count	29
7.13.1.4 entry_ids	29
7.13.1.5 entry_pwr_diff_q8	29
7.13.1.6 IMU_vel_horiz	30
7.13.1.7 IMU_vel_vert	30
7.13.1.8 passive_tx_ts	30
7.13.1.9 poll_antenna_unreliable	30
7.13.1.10 poll_pwr_diff_q8	30
7.13.1.11 poll_rx_ts	30
7.13.1.12 resp_antenna_unreliable	30
7.13.1.13 resp_pwr_diff_q8	30
7.13.1.14 resp_rx_ts	31
7.13.1.15 seq_num	31
7.14 msg_poll_t Struct Reference	31
7.14.1 Detailed Description	31
7.14.2 Field Documentation	31
7.14.2.1 seq_num	31
7.15 msg_response_t Struct Reference	32
7.15.1 Detailed Description	32
7.15.2 Field Documentation	32
7.15.2.1 seq_num	32
7.16 msg_share_t Struct Reference	32
7.16.1 Detailed Description	33
7.16.2 Field Documentation	33

7.16.2.1 accuracy	33
7.16.2.2 distance_mm	33
7.16.2.3 node_count	33
7.16.2.4 node_ids	33
7.16.2.5 seq_num	33
7.16.2.6 sleep_time	33
7.16.2.7 vel_horiz	34
7.16.2.8 vel_vert	34
7.17 msg_sync_t Struct Reference	34
7.17.1 Detailed Description	34
7.17.2 Field Documentation	34
7.17.2.1 peer_count	34
7.17.2.2 peer_ids	35
7.17.2.3 seq_num	35
7.18 msg_t Struct Reference	35
7.18.1 Detailed Description	35
7.18.2 Field Documentation	36
7.18.2.1 [union]	36
7.18.2.2 final	36
7.18.2.3 len	36
7.18.2.4 passive	36
7.18.2.5 poll	36
7.18.2.6 receiver	36
7.18.2.7 response	36
7.18.2.8 sender	36
7.18.2.9 share	37
7.18.2.10 sync	37
7.18.2.11 type	37
7.19 network_t Struct Reference	37
7.19.1 Detailed Description	37
7.19.2 Field Documentation	38
7.19.2.1 acknowledged	38
7.19.2.2 count	38
7.19.2.3 expected_seq_num	38
7.19.2.4 master_id	38
7.19.2.5 measurements	38
7.19.2.6 peers	38
7.19.2.7 self	38
7.20 node_peer_state_t Struct Reference	39
7.20.1 Detailed Description	39
7.20.2 Field Documentation	39
7.20.2.1 certainty	39

7.20.2.2 distance_scaled	39
7.20.2.3 peer_id	39
7.21 node_t Struct Reference	39
7.21.1 Field Documentation	40
7.21.1.1 id	40
7.21.1.2 imu_vel_horiz	40
7.21.1.3 imu_vel_vert	40
7.21.1.4 peers	40
7.21.1.5 pos	40
7.22 print_msg_t Struct Reference	40
7.22.1 Detailed Description	41
7.22.2 Field Documentation	41
7.22.2.1 data	41
7.22.2.2 len	41
7.22.2.3 tick	41
7.23 second_order_t Struct Reference	41
7.23.1 Field Documentation	42
7.23.1.1 answer_rx_unreliable	42
7.23.1.2 init_rx_unreliable	42
7.23.1.3 initiator_id	42
7.23.1.4 responder_id	42
7.23.1.5 result_distance_tick	42
7.23.1.6 twr	42
7.23.1.7 twr_observation	42
7.24 ss_twr_t Struct Reference	42
7.24.1 Field Documentation	43
7.24.1.1 answer_rx	43
7.24.1.2 answer_tx	43
7.24.1.3 init_rx	43
7.24.1.4 init_tx	43
7.25 tdoa_twr_t Struct Reference	43
7.25.1 Field Documentation	44
7.25.1.1 answer_rx	44
7.25.1.2 answer_tx	44
7.25.1.3 init_rx	44
7.25.1.4 init_tx	44
7.25.1.5 master_init_rx	44
7.26 third_order_t Struct Reference	44
7.26.1 Field Documentation	45
7.26.1.1 answer_rx_unreliable	45
7.26.1.2 initiator_id	45
7.26.1.3 responder_id	45

7.26.1.4 result_distance_tick	45
7.26.1.5 twr	45
7.26.1.6 twr_observation_c	45
7.26.1.7 twr_observation_d	45
7.27 timestamps_t Struct Reference	45
7.27.1 Field Documentation	46
7.27.1.1 first	46
7.27.1.2 second	46
7.27.1.3 second_count	46
7.27.1.4 third	46
7.27.1.5 third_count	46
7.28 twr_observation_t Struct Reference	46
7.28.1 Field Documentation	47
7.28.1.1 poll_rx	47
7.28.1.2 poll_rx_unreliable	47
7.28.1.3 resp_rx	47
7.28.1.4 resp_rx_unreliable	47
7.29 twr_timestamps_t Struct Reference	47
7.29.1 Field Documentation	47
7.29.1.1 final_rx	47
7.29.1.2 final_rx_unreliable	48
7.29.1.3 final_tx	48
7.29.1.4 poll_rx	48
7.29.1.5 poll_rx_unreliable	48
7.29.1.6 poll_tx	48
7.29.1.7 resp_rx	48
7.29.1.8 resp_rx_unreliable	48
7.29.1.9 resp_tx	48
7.30 uwb_rx_meas_t Struct Reference	48
7.30.1 Detailed Description	49
7.30.2 Field Documentation	49
7.30.2.1 pwr_diff_q8	49
7.30.2.2 ts	49
8 File Documentation	51
8.1 App/Accelerometer/imu.c File Reference	51
8.1.1 Macro Definition Documentation	52
8.1.1.1 LSM6DSV_FIFO_DATA_OUT_TAG_ADDR	52
8.1.2 Function Documentation	52
8.1.2.1 imu_calibrate()	52
8.1.2.2 imu_init()	52
8.1.2.3 imu_integrate()	53

8.1.2.4 imu_read_fifo()	53
8.1.3 Variable Documentation	54
8.1.3.1 dev_ctx	54
8.2 App/Accelerometer/imu.h File Reference	54
8.2.1 Macro Definition Documentation	55
8.2.1.1 IMU_ACCEL_X_MS2	55
8.2.1.2 IMU_ACCEL_Y_MS2	55
8.2.1.3 IMU_ACCEL_Z_MS2	55
8.2.1.4 IMU_GRAV_X	55
8.2.1.5 IMU_GRAV_Y	55
8.2.1.6 IMU_GRAV_Z	56
8.2.2 Function Documentation	56
8.2.2.1 imu_calibrate()	56
8.2.2.2 imu_init()	56
8.2.2.3 imu_integrate()	57
8.2.2.4 imu_read_fifo()	57
8.3 imu.h	58
8.4 App/Accelerometer/imu_setup.h File Reference	58
8.4.1 Macro Definition Documentation	59
8.4.1.1 IMU_BOOT_TIME_MS	59
8.4.1.2 IMU_CAL_ODR_HZ	59
8.4.1.3 IMU_CAL_SAMPLES	59
8.4.1.4 IMU_FIFO_GY_BATCH	59
8.4.1.5 IMU_FIFO_RAW_BUF_ENTRIES	59
8.4.1.6 IMU_FIFO_TS_DEC	59
8.4.1.7 IMU_FIFO_XL_BATCH	59
8.4.1.8 IMU_GY_FULL_SCALE	60
8.4.1.9 IMU_GY_LP1_BW	60
8.4.1.10 IMU_GY_ODR	60
8.4.1.11 IMU_MAX_SAMPLES	60
8.4.1.12 IMU_SFLP_BATCH_GBIAS	60
8.4.1.13 IMU_SFLP_BATCH_GRAVITY	60
8.4.1.14 IMU_SFLP_BATCH_QUAT	60
8.4.1.15 IMU_SFLP_ODR	60
8.4.1.16 IMU_XL_FULL_SCALE	60
8.4.1.17 IMU_XL_LP2_BW	60
8.4.1.18 IMU_XL_ODR	61
8.5 imu_setup.h	61
8.6 App/Accelerometer/lsm6dsv.c File Reference	63
8.6.1 Function Documentation	63
8.6.1.1 imu_get_results()	63
8.6.1.2 StartAcc()	64

8.7 App/Accelerometer/lsm6dsv.h File Reference	64
8.7.1 Function Documentation	65
8.7.1.1 imu_get_results()	65
8.7.1.2 StartAcc()	65
8.8 lsm6dsv.h	66
8.9 App/Accelerometer/lsm6dsv_self_test.c File Reference	66
8.9.1 Macro Definition Documentation	67
8.9.1.1 BOOT_TIME	67
8.9.1.2 MAX_ST_LIMIT_mdps	67
8.9.1.3 MAX_ST_LIMIT_mg	67
8.9.1.4 MIN_ST_LIMIT_mdps	67
8.9.1.5 MIN_ST_LIMIT_mg	67
8.9.1.6 ST_FAIL	67
8.9.1.7 ST_PASS	67
8.9.2 Function Documentation	67
8.9.2.1 SelfTest()	67
8.9.3 Variable Documentation	68
8.9.3.1 tx_buffer	68
8.10 App/Accelerometer/lsm6dsv_ST_example.h File Reference	68
8.10.1 Function Documentation	68
8.10.1.1 lsm6dsv_read_data_polling()	68
8.10.1.2 SelfTest()	68
8.11 lsm6dsv_ST_example.h	68
8.12 App/Accelerometer/read_data_polling.c File Reference	69
8.12.1 Macro Definition Documentation	69
8.12.1.1 BOOT_TIME	69
8.12.2 Function Documentation	69
8.12.2.1 lsm6dsv_read_data_polling()	69
8.13 App/Blink/blink.c File Reference	69
8.13.1 Function Documentation	70
8.13.1.1 StartBlink()	70
8.14 App/Blink/blink.h File Reference	70
8.14.1 Function Documentation	70
8.14.1.1 StartBlink()	70
8.15 blink.h	70
8.16 App/Calculations/calibrate.c File Reference	71
8.16.1 Function Documentation	71
8.16.1.1 calibrate_get_k()	71
8.16.1.2 calibrate_rssi_antenna()	72
8.16.1.3 calibrate_rx_timestamp()	72
8.16.1.4 calibrate_set_clock_offset_poll()	73
8.16.1.5 calibrate_set_clock_offset_sync()	73

8.16.1.6	calibrate_set_pitch()	74
8.16.1.7	calibrate_tx_timestamp()	74
8.17	App/Calculations/calibrate.h File Reference	74
8.17.1	Macro Definition Documentation	75
8.17.1.1	CALIB_ANTENNA_MIN_DIST_TICKS	75
8.17.1.2	CALIB_ANTENNA_N	75
8.17.1.3	CALIB_ANTENNA_NULL_THRESHOLD_DEG	75
8.17.1.4	CALIB_ANTENNA_SIMILARITY_DEG	75
8.17.1.5	CALIB_ANTENNA_TILT_LIMIT_DEG	76
8.17.1.6	CALIB_RX_TS_A	76
8.17.1.7	CFO_RAW_TO_K	76
8.17.1.8	CFO_RAW_TO_PPM	76
8.17.1.9	PPM_TO_K	76
8.17.2	Function Documentation	76
8.17.2.1	calibrate_get_k()	76
8.17.2.2	calibrate_rx_timestamp()	77
8.17.2.3	calibrate_set_clock_offset_poll()	77
8.17.2.4	calibrate_set_clock_offset_sync()	78
8.17.2.5	calibrate_set_pitch()	78
8.17.2.6	calibrate_tx_timestamp()	79
8.18	calibrate.h	79
8.19	App/Calculations/distance.c File Reference	80
8.19.1	Macro Definition Documentation	81
8.19.1.1	CLAMP_PDIFF	81
8.19.1.2	PDIFF_MAX_DB	81
8.19.2	Function Documentation	81
8.19.2.1	compute_certainty()	81
8.19.2.2	dist_scale_to_ticks()	82
8.19.2.3	dist_ticks_to_scale()	82
8.19.2.4	distance_calculate()	82
8.19.2.5	populate_computation_structs()	83
8.19.2.6	vel_horiz_to_u8()	84
8.19.2.7	vel_horiz_u8_to_ms()	84
8.19.2.8	vel_vert_to_u8()	84
8.19.2.9	vel_vert_u8_to_ms()	85
8.19.3	Variable Documentation	85
8.19.3.1	timestamps	85
8.20	App/Calculations/distance.h File Reference	85
8.20.1	Macro Definition Documentation	87
8.20.1.1	CERTAINTY_BASE_DSTWR	87
8.20.1.2	CERTAINTY_BASE_SSTWR	87
8.20.1.3	CERTAINTY_BASE_TDOA	87

8.20.1.4 CERTAINTY_DELTA0_Q8	87
8.20.1.5 CERTAINTY_KN_Q8	87
8.20.1.6 CERTAINTY_KT	87
8.20.1.7 CERTAINTY_LOS_MAX	87
8.20.1.8 CERTAINTY_LOS_MIN	87
8.20.1.9 CERTAINTY_TIMING_MAX	87
8.20.1.10 CERTAINTY_TIMING_MIN	87
8.20.1.11 CERTAINTY_TREF_US	88
8.20.1.12 CERTAINTY_UNRELIABLE_PENALTY	88
8.20.1.13 DIST_SHARE_DIST_MAX_TICKS	88
8.20.1.14 DIST_SHARE_MAX_M	88
8.20.1.15 DIST_SHARE_TICKS_PER_LSB	88
8.20.1.16 DWT_TICK_TO_US	88
8.20.1.17 MAX_PASSIVE	88
8.20.1.18 MAX_SECOND_ORDER	88
8.20.1.19 MAX_THIRD_ORDER	88
8.20.1.20 METERS_PER_TICK	88
8.20.1.21 SPEED_OF_LIGHT	89
8.20.1.22 UWB_40BIT_MASK	89
8.20.1.23 VEL_HORIZ_MAX_MS	89
8.20.1.24 VEL_HORIZ_MS_PER_LSB	89
8.20.1.25 VEL_VERT_MS_PER_LSB	89
8.20.1.26 VEL_VERT_RANGE_MS	89
8.20.2 Enumeration Type Documentation	89
8.20.2.1 MeasurementType	89
8.20.3 Function Documentation	90
8.20.3.1 dist_scale_to_ticks()	90
8.20.3.2 dist_ticks_to_scale()	90
8.20.3.3 distance_calculate()	90
8.20.3.4 position_calibrate_timestamp()	91
8.20.3.5 vel_horiz_to_u8()	91
8.20.3.6 vel_horiz_u8_to_ms()	91
8.20.3.7 vel_vert_to_u8()	92
8.20.3.8 vel_vert_u8_to_ms()	92
8.21 distance.h	92
8.22 App/Calculations/ekf.c File Reference	95
8.22.1 Macro Definition Documentation	96
8.22.1.1 EKF_INIT_RAND_M	96
8.22.1.2 EKF_TAG_HARDCODED_Z	96
8.22.2 Function Documentation	96
8.22.2.1 ekf_anchor_ready()	96
8.22.2.2 ekf_certainty_to_sigma()	97

8.22.2.3 ekf_get_anchor_positions()	97
8.22.2.4 ekf_get_peer_pos()	97
8.22.2.5 ekf_get_phase()	98
8.22.2.6 ekf_get_state()	98
8.22.2.7 ekf_init()	99
8.22.2.8 ekf_step()	99
8.22.2.9 HAL_GetTick()	100
8.22.3 Variable Documentation	100
8.22.3.1 EKF_ANCHOR_IDS	100
8.23 App/Calculations/ekf.h File Reference	100
8.23.1 Macro Definition Documentation	101
8.23.1.1 EKF_ANCHOR_MIN_STEPS	101
8.23.1.2 EKF_ANCHOR_P_CONVERGED	102
8.23.1.3 EKF_DT_MAX_S	102
8.23.1.4 EKF_DT_NOM_S	102
8.23.1.5 EKF_HDG_MIN_DELTA_M	102
8.23.1.6 EKF_IMU_DEADBAND_MS	102
8.23.1.7 EKF_INIT_P_POS	102
8.23.1.8 EKF_MAX_PEERS	102
8.23.1.9 EKF_MAX_STATE	102
8.23.1.10 EKF_MAX_STEP_M	102
8.23.1.11 EKF_MAX_UPDATE_M	103
8.23.1.12 EKF_NUM_ANCHORS	103
8.23.1.13 EKF_OUTLIER_GATE_NEG	103
8.23.1.14 EKF_OUTLIER_GATE_POS	103
8.23.1.15 EKF_P_MAX	103
8.23.1.16 EKF_Q_H_FLOOR	103
8.23.1.17 EKF_Q_H_FLOOR_STATIONARY	103
8.23.1.18 EKF_Q_H_VEL_SCALE	103
8.23.1.19 EKF_Q_Z_POS	103
8.23.1.20 EKF_SIGMA_MAX	103
8.23.1.21 EKF_SIGMA_MIN	104
8.23.1.22 EKF_STATIONARY_STEPS	104
8.23.1.23 EKF_VEL_EMA_ALPHA	104
8.23.2 Enumeration Type Documentation	104
8.23.2.1 ekf_phase_t	104
8.23.3 Function Documentation	104
8.23.3.1 ekf_anchor_ready()	104
8.23.3.2 ekf_certainty_to_sigma()	105
8.23.3.3 ekf_get_anchor_positions()	105
8.23.3.4 ekf_get_peer_pos()	105
8.23.3.5 ekf_get_phase()	106

8.23.3.6 ekf_get_state()	106
8.23.3.7 ekf_init()	107
8.23.3.8 ekf_step()	107
8.23.4 Variable Documentation	108
8.23.4.1 EKF_ANCHOR_IDS	108
8.24 ekf.h	108
8.25 App/DWM/DWM3000.c File Reference	110
8.25.1 Macro Definition Documentation	111
8.25.1.1 MPRINT_TS	111
8.25.1.2 U64_HI	111
8.25.1.3 U64_LO	111
8.25.2 Function Documentation	112
8.25.2.1 low_battery_check()	112
8.25.2.2 StartRangingTask()	112
8.25.2.3 tx_err_watchdog()	112
8.26 App/DWM/DWM3000.h File Reference	113
8.26.1 Function Documentation	113
8.26.1.1 StartDWM()	113
8.27 DWM3000.h	113
8.28 App/DWM/DWM3000_driver.c File Reference	113
8.28.1 Function Documentation	115
8.28.1.1 cb_rx_err()	115
8.28.1.2 cb_rx_ok()	115
8.28.1.3 cb_rx_to()	116
8.28.1.4 cb_spi_rdy()	116
8.28.1.5 cb_tx_done()	116
8.28.1.6 dwm_configure()	116
8.28.1.7 dwm_enter_idle_rc()	117
8.28.1.8 dwm_exit_idle_rc()	117
8.28.1.9 dwm_get_addr()	117
8.28.1.10 dwm_get_ant_delay()	117
8.28.1.11 dwm_init()	117
8.28.1.12 dwm_init_queues()	117
8.28.1.13 dwm_restore_runtime()	118
8.28.1.14 dwm_rx()	118
8.28.1.15 dwm_rx_flush()	118
8.28.1.16 dwm_selftest()	118
8.28.1.17 dwm_sleep()	119
8.28.1.18 dwm_tx()	119
8.28.1.19 dwm_tx_delayed()	119
8.28.1.20 dwm_wakeup()	120
8.28.1.21 ts_buf_to_u64()	120

8.28.2 Variable Documentation	120
8.28.2.1 dw3000_probe_intf	120
8.28.2.2 hspi1	120
8.28.2.3 rx_queue	120
8.28.2.4 tx_queue	120
8.28.2.5 wakeup_queue	120
8.29 App/DWM/DWM3000_driver.h File Reference	121
8.29.1 Macro Definition Documentation	122
8.29.1.1 MS_TO_DWT_TIME	122
8.29.1.2 TX_DELAY_500US	122
8.29.1.3 US_TO_DWT_TIME	122
8.29.2 Enumeration Type Documentation	122
8.29.2.1 dwm_rx_event_type_t	122
8.29.2.2 dwm_tx_event_type_t	123
8.29.3 Function Documentation	123
8.29.3.1 cb_rx_err()	123
8.29.3.2 cb_rx_ok()	123
8.29.3.3 cb_rx_to()	123
8.29.3.4 cb_spi_rdy()	124
8.29.3.5 cb_tx_done()	124
8.29.3.6 dwm_configure()	124
8.29.3.7 dwm_enter_idle_rc()	124
8.29.3.8 dwm_exit_idle_rc()	124
8.29.3.9 dwm_get_addr()	125
8.29.3.10 dwm_init()	125
8.29.3.11 dwm_rx()	125
8.29.3.12 dwm_rx_continuous_sleep()	125
8.29.3.13 dwm_rx_flush()	125
8.29.3.14 dwm_selftest()	126
8.29.3.15 dwm_sleep()	126
8.29.3.16 dwm_tx()	126
8.29.3.17 dwm_tx_continuous()	126
8.29.3.18 dwm_tx_continuous_delayed()	126
8.29.3.19 dwm_tx_delayed()	127
8.29.3.20 dwm_wakeup()	127
8.29.3.21 ts_buf_to_u64()	127
8.30 DWM3000_driver.h	128
8.31 App/DWM/DWM3000_setup.h File Reference	129
8.31.1 Macro Definition Documentation	129
8.31.1.1 DWM_ANT_DELAY	129
8.31.1.2 DWM_CRC_LEN	129
8.31.1.3 DWM_IRQ_MASK	129

8.31.1.4 DWM_MAX_FRAME_LEN	130
8.31.1.5 DWM_PAN_ID	130
8.31.1.6 DWM_RX_TIMEOUT_1S	130
8.31.1.7 DWM_TX_PGDLy	130
8.31.1.8 DWM_UWB_CHANNEL	130
8.31.1.9 DWM_UWB_DATA_RATE	130
8.31.1.10 DWM_UWB_PAC	130
8.31.1.11 DWM_UWB_PDOA_MODE	130
8.31.1.12 DWM_UWB_PHR_MODE	130
8.31.1.13 DWM_UWB_PHR_RATE	130
8.31.1.14 DWM_UWB_PREAMBLE_LEN	130
8.31.1.15 DWM_UWB_RX_CODE	130
8.31.1.16 DWM_UWB_SFD_TO	130
8.31.1.17 DWM_UWB_SFD_TYPE	130
8.31.1.18 DWM_UWB_STS_LENGTH	130
8.31.1.19 DWM_UWB_STS_MODE	131
8.31.1.20 DWM_UWB_TX_CODE	131
8.31.1.21 DWT_DEVICE_ID	131
8.31.1.22 TX_RF_PGcount	131
8.31.1.23 TX_RF_PGdly	131
8.31.1.24 TX_RF_Power	131
8.32 DWM3000_setup.h	131
8.33 App/Generic/my_print.c File Reference	132
8.33.1 Macro Definition Documentation	132
8.33.1.1 PRINT_BUF_SIZE	132
8.33.1.2 PRINT_QUEUE_DEPTH	132
8.33.2 Function Documentation	133
8.33.2.1 get_usb_ready()	133
8.33.2.2 mprintf()	133
8.33.2.3 PrintTask()	133
8.33.2.4 PrintTask_TxCpltCallback()	134
8.33.3 Variable Documentation	134
8.33.3.1 hUsbDeviceFS	134
8.34 App/Generic/my_print.h File Reference	134
8.34.1 Macro Definition Documentation	134
8.34.1.1 Both	134
8.34.1.2 PRINT_MODE	134
8.34.1.3 STlink	135
8.34.1.4 USB_CDC	135
8.34.2 Function Documentation	135
8.34.2.1 get_usb_ready()	135
8.34.2.2 mprintf()	135

8.34.2.3 PrintTask()	135
8.35 my_print.h	136
8.36 App/Generic/power.c File Reference	136
8.36.1 Macro Definition Documentation	137
8.36.1.1 ADC_FULL_SCALE	137
8.36.1.2 DIVIDER_RATIO	137
8.36.1.3 VDDA_MV	137
8.36.2 Function Documentation	137
8.36.2.1 power_get_status()	137
8.36.2.2 power_read()	137
8.37 App/Generic/power.h File Reference	137
8.37.1 Function Documentation	138
8.37.1.1 power_get_status()	138
8.37.1.2 power_read()	138
8.38 power.h	138
8.39 App/Generic/sleep.c File Reference	139
8.39.1 Function Documentation	139
8.39.1.1 HAL_LPTIM_AutoReloadMatchCallback()	139
8.39.1.2 sleep_stop1_ms()	139
8.39.2 Variable Documentation	140
8.39.2.1 hlptim1	140
8.40 App/Generic/sleep.h File Reference	140
8.40.1 Function Documentation	140
8.40.1.1 sleep_stop1_ms()	140
8.41 sleep.h	140
8.42 App/Generic/UART.c File Reference	141
8.42.1 Macro Definition Documentation	141
8.42.1.1 UART_RST_BUF_SIZE	141
8.42.1.2 UART_RST_BYTE	141
8.42.2 Function Documentation	142
8.42.2.1 HAL_UART_ErrorCallback()	142
8.42.2.2 HAL_UARTEx_RxEventCallback()	142
8.42.2.3 uart_boot_config_read()	142
8.42.2.4 uart_boot_start()	142
8.42.2.5 uart_periodic_poll()	143
8.42.2.6 uart_print_power()	143
8.42.2.7 uart_rst_arm()	143
8.43 App/Generic/UART.h File Reference	143
8.43.1 Macro Definition Documentation	144
8.43.1.1 UART_BOOT_RX_LEN	144
8.43.2 Function Documentation	144
8.43.2.1 HAL_UART_ErrorCallback()	144

8.43.2.2 HAL_UART_RxCpltCallback()	144
8.43.2.3 uart_boot_config_read()	144
8.43.2.4 uart_boot_start()	145
8.43.2.5 uart_periodic_poll()	145
8.43.2.6 uart_print_power()	145
8.43.2.7 uart_rst_arm()	145
8.44 UART.h	145
8.45 App/UWB_app/messages.c File Reference	146
8.45.1 Detailed Description	146
8.45.2 Function Documentation	146
8.45.2.1 msg_decode()	146
8.45.2.2 msg_encode()	147
8.45.2.3 u64_to_ts_buf()	147
8.46 App/UWB_app/messages.h File Reference	148
8.46.1 Detailed Description	149
8.46.2 Macro Definition Documentation	149
8.46.2.1 MSG_HEADER_LEN	149
8.46.2.2 MSG_OFFSET_RECEIVER	149
8.46.2.3 MSG_OFFSET_SENDER	149
8.46.2.4 MSG_PAYLOAD_OFFSET_RESP_FP	150
8.46.2.5 MSG_PAYLOAD_OFFSET_RESP_RSSI	150
8.46.2.6 MSG_PAYLOAD_OFFSET_SEQ	150
8.46.2.7 MSG_PAYLOAD_OFFSET_SHARE_SLEEP	150
8.46.2.8 MSG_PAYLOAD_OFFSET_TS_0	150
8.46.2.9 MSG_PAYLOAD_OFFSET_TS_1	150
8.46.2.10 MSG_PAYLOAD_OFFSET_TS_2	150
8.46.2.11 MSG_START_LOCATION	150
8.46.2.12 MSG_TS_LEN	150
8.46.2.13 NETWORK_MAX_PAIRS	150
8.46.2.14 NETWORK_MAX_PEERS	150
8.46.3 Enumeration Type Documentation	151
8.46.3.1 msg_type_t	151
8.46.4 Function Documentation	151
8.46.4.1 msg_decode()	151
8.46.4.2 msg_encode()	151
8.46.4.3 u64_to_ts_buf()	152
8.47 messages.h	152
8.48 App/UWB_app/uwb_exchange.c File Reference	154
8.48.1 Macro Definition Documentation	155
8.48.1.1 MPRINT_TS	155
8.48.1.2 RX_MEAS_FROM_FRAME	156
8.48.1.3 SYNC_PAD	156

8.48.1.4 U64_HI	156
8.48.1.5 U64_LO	156
8.48.2 Function Documentation	156
8.48.2.1 uwb_build_share()	156
8.48.2.2 uwb_extended_twr()	157
8.48.2.3 uwb_read_share()	157
8.48.2.4 uwb_share()	157
8.48.2.5 uwb_sync()	158
8.48.2.6 uwb_unexpected_message_handler()	158
8.49 App/UWB_app/uwb_exchange.h File Reference	159
8.49.1 Detailed Description	160
8.49.2 Enumeration Type Documentation	160
8.49.2.1 uwb_etwr_result_t	160
8.49.2.2 uwb_sync_result_t	161
8.49.3 Function Documentation	161
8.49.3.1 uwb_extended_twr()	161
8.49.3.2 uwb_share()	162
8.49.3.3 uwb_sync()	162
8.50 uwb_exchange.h	163
8.51 App/UWB_app/uwb_network.c File Reference	164
8.51.1 Detailed Description	166
8.51.2 Macro Definition Documentation	166
8.51.2.1 NETWORK_DISTANCE_INVALID	166
8.51.3 Function Documentation	166
8.51.3.1 find_peer()	166
8.51.3.2 network_add_peer()	166
8.51.3.3 network_bump_certainty()	166
8.51.3.4 network_fill_peer_ids()	167
8.51.3.5 network_get_certainty()	167
8.51.3.6 network_get_count()	168
8.51.3.7 network_get_distance()	168
8.51.3.8 network_get_expected_seq_num()	168
8.51.3.9 network_get_highest_uncertainty()	168
8.51.3.10 network_get_master()	169
8.51.3.11 network_get_measurements()	169
8.51.3.12 network_get_network()	169
8.51.3.13 network_get_ownid()	169
8.51.3.14 network_get_passive_count()	170
8.51.3.15 network_get_peer_index()	170
8.51.3.16 network_get_peer_state()	170
8.51.3.17 network_get_peers()	171
8.51.3.18 network_init()	171

8.51.3.19 network_is_acknowledged()	172
8.51.3.20 network_is_master()	172
8.51.3.21 network_print_certainty()	172
8.51.3.22 network_print_positions()	172
8.51.3.23 network_remove_peer()	172
8.51.3.24 network_reset_certainty()	173
8.51.3.25 network_reset_measurements()	173
8.51.3.26 network_set_acknowledged()	173
8.51.3.27 network_set_distance()	173
8.51.3.28 network_set_expected_seq_num()	174
8.51.3.29 network_set_master()	174
8.51.3.30 network_set_self_pos()	174
8.51.3.31 network_store_final()	175
8.51.3.32 network_store_passive()	175
8.51.3.33 network_store_poll()	175
8.51.3.34 network_store_resp_tx()	176
8.51.3.35 network_update_certainty()	176
8.51.3.36 network_update_peers_from_sync()	176
8.52 App/UWB_app/uwb_network.h File Reference	177
8.52.1 Detailed Description	179
8.52.2 Function Documentation	179
8.52.2.1 find_peer()	179
8.52.2.2 network_add_peer()	179
8.52.2.3 network_bump_certainty()	179
8.52.2.4 network_fill_peer_ids()	180
8.52.2.5 network_get_certainty()	180
8.52.2.6 network_get_count()	180
8.52.2.7 network_get_distance()	181
8.52.2.8 network_get_expected_seq_num()	181
8.52.2.9 network_get_highest_uncertainty()	181
8.52.2.10 network_get_k()	182
8.52.2.11 network_get_master()	182
8.52.2.12 network_get_network()	182
8.52.2.13 network_get_ownid()	182
8.52.2.14 network_get_passive_count()	182
8.52.2.15 network_get_peer_index()	183
8.52.2.16 network_get_peer_state()	183
8.52.2.17 network_get_peers()	183
8.52.2.18 network_init()	184
8.52.2.19 network_is_acknowledged()	184
8.52.2.20 network_is_master()	185
8.52.2.21 network_print_certainty()	185

8.52.2.22 network_print_positions()	185
8.52.2.23 network_remove_peer()	185
8.52.2.24 network_reset_certainty()	185
8.52.2.25 network_reset_measurements()	186
8.52.2.26 network_set_acknowledged()	186
8.52.2.27 network_set_distance()	186
8.52.2.28 network_set_expected_seq_num()	186
8.52.2.29 network_set_k()	187
8.52.2.30 network_set_master()	187
8.52.2.31 network_set_self_pos()	187
8.52.2.32 network_store_final()	188
8.52.2.33 network_store_passive()	188
8.52.2.34 network_store_poll()	188
8.52.2.35 network_store_resp_tx()	189
8.52.2.36 network_update_certainty()	189
8.52.2.37 network_update_peers_from_sync()	189
8.53 uwb_network.h	190

Index	193
--------------	------------

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

Timing constants	9
----------------------------	---

Chapter 2

Directory Hierarchy

2.1 Directories

Accelerometer	13
imu.c	51
imu.h	54
imu_setup.h	58
lsm6dsv.c	63
lsm6dsv.h	64
lsm6dsv_self_test.c	66
lsm6dsv_ST_example.h	68
read_data_polling.c	69
App	13
Accelerometer	13
imu.c	51
imu.h	54
imu_setup.h	58
lsm6dsv.c	63
lsm6dsv.h	64
lsm6dsv_self_test.c	66
lsm6dsv_ST_example.h	68
read_data_polling.c	69
Blink	13
blink.c	69
blink.h	70
Calculations	14
calibrate.c	71
calibrate.h	74
distance.c	80
distance.h	85
ekf.c	95
ekf.h	100
DWM	14
DWM3000.c	110
DWM3000.h	113
DWM3000_driver.c	113
DWM3000_driver.h	121
DWM3000_setup.h	129
Generic	14
my_print.c	132

my_print.h	134
power.c	136
power.h	137
sleep.c	139
sleep.h	140
UART.c	141
UART.h	143
UWB_app	14
messages.c	146
messages.h	148
uwb_exchange.c	154
uwb_exchange.h	159
uwb_network.c	164
uwb_network.h	177
Blink	13
blink.c	69
blink.h	70
Calculations	14
calibrate.c	71
calibrate.h	74
distance.c	80
distance.h	85
ekf.c	95
ekf.h	100
DWM	14
DWM3000.c	110
DWM3000.h	113
DWM3000_driver.c	113
DWM3000_driver.h	121
DWM3000_setup.h	129
Generic	14
my_print.c	132
my_print.h	134
power.c	136
power.h	137
sleep.c	139
sleep.h	140
UART.c	141
UART.h	143
UWB_app	14
messages.c	146
messages.h	148
uwb_exchange.c	154
uwb_exchange.h	159
uwb_network.c	164
uwb_network.h	177

Chapter 3

Data Structure Index

3.1 Data Structures

Here are the data structures with brief descriptions:

BatteryStatus_t	15
boot_config_t	16
coop_ekf_t	16
dwm_rx_frame_t	18
dwm_rx_raw_frame_t	19
dwm_tx_frame_t	20
ekf_node_hint_t	21
first_order_t	22
imu_fifo_data_t	
Batch of samples drained from the IMU FIFO in one call to imu_read_fifo	23
imu_sample_t	
One IMU FIFO sample — accel (raw int16) paired with SFLP gravity (f16)	23
measurements_t	24
msg_final_t	26
msg_passive_t	28
msg_poll_t	
Payload for <code>MSG_TYPE_POLL</code>	31
msg_response_t	
Payload for <code>MSG_TYPE_RESPONSE</code>	32
msg_share_t	32
msg_sync_t	
Payload for <code>MSG_TYPE_SYNC</code>	34
msg_t	
Container for a fully decoded UWB message of any type	35
network_t	
Global network state for this device	37
node_peer_state_t	
Identity and position state of one UWB network node	39
node_t	39
print_msg_t	
One queued print message — tick-stamped at enqueue time	40
second_order_t	41
ss_twr_t	42
tdoa_twr_t	43
third_order_t	44

timestamps_t	45
twr_observation_t	46
twr_timestamps_t	47
uwb_rx_meas_t A single received UWB frame measurement	48

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

App/Accelerometer/ imu.c	51
App/Accelerometer/ imu.h	54
App/Accelerometer/ imu_setup.h	58
App/Accelerometer/ lsm6dsv.c	63
App/Accelerometer/ lsm6dsv.h	64
App/Accelerometer/ lsm6dsv_self_test.c	66
App/Accelerometer/ lsm6dsv_ST_example.h	68
App/Accelerometer/ read_data_polling.c	69
App/Blink/ blink.c	69
App/Blink/ blink.h	70
App/Calculations/ calibrate.c	71
App/Calculations/ calibrate.h	74
App/Calculations/ distance.c	80
App/Calculations/ distance.h	85
App/Calculations/ ekf.c	95
App/Calculations/ ekf.h	100
App/DWM/ DWM3000.c	110
App/DWM/ DWM3000.h	113
App/DWM/ DWM3000_driver.c	113
App/DWM/ DWM3000_driver.h	121
App/DWM/ DWM3000_setup.h	129
App/Generic/ my_print.c	132
App/Generic/ my_print.h	134
App/Generic/ power.c	136
App/Generic/ power.h	137
App/Generic/ sleep.c	139
App/Generic/ sleep.h	140
App/Generic/ UART.c	141
App/Generic/ UART.h	143
App/UWB_app/ messages.c	
UWB TWR message encode/decode implementation	146
App/UWB_app/ messages.h	
UWB TWR message definitions, offsets, and decoded data structures	148
App/UWB_app/ uwb_exchange.c	154
App/UWB_app/ uwb_exchange.h	
Public API and configuration constants for the UWB ranging exchange	159

App/UWB_app/ uwb_network.c	
Implementation of the UWB network state management API	164
App/UWB_app/ uwb_network.h	
Network state management for the UWB ranging system	177

Chapter 5

Topic Documentation

5.1 Timing constants

Macros

- #define DEEP_SLEEP 200U
DW3xxx deep-sleep duration (ms).
- #define T_SYNC_RX_SET 200U
Slave RX window for the initial SYNC broadcast (ms).
- #define T_SYNC_RX_ANSWER 10U
Master RX window waiting for slave join replies after SYNC (ms).
- #define T_SHARE_RX_WAIT 50U
Slave RX window waiting for a SHARE broadcast (ms).
- #define T_EARLY_WKUP 5U
Early-wakeup margin before a scheduled radio event (ms).
- #define UWB_US_TO_TICKS(us)
Convert microseconds to DW3xxx 40-bit timestamp ticks.
- #define T_RESPONSE_RX_WAIT 100U
Master RX window waiting for RESPONSE after POLL (ms).
- #define T_FINAL_RX_WAIT 100U
Slave RX window waiting for FINAL (ms).
- #define T_TWR_RX_WAIT 300U
Slave RX window covering the entire TWR exchange (ms).
- #define T_FINAL_TX_ASAP_TICKS UWB_US_TO_TICKS(800)
Delay from RESPONSE reception to FINAL transmission (ticks).
- #define T_PASSIVE_TX_ASAP_TICKS UWB_US_TO_TICKS(1000)
Minimum delay before a passive observer's TX (ticks).
- #define T_FINAL_TX_WAIT_MS 10U
Timeout waiting for TXFRS after scheduling the delayed FINAL (ms).
- #define T_PASSIVE_TX_WAIT_MS 10U
Timeout waiting for TXFRS after scheduling the passive TX (ms).
- #define SYNC_TIMEOUT_MAX 5U
Consecutive SYNC timeouts before a slave promotes itself to master.
- #define ALL_ID 0xFFFFU
Broadcast destination address.

5.1.1 Detailed Description

5.1.2 Macro Definition Documentation

5.1.2.1 ALL_ID

```
#define ALL_ID 0xFFFFU
```

Broadcast destination address.

5.1.2.2 DEEP_SLEEP

```
#define DEEP_SLEEP 200U
```

DW3xxx deep-sleep duration (ms).

5.1.2.3 SYNC_TIMEOUT_MAX

```
#define SYNC_TIMEOUT_MAX 5U
```

Consecutive SYNC timeouts before a slave promotes itself to master.

5.1.2.4 T_EARLY_WKUP

```
#define T_EARLY_WKUP 5U
```

Early-wakeup margin before a scheduled radio event (ms).

5.1.2.5 T_FINAL_RX_WAIT

```
#define T_FINAL_RX_WAIT 100U
```

Slave RX window waiting for FINAL (ms).

5.1.2.6 T_FINAL_TX_ASAP_TICKS

```
#define T_FINAL_TX_ASAP_TICKS UWB_US_TO_TICKS(800)
```

Delay from RESPONSE reception to FINAL transmission (ticks).

5.1.2.7 T_FINAL_TX_WAIT_MS

```
#define T_FINAL_TX_WAIT_MS 10U
```

Timeout waiting for TXFRS after scheduling the delayed FINAL (ms).

5.1.2.8 T_PASSIVE_TX_ASAP_TICKS

```
#define T_PASSIVE_TX_ASAP_TICKS UWB_US_TO_TICKS(1000)
```

Minimum delay before a passive observer's TX (ticks).

5.1.2.9 T_PASSIVE_TX_WAIT_MS

```
#define T_PASSIVE_TX_WAIT_MS 10U
```

Timeout waiting for TXFRS after scheduling the passive TX (ms).

5.1.2.10 T_RESPONSE_RX_WAIT

```
#define T_RESPONSE_RX_WAIT 100U
```

Master RX window waiting for RESPONSE after POLL (ms).

5.1.2.11 T_SHARE_RX_WAIT

```
#define T_SHARE_RX_WAIT 50U
```

Slave RX window waiting for a SHARE broadcast (ms).

5.1.2.12 T_SYNC_RX_ANSWER

```
#define T_SYNC_RX_ANSWER 10U
```

Master RX window waiting for slave join replies after SYNC (ms).

5.1.2.13 T_SYNC_RX_SET

```
#define T_SYNC_RX_SET 200U
```

Slave RX window for the initial SYNC broadcast (ms).

5.1.2.14 T_TWR_RX_WAIT

```
#define T_TWR_RX_WAIT 300U
```

Slave RX window covering the entire TWR exchange (ms).

5.1.2.15 UWB_US_TO_TICKS

```
#define UWB_US_TO_TICKS(  
    us)
```

Value:

```
((uint64_t)((us) * 63898ULL))
```

Convert microseconds to DW3xxx 40-bit timestamp ticks.

Chapter 6

Directory Documentation

6.1 App/Accelerometer Directory Reference

Files

- file [imu.c](#)
- file [imu.h](#)
- file [imu_setup.h](#)
- file [lsm6dsv.c](#)
- file [lsm6dsv.h](#)
- file [lsm6dsv_self_test.c](#)
- file [lsm6dsv_ST_example.h](#)
- file [read_data_polling.c](#)

6.2 App Directory Reference

Directories

- directory [Accelerometer](#)
- directory [Blink](#)
- directory [Calculations](#)
- directory [DWM](#)
- directory [Generic](#)
- directory [UWB_app](#)

6.3 App/Blink Directory Reference

Files

- file [blink.c](#)
- file [blink.h](#)

6.4 App/Calculations Directory Reference

Files

- file [calibrate.c](#)
- file [calibrate.h](#)
- file [distance.c](#)
- file [distance.h](#)
- file [ekf.c](#)
- file [ekf.h](#)

6.5 App/DWM Directory Reference

Files

- file [DWM3000.c](#)
- file [DWM3000.h](#)
- file [DWM3000_driver.c](#)
- file [DWM3000_driver.h](#)
- file [DWM3000_setup.h](#)

6.6 App/Generic Directory Reference

Files

- file [my_print.c](#)
- file [my_print.h](#)
- file [power.c](#)
- file [power.h](#)
- file [sleep.c](#)
- file [sleep.h](#)
- file [UART.c](#)
- file [UART.h](#)

6.7 App/UWB_app Directory Reference

Files

- file [messages.c](#)
UWB TWR message encode/decode implementation.
- file [messages.h](#)
UWB TWR message definitions, offsets, and decoded data structures.
- file [uwb_exchange.c](#)
- file [uwb_exchange.h](#)
Public API and configuration constants for the UWB ranging exchange.
- file [uwb_network.c](#)
Implementation of the UWB network state management API.
- file [uwb_network.h](#)
Network state management for the UWB ranging system.

Chapter 7

Data Structure Documentation

7.1 BatteryStatus_t Struct Reference

```
#include <power.h>
```

Data Fields

- float [battery_voltage_V](#)
- bool [usb_connected](#)
- bool [is_charging](#)
- bool [charge_complete](#)

7.1.1 Field Documentation

7.1.1.1 battery_voltage_V

```
float battery_voltage_V
```

7.1.1.2 charge_complete

```
bool charge_complete
```

7.1.1.3 is_charging

```
bool is_charging
```

7.1.1.4 usb_connected

```
bool usb_connected
```

The documentation for this struct was generated from the following file:

- App/Generic/[power.h](#)

7.2 boot_config_t Struct Reference

```
#include <UART.h>
```

Data Fields

- bool [charge_only](#)
- [ekf_node_hint_t](#) [hints](#) [4]
- bool [valid](#)

7.2.1 Field Documentation

7.2.1.1 charge_only

```
bool charge_only
```

7.2.1.2 hints

```
ekf\_node\_hint\_t hints[4]
```

7.2.1.3 valid

```
bool valid
```

The documentation for this struct was generated from the following file:

- App/Generic/[UART.h](#)

7.3 coop_ekf_t Struct Reference

```
#include <ekf.h>
```

Data Fields

- float [x](#) [[EKF_MAX_STATE](#)]
- float [P](#) [[EKF_MAX_STATE](#)][[EKF_MAX_STATE](#)]
- float [Q](#) [[EKF_MAX_STATE](#)][[EKF_MAX_STATE](#)]
- [uint8_t](#) [n_peers](#)
- [uint16_t](#) [peer_ids](#) [[EKF_MAX_PEERS](#)]
- bool [initialised](#)
- bool [peer_seeded](#) [[EKF_MAX_PEERS](#)]
- bool [peer_away](#) [[EKF_MAX_PEERS](#)]
- bool [peer_imu_valid](#) [[EKF_MAX_PEERS](#)]
- [uint32_t](#) [last_tick_ms](#)
- [uint16_t](#) [stationary_count](#) [[EKF_MAX_PEERS](#)]
- [int8_t](#) [self_slot](#)

7.3.1 Field Documentation

7.3.1.1 initialised

`bool initialised`

True after first `ekf_step()`

7.3.1.2 last_tick_ms

`uint32_t last_tick_ms`

Timestamp of last step

7.3.1.3 n_peers

`uint8_t n_peers`

Active slots (incl. away)

7.3.1.4 P

`float P[EKF_MAX_STATE][EKF_MAX_STATE]`

Joint covariance matrix

7.3.1.5 peer_away

`bool peer_away[EKF_MAX_PEERS]`

Peer absent from network

7.3.1.6 peer_ids

`uint16_t peer_ids[EKF_MAX_PEERS]`

Network ID per slot

7.3.1.7 peer_imu_valid

`bool peer_imu_valid[EKF_MAX_PEERS]`

≥ 1 valid IMU packet seen

7.3.1.8 peer_seeded

```
bool peer_seeded[EKF_MAX_PEERS]
```

First-fix seed done

7.3.1.9 Q

```
float Q[EKF_MAX_STATE][EKF_MAX_STATE]
```

Process noise (diagonal)

7.3.1.10 self_slot

```
int8_t self_slot
```

Slot index for self; -1=none

7.3.1.11 stationary_count

```
uint16_t stationary_count[EKF_MAX_PEERS]
```

Steps below IMU deadband

7.3.1.12 x

```
float x[EKF_MAX_STATE]
```

Joint state vector

The documentation for this struct was generated from the following file:

- App/Calculations/[ekf.h](#)

7.4 dwm_rx_frame_t Struct Reference

```
#include <DWM3000_driver.h>
```

Data Fields

- [dwm_rx_event_type_t](#) type
- [uint32_t](#) status
- [uint8_t](#) data [DWM_MAX_FRAME_LEN]
- [uint16_t](#) len
- [uint64_t](#) rx_timestamp
- [int16_t](#) rssi_q8
- [int16_t](#) fp_q8

7.4.1 Field Documentation

7.4.1.1 data

```
uint8_t data[DWM_MAX_FRAME_LEN]
```

7.4.1.2 fp_q8

```
int16_t fp_q8
```

7.4.1.3 len

```
uint16_t len
```

7.4.1.4 rssi_q8

```
int16_t rssi_q8
```

7.4.1.5 rx_timestamp

```
uint64_t rx_timestamp
```

7.4.1.6 status

```
uint32_t status
```

7.4.1.7 type

```
dwm_rx_event_type_t type
```

The documentation for this struct was generated from the following file:

- App/DWM/[DWM3000_driver.h](#)

7.5 dwm_rx_raw_frame_t Struct Reference

```
#include <DWM3000_driver.h>
```

Data Fields

- [dwm_rx_event_type_t](#) type
- [uint32_t](#) status
- [uint8_t](#) data [[DWM_MAX_FRAME_LEN](#)]
- [uint16_t](#) len
- [uint8_t](#) ts_buf [5]
- [dwt_cirdiags_t](#) rx_diag

7.5.1 Field Documentation

7.5.1.1 data

```
uint8_t data[DWM_MAX_FRAME_LEN]
```

7.5.1.2 len

```
uint16_t len
```

7.5.1.3 rx_diag

```
dwt_cirdiags_t rx_diag
```

7.5.1.4 status

```
uint32_t status
```

7.5.1.5 ts_buf

```
uint8_t ts_buf[5]
```

7.5.1.6 type

```
dwm\_rx\_event\_type\_t type
```

The documentation for this struct was generated from the following file:

- [App/DWM/DWM3000_driver.h](#)

7.6 dwm_tx_frame_t Struct Reference

```
#include <DWM3000_driver.h>
```

Data Fields

- uint8_t * [data](#)
- uint16_t [len](#)
- uint64_t [tx_timestamp](#)

7.6.1 Field Documentation

7.6.1.1 data

```
uint8_t* data
```

7.6.1.2 len

```
uint16_t len
```

7.6.1.3 tx_timestamp

```
uint64_t tx_timestamp
```

The documentation for this struct was generated from the following file:

- App/DWM/[DWM3000_driver.h](#)

7.7 ekf_node_hint_t Struct Reference

```
#include <UART.h>
```

Data Fields

- uint16_t [id](#)
- float [x](#)
- float [y](#)
- float [z](#)

7.7.1 Field Documentation

7.7.1.1 id

```
uint16_t id
```

7.7.1.2 x

float x

7.7.1.3 y

float y

7.7.1.4 z

float z

The documentation for this struct was generated from the following file:

- App/Generic/[UART.h](#)

7.8 first_order_t Struct Reference

```
#include <distance.h>
```

Data Fields

- uint16_t [initiator_id](#)
- uint16_t [responder_id](#)
- [twr_timestamps_t](#) twr
- double [result_distance_tick](#)

7.8.1 Field Documentation

7.8.1.1 initiator_id

uint16_t initiator_id

7.8.1.2 responder_id

uint16_t responder_id

7.8.1.3 result_distance_tick

double result_distance_tick

7.8.1.4 twr

```
twr_timestamps_t twr
```

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.9 imu_fifo_data_t Struct Reference

Batch of samples drained from the IMU FIFO in one call to [imu_read_fifo](#).

```
#include <imu.h>
```

Data Fields

- [imu_sample_t](#) `samples` [IMU_MAX_SAMPLES]
- `uint16_t` `count`

7.9.1 Detailed Description

Batch of samples drained from the IMU FIFO in one call to [imu_read_fifo](#).

7.9.2 Field Documentation

7.9.2.1 count

```
uint16_t count
```

Number of valid entries in `samples`.

7.9.2.2 samples

```
imu_sample_t samples[IMU_MAX_SAMPLES]
```

Sample array.

The documentation for this struct was generated from the following file:

- App/Accelerometer/[imu.h](#)

7.10 imu_sample_t Struct Reference

One IMU FIFO sample — accel (raw int16) paired with SFLP gravity (f16).

```
#include <imu.h>
```

Data Fields

- `int16_t accel_raw` [3]
- `uint16_t gravity_raw` [3]
- `uint32_t timestamp_ticks`

7.10.1 Detailed Description

One IMU FIFO sample — accel (raw int16) paired with SFLP gravity (f16).

7.10.2 Field Documentation

7.10.2.1 accel_raw

```
int16_t accel_raw[3]
```

Raw accelerometer counts; convert with `lsm6dsv_from_fs2_to_mg()`.

7.10.2.2 gravity_raw

```
uint16_t gravity_raw[3]
```

SFLP gravity vector in f16 (x, y, z).

7.10.2.3 timestamp_ticks

```
uint32_t timestamp_ticks
```

LSM6DSV hardware timestamp; 25 μ s per tick.

The documentation for this struct was generated from the following file:

- [App/Accelerometer/imu.h](#)

7.11 measurements_t Struct Reference

```
#include <uwb_network.h>
```

Data Fields

- [msg_poll_t](#) poll
- [uwb_rx_meas_t](#) poll_rx
- bool [poll_antenna_unreliable](#)
- [msg_response_t](#) resp
- [uint64_t](#) resp_tx
- [msg_final_t](#) final
- [uwb_rx_meas_t](#) final_rx
- bool [final_antenna_unreliable](#)
- [uint8_t](#) passive_count
- [msg_passive_t](#) passive [NETWORK_MAX_PEERS - 2]
- [uwb_rx_meas_t](#) passive_rx [NETWORK_MAX_PEERS - 2]
- [uint16_t](#) passive_device_id [NETWORK_MAX_PEERS - 2]
- bool [passive_antenna_unreliable](#) [NETWORK_MAX_PEERS - 2]

7.11.1 Field Documentation

7.11.1.1 final

[msg_final_t](#) final

7.11.1.2 final_antenna_unreliable

bool [final_antenna_unreliable](#)

7.11.1.3 final_rx

[uwb_rx_meas_t](#) final_rx

7.11.1.4 passive

[msg_passive_t](#) passive[[NETWORK_MAX_PEERS](#) - 2]

7.11.1.5 passive_antenna_unreliable

bool [passive_antenna_unreliable](#)[[NETWORK_MAX_PEERS](#) - 2]

7.11.1.6 passive_count

[uint8_t](#) passive_count

7.11.1.7 passive_device_id

[uint16_t](#) passive_device_id[[NETWORK_MAX_PEERS](#) - 2]

7.11.1.8 passive_rx

```
uwb_rx_meas_t passive_rx[NETWORK_MAX_PEERS - 2]
```

7.11.1.9 poll

```
msg_poll_t poll
```

7.11.1.10 poll_antenna_unreliable

```
bool poll_antenna_unreliable
```

7.11.1.11 poll_rx

```
uwb_rx_meas_t poll_rx
```

7.11.1.12 resp

```
msg_response_t resp
```

7.11.1.13 resp_tx

```
uint64_t resp_tx
```

The documentation for this struct was generated from the following file:

- App/UWB_app/[uwb_network.h](#)

7.12 msg_final_t Struct Reference

```
#include <messages.h>
```

Data Fields

- uint8_t [seq_num](#)
- uint64_t [poll_tx_ts](#)
- uint64_t [resp_rx_ts](#)
- uint64_t [final_tx_ts](#)
- int16_t [resp_pwr_diff_q8](#)
- bool [resp_antenna_unreliable](#)
- uint8_t [entry_count](#)
- uint64_t [entries](#) [NETWORK_MAX_PEERS - 2]
- int16_t [entry_pwr_diff_q8](#) [NETWORK_MAX_PEERS - 2]
- bool [entry_antenna_unreliable](#) [NETWORK_MAX_PEERS - 2]
- uint16_t [entry_id](#) [NETWORK_MAX_PEERS - 2]
- float [IMU_vel_horiz](#)
- float [IMU_vel_vert](#)

7.12.1 Field Documentation

7.12.1.1 entries

```
uint64_t entries[NETWORK_MAX_PEERS - 2]
```

TX timestamps of preceding PASSIVE frames. The TWR pair (initiator + responder) never send passive frames, hence the -2.

7.12.1.2 entry_antenna_unreliable

```
bool entry_antenna_unreliable[NETWORK_MAX_PEERS - 2]
```

7.12.1.3 entry_count

```
uint8_t entry_count
```

Number of valid entries in `entries`.

7.12.1.4 entry_id

```
uint16_t entry_id[NETWORK_MAX_PEERS - 2]
```

7.12.1.5 entry_pwr_diff_q8

```
int16_t entry_pwr_diff_q8[NETWORK_MAX_PEERS - 2]
```

7.12.1.6 final_tx_ts

```
uint64_t final_tx_ts
```

Predicted TX timestamp of this FINAL (40-bit).

7.12.1.7 IMU_vel_horiz

```
float IMU_vel_horiz
```

7.12.1.8 IMU_vel_vert

```
float IMU_vel_vert
```

7.12.1.9 poll_tx_ts

```
uint64_t poll_tx_ts
```

Actual TX timestamp of the POLL (40-bit, little-endian).

7.12.1.10 resp_antenna_unreliable

```
bool resp_antenna_unreliable
```

True if the RESPONSE frame was received with an unreliable antenna (e.g. due to blockage).

7.12.1.11 resp_pwr_diff_q8

```
int16_t resp_pwr_diff_q8
```

RSSI measured by initiator on RESPONSE reception (Q8).

7.12.1.12 resp_rx_ts

```
uint64_t resp_rx_ts
```

Initiator's RX timestamp of the RESPONSE (40-bit).

7.12.1.13 seq_num

```
uint8_t seq_num
```

Sequence number.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.13 msg_passive_t Struct Reference

```
#include <messages.h>
```

Data Fields

- uint8_t seq_num
- uint64_t poll_rx_ts
- int16_t poll_pwr_diff_q8
- bool poll_antenna_unreliable
- uint64_t resp_rx_ts
- int16_t resp_pwr_diff_q8
- bool resp_antenna_unreliable
- uint64_t passive_tx_ts
- uint8_t entry_count
- uint64_t entries [NETWORK_MAX_PEERS - 2]
- int16_t entry_pwr_diff_q8 [NETWORK_MAX_PEERS - 2]
- bool entry_antenna_unreliable [NETWORK_MAX_PEERS - 2]
- uint16_t entry_ids [NETWORK_MAX_PEERS - 2]
- float IMU_vel_horiz
- float IMU_vel_vert

7.13.1 Field Documentation

7.13.1.1 entries

```
uint64_t entries[NETWORK_MAX_PEERS - 2]
```

TX timestamps of preceding PASSIVE frames. The TWR pair (initiator + responder) never send passive frames, hence the -2.

7.13.1.2 entry_antenna_unreliable

```
bool entry_antenna_unreliable[NETWORK_MAX_PEERS - 2]
```

7.13.1.3 entry_count

```
uint8_t entry_count
```

Number of valid entries in `entries`.

7.13.1.4 entry_ids

```
uint16_t entry_ids[NETWORK_MAX_PEERS - 2]
```

Network IDs of the preceding PASSIVE frames, in the same order as `entries`.

7.13.1.5 entry_pwr_diff_q8

```
int16_t entry_pwr_diff_q8[NETWORK_MAX_PEERS - 2]
```

RSSI of the received PASSIVE frames at this node (Q8).

7.13.1.6 IMU_vel_horiz

`float IMU_vel_horiz`

7.13.1.7 IMU_vel_vert

`float IMU_vel_vert`

7.13.1.8 passive_tx_ts

`uint64_t passive_tx_ts`

Predicted TX timestamp of this PASSIVE frame (40-bit).

7.13.1.9 poll_antenna_unreliable

`bool poll_antenna_unreliable`

True if the POLL frame was received with an unreliable antenna (e.g. due to blockage).

7.13.1.10 poll_pwr_diff_q8

`int16_t poll_pwr_diff_q8`

RSSI of the received POLL at this node (Q8).

7.13.1.11 poll_rx_ts

`uint64_t poll_rx_ts`

RX timestamp of POLL at this passive node (40-bit).

7.13.1.12 resp_antenna_unreliable

`bool resp_antenna_unreliable`

7.13.1.13 resp_pwr_diff_q8

`int16_t resp_pwr_diff_q8`

RSSI of the received RESPONSE at this node (Q8).

7.13.1.14 resp_rx_ts

```
uint64_t resp_rx_ts
```

RX timestamp of RESPONSE at this passive node (40-bit).

7.13.1.15 seq_num

```
uint8_t seq_num
```

Sequence number matching the TWR exchange observed.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.14 msg_poll_t Struct Reference

Payload for MSG_TYPE_POLL.

```
#include <messages.h>
```

Data Fields

- uint8_t [seq_num](#)

7.14.1 Detailed Description

Payload for MSG_TYPE_POLL.

Sent by the initiator to start a ranging exchange. Only the sequence number is transmitted on the wire; the signal quality metrics and RX timestamp are populated from the DWM3000 frame metadata on the receiver side.

Note

Fields below the "Not in message" marker are not on the wire.

7.14.2 Field Documentation

7.14.2.1 seq_num

```
uint8_t seq_num
```

Sequence number.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.15 msg_response_t Struct Reference

Payload for MSG_TYPE_RESPONSE.

```
#include <messages.h>
```

Data Fields

- uint8_t [seq_num](#)

7.15.1 Detailed Description

Payload for MSG_TYPE_RESPONSE.

Sent by the responder after receiving a POLL. The RESPONSE carries no timestamps on the wire; the responder's RX/TX timestamps are delivered by the initiator inside the FINAL message. Signal quality metrics are populated from the DWM3000 frame metadata.

Note

Fields below the "Not in message" marker are not on the wire.

7.15.2 Field Documentation

7.15.2.1 seq_num

```
uint8_t seq_num
```

Sequence number.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.16 msg_share_t Struct Reference

```
#include <messages.h>
```

Data Fields

- uint8_t [seq_num](#)
- uint32_t [sleep_time](#)
- uint8_t [node_count](#)
- uint16_t [node_ids](#) [[NETWORK_MAX_PEERS](#)]
- uint8_t [vel_vert](#) [[NETWORK_MAX_PEERS](#)]
- uint8_t [vel_horiz](#) [[NETWORK_MAX_PEERS](#)]
- uint16_t [distance_mm](#) [[NETWORK_MAX_PAIRS](#)]
- uint8_t [accuracy](#) [[NETWORK_MAX_PAIRS](#)]

7.16.1 Detailed Description

SHARE broadcast payload.

The `node_ids[]` array defines the canonical ordering A, B, C, D... Pair entries follow the upper-triangle order: $\leftrightarrow (0,1),(0,2),(0,3),(1,2),(1,3),(2,3)$ i.e. for i in $0..\text{node_count}-1$, for j in $i+1..\text{node_count}-1 \rightarrow \text{pair index} = i*(2n-i-1)/2 + (j-i-1)$ Both sides encode/decode using the same deterministic formula — no IDs needed per pair.

7.16.2 Field Documentation

7.16.2.1 accuracy

```
uint8_t accuracy[NETWORK_MAX_PAIRS]
```

Certainty score (0=unknown, 255=best).

7.16.2.2 distance_mm

```
uint16_t distance_mm[NETWORK_MAX_PAIRS]
```

Distance 0–200 m in ~ 3.05 mm/LSB. wrong name

7.16.2.3 node_count

```
uint8_t node_count
```

Number of nodes in `node_ids[]`.

7.16.2.4 node_ids

```
uint16_t node_ids[NETWORK_MAX_PEERS]
```

Canonical node order: A, B, C, D...

7.16.2.5 seq_num

```
uint8_t seq_num
```

Sequence number.

7.16.2.6 sleep_time

```
uint32_t sleep_time
```

Planned sleep duration in ms.

7.16.2.7 vel_horiz

```
uint8_t vel_horiz[NETWORK_MAX_PEERS]
```

Horizontal speed, offset-binary.

7.16.2.8 vel_vert

```
uint8_t vel_vert[NETWORK_MAX_PEERS]
```

Vertical velocity, offset-binary.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.17 msg_sync_t Struct Reference

Payload for MSG_TYPE_SYNC.

```
#include <messages.h>
```

Data Fields

- uint8_t [seq_num](#)
- uint8_t [peer_count](#)
- uint16_t [peer_ids](#) [NETWORK_MAX_PEERS]

7.17.1 Detailed Description

Payload for MSG_TYPE_SYNC.

Announces presence and carries the list of all currently known peers in the network. Unjoined devices use this list to determine whether they need to request joining.

7.17.2 Field Documentation

7.17.2.1 peer_count

```
uint8_t peer_count
```

Number of peers in the list.

7.17.2.2 peer_ids

```
uint16_t peer_ids[NETWORK_MAX_PEERS]
```

IDs of all known active peers, excludes sender.

7.17.2.3 seq_num

```
uint8_t seq_num
```

Sequence number.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.18 msg_t Struct Reference

Container for a fully decoded UWB message of any type.

```
#include <messages.h>
```

Data Fields

- [msg_type_t](#) type
- [uint16_t](#) len
- [uint16_t](#) sender
- [uint16_t](#) receiver
- union {
 - [msg_sync_t](#) sync
 - [msg_poll_t](#) poll
 - [msg_response_t](#) response
 - [msg_final_t](#) final
 - [msg_share_t](#) share
 - [msg_passive_t](#) passive
- [data](#)

7.18.1 Detailed Description

Container for a fully decoded UWB message of any type.

After a successful call to [msg_decode](#), `type` identifies which union member is valid. Always check `type` before accessing `data`.

The union is sized to the largest member ([msg_passive_t](#)), so this struct is safe to allocate on the stack.

7.18.2 Field Documentation

7.18.2.1 [union]

```
union { ... } data
```

7.18.2.2 final

```
msg_final_t final
```

Valid when `type == MSG_TYPE_FINAL`.

7.18.2.3 len

```
uint16_t len
```

Size in bytes of the active union member.

7.18.2.4 passive

```
msg_passive_t passive
```

Valid when `type == MSG_TYPE_PASSIVE`.

7.18.2.5 poll

```
msg_poll_t poll
```

Valid when `type == MSG_TYPE_POLL`.

7.18.2.6 receiver

```
uint16_t receiver
```

Receiver device ID decoded from the frame header.

7.18.2.7 response

```
msg_response_t response
```

Valid when `type == MSG_TYPE_RESPONSE`.

7.18.2.8 sender

```
uint16_t sender
```

Sender device ID decoded from the frame header.

7.18.2.9 share

`msg_share_t` share

Valid when `type == MSG_TYPE_SHARE`.

7.18.2.10 sync

`msg_sync_t` sync

Valid when `type == MSG_TYPE_SYNC`.

7.18.2.11 type

`msg_type_t` type

Decoded message type — determines valid union member.

The documentation for this struct was generated from the following file:

- App/UWB_app/[messages.h](#)

7.19 network_t Struct Reference

Global network state for this device.

```
#include <uwb_network.h>
```

Data Fields

- [node_t](#) peers [NETWORK_MAX_PEERS]
- [uint8_t](#) expected_seq_num
- [uint8_t](#) count
- [uint16_t](#) master_id
- [node_t](#) self
- [bool](#) acknowledged
- [measurements_t](#) measurements

7.19.1 Detailed Description

Global network state for this device.

Managed exclusively through the network API. Direct access to the underlying instance is not exposed.

7.19.2 Field Documentation

7.19.2.1 acknowledged

`bool acknowledged`

`true` once the master has included `self.id` in a SYNC peer list.

7.19.2.2 count

`uint8_t count`

Number of valid entries in `peers`.

7.19.2.3 expected_seq_num

`uint8_t expected_seq_num`

Sequence number expected in the next incoming UWB frame.

7.19.2.4 master_id

`uint16_t master_id`

ID of the current master. Equals `self.id` when this device is master.

7.19.2.5 measurements

`measurements_t measurements`

All measurements for the current exchange. Reset at the start of each round.

7.19.2.6 peers

`node_t peers[NETWORK_MAX_PEERS]`

7.19.2.7 self

`node_t self`

This device's own identity and position.

The documentation for this struct was generated from the following file:

- `App/UWB_app/uwb_network.h`

7.20 node_peer_state_t Struct Reference

Identity and position state of one UWB network node.

```
#include <uwb_network.h>
```

Data Fields

- uint16_t [peer_id](#)
- uint16_t [distance_scaled](#)
- uint8_t [certainty](#)

7.20.1 Detailed Description

Identity and position state of one UWB network node.

Used for both peers ([network_t.peers](#)) and self ([network_t.self](#)). The `certainty` field drives DS-TWR target selection: the peer with the lowest certainty is chosen as the next ranging target.

7.20.2 Field Documentation

7.20.2.1 certainty

```
uint8_t certainty
```

Certainty score for this peer's distance (0-255).

7.20.2.2 distance_scaled

```
uint16_t distance_scaled
```

7.20.2.3 peer_id

```
uint16_t peer_id
```

Network ID of the peer (0 = slot unused).

The documentation for this struct was generated from the following file:

- App/UWB_app/[uwb_network.h](#)

7.21 node_t Struct Reference

```
#include <uwb_network.h>
```

Data Fields

- `uint16_t id`
- `float pos [3]`
- `uint8_t imu_vel_vert`
- `uint8_t imu_vel_horiz`
- `node_peer_state_t peers [NETWORK_MAX_PEERS]`

7.21.1 Field Documentation

7.21.1.1 id

```
uint16_t id
```

Network ID.

7.21.1.2 imu_vel_horiz

```
uint8_t imu_vel_horiz
```

7.21.1.3 imu_vel_vert

```
uint8_t imu_vel_vert
```

7.21.1.4 peers

```
node_peer_state_t peers [NETWORK_MAX_PEERS]
```

7.21.1.5 pos

```
float pos [3]
```

Position estimate [x, y, z] in metres. Zero-initialised until first fix.

The documentation for this struct was generated from the following file:

- `App/UWB_app/uwb_network.h`

7.22 print_msg_t Struct Reference

One queued print message — tick-stamped at enqueue time.

Data Fields

- uint32_t [tick](#)
- uint16_t [len](#)
- char [data](#) [[PRINT_BUF_SIZE](#)]

7.22.1 Detailed Description

One queued print message — tick-stamped at enqueue time.

Kept small so the full queue fits in a predictable amount of heap.

7.22.2 Field Documentation

7.22.2.1 data

```
char data[PRINT_BUF_SIZE]
```

Formatted string.

7.22.2.2 len

```
uint16_t len
```

Valid byte count in `data` (without NUL).

7.22.2.3 tick

```
uint32_t tick
```

RTOS tick count at time of `mprintf` call.

The documentation for this struct was generated from the following file:

- App/Generic/[my_print.c](#)

7.23 second_order_t Struct Reference

```
#include <distance.h>
```

Data Fields

- uint16_t [initiator_id](#)
- uint16_t [responder_id](#)
- [twr_observation_t](#) [twr_observation](#)
- [ss_twr_t](#) [twr](#)
- bool [init_rx_unreliable](#)
- bool [answer_rx_unreliable](#)
- double [result_distance_tick](#)

7.23.1 Field Documentation

7.23.1.1 answer_rx_unreliable

```
bool answer_rx_unreliable
```

7.23.1.2 init_rx_unreliable

```
bool init_rx_unreliable
```

7.23.1.3 initiator_id

```
uint16_t initiator_id
```

7.23.1.4 responder_id

```
uint16_t responder_id
```

7.23.1.5 result_distance_tick

```
double result_distance_tick
```

7.23.1.6 twr

```
ss_twr_t twr
```

7.23.1.7 twr_observation

```
twr_observation_t twr_observation
```

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.24 ss_twr_t Struct Reference

```
#include <distance.h>
```


Data Fields

- uint64_t [init_tx](#)
- [uwb_rx_meas_t](#) [init_rx](#)
- uint64_t [answer_tx](#)
- [uwb_rx_meas_t](#) [answer_rx](#)

7.24.1 Field Documentation

7.24.1.1 answer_rx

[uwb_rx_meas_t](#) [answer_rx](#)

7.24.1.2 answer_tx

uint64_t [answer_tx](#)

7.24.1.3 init_rx

[uwb_rx_meas_t](#) [init_rx](#)

7.24.1.4 init_tx

uint64_t [init_tx](#)

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.25 tdoa_twr_t Struct Reference

```
#include <distance.h>
```

Data Fields

- uint64_t [init_tx](#)
- [uwb_rx_meas_t](#) [init_rx](#)
- [uwb_rx_meas_t](#) [master_init_rx](#)
- uint64_t [answer_tx](#)
- [uwb_rx_meas_t](#) [answer_rx](#)

7.25.1 Field Documentation

7.25.1.1 answer_rx

`uwb_rx_meas_t` answer_rx

7.25.1.2 answer_tx

`uint64_t` answer_tx

7.25.1.3 init_rx

`uwb_rx_meas_t` init_rx

7.25.1.4 init_tx

`uint64_t` init_tx

7.25.1.5 master_init_rx

`uwb_rx_meas_t` master_init_rx

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.26 third_order_t Struct Reference

```
#include <distance.h>
```

Data Fields

- `uint16_t` [initiator_id](#)
- `uint16_t` [responder_id](#)
- `twr_observation_t` [twr_observation_c](#)
- `twr_observation_t` [twr_observation_d](#)
- `ss_twr_t` [twr](#)
- `bool` [answer_rx_unreliable](#)
- `double` [result_distance_tick](#)

7.26.1 Field Documentation

7.26.1.1 answer_rx_unreliable

```
bool answer_rx_unreliable
```

7.26.1.2 initiator_id

```
uint16_t initiator_id
```

7.26.1.3 responder_id

```
uint16_t responder_id
```

7.26.1.4 result_distance_tick

```
double result_distance_tick
```

7.26.1.5 twr

```
ss_twr_t twr
```

7.26.1.6 twr_observation_c

```
twr_observation_t twr_observation_c
```

7.26.1.7 twr_observation_d

```
twr_observation_t twr_observation_d
```

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.27 timestamps_t Struct Reference

```
#include <distance.h>
```

Data Fields

- [first_order_t](#) first
- [second_order_t](#) second [MAX_SECOND_ORDER]
- [uint8_t](#) second_count
- [third_order_t](#) third [MAX_THIRD_ORDER]
- [uint8_t](#) third_count

7.27.1 Field Documentation

7.27.1.1 first

```
first_order_t first
```

7.27.1.2 second

```
second_order_t second[MAX_SECOND_ORDER]
```

7.27.1.3 second_count

```
uint8_t second_count
```

7.27.1.4 third

```
third_order_t third[MAX_THIRD_ORDER]
```

7.27.1.5 third_count

```
uint8_t third_count
```

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.28 twr_observation_t Struct Reference

```
#include <distance.h>
```

Data Fields

- [uwb_rx_meas_t](#) poll_rx
- [uwb_rx_meas_t](#) resp_rx
- [bool](#) poll_rx_unreliable
- [bool](#) resp_rx_unreliable

7.28.1 Field Documentation

7.28.1.1 poll_rx

`uwb_rx_meas_t poll_rx`

7.28.1.2 poll_rx_unreliable

`bool poll_rx_unreliable`

7.28.1.3 resp_rx

`uwb_rx_meas_t resp_rx`

7.28.1.4 resp_rx_unreliable

`bool resp_rx_unreliable`

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.29 twr_timestamps_t Struct Reference

```
#include <distance.h>
```

Data Fields

- `uint64_t poll_tx`
- `uwb_rx_meas_t poll_rx`
- `bool poll_rx_unreliable`
- `uint64_t resp_tx`
- `uwb_rx_meas_t resp_rx`
- `bool resp_rx_unreliable`
- `uint64_t final_tx`
- `uwb_rx_meas_t final_rx`
- `bool final_rx_unreliable`

7.29.1 Field Documentation

7.29.1.1 final_rx

`uwb_rx_meas_t final_rx`

7.29.1.2 final_rx_unreliable

```
bool final_rx_unreliable
```

7.29.1.3 final_tx

```
uint64_t final_tx
```

7.29.1.4 poll_rx

```
uw_b_rx_meas_t poll_rx
```

7.29.1.5 poll_rx_unreliable

```
bool poll_rx_unreliable
```

7.29.1.6 poll_tx

```
uint64_t poll_tx
```

7.29.1.7 resp_rx

```
uw_b_rx_meas_t resp_rx
```

7.29.1.8 resp_rx_unreliable

```
bool resp_rx_unreliable
```

7.29.1.9 resp_tx

```
uint64_t resp_tx
```

The documentation for this struct was generated from the following file:

- App/Calculations/[distance.h](#)

7.30 uw_b_rx_meas_t Struct Reference

A single received UWB frame measurement.

```
#include <uw_b_network.h>
```

Data Fields

- uint64_t [ts](#)
- int16_t [pwr_diff_q8](#)

7.30.1 Detailed Description

A single received UWB frame measurement.

Bundles the hardware timestamp with the signal quality metrics returned by the DW3xxx driver. TX events use plain `uint64_t` since no signal measurement exists on transmission.

7.30.2 Field Documentation

7.30.2.1 `pwr_diff_q8`

```
int16_t pwr_diff_q8
```

Total received power, Q8 fixed-point (dBm × 256).

7.30.2.2 `ts`

```
uint64_t ts
```

UWB hardware timestamp (~15 ps resolution).

The documentation for this struct was generated from the following file:

- App/UWB_app/[uwb_network.h](#)

Chapter 8

File Documentation

8.1 App/Accelerometer/imu.c File Reference

```
#include <math.h>
#include <string.h>
#include "cmsis_os2.h"
#include "lsm6dsv_reg.h"
#include "lsm6dsv_platform.h"
#include "../Generic/my_print.h"
#include "imu_setup.h"
#include "lsm6dsv_ST_example.h"
#include "imu.h"
#include "main.h"
```

Macros

- `#define LSM6DSV_FIFO_DATA_OUT_TAG_ADDR 0x78U`

Functions

- void `imu_calibrate` (void)
Collect and average accelerometer bias over IMU_CAL_SAMPLES samples.
- void `imu_init` (void)
Initialise the LSM6DSV IMU and configure it for FIFO-based operation.
- void `imu_read_fifo` (`imu_fifo_data_t` *fifo)
Drain the IMU FIFO into fifo in a single burst bus transaction.
- void `imu_integrate` (const `imu_fifo_data_t` *fifo, float *out_pitch_rad, float *out_speed_horiz, float *out_vel_z, uint32_t *out_ticks_elapsed)
Integrate a FIFO batch into velocity and pitch estimates.

Variables

- stmdev_ctx_t `dev_ctx`

8.1.1 Macro Definition Documentation

8.1.1.1 LSM6DSV_FIFO_DATA_OUT_TAG_ADDR

```
#define LSM6DSV_FIFO_DATA_OUT_TAG_ADDR 0x78U
```

8.1.2 Function Documentation

8.1.2.1 imu_calibrate()

```
void imu_calibrate (  
    void )
```

Collect and average accelerometer bias over IMU_CAL_SAMPLES samples.

The device must be stationary and level during calibration. The function signals the calibration window to the operator via LED_R:

- LED toggles ON — 2-second settling window (place the device flat).
- LED toggles OFF — 1-second final settling period.
- LED toggles ON during sampling, then OFF on completion.

The FIFO is flushed via a bypass/stream cycle before sampling to discard any frames that arrived before the device was stationary. `last_ts` is reset so [imu_integrate](#) does not use a stale reference after calibration.

Gravity is subtracted sample-by-sample using the SFLP gravity vector, so the bias estimate is independent of sensor orientation.

Results are stored in `s_accel_bias[]` and `s_calibrated` is set to true.

8.1.2.2 imu_init()

```
void imu_init (  
    void )
```

Initialise the LSM6DSV IMU and configure it for FIFO-based operation.

Sets up the ST driver context (bus handle selected by `UWB_BOARD_V1_1`), verifies the `WHO_AM_I` register, applies a soft reset, then configures:

- Full-scale ranges and low-pass filters for accel and gyro.
- Hardware timestamp at `IMU_FIFO_TS_DEC` decimation (25 μ s/tick).
- FIFO in STREAM mode: XL batched, gyro bypassed (feeds SFLP directly), gravity vector output enabled.
- Output data rates for XL, gyro, and SFLP.

Calls `vTaskDelete(NULL)` and loops if the device ID is not detected, preventing silent misconfiguration.

8.1.2.3 imu_integrate()

```
void imu_integrate (
    const imu_fifo_data_t * fifo,
    float * out_pitch_rad,
    float * out_speed_horiz,
    float * out_vel_z,
    uint32_t * out_ticks_elapsed)
```

Integrate a FIFO batch into velocity and pitch estimates.

Pitch is computed once from the last sample's SFLP gravity vector (gravity changes slowly; per-sample recomputation wastes cycles). The velocity integration loop is kept tight — no transcendental calls inside.

Z velocity is projected along the gravity axis; horizontal speed is the magnitude of the residual after subtracting the vertical component. Both `last_ts` (module-static) and `s_accel_bias[]` are applied.

Samples with `dt <= 0` or `dt > 1 s` are skipped to guard against timestamp glitches or first-sample stale-reference conditions.

Parameters

in	<i>fifo</i>	Populated FIFO batch from imu_read_fifo .
out	<i>out_pitch_rad</i>	Pitch angle in radians (angle from vertical).
out	<i>out_speed_horiz</i>	Horizontal speed magnitude in m/s (≥ 0).
out	<i>out_vel_z</i>	Vertical velocity in m/s (signed).
out	<i>out_ticks_elapsed</i>	Hardware ticks spanned by this batch (0 if empty).

8.1.2.4 imu_read_fifo()

```
void imu_read_fifo (
    imu_fifo_data_t * fifo)
```

Drain the IMU FIFO into `fifo` in a single burst bus transaction.

FIFO layout at 120 Hz XL = 120 Hz SFLP (1:1 ratio) with `TMSTMP_DEC_8`:

[TIMESTAMP] [XL]×8 [GRAVITY]×8

One timestamp entry appears per group of 8 paired XL + GRAVITY entries.

Parsing strategy:

1. Read FIFO level (2 register bytes — fast).
2. Burst-read all `num×7` bytes in one SPI/I2C transaction.
3. Walk the raw buffer, pairing each `XL_NC_TAG` entry with the immediately following `SFLP_GRAVITY_VECTOR_TAG` entry.
4. Timestamps are back-filled uniformly across the 8-sample group once the closing timestamp arrives.

Parameters

out	fifo	Output struct; <code>count</code> and <code>samples</code> are populated. Must not be NULL.
-----	------	---

8.1.3 Variable Documentation**8.1.3.1 dev_ctx**

```
stmdev_ctx_t dev_ctx
```

8.2 App/Accelerometer/imu.h File Reference

```
#include "imu_setup.h"
#include "main.h"
```

Data Structures

- struct [imu_sample_t](#)
One IMU FIFO sample — accel (raw int16) paired with SFLP gravity (f16).
- struct [imu_fifo_data_t](#)
Batch of samples drained from the IMU FIFO in one call to [imu_read_fifo](#).

Macros

- #define [IMU_ACCEL_X_MS2\(s\)](#)
Accelerometer X axis in m/s^2 (raw $\rightarrow$ mg $\rightarrow$ m/s^2).
- #define [IMU_ACCEL_Y_MS2\(s\)](#)
Accelerometer Y axis in m/s^2 .
- #define [IMU_ACCEL_Z_MS2\(s\)](#)
Accelerometer Z axis in m/s^2 .
- #define [IMU_GRAV_X\(s\)](#)
SFLP gravity X component in m/s^2 (f16 $\rightarrow$ mg $\rightarrow$ m/s^2).
- #define [IMU_GRAV_Y\(s\)](#)
SFLP gravity Y component in m/s^2 .
- #define [IMU_GRAV_Z\(s\)](#)
SFLP gravity Z component in m/s^2 .

Functions

- void [imu_init](#) (void)
Initialise the LSM6DSV IMU and configure it for FIFO-based operation.
- void [imu_read_fifo](#) ([imu_fifo_data_t](#) *fifo)
Drain the IMU FIFO into `fifo` in a single burst bus transaction.
- void [imu_integrate](#) (const [imu_fifo_data_t](#) *fifo, float *out_pitch_rad, float *out_speed_horiz, float *out_vel_z, uint32_t *out_ticks_elapsed)
Integrate a FIFO batch into velocity and pitch estimates.
- void [imu_calibrate](#) (void)
Collect and average accelerometer bias over `IMU_CAL_SAMPLES` samples.

8.2.1 Macro Definition Documentation

8.2.1.1 IMU_ACCEL_X_MS2

```
#define IMU_ACCEL_X_MS2(  
    s)
```

Value:

```
(lsm6dsv_from_fs2_to_mg((s)->accel_raw[0]) * 0.00981f)
```

Accelerometer X axis in m/s^2 (raw $\rightarrow$ mg $\rightarrow$ m/s^2).

8.2.1.2 IMU_ACCEL_Y_MS2

```
#define IMU_ACCEL_Y_MS2(  
    s)
```

Value:

```
(lsm6dsv_from_fs2_to_mg((s)->accel_raw[1]) * 0.00981f)
```

Accelerometer Y axis in m/s^2 .

8.2.1.3 IMU_ACCEL_Z_MS2

```
#define IMU_ACCEL_Z_MS2(  
    s)
```

Value:

```
(lsm6dsv_from_fs2_to_mg((s)->accel_raw[2]) * 0.00981f)
```

Accelerometer Z axis in m/s^2 .

8.2.1.4 IMU_GRAV_X

```
#define IMU_GRAV_X(  
    s)
```

Value:

```
(lsm6dsv_from_sflp_to_mg((s)->gravity_raw[0]) * 0.00981f)
```

SFLP gravity X component in m/s^2 (f16 $\rightarrow$ mg $\rightarrow$ m/s^2).

8.2.1.5 IMU_GRAV_Y

```
#define IMU_GRAV_Y(  
    s)
```

Value:

```
(lsm6dsv_from_sflp_to_mg((s)->gravity_raw[1]) * 0.00981f)
```

SFLP gravity Y component in m/s^2 .

8.2.1.6 IMU_GRAV_Z

```
#define IMU_GRAV_Z(  
    s)
```

Value:

```
(lsm6dsv_from_sflp_to_mg((s)->gravity_raw[2]) * 0.00981f)
```

SFLP gravity Z component in m/s^2 .

8.2.2 Function Documentation

8.2.2.1 imu_calibrate()

```
void imu_calibrate (  
    void )
```

Collect and average accelerometer bias over `IMU_CAL_SAMPLES` samples.

The device must be stationary and level during calibration. The function signals the calibration window to the operator via LED_R:

- LED toggles ON — 2-second settling window (place the device flat).
- LED toggles OFF — 1-second final settling period.
- LED toggles ON during sampling, then OFF on completion.

The FIFO is flushed via a bypass/stream cycle before sampling to discard any frames that arrived before the device was stationary. `last_ts` is reset so [imu_integrate](#) does not use a stale reference after calibration.

Gravity is subtracted sample-by-sample using the SFLP gravity vector, so the bias estimate is independent of sensor orientation.

Results are stored in `s_accel_bias[]` and `s_calibrated` is set to true.

8.2.2.2 imu_init()

```
void imu_init (  
    void )
```

Initialise the LSM6DSV IMU and configure it for FIFO-based operation.

Sets up the ST driver context (bus handle selected by `UWB_BOARD_V1_1`), verifies the `WHO_AM_I` register, applies a soft reset, then configures:

- Full-scale ranges and low-pass filters for accel and gyro.
- Hardware timestamp at `IMU_FIFO_TS_DEC` decimation (25 μs /tick).
- FIFO in STREAM mode: XL batched, gyro bypassed (feeds SFLP directly), gravity vector output enabled.
- Output data rates for XL, gyro, and SFLP.

Calls `vTaskDelete(NULL)` and loops if the device ID is not detected, preventing silent misconfiguration.

8.2.2.3 imu_integrate()

```
void imu_integrate (
    const imu_fifo_data_t * fifo,
    float * out_pitch_rad,
    float * out_speed_horiz,
    float * out_vel_z,
    uint32_t * out_ticks_elapsed)
```

Integrate a FIFO batch into velocity and pitch estimates.

Pitch is computed once from the last sample's SFLP gravity vector (gravity changes slowly; per-sample recomputation wastes cycles). The velocity integration loop is kept tight — no transcendental calls inside.

Z velocity is projected along the gravity axis; horizontal speed is the magnitude of the residual after subtracting the vertical component. Both `last_ts` (module-static) and `s_accel_bias[]` are applied.

Samples with `dt <= 0` or `dt > 1 s` are skipped to guard against timestamp glitches or first-sample stale-reference conditions.

Parameters

in	<i>fifo</i>	Populated FIFO batch from imu_read_fifo .
out	<i>out_pitch_rad</i>	Pitch angle in radians (angle from vertical).
out	<i>out_speed_horiz</i>	Horizontal speed magnitude in m/s (≥ 0).
out	<i>out_vel_z</i>	Vertical velocity in m/s (signed).
out	<i>out_ticks_elapsed</i>	Hardware ticks spanned by this batch (0 if empty).

8.2.2.4 imu_read_fifo()

```
void imu_read_fifo (
    imu_fifo_data_t * fifo)
```

Drain the IMU FIFO into `fifo` in a single burst bus transaction.

FIFO layout at 120 Hz XL = 120 Hz SFLP (1:1 ratio) with `TMSTMP_DEC_8`:

[TIMESTAMP] [XL]×8 [GRAVITY]×8

One timestamp entry appears per group of 8 paired XL + GRAVITY entries.

Parsing strategy:

1. Read FIFO level (2 register bytes — fast).
2. Burst-read all `num×7` bytes in one SPI/I2C transaction.
3. Walk the raw buffer, pairing each `XL_NC_TAG` entry with the immediately following `SFLP_GRAVITY_VECTOR_TAG` entry.
4. Timestamps are back-filled uniformly across the 8-sample group once the closing timestamp arrives.

Parameters

out	<i>fifo</i>	Output struct; count and samples are populated. Must not be NULL.
-----	-------------	---

8.3 imu.h

[Go to the documentation of this file.](#)

```

00001 #ifndef IMU_H
00002 #define IMU_H
00003
00004 #include "imu_setup.h"
00005 #include "main.h"
00006
00007 typedef struct {
00008     int16_t accel_raw[3];
00009     uint16_t gravity_raw[3];
00010     uint32_t timestamp_ticks;
00011 } imu_sample_t;
00012
00013 typedef struct {
00014     imu_sample_t samples[IMU_MAX_SAMPLES];
00015     uint16_t count;
00016 } imu_fifo_data_t;
00017
00018 static inline float f16_to_f32(uint16_t h)
00019 {
00020     union { float f; uint32_t u; } conv;
00021     conv.u = lsm6dsv_from_f16_to_f32(h);
00022     return conv.f;
00023 }
00024
00025 #define IMU_ACCEL_X_MS2(s) (lsm6dsv_from_fs2_to_mg((s)->accel_raw[0]) * 0.00981f)
00026 #define IMU_ACCEL_Y_MS2(s) (lsm6dsv_from_fs2_to_mg((s)->accel_raw[1]) * 0.00981f)
00027 #define IMU_ACCEL_Z_MS2(s) (lsm6dsv_from_fs2_to_mg((s)->accel_raw[2]) * 0.00981f)
00028
00029 #define IMU_GRAV_X(s) (lsm6dsv_from_sflp_to_mg((s)->gravity_raw[0]) * 0.00981f)
00030 #define IMU_GRAV_Y(s) (lsm6dsv_from_sflp_to_mg((s)->gravity_raw[1]) * 0.00981f)
00031 #define IMU_GRAV_Z(s) (lsm6dsv_from_sflp_to_mg((s)->gravity_raw[2]) * 0.00981f)
00032
00033 void imu_init(void);
00034 void imu_read_fifo(imu_fifo_data_t *fifo);
00035
00036 void imu_integrate(const imu_fifo_data_t *fifo,
00037                   float *out_pitch_rad,
00038                   float *out_speed_horiz,
00039                   float *out_vel_z,
00040                   uint32_t *out_ticks_elapsed);
00041
00042 void imu_calibrate(void);
00043
00044 #ifdef UWB_DEBUG
00045 void imu_print_results(float pitch_rad,
00046                       float speed_horiz,
00047                       float vel_z,
00048                       uint32_t ticks_elapsed);
00049 #endif /* UWB_DEBUG */
00050
00051 #endif /* IMU_H */

```

8.4 App/Accelerometer/imu_setup.h File Reference

```
#include "lsm6dsv_reg.h"
```

Macros

- `#define IMU_FIFO_RAW_BUF_ENTRIES (IMU_MAX_SAMPLES * 2 + 8)`
- `#define IMU_BOOT_TIME_MS 10`
- `#define IMU_XL_ODR LSM6DSV_ODR_AT_60Hz`

- `#define IMU_XL_FULL_SCALE` LSM6DSV_2g
- `#define IMU_XL_LP2_BW` LSM6DSV_XL_STRONG
- `#define IMU_GY_ODR` LSM6DSV_ODR_AT_60Hz
- `#define IMU_GY_FULL_SCALE` LSM6DSV_2000dps
- `#define IMU_GY_LP1_BW` LSM6DSV_GY_ULTRA_LIGHT
- `#define IMU_SFLP_ODR` LSM6DSV_SFLP_30Hz
- `#define IMU_SFLP_BATCH_QUAT` PROPERTY_ENABLE
- `#define IMU_SFLP_BATCH_GRAVITY` PROPERTY_ENABLE
- `#define IMU_SFLP_BATCH_GBIAS` PROPERTY_DISABLE
- `#define IMU_FIFO_XL_BATCH` LSM6DSV_XL_BATCHED_AT_30Hz
- `#define IMU_FIFO_GY_BATCH` LSM6DSV_GY_BATCHED_AT_30Hz
- `#define IMU_FIFO_TS_DEC` LSM6DSV_TMSTMP_DEC_1
- `#define IMU_MAX_SAMPLES` 32
- `#define IMU_CAL_SAMPLES` 32
- `#define IMU_CAL_ODR_HZ` 30

8.4.1 Macro Definition Documentation

8.4.1.1 IMU_BOOT_TIME_MS

```
#define IMU_BOOT_TIME_MS 10
```

8.4.1.2 IMU_CAL_ODR_HZ

```
#define IMU_CAL_ODR_HZ 30
```

8.4.1.3 IMU_CAL_SAMPLES

```
#define IMU_CAL_SAMPLES 32
```

8.4.1.4 IMU_FIFO_GY_BATCH

```
#define IMU_FIFO_GY_BATCH LSM6DSV_GY_BATCHED_AT_30Hz
```

8.4.1.5 IMU_FIFO_RAW_BUF_ENTRIES

```
#define IMU_FIFO_RAW_BUF_ENTRIES (IMU_MAX_SAMPLES * 2 + 8)
```

8.4.1.6 IMU_FIFO_TS_DEC

```
#define IMU_FIFO_TS_DEC LSM6DSV_TMSTMP_DEC_1
```

8.4.1.7 IMU_FIFO_XL_BATCH

```
#define IMU_FIFO_XL_BATCH LSM6DSV_XL_BATCHED_AT_30Hz
```

8.4.1.8 IMU_GY_FULL_SCALE

```
#define IMU_GY_FULL_SCALE LSM6DSV_2000dps
```

8.4.1.9 IMU_GY_LP1_BW

```
#define IMU_GY_LP1_BW LSM6DSV_GY_ULTRA_LIGHT
```

8.4.1.10 IMU_GY_ODR

```
#define IMU_GY_ODR LSM6DSV_ODR_AT_60Hz
```

8.4.1.11 IMU_MAX_SAMPLES

```
#define IMU_MAX_SAMPLES 32
```

8.4.1.12 IMU_SFLP_BATCH_GBIAS

```
#define IMU_SFLP_BATCH_GBIAS PROPERTY_DISABLE
```

8.4.1.13 IMU_SFLP_BATCH_GRAVITY

```
#define IMU_SFLP_BATCH_GRAVITY PROPERTY_ENABLE
```

8.4.1.14 IMU_SFLP_BATCH_QUAT

```
#define IMU_SFLP_BATCH_QUAT PROPERTY_ENABLE
```

8.4.1.15 IMU_SFLP_ODR

```
#define IMU_SFLP_ODR LSM6DSV_SFLP_30Hz
```

8.4.1.16 IMU_XL_FULL_SCALE

```
#define IMU_XL_FULL_SCALE LSM6DSV_2g
```

8.4.1.17 IMU_XL_LP2_BW

```
#define IMU_XL_LP2_BW LSM6DSV_XL_STRONG
```

8.4.1.18 IMU_XL_ODR

```
#define IMU_XL_ODR LSM6DSV_ODR_AT_60Hz
```

8.5 imu_setup.h

[Go to the documentation of this file.](#)

```
00001 #ifndef IMU_SETUP_H
00002 #define IMU_SETUP_H
00003
00004 #include "lsm6dsv_reg.h"
00005
00006 /* =====
00007  * imu_setup.h
00008  * Central configuration for the LSM6DSV IMU.
00009  * Change values here only -- never edit imu_init() directly.
00010  * ===== */
00011 #define IMU_FIFO_RAW_BUF_ENTRIES (IMU_MAX_SAMPLES * 2 + 8)
00012
00013 /* -- Boot ----- */
00014
00015 #define IMU_BOOT_TIME_MS      10      // sensor startup delay, do not go below 10
00016
00017
00018 /* -- Accelerometer ----- */
00019
00020 /* Output data rate -- how often a new accel sample is produced
00021  * Must match IMU_FIFO_XL_BATCH below.
00022  *
00023  * Options (with I2C read time for 200ms FIFO window):
00024  *   LSM6DSV_ODR_AT_7Hz5  → ~1-2 samples,   I2C Fast ~1.0ms  ← current
00025  *   LSM6DSV_ODR_AT_15Hz  → ~3 samples,     I2C Fast ~1.5ms
00026  *   LSM6DSV_ODR_AT_30Hz  → ~6 samples,     I2C Fast ~2.5ms
00027  *   LSM6DSV_ODR_AT_60Hz  → ~12 samples,    I2C Fast ~4.5ms
00028  *   LSM6DSV_ODR_AT_120Hz → ~24 samples,    I2C Fast ~8.5ms  (SPI recommended)
00029  */
00030
00031 #ifdef UWB_BOARD_V1_1
00032     #define IMU_XL_ODR          LSM6DSV_ODR_AT_240Hz
00033 #else
00034     #define IMU_XL_ODR          LSM6DSV_ODR_AT_60Hz
00035 #endif
00036
00037 /* Full scale -- maximum measurable acceleration
00038  *   LSM6DSV_2g  → 0.061 mg/LSB  best precision, clips above 2g  ← current
00039  *   LSM6DSV_4g  → 0.122 mg/LSB
00040  *   LSM6DSV_8g  → 0.244 mg/LSB
00041  *   LSM6DSV_16g → 0.488 mg/LSB  use for high-impact / vibration
00042  */
00043 #define IMU_XL_FULL_SCALE      LSM6DSV_2g
00044
00045 /* Low-pass filter 2 bandwidth -- higher = less filtering, less phase lag
00046  * At low ODR use STRONG. At high ODR switch to LIGHT to preserve signal.
00047  *   LSM6DSV_XL_ULTRA_LIGHT → ODR/2   minimal filtering
00048  *   LSM6DSV_XL_VERY_LIGHT  → ODR/4
00049  *   LSM6DSV_XL_LIGHT       → ODR/5
00050  *   LSM6DSV_XL_MEDIUM      → ODR/7
00051  *   LSM6DSV_XL_STRONG      → ODR/9   ← current (good for 7.5-30 Hz)
00052  *   LSM6DSV_XL_VERY_STRONG → ODR/20
00053  */
00054 #ifdef UWB_BOARD_V1_1
00055     #define IMU_XL_LP2_BW      LSM6DSV_XL_ULTRA_LIGHT
00056 #else
00057     #define IMU_XL_LP2_BW      LSM6DSV_XL_STRONG
00058 #endif
00059
00060
00061
00062 /* -- Gyroscope ----- */
00063
00064 /* Output data rate -- should match IMU_XL_ODR in most cases
00065  * Same options as accel ODR above.
00066  */
00067 #ifdef UWB_BOARD_V1_1
00068     #define IMU_GY_ODR          LSM6DSV_ODR_AT_240Hz
00069 #else
00070     #define IMU_GY_ODR          LSM6DSV_ODR_AT_60Hz
00071 #endif
00072
```

```

00073
00074 /* Full scale -- maximum measurable angular rate
00075 *   LSM6DSV_125dps → 4.375 mdps/LSB highest precision
00076 *   LSM6DSV_250dps → 8.75 mdps/LSB
00077 *   LSM6DSV_500dps → 17.5 mdps/LSB good for slow human motion
00078 *   LSM6DSV_1000dps → 35 mdps/LSB
00079 *   LSM6DSV_2000dps → 70 mdps/LSB ← current (safe for any motion)
00080 *   LSM6DSV_4000dps → 140 mdps/LSB motorsport / robotics
00081 */
00082 #define IMU_GY_FULL_SCALE LSM6DSV_2000dps
00083
00084 /* Low-pass filter 1 bandwidth -- keep ULTRA_LIGHT for integration.
00085 * Heavier settings introduce phase lag that corrupts quaternion output.
00086 *   LSM6DSV_GY_ULTRA_LIGHT → widest BW, minimal lag ← current
00087 *   LSM6DSV_GY_VERY_LIGHT
00088 *   LSM6DSV_GY_LIGHT
00089 *   LSM6DSV_GY_MEDIUM
00090 *   LSM6DSV_GY_STRONG → narrowest BW, most lag
00091 */
00092 #define IMU_GY_LP1_BW LSM6DSV_GY_ULTRA_LIGHT
00093
00094
00095 /* -- SFLP (Sensor Fusion Low Power) ----- */
00096
00097 /* SFLP output data rate -- independent of accel/gyro ODR
00098 * Minimum recommended: 2× your main ODR.
00099 *   LSM6DSV_SFLP_15Hz ← current
00100 *   LSM6DSV_SFLP_30Hz
00101 *   LSM6DSV_SFLP_60Hz
00102 *   LSM6DSV_SFLP_120Hz
00103 */
00104 #ifdef UWB_BOARD_V1_1
00105     #define IMU_SFLP_ODR LSM6DSV_SFLP_120Hz
00106 #else
00107     #define IMU_SFLP_ODR LSM6DSV_SFLP_30Hz
00108 #endif
00109
00110
00111 /* Which SFLP outputs to batch into FIFO */
00112 #define IMU_SFLP_BATCH_QUAT PROPERTY_ENABLE // game rotation quaternion x,y,z
00113 #define IMU_SFLP_BATCH_GRAVITY PROPERTY_ENABLE // gravity vector -- needed for accel compensation
00114 #define IMU_SFLP_BATCH_GBIAIS PROPERTY_DISABLE // gyro bias -- enable if you want raw bias values
00115
00116
00117 /* -- FIFO batching ----- */
00118
00119 /* Must match IMU_XL_ODR / IMU_GY_ODR above.
00120 * Accel options: LSM6DSV_XL_NOT_BATCHED, LSM6DSV_XL_BATCHED_AT_7Hz5 ... _AT_240Hz
00121 * Gyro options: LSM6DSV_GY_NOT_BATCHED, LSM6DSV_GY_BATCHED_AT_7Hz5 ... _AT_240Hz
00122 */
00123 #ifdef UWB_BOARD_V1_1
00124     #define IMU_FIFO_XL_BATCH LSM6DSV_XL_BATCHED_AT_120Hz
00125     #define IMU_FIFO_GY_BATCH LSM6DSV_GY_BATCHED_AT_120Hz
00126 #else
00127     #define IMU_FIFO_XL_BATCH LSM6DSV_XL_BATCHED_AT_30Hz
00128     #define IMU_FIFO_GY_BATCH LSM6DSV_GY_BATCHED_AT_30Hz
00129 #endif
00130
00131
00132 /* Timestamp decimation -- how often a timestamp entry appears in FIFO
00133 * relative to the accel/gyro batch rate.
00134 *   LSM6DSV_TMSTMP_DEC_1 → every sample ← current
00135 *   LSM6DSV_TMSTMP_DEC_8 → every 8 samples
00136 *   LSM6DSV_TMSTMP_DEC_32 → every 32 samples
00137 */
00138 #define IMU_FIFO_TS_DEC LSM6DSV_TMSTMP_DEC_1
00139
00140 /* Maximum samples to read per FIFO drain.
00141 * At 7.5 Hz over 200ms: 2 accel + 2 gyro + 3 quat + 3 gravity + 2 ts = ~12 entries
00142 * Set higher when increasing ODR.
00143 *   7.5 Hz → 16
00144 *   15 Hz → 32
00145 *   52 Hz → 64
00146 *   104 Hz → 128
00147 */
00148 #ifdef UWB_BOARD_V1_1
00149     #define IMU_MAX_SAMPLES 32 //TODO should be different number
00150 #else
00151     #define IMU_MAX_SAMPLES 32
00152 #endif
00153
00154
00155 /* -- Calibration ----- */
00156 #ifdef UWB_BOARD_V1_1
00157     #define IMU_CAL_SAMPLES 240
00158     #define IMU_CAL_ODR_HZ 256
00159 #else

```

```

00160     #define IMU_CAL_SAMPLES    32
00161     #define IMU_CAL_ODR_HZ     30
00162 #endif
00163
00164
00165 #endif /* IMU_SETUP_H */

```

8.6 App/Accelerometer/lsm6dsv.c File Reference

```

#include "cmsis_os.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include "cmsis_os2.h"
#include "lsm6dsv_ST_example.h"
#include "main.h"
#include "../Generic/my_print.h"
#include "stm32l4xx_hal.h"
#include "imu.h"

```

Functions

- void [StartAcc](#) (void *argument)
FreeRTOS task entry point for the IMU pipeline.
- void [imu_get_results](#) (float *out_pitch_rad, float *out_speed_horiz, float *out_vel_z)
Read back the most recent IMU integration results.

8.6.1 Function Documentation

8.6.1.1 imu_get_results()

```

void imu_get_results (
    float * out_pitch_rad,
    float * out_speed_horiz,
    float * out_vel_z)

```

Read back the most recent IMU integration results.

Parameters

out	<i>out_pitch_rad</i>	Pitch angle in radians.
out	<i>out_speed_horiz</i>	Horizontal speed magnitude in m/s.
out	<i>out_vel_z</i>	Vertical velocity in m/s.

All three outputs are updated atomically from the module-static variables written by [StartAcc](#) after each [imu_integrate](#) call.

Parameters

out	<i>out_pitch_rad</i>	Pitch angle in radians.
out	<i>out_speed_horiz</i>	Horizontal speed magnitude in m/s (≥ 0).
out	<i>out_vel_z</i>	Vertical velocity in m/s (signed).

8.6.1.2 StartAcc()

```
void StartAcc (
    void * argument)
```

FreeRTOS task entry point for the IMU pipeline.

Startup sequence:

1. LSM6DSV self-test — task deleted on failure.
2. [imu_init](#) — configures FIFO, filters, ODR.
3. 200 ms settle delay.
4. [imu_calibrate](#) — collects accelerometer bias.
5. Signals the ranging task (flag 0x01) that the IMU is ready.

Loop (triggered by ranging task flag 0x01):

- Drains the FIFO, integrates velocity and pitch.
- Signals the ranging task (flag 0x02) that results are ready.

Parameters

<i>argument</i>	Unused; required by the CMSIS-RTOS2 task signature.
-----------------	---

Called by `osThreadNew()` in `main.c`. Runs the LSM6DSV self-test, initialises the IMU, calibrates the accelerometer bias, signals the ranging task that it is ready, then loops waiting for ranging-task trigger flags.

Parameters

<i>argument</i>	Unused; required by the CMSIS-RTOS2 task signature.
-----------------	---

8.7 App/Accelerometer/lsm6dsv.h File Reference**Functions**

- void [StartAcc](#) (void *argument)
FreeRTOS task entry point for the IMU pipeline.
- void [imu_get_results](#) (float *out_pitch_rad, float *out_speed_horiz, float *out_vel_z)
Read back the most recent IMU integration results.

8.7.1 Function Documentation

8.7.1.1 imu_get_results()

```
void imu_get_results (
    float * out_pitch_rad,
    float * out_speed_horiz,
    float * out_vel_z)
```

Read back the most recent IMU integration results.

All three outputs are updated atomically from the module-static variables written by [StartAcc](#) after each [imu_integrate](#) call.

Parameters

out	<i>out_pitch_rad</i>	Pitch angle in radians.
out	<i>out_speed_horiz</i>	Horizontal speed magnitude in m/s (≥ 0).
out	<i>out_vel_z</i>	Vertical velocity in m/s (signed).
out	<i>out_pitch_rad</i>	Pitch angle in radians.
out	<i>out_speed_horiz</i>	Horizontal speed magnitude in m/s.
out	<i>out_vel_z</i>	Vertical velocity in m/s.

8.7.1.2 StartAcc()

```
void StartAcc (
    void * argument)
```

FreeRTOS task entry point for the IMU pipeline.

Called by `osThreadNew()` in `main.c`. Runs the LSM6DSV self-test, initialises the IMU, calibrates the accelerometer bias, signals the ranging task that it is ready, then loops waiting for ranging-task trigger flags.

Parameters

<i>argument</i>	Unused; required by the CMSIS-RTOS2 task signature.
-----------------	---

Startup sequence:

1. LSM6DSV self-test — task deleted on failure.
2. [imu_init](#) — configures FIFO, filters, ODR.
3. 200 ms settle delay.
4. [imu_calibrate](#) — collects accelerometer bias.
5. Signals the ranging task (flag 0x01) that the IMU is ready.

Loop (triggered by ranging task flag 0x01):

- Drains the FIFO, integrates velocity and pitch.
- Signals the ranging task (flag 0x02) that results are ready.

Parameters

<i>argument</i>	Unused; required by the CMSIS-RTOS2 task signature.
-----------------	---

8.8 lsm6dsv.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #ifndef __cplusplus
00004 extern "C" {
00005 #endif
00006
00017 void StartAcc(void *argument);
00018
00029 void imu_get_results(float *out_pitch_rad,
00030                     float *out_speed_horiz,
00031                     float *out_vel_z);
00032
00033 #ifdef __cplusplus
00034 }
00035 #endif

```

8.9 App/Accelerometer/lsm6dsv_self_test.c File Reference

```

#include "lsm6dsv_reg.h"
#include "lsm6dsv_platform.h"
#include <string.h>
#include <stdio.h>
#include <stdbool.h>
#include "../Generic/my_print.h"
#include "lsm6dsv_ST_example.h"

```

Macros

- #define `BOOT_TIME` 100
- #define `MIN_ST_LIMIT_mg` 50.0f
- #define `MAX_ST_LIMIT_mg` 1700.0f
- #define `MIN_ST_LIMIT_mdps` 150000.0f
- #define `MAX_ST_LIMIT_mdps` 700000.0f
- #define `ST_PASS` 1U
- #define `ST_FAIL` 0U

Functions

- bool `SelfTest` (void)

Variables

- `uint8_t tx_buffer [1000]`

8.9.1 Macro Definition Documentation

8.9.1.1 BOOT_TIME

```
#define BOOT_TIME 100
```

8.9.1.2 MAX_ST_LIMIT_mdps

```
#define MAX_ST_LIMIT_mdps 700000.0f
```

8.9.1.3 MAX_ST_LIMIT_mg

```
#define MAX_ST_LIMIT_mg 1700.0f
```

8.9.1.4 MIN_ST_LIMIT_mdps

```
#define MIN_ST_LIMIT_mdps 150000.0f
```

8.9.1.5 MIN_ST_LIMIT_mg

```
#define MIN_ST_LIMIT_mg 50.0f
```

8.9.1.6 ST_FAIL

```
#define ST_FAIL 0U
```

8.9.1.7 ST_PASS

```
#define ST_PASS 1U
```

8.9.2 Function Documentation

8.9.2.1 SelfTest()

```
bool SelfTest (  
    void )
```

8.9.3 Variable Documentation

8.9.3.1 tx_buffer

```
uint8_t tx_buffer[1000]
```

8.10 App/Accelerometer/lsm6dsv_ST_example.h File Reference

```
#include <stdbool.h>
```

Functions

- bool [SelfTest](#) (void)
- void [lsm6dsv_read_data_polling](#) (void)

8.10.1 Function Documentation

8.10.1.1 lsm6dsv_read_data_polling()

```
void lsm6dsv_read_data_polling (  
    void )
```

8.10.1.2 SelfTest()

```
bool SelfTest (  
    void )
```

8.11 lsm6dsv_ST_example.h

[Go to the documentation of this file.](#)

```
00001 #ifndef SENSOR_INTERFACE_H
00002 #define SENSOR_INTERFACE_H
00003
00004 #ifdef __cplusplus
00005 extern "C" {
00006 #endif
00007
00008 #include <stdbool.h>
00009
00010 // Defined in lsm6dsv_self_test.c (or similar)
00011 bool SelfTest(void);
00012
00013 // Defined in lsm6dsv_read_data_polling.c (or similar)
00014 void lsm6dsv_read_data_polling(void);
00015
00016 #ifdef __cplusplus
00017 }
00018 #endif
00019
00020 #endif // SENSOR_INTERFACE_H
```

8.12 App/Accelerometer/read_data_polling.c File Reference

```
#include <string.h>
#include <stdio.h>
#include "lsm6dsv_reg.h"
#include "lsm6dsv_platform.h"
#include "../Generic/my_print.h"
#include "lsm6dsv_ST_example.h"
```

Macros

- #define [BOOT_TIME](#) 10

Functions

- void [lsm6dsv_read_data_polling](#) (void)

8.12.1 Macro Definition Documentation

8.12.1.1 [BOOT_TIME](#)

```
#define BOOT_TIME 10
```

8.12.2 Function Documentation

8.12.2.1 [lsm6dsv_read_data_polling\(\)](#)

```
void lsm6dsv_read_data_polling (
    void )
```

8.13 App/Blink/blink.c File Reference

```
#include "main.h"
#include "cmsis_os.h"
#include <stdbool.h>
#include <stdint.h>
#include "../Generic/my_print.h"
```

Functions

- void [StartBlink](#) (void *argument)
FreeRTOS watchdog-refresh task.

8.13.1 Function Documentation

8.13.1.1 StartBlink()

```
void StartBlink (
    void * argument)
```

FreeRTOS watchdog-refresh task.

Feeds the IWDG every 1 000 ms. The IWDG timeout is configured for 3 seconds in CubeMX, so this task must not be starved for longer than that or the device will reset.

Also serves as a 1 Hz heartbeat indicator: when `UWB_DEBUG` is defined, a counter value and board variant are printed each cycle.

Parameters

<i>argument</i>	Unused; required by the CMSIS-RTOS2 task signature.
-----------------	---

8.14 App/Blink/blink.h File Reference

Functions

- void [StartBlink](#) (void *argument)
FreeRTOS watchdog-refresh task.

8.14.1 Function Documentation

8.14.1.1 StartBlink()

```
void StartBlink (
    void * argument)
```

FreeRTOS watchdog-refresh task.

Feeds the IWDG every 1 000 ms. The IWDG timeout is configured for 3 seconds in CubeMX, so this task must not be starved for longer than that or the device will reset.

Also serves as a 1 Hz heartbeat indicator: when `UWB_DEBUG` is defined, a counter value and board variant are printed each cycle.

Parameters

<i>argument</i>	Unused; required by the CMSIS-RTOS2 task signature.
-----------------	---

8.15 blink.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003
00004
00005 /* FreeRTOS task entry point -- called by osThreadNew() in main.c */
00006 void StartBlink(void *argument);
00007
00008
```

8.16 App/Calculations/calibrate.c File Reference

```
#include "cmsis_os.h"
#include "cmsis_os2.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <float.h>
#include <stdlib.h>
#include "../Generic/my_print.h"
#include "calibrate.h"
#include "../UWB_app/uwb_network.h"
#include "distance.h"
```

Functions

- void [calibrate_set_clock_offset_sync](#) (int16_t cfo)
Update the clock-rate correction factor from a SYNC CFO reading.
- void [calibrate_set_clock_offset_poll](#) (int16_t cfo)
Refine the clock-rate correction factor from a POLL CFO reading.
- float [calibrate_get_k](#) (void)
Return the current clock-rate correction factor.
- void [calibrate_set_pitch](#) (float pitch_rad)
Store the IMU pitch angle used for antenna pattern correction.
- float [calibrate_rssi_antenna](#) (float rssi_dbm, uint16_t target_id, bool *antenna_unreliable)
Apply antenna pattern gain correction to a raw RSSI value.
- uint64_t [calibrate_rx_timestamp](#) (uint64_t rx_timestamp, int16_t rssi_q8, uint16_t target_id, bool *antenna_unreliable)
Apply full calibration pipeline to a raw RX timestamp.
- uint64_t [calibrate_tx_timestamp](#) (uint64_t tx_timestamp)
Apply clock-rate correction to a raw TX timestamp.

8.16.1 Function Documentation

8.16.1.1 [calibrate_get_k\(\)](#)

```
float calibrate_get_k (
    void )
```

Return the current clock-rate correction factor.

Return the current clock correction factor k. Returns 1.0f if no calibration has been performed yet.

Returns 1.0 when no CFO sample has been received yet, so timestamps pass through uncorrected rather than being scaled to zero.

Returns

Stored k value, or 1.0f if uninitialised.

Return the current clock-rate correction factor.

8.16.1.2 `calibrate_rssi_antenna()`

```
float calibrate_rssi_antenna (
    float rssi_dbm,
    uint16_t target_id,
    bool * antenna_unreliable)
```

Apply antenna pattern gain correction to a raw RSSI value.

Models the dipole-like $\cos^N(\theta)$ gain pattern of the UWB antenna. The elevation angle to the target is computed from stored EKF positions. If the target is near the antenna null (elevation close to 90° - pitch), the measurement is flagged as unreliable and the uncorrected RSSI is returned.

Returns the uncorrected `rssi_dbm` unchanged when:

- IMU pitch has not been initialised (`s_pitch_rad == 0`),
- the target peer is not in the active peer list,
- the stored distance is invalid or below `CALIB_ANTENNA_MIN_DIST_TICKS`,
- the inter-node separation is below 1 mm.

Parameters

in	<i>rssi_dbm</i>	Raw RSSI converted to dBm.
in	<i>target_id</i>	Node ID of the transmitting peer.
out	<i>antenna_unreliable</i>	Set to true if the target is near the antenna null.

Returns

Corrected RSSI in dBm, or `rssi_dbm` if correction is skipped.

8.16.1.3 `calibrate_rx_timestamp()`

```
uint64_t calibrate_rx_timestamp (
    uint64_t rx_timestamp,
    int16_t rssi_q8,
    uint16_t target_id,
    bool * antenna_unreliable)
```

Apply full calibration pipeline to a raw RX timestamp.

Performs three sequential corrections:

1. Antenna pattern correction via [calibrate_rssi_antenna](#) (skipped if IMU pitch is uninitialised).
2. Power-dependent leading-edge correction: $\delta = \text{CALIB_RX_TS_A} \times \text{RSSI_dBm}$.
3. Clock-rate correction using the CFO-derived factor `k`.

Note

The clock-rate multiplication uses `double` to preserve all 40 bits of the DWM3000 timestamp. A `float` cast would discard the lower ~ 16 bits (~ 150 m equivalent), and this noise does not cancel when two calibrated timestamps are subtracted in `distance.c`.

Parameters

in	<code>rx_timestamp</code>	Raw 40-bit RX timestamp from the DWM3000.
in	<code>rssiq8</code>	Received signal strength in Q8 fixed-point (dBm $\times$ 256).
in	<code>target_id</code>	Node ID of the transmitting peer.
out	<code>antenna_unreliable</code>	Set to true if the antenna null flag was triggered.

Returns

Fully calibrated RX timestamp.

8.16.1.4 calibrate_set_clock_offset_poll()

```
void calibrate_set_clock_offset_poll (
    int16_t cfo)
```

Refine the clock-rate correction factor from a POLL CFO reading.

Refine `k` by averaging it with a new CFO sample. On the first call (uninitialised), behaves like `sync`.

On the first call after boot (`s_k == 0`) the value is set directly. On subsequent calls it is averaged with the current estimate. The `0.0f` sentinel is safe because `k` is a clock-rate ratio $\sim 1 \pm 20$ ppm and can never legitimately equal zero.

Parameters

<code>cfo</code>	Raw carrier-frequency-offset value from the DWM3000 RX frame.
------------------	---

Refine the clock-rate correction factor from a POLL CFO reading.

Parameters

<code>cfo</code>	Raw <code>dwt_readclockoffset()</code> value.
------------------	---

8.16.1.5 calibrate_set_clock_offset_sync()

```
void calibrate_set_clock_offset_sync (
    int16_t cfo)
```

Update the clock-rate correction factor from a SYNC CFO reading.

Overwrite `k` from scratch using a single CFO sample.

Replaces the stored `s_k` directly. Use this on the first CFO sample of a round, before any POLL-based averaging has occurred.

Parameters

<i>cfo</i>	Raw carrier-frequency-offset value from the DWM3000 RX frame.
------------	---

Update the clock-rate correction factor from a SYNC CFO reading.

Parameters

<i>cfo</i>	Raw <code>dwt_readclockoffset()</code> value.
------------	---

8.16.1.6 `calibrate_set_pitch()`

```
void calibrate_set_pitch (
    float pitch_rad)
```

Store the IMU pitch angle used for antenna pattern correction.

A value of 0.0f disables antenna correction in [calibrate_rssi_antenna](#) and [calibrate_rx_timestamp](#).

Parameters

<i>pitch_rad</i>	Pitch angle in radians from the IMU.
------------------	--------------------------------------

8.16.1.7 `calibrate_tx_timestamp()`

```
uint64_t calibrate_tx_timestamp (
    uint64_t tx_timestamp)
```

Apply clock-rate correction to a raw TX timestamp.

Scales the timestamp by the CFO-derived factor `k` using `double` arithmetic for the same precision reasons as [calibrate_rx_timestamp](#).

Parameters

<i>tx_timestamp</i>	Raw 40-bit TX timestamp from the DWM3000.
---------------------	---

Returns

Clock-rate corrected TX timestamp.

8.17 App/Calculations/calibrate.h File Reference

```
#include <stdint.h>
#include "distance.h"
```


Macros

- `#define CFO_RAW_TO_PPM(raw)`
- `#define PPM_TO_K(ppm)`
- `#define CFO_RAW_TO_K(raw)`
- `#define CALIB_RX_TS_A 0.0f /* TODO: measure — ticks per dBm */`
- `#define CALIB_ANTENNA_MIN_DIST_TICKS (2.0 * (499.2e6 * 128.0 * 2.0) / SPEED_OF_LIGHT)`
- `#define CALIB_ANTENNA_TILT_LIMIT_DEG 15.0f`
- `#define CALIB_ANTENNA_SIMILARITY_DEG 15.0f`
- `#define CALIB_ANTENNA_NULL_THRESHOLD_DEG 70.0f`
- `#define CALIB_ANTENNA_N 2.0f`

Functions

- `void calibrate_set_clock_offset_sync (int16_t cfo)`
Overwrite k from scratch using a single CFO sample.
- `void calibrate_set_clock_offset_poll (int16_t cfo)`
Refine k by averaging it with a new CFO sample. On the first call (uninitialised), behaves like sync.
- `float calibrate_get_k (void)`
Return the current clock correction factor k . Returns 1.0f if no calibration has been performed yet.
- `void calibrate_set_pitch (float pitch_rad)`
Store the IMU pitch angle used for antenna pattern correction.
- `uint64_t calibrate_rx_timestamp (uint64_t rx_timestamp, int16_t rssi_q8, uint16_t target_id, bool *antenna_unreliable)`
Apply full calibration pipeline to a raw RX timestamp.
- `uint64_t calibrate_tx_timestamp (uint64_t tx_timestamp)`
Apply clock-rate correction to a raw TX timestamp.

8.17.1 Macro Definition Documentation

8.17.1.1 CALIB_ANTENNA_MIN_DIST_TICKS

```
#define CALIB_ANTENNA_MIN_DIST_TICKS (2.0 * (499.2e6 * 128.0 * 2.0) / SPEED_OF_LIGHT)
```

8.17.1.2 CALIB_ANTENNA_N

```
#define CALIB_ANTENNA_N 2.0f
```

8.17.1.3 CALIB_ANTENNA_NULL_THRESHOLD_DEG

```
#define CALIB_ANTENNA_NULL_THRESHOLD_DEG 70.0f
```

8.17.1.4 CALIB_ANTENNA_SIMILARITY_DEG

```
#define CALIB_ANTENNA_SIMILARITY_DEG 15.0f
```

8.17.1.5 CALIB_ANTENNA_TILT_LIMIT_DEG

```
#define CALIB_ANTENNA_TILT_LIMIT_DEG 15.0f
```

8.17.1.6 CALIB_RX_TS_A

```
#define CALIB_RX_TS_A 0.0f /* TODO: measure -- ticks per dBm */
```

8.17.1.7 CFO_RAW_TO_K

```
#define CFO_RAW_TO_K(  
    raw)
```

Value:

```
PPM_TO_K(CFO_RAW_TO_PPM(raw))
```

8.17.1.8 CFO_RAW_TO_PPM

```
#define CFO_RAW_TO_PPM(  
    raw)
```

Value:

```
((float)(raw) / (float)(1 << 26) * 1e6f)
```

8.17.1.9 PPM_TO_K

```
#define PPM_TO_K(  
    ppm)
```

Value:

```
(1.0f + (ppm) / 1e6f)
```

8.17.2 Function Documentation

8.17.2.1 calibrate_get_k()

```
float calibrate_get_k (  
    void )
```

Return the current clock correction factor k. Returns 1.0f if no calibration has been performed yet.

Return the current clock-rate correction factor.

Return the current clock correction factor k. Returns 1.0f if no calibration has been performed yet.

Returns 1.0 when no CFO sample has been received yet, so timestamps pass through uncorrected rather than being scaled to zero.

Returns

Stored k value, or 1.0f if uninitialised.

8.17.2.2 `calibrate_rx_timestamp()`

```
uint64_t calibrate_rx_timestamp (
    uint64_t rx_timestamp,
    int16_t rssi_q8,
    uint16_t target_id,
    bool * antenna_unreliable)
```

Apply full calibration pipeline to a raw RX timestamp.

Performs three sequential corrections:

1. Antenna pattern correction via [calibrate_rssi_antenna](#) (skipped if IMU pitch is uninitialised).
2. Power-dependent leading-edge correction: $\delta = \text{CALIB_RX_TS_A} \times \text{RSSI_dBm}$.
3. Clock-rate correction using the CFO-derived factor k .

Note

The clock-rate multiplication uses `double` to preserve all 40 bits of the DWM3000 timestamp. A `float` cast would discard the lower ~ 16 bits (~ 150 m equivalent), and this noise does not cancel when two calibrated timestamps are subtracted in [distance.c](#).

Parameters

in	<i>rx_timestamp</i>	Raw 40-bit RX timestamp from the DWM3000.
in	<i>rssi_q8</i>	Received signal strength in Q8 fixed-point (dBm $\times$ 256).
in	<i>target_id</i>	Node ID of the transmitting peer.
out	<i>antenna_unreliable</i>	Set to true if the antenna null flag was triggered.

Returns

Fully calibrated RX timestamp.

8.17.2.3 `calibrate_set_clock_offset_poll()`

```
void calibrate_set_clock_offset_poll (
    int16_t cfo)
```

Refine k by averaging it with a new CFO sample. On the first call (uninitialised), behaves like sync.

Refine the clock-rate correction factor from a POLL CFO reading.

Parameters

<i>cfo</i>	Raw <code>dwt_readclockoffset()</code> value.
------------	---

Refine k by averaging it with a new CFO sample. On the first call (uninitialised), behaves like sync.

On the first call after boot (`s_k == 0`) the value is set directly. On subsequent calls it is averaged with the current estimate. The `0.0f` sentinel is safe because `k` is a clock-rate ratio $\approx 1 \pm 20$ ppm and can never legitimately equal zero.

Parameters

<i>cfo</i>	Raw carrier-frequency-offset value from the DWM3000 RX frame.
------------	---

8.17.2.4 `calibrate_set_clock_offset_sync()`

```
void calibrate_set_clock_offset_sync (
    int16_t cfo)
```

Overwrite `k` from scratch using a single CFO sample.

Update the clock-rate correction factor from a SYNC CFO reading.

Parameters

<i>cfo</i>	Raw <code>dwt_readclockoffset()</code> value.
------------	---

Overwrite `k` from scratch using a single CFO sample.

Replaces the stored `s_k` directly. Use this on the first CFO sample of a round, before any POLL-based averaging has occurred.

Parameters

<i>cfo</i>	Raw carrier-frequency-offset value from the DWM3000 RX frame.
------------	---

8.17.2.5 `calibrate_set_pitch()`

```
void calibrate_set_pitch (
    float pitch_rad)
```

Store the IMU pitch angle used for antenna pattern correction.

A value of `0.0f` disables antenna correction in [calibrate_rssi_antenna](#) and [calibrate_rx_timestamp](#).

Parameters

<i>pitch_rad</i>	Pitch angle in radians from the IMU.
------------------	--------------------------------------

8.17.2.6 calibrate_tx_timestamp()

```
uint64_t calibrate_tx_timestamp (
    uint64_t tx_timestamp)
```

Apply clock-rate correction to a raw TX timestamp.

Scales the timestamp by the CFO-derived factor *k* using double arithmetic for the same precision reasons as [calibrate_rx_timestamp](#).

Parameters

<i>tx_timestamp</i>	Raw 40-bit TX timestamp from the DWM3000.
---------------------	---

Returns

Clock-rate corrected TX timestamp.

8.18 calibrate.h

[Go to the documentation of this file.](#)

```
00001 /* CHANGES:
00002  * CALIB_ANTENNA_MIN_DIST_TICKS: replaced literal 299792458.0 with SPEED_OF_LIGHT
00003  * from distance.h, per task constraint that all SoL uses go through that macro.
00004  * Numerical impact is negligible (~0.030% on a 3 m threshold), but the file is
00005  * included alongside distance.h so the macro is in scope.
00006  */
00007 #pragma once
00008
00009 #include <stdint.h>
00010 #include "distance.h" /* SPEED_OF_LIGHT */
00011
00012 #define CFO_RAW_TO_PPM(raw) ((float)(raw) / (float)(1 « 26) * 1e6f)
00013 #define PPM_TO_K(ppm) (1.0f + (ppm) / 1e6f)
00014 #define CFO_RAW_TO_K(raw) PPM_TO_K(CFO_RAW_TO_PPM(raw))
00015
00016 /* RX timestamp power correction coefficient.
00017  * Model: delta_ticks = CALIB_RX_TS_A * rssi_dbm
00018  *
00019  * Corrects leading-edge detection bias that varies with signal level.
00020  * Assumes static antenna delay is already calibrated (dwt_setrxantennadelay).
00021  * Typical range: -0.1 to -0.5 ticks per dBm. Must be measured per board. */
00022 #define CALIB_RX_TS_A 0.0f /* TODO: measure -- ticks per dBm */
00023
00024 /* 3 metres expressed in DW3000 ticks (one-way, two-way already halved in DS-TWR):
00025  * ticks = metres / (c / (499.2e6 * 128 * 2)) */
00026 #define CALIB_ANTENNA_MIN_DIST_TICKS \
00027     (2.0 * (499.2e6 * 128.0 * 2.0) / SPEED_OF_LIGHT)
00028
00029 /* Device tilt (from vertical) above which correction is skipped entirely.
00030  * 15° keeps the board nearly flat -- null axis is close to true vertical,
00031  * donut approximation is most valid here. */
00032 #define CALIB_ANTENNA_TILT_LIMIT_DEG 15.0f
00033
00034 /* Within TILT_LIMIT, if the target elevation is within this many degrees
00035  * of the null axis elevation -- flag unreliable instead of correcting.
00036  * Tight window: only trigger when alignment is very close to the null. */
00037 #define CALIB_ANTENNA_SIMILARITY_DEG 15.0f
00038
00039 /* Target elevation above which the donut null correction is applied.
00040  * Below this angle the antenna is near its gain maximum -- no correction.
00041  * 70° is conservative -- only correct deep in the null zone. */
00042 #define CALIB_ANTENNA_NULL_THRESHOLD_DEG 70.0f
00043
00044 /* Donut pattern exponent. n=2 for dipole-like pattern. */
00045 #define CALIB_ANTENNA_N 2.0f
00046
00051 void calibrate_set_clock_offset_sync(int16_t cfo);
00052
00058 void calibrate_set_clock_offset_poll(int16_t cfo);
00059
```

```

00064 float calibrate_get_k(void);
00065
00066 void calibrate_set_pitch(float pitch_rad);
00067
00068 uint64_t calibrate_rx_timestamp(uint64_t rx_timestamp,
00069                                int16_t rssi_q8,
00070                                uint16_t target_id,
00071                                bool *antenna_unreliable);
00072
00073 uint64_t calibrate_tx_timestamp(uint64_t tx_timestamp);

```

8.19 App/Calculations/distance.c File Reference

```

#include "cmsis_os.h"
#include "cmsis_os2.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <math.h>
#include <stdlib.h>
#include <stdio.h>
#include "../Generic/my_print.h"
#include "DWM3000_setup.h"
#include "DWM3000_driver.h"
#include "../UWB_app/messages.h"
#include "../UWB_app/uwb_network.h"
#include "../UWB_app/uwb_exchange.h"
#include "distance.h"
#include "lsm6dsv.h"

```

Macros

- #define `PDIFF_MAX_DB` 40.0
- #define `CLAMP_PDIFF(x)`

Functions

- uint8_t `vel_vert_to_u8` (float v)
Saturate-and-encode a vertical velocity to a uint8 LSB value.
- float `vel_vert_u8_to_ms` (uint8_t u)
Decode a uint8 LSB value back to vertical velocity in m/s.
- uint8_t `vel_horiz_to_u8` (float v)
Encode an unsigned horizontal speed to a uint8 LSB value.
- float `vel_horiz_u8_to_ms` (uint8_t u)
Decode a uint8 LSB value back to horizontal speed in m/s.
- void `populate_computation_structs` (timestamps_t *ts)
Populate ts from the current network measurement snapshot.
- uint16_t `dist_ticks_to_scale` (double ticks)
Encode a ToF tick count to the compact 16-bit network distance format.
- double `dist_scale_to_ticks` (uint16_t encoded)
Decode a compact 16-bit network distance value back to ticks.
- uint8_t `compute_certainty` (MeasurementType type, float t_gap_us, int16_t pwr_diff_q8, uint8_t unreliable_count)
Compute a composite measurement certainty score in [1, 254].
- void `distance_calculate` (uwb_etwr_result_t result)
Run the full three-order ranging pipeline for one TWR exchange.

Variables

- [timestamps_t timestamps](#)

Global timestamp workspace — reused each call to [distance_calculate\(\)](#).

8.19.1 Macro Definition Documentation

8.19.1.1 CLAMP_PDIFF

```
#define CLAMP_PDIFF(  
    x)
```

Value:

```
do { if ((x) < 0.0) (x) = 0.0; \  
if ((x) > PDIFF_MAX_DB) (x) = PDIFF_MAX_DB; } while(0)
```

8.19.1.2 PDIFF_MAX_DB

```
#define PDIFF_MAX_DB 40.0
```

8.19.2 Function Documentation

8.19.2.1 compute_certainty()

```
uint8_t compute_certainty (  
    MeasurementType type,  
    float t_gap_us,  
    int16_t pwr_diff_q8,  
    uint8_t unreliable_count)
```

Compute a composite measurement certainty score in [1, 254].

Starts from a type-dependent base score, adds a timing bonus/penalty and an LOS/NLOS bonus/penalty, then subtracts CERTAINTY_UNRELIABLE_PENALTY per unreliable antenna observation. Returns 0 for MEAS_↵NONE.

Parameters

<i>type</i>	Measurement type (MEAS_DSTWR, MEAS_SSTWR, MEAS_TDOA_SSTWR).
<i>t_gap_us</i>	SS-TWR turnaround in μs; ignored for MEAS_DSTWR.
<i>pwr_diff_q8</i>	RSSI - first-path power in Q8 (dBm × 256).
<i>unreliable_count</i>	Number of per-frame antenna-unreliable flags set.

Returns

Certainty score in [1, 254], or 0 for MEAS_NONE.

8.19.2.2 `dist_scale_to_ticks()`

```
double dist_scale_to_ticks (
    uint16_t encoded)
```

Decode a compact 16-bit network distance value back to ticks.

Parameters

<i>encoded</i>	Packed distance from the network layer.
----------------	---

Returns

Time-of-flight in DWM3000 ticks, or -1.0 for the sentinel 0xFFFF.

8.19.2.3 `dist_ticks_to_scale()`

```
uint16_t dist_ticks_to_scale (
    double ticks)
```

Encode a ToF tick count to the compact 16-bit network distance format.

The sentinel value 0xFFFF indicates an invalid or out-of-range distance. Values that would round to 0xFFFF are clamped to 0xFFFE.

Parameters

<i>ticks</i>	Time-of-flight in DWM3000 ticks (< 0 or > DIST_SHARE_DIST_MAX_TICKS → invalid).
--------------	---

Returns

Encoded uint16 value, or 0xFFFF if invalid.

8.19.2.4 `distance_calculate()`

```
void distance_calculate (
    uwb_etwr_result_t result)
```

Run the full three-order ranging pipeline for one TWR exchange.

Called after `uwb_extended_twr` completes with UWB_TWR_RECEIVED. Executes in order:

1. Populates `timestamps` from the current network measurement snapshot.
2. First-order DS-TWR (A <-> B distance).
3. Second-order SS-TWR with iterative clock correction (A/B <-> each passive C).
4. Third-order TDOA (each passive-pair C <-> D).

Updates `network_set_distance()` and `network_update_certainty()` for every computed pair. Pads execution time to at least 10 ms to give the FreeRTOS scheduler a consistent call period.

Does nothing if `result` is not `UWB_TWR_RECEIVED`.

Parameters

<code>result</code>	Return value from the preceding <code>uwb_extended_twr</code> call.
---------------------	---

8.19.2.5 populate_computation_structs()

```
void populate_computation_structs (
    timestamps_t * ts)
```

Populate `ts` from the current network measurement snapshot.

Assembles all three orders of timestamp structures required by the distance calculation pipeline. The caller must pass a valid `timestamps_t`; the struct is zero-filled at entry so any field still zero after return was never written.

Timestamp origin summary (A = Initiator/Master, B = Responder/Self, C/D = Passive observers):

First order (DS-TWR, A <-> B):

- `poll_tx` — A-clock, embedded in `msg_final_t.poll_tx_ts`
- `poll_rx` — B-clock, `calibrate_rx_timestamp()` on POLL reception
- `resp_tx` — B-clock, `calibrate_tx_timestamp()` in `uwb_send_RESPONSE()`
- `resp_rx` — A-clock, `calibrate_rx_timestamp()` in `uwb_wait_for_RESPONSE()`, shipped in FINAL
- `final_tx` — A-clock, 9-bit-aligned `final_tx_exact` in `uwb_send_FINAL()`, shipped in FINAL
- `final_rx` — B-clock, `calibrate_rx_timestamp()` on FINAL reception

Second order (SS-TWR, A <-> C and B <-> C):

- `so_a`: `init_tx` = A's POLL TX; `init_rx` = C's RX of POLL; `answer_tx` = C's PASSIVE TX; `answer_rx` = A's RX of C's PASSIVE
- `so_b`: `init_tx` = B's RESPONSE TX; `init_rx` = C's RX of RESPONSE; `answer_tx` = C's PASSIVE TX; `answer_rx` = B's RX of C's PASSIVE

Third order (TDOA, C <-> D):

- `init_tx` = C's PASSIVE TX; `answer_rx` = D's RX of C's PASSIVE; `answer_tx` = D's PASSIVE TX

Parameters

<code>out</code>	<code>ts</code>	Timestamps struct to populate. Must not be NULL.
------------------	-----------------	--

8.19.2.6 vel_horiz_to_u8()

```
uint8_t vel_horiz_to_u8 (  
    float v)
```

Encode an unsigned horizontal speed to a uint8 LSB value.

Direction is discarded — only magnitude is stored. Values above `VEL_HORIZ_MAX_MS` are clamped.

Parameters

<i>v</i>	Horizontal velocity in m/s (sign ignored).
----------	--

Returns

Encoded uint8 value.

8.19.2.7 vel_horiz_u8_to_ms()

```
float vel_horiz_u8_to_ms (  
    uint8_t u)
```

Decode a uint8 LSB value back to horizontal speed in m/s.

Always returns a non-negative value (magnitude only).

Parameters

<i>u</i>	Encoded uint8 value.
----------	----------------------

Returns

Horizontal speed in m/s (≥ 0).

8.19.2.8 vel_vert_to_u8()

```
uint8_t vel_vert_to_u8 (  
    float v)
```

Saturate-and-encode a vertical velocity to a uint8 LSB value.

Maps the signed range `[-VEL_VERT_RANGE_MS, +VEL_VERT_RANGE_MS]` linearly to `[0, 255]`. Values outside the range are clamped.

Parameters

<i>v</i>	Vertical velocity in m/s.
----------	---------------------------

Returns

Encoded uint8 value.

8.19.2.9 vel_vert_u8_to_ms()

```
float vel_vert_u8_to_ms (
    uint8_t u)
```

Decode a uint8 LSB value back to vertical velocity in m/s.

Inverse of [vel_vert_to_u8](#).

Parameters

<i>u</i>	Encoded uint8 value.
----------	----------------------

Returns

Vertical velocity in m/s.

8.19.3 Variable Documentation

8.19.3.1 timestamps

[timestamps_t](#) timestamps

Global timestamp workspace — reused each call to [distance_calculate\(\)](#).

8.20 App/Calculations/distance.h File Reference

```
#include "../UWB_app/uwb_exchange.h"
#include "../UWB_app/messages.h"
#include "../UWB_app/uwb_network.h"
```

Data Structures

- struct [twr_timestamps_t](#)
- struct [twr_observation_t](#)
- struct [ss_twr_t](#)
- struct [tdoa_twr_t](#)
- struct [first_order_t](#)
- struct [second_order_t](#)
- struct [third_order_t](#)
- struct [timestamps_t](#)

Macros

- `#define UWB_40BIT_MASK 0xFFFFFFFFFULL`
- `#define SPEED_OF_LIGHT 299702547.0`
- `#define METERS_PER_TICK ((float)(DWT_TIME_UNITS * SPEED_OF_LIGHT)) /* ~4.6917e-3 */`
- `#define MAX_PASSIVE (NETWORK_MAX_PEERS - 2) /* 5 */`
- `#define MAX_SECOND_ORDER (2 * MAX_PASSIVE) /* 10 */`
- `#define MAX_THIRD_ORDER (MAX_PASSIVE * (MAX_PASSIVE - 1) / 2) /* 10 */`
- `#define DIST_SHARE_MAX_M 308.26f`
- `#define DIST_SHARE_DIST_MAX_TICKS (DIST_SHARE_MAX_M / (SPEED_OF_LIGHT * DWT_TIME_UNITS))`
- `#define DIST_SHARE_TICKS_PER_LSB (DIST_SHARE_DIST_MAX_TICKS / 65535.0)`
- `#define DWT_TICK_TO_US (1.0f / 63897.6f)`
- `#define VEL_VERT_RANGE_MS 4.0f`
- `#define VEL_VERT_MS_PER_LSB (2.0f * VEL_VERT_RANGE_MS / 255.0f) /* ~31.4 mm/s */`
- `#define VEL_HORIZ_MAX_MS 4.0f`
- `#define VEL_HORIZ_MS_PER_LSB (VEL_HORIZ_MAX_MS / 255.0f) /* ~15.7 mm/s */`
- `#define CERTAINTY_UNRELIABLE_PENALTY 20`
- `#define CERTAINTY_BASE_DSTWR 224`
- `#define CERTAINTY_BASE_SSTWR 127`
- `#define CERTAINTY_BASE_TDOA 60`
- `#define CERTAINTY_TREF_US 1000.0f`
- `#define CERTAINTY_KT 0.015f`
- `#define CERTAINTY_TIMING_MAX 15`
- `#define CERTAINTY_TIMING_MIN -30`
- `#define CERTAINTY_DELTA0_Q8 1024`
- `#define CERTAINTY_KN_Q8 0.594f`
- `#define CERTAINTY_LOS_MAX 30`
- `#define CERTAINTY_LOS_MIN -40`

Enumerations

- enum `MeasurementType` { `MEAS_NONE` = 0 , `MEAS_TDOA_SSTWR` = 1 , `MEAS_SSTWR` = 2 , `MEAS_DSTWR` = 3 }

Functions

- void `distance_calculate` (`uwb_etwr_result_t` result)
Run the full three-order ranging pipeline for one TWR exchange.
- `uint64_t` `position_calibrate_timestamp` (`uint64_t` orig_timestamp)
- `uint16_t` `dist_ticks_to_scale` (double ticks)
Encode a ToF tick count to the compact 16-bit network distance format.
- double `dist_scale_to_ticks` (`uint16_t` encoded)
Decode a compact 16-bit network distance value back to ticks.
- `uint8_t` `vel_vert_to_u8` (float v)
Saturate-and-encode a vertical velocity to a uint8 LSB value.
- float `vel_vert_u8_to_ms` (`uint8_t` u)
Decode a uint8 LSB value back to vertical velocity in m/s.
- `uint8_t` `vel_horiz_to_u8` (float v)
Encode an unsigned horizontal speed to a uint8 LSB value.
- float `vel_horiz_u8_to_ms` (`uint8_t` u)
Decode a uint8 LSB value back to horizontal speed in m/s.

8.20.1 Macro Definition Documentation

8.20.1.1 CERTAINTY_BASE_DSTWR

```
#define CERTAINTY_BASE_DSTWR 224
```

8.20.1.2 CERTAINTY_BASE_SSTWR

```
#define CERTAINTY_BASE_SSTWR 127
```

8.20.1.3 CERTAINTY_BASE_TDOA

```
#define CERTAINTY_BASE_TDOA 60
```

8.20.1.4 CERTAINTY_DELTA0_Q8

```
#define CERTAINTY_DELTA0_Q8 1024
```

8.20.1.5 CERTAINTY_KN_Q8

```
#define CERTAINTY_KN_Q8 0.594f
```

8.20.1.6 CERTAINTY_KT

```
#define CERTAINTY_KT 0.015f
```

8.20.1.7 CERTAINTY_LOS_MAX

```
#define CERTAINTY_LOS_MAX 30
```

8.20.1.8 CERTAINTY_LOS_MIN

```
#define CERTAINTY_LOS_MIN -40
```

8.20.1.9 CERTAINTY_TIMING_MAX

```
#define CERTAINTY_TIMING_MAX 15
```

8.20.1.10 CERTAINTY_TIMING_MIN

```
#define CERTAINTY_TIMING_MIN -30
```

8.20.1.11 CERTAINTY_TREF_US

```
#define CERTAINTY_TREF_US 1000.0f
```

8.20.1.12 CERTAINTY_UNRELIABLE_PENALTY

```
#define CERTAINTY_UNRELIABLE_PENALTY 20
```

8.20.1.13 DIST_SHARE_DIST_MAX_TICKS

```
#define DIST_SHARE_DIST_MAX_TICKS (DIST_SHARE_MAX_M / (SPEED_OF_LIGHT * DWT_TIME_UNITS))
```

8.20.1.14 DIST_SHARE_MAX_M

```
#define DIST_SHARE_MAX_M 308.26f
```

8.20.1.15 DIST_SHARE_TICKS_PER_LSB

```
#define DIST_SHARE_TICKS_PER_LSB (DIST_SHARE_DIST_MAX_TICKS / 65535.0)
```

8.20.1.16 DWT_TICK_TO_US

```
#define DWT_TICK_TO_US (1.0f / 63897.6f)
```

8.20.1.17 MAX_PASSIVE

```
#define MAX_PASSIVE (NETWORK_MAX_PEERS - 2) /* 5 */
```

8.20.1.18 MAX_SECOND_ORDER

```
#define MAX_SECOND_ORDER (2 * MAX_PASSIVE) /* 10 */
```

8.20.1.19 MAX_THIRD_ORDER

```
#define MAX_THIRD_ORDER (MAX_PASSIVE * (MAX_PASSIVE - 1) / 2) /* 10 */
```

8.20.1.20 METERS_PER_TICK

```
#define METERS_PER_TICK ((float)(DWT_TIME_UNITS * SPEED_OF_LIGHT)) /* ~4.6917e-3 */
```

8.20.1.21 SPEED_OF_LIGHT

```
#define SPEED_OF_LIGHT 299702547.0
```

8.20.1.22 UWB_40BIT_MASK

```
#define UWB_40BIT_MASK 0xFFFFFFFFFULL
```

8.20.1.23 VEL_HORIZ_MAX_MS

```
#define VEL_HORIZ_MAX_MS 4.0f
```

8.20.1.24 VEL_HORIZ_MS_PER_LSB

```
#define VEL_HORIZ_MS_PER_LSB (VEL_HORIZ_MAX_MS / 255.0f) /* ~15.7 mm/s */
```

8.20.1.25 VEL_VERT_MS_PER_LSB

```
#define VEL_VERT_MS_PER_LSB (2.0f * VEL_VERT_RANGE_MS / 255.0f) /* ~31.4 mm/s */
```

8.20.1.26 VEL_VERT_RANGE_MS

```
#define VEL_VERT_RANGE_MS 4.0f
```

8.20.2 Enumeration Type Documentation

8.20.2.1 MeasurementType

```
enum MeasurementType
```

Enumerator

MEAS_NONE	
MEAS_TDOA_SSTWR	
MEAS_SSTWR	
MEAS_DSTWR	

8.20.3 Function Documentation

8.20.3.1 `dist_scale_to_ticks()`

```
double dist_scale_to_ticks (
    uint16_t encoded)
```

Decode a compact 16-bit network distance value back to ticks.

Parameters

<i>encoded</i>	Packed distance from the network layer.
----------------	---

Returns

Time-of-flight in DWM3000 ticks, or -1.0 for the sentinel 0xFFFF.

8.20.3.2 `dist_ticks_to_scale()`

```
uint16_t dist_ticks_to_scale (
    double ticks)
```

Encode a ToF tick count to the compact 16-bit network distance format.

The sentinel value 0xFFFF indicates an invalid or out-of-range distance. Values that would round to 0xFFFF are clamped to 0xFFFE.

Parameters

<i>ticks</i>	Time-of-flight in DWM3000 ticks (< 0 or > DIST_SHARE_DIST_MAX_TICKS → invalid).
--------------	---

Returns

Encoded uint16 value, or 0xFFFF if invalid.

8.20.3.3 `distance_calculate()`

```
void distance_calculate (
    uwb_etwr_result_t result)
```

Run the full three-order ranging pipeline for one TWR exchange.

Called after `uwb_extended_twr` completes with UWB_TWR_RECEIVED. Executes in order:

1. Populates `timestamps` from the current network measurement snapshot.
2. First-order DS-TWR (A <-> B distance).
3. Second-order SS-TWR with iterative clock correction (A/B <-> each passive C).

4. Third-order TDOA (each passive-pair C <-> D).

Updates `network_set_distance()` and `network_update_certainty()` for every computed pair. Pads execution time to at least 10 ms to give the FreeRTOS scheduler a consistent call period.

Does nothing if `result` is not `UWB_TWR_RECEIVED`.

Parameters

<code>result</code>	Return value from the preceding <code>uwb_extended_twr</code> call.
---------------------	---

8.20.3.4 `position_calibrate_timestamp()`

```
uint64_t position_calibrate_timestamp (
    uint64_t orig_timestamp)
```

8.20.3.5 `vel_horiz_to_u8()`

```
uint8_t vel_horiz_to_u8 (
    float v)
```

Encode an unsigned horizontal speed to a uint8 LSB value.

Direction is discarded — only magnitude is stored. Values above `VEL_HORIZ_MAX_MS` are clamped.

Parameters

<code>v</code>	Horizontal velocity in m/s (sign ignored).
----------------	--

Returns

Encoded uint8 value.

8.20.3.6 `vel_horiz_u8_to_ms()`

```
float vel_horiz_u8_to_ms (
    uint8_t u)
```

Decode a uint8 LSB value back to horizontal speed in m/s.

Always returns a non-negative value (magnitude only).

Parameters

<code>u</code>	Encoded uint8 value.
----------------	----------------------

Returns

Horizontal speed in m/s (≥ 0).

8.20.3.7 vel_vert_to_u8()

```
uint8_t vel_vert_to_u8 (
    float v)
```

Saturate-and-encode a vertical velocity to a uint8 LSB value.

Maps the signed range [-VEL_VERT_RANGE_MS, +VEL_VERT_RANGE_MS] linearly to [0, 255]. Values outside the range are clamped.

Parameters

<i>v</i>	Vertical velocity in m/s.
----------	---------------------------

Returns

Encoded uint8 value.

8.20.3.8 vel_vert_u8_to_ms()

```
float vel_vert_u8_to_ms (
    uint8_t u)
```

Decode a uint8 LSB value back to vertical velocity in m/s.

Inverse of [vel_vert_to_u8](#).

Parameters

<i>u</i>	Encoded uint8 value.
----------	----------------------

Returns

Vertical velocity in m/s.

8.21 distance.h

[Go to the documentation of this file.](#)

```
00001 /* distance.h -- unchanged from original; all fixes are in distance.c */
00002 #pragma once
00003
00004 #include "../UWB_app/uwb_exchange.h" /* uwb_etwr_result_t */
00005 #include "../UWB_app/messages.h"
00006 #include "../UWB_app/uwb_network.h"
00007
00008 void distance_calculate(uwb_etwr_result_t result);
00009
00010 uint64_t position_calibrate_timestamp(uint64_t orig_timestamp);
00011
00012 // #define DEBUG_distance_populate
00013
00014 #define UWB_40BIT_MASK 0xFFFFFFFFFULL
00015
```

```

00016 #define SPEED_OF_LIGHT    299702547.0
00017 #define METERS_PER_TICK    ((float)(DWT_TIME_UNITS * SPEED_OF_LIGHT)) /* ~4.6917e-3 */
00018
00019 #define MAX_PASSIVE          (NETWORK_MAX_PEERS - 2)           /* 5 */
00020 #define MAX_SECOND_ORDER     (2 * MAX_PASSIVE)                /* 10 */
00021 #define MAX_THIRD_ORDER      (MAX_PASSIVE * (MAX_PASSIVE - 1) / 2) /* 10 */
00022
00023 #define DIST_SHARE_MAX_M      308.26f
00024 #define DIST_SHARE_DIST_MAX_TICKS (DIST_SHARE_MAX_M / (SPEED_OF_LIGHT * DWT_TIME_UNITS))
00025 #define DIST_SHARE_TICKS_PER_LSB (DIST_SHARE_DIST_MAX_TICKS / 65535.0)
00026 #define DWT_TICK_TO_US        (1.0f / 63897.6f)
00027
00028
00029 /* vel_vert : signed ±4 m/s, offset-binary (direction matters) */
00030 #define VEL_VERT_RANGE_MS      4.0f
00031 #define VEL_VERT_MS_PER_LSB    (2.0f * VEL_VERT_RANGE_MS / 255.0f) /* ~31.4 mm/s */
00032
00033 /* vel_horiz : unsigned 0..4 m/s (direction irrelevant, 2x resolution) */
00034 #define VEL_HORIZ_MAX_MS       4.0f
00035 #define VEL_HORIZ_MS_PER_LSB   (VEL_HORIZ_MAX_MS / 255.0f)          /* ~15.7 mm/s */
00036
00037 #define CERTAINTY_UNRELIABLE_PENALTY 20
00038
00039
00040 /*
00041  * CERTAINTY SYSTEM -- UWB Position Measurement Quality Score
00042  *
00043  * Produces a uint8_t score [0..254] each measurement round (200 ms).
00044  * Score is recomputed fresh every round -- no accumulation.
00045  *
00046  * Score anchors:
00047  *   0 -- node not heard this round
00048  *  127 -- SS-TWR, average turnaround, average LOS (the "neutral" node)
00049  *  254 -- DS-TWR, perfect LOS
00050  */
00051
00052 /* ----- */
00053 /* TUNING DEFINES -- adjust these to calibrate your environment */
00054 /* ----- */
00055
00056 /*
00057  * Base certainty scores per measurement method.
00058  * These are the starting values BEFORE timing and LOS adjustments.
00059  * Gap between methods must exceed the max combined adjustment range
00060  * so that a better method always outranks a worse one.
00061  *
00062  *   DS-TWR base: 224 (+30 LOS headroom → 254 max)
00063  *   SS-TWR base: 127 (neutral midpoint → 127 at average conditions)
00064  *   TDOA base: 60 (indirect geometry, lower ceiling)
00065  */
00066 #define CERTAINTY_BASE_DSTWR 224
00067 #define CERTAINTY_BASE_SSTWR 127
00068 #define CERTAINTY_BASE_TDOA 60
00069
00070 /*
00071  * SS-TWR timing adjustment -- linear, centered at CERTAINTY_TREF_US.
00072  *
00073  * CERTAINTY_TREF_US: turnaround time (µs) considered "average".
00074  * At this value timing offset = 0.
00075  * Faster → positive bonus (up to CERTAINTY_TIMING_MAX).
00076  * Slower → negative penalty (down to CERTAINTY_TIMING_MIN).
00077  *
00078  * CERTAINTY_KT: slope in offset-units per µs deviation.
00079  * Example: KT = 0.01 means 1000 µs off → ±10 points.
00080  */
00081 #define CERTAINTY_TREF_US 1000.0f
00082 #define CERTAINTY_KT 0.015f
00083 #define CERTAINTY_TIMING_MAX 15
00084 #define CERTAINTY_TIMING_MIN -30
00085
00086 /*
00087  * LOS adjustment -- sqrt nonlinearity, centered at CERTAINTY_DELTA0_DB.
00088  *
00089  * CERTAINTY_DELTA0_DB: the RSSI-FP gap (dB) considered "average" LOS.
00090  * At this value LOS offset = 0.
00091  * Below → positive bonus (good LOS), up to CERTAINTY_LOS_MAX.
00092  * Above → penalty (NLOS), steep in first ~10 dB then flattening,
00093  * down to CERTAINTY_LOS_MIN.
00094  *
00095  * CERTAINTY_KN: controls steepness of the sqrt curve.
00096  * KN = 9.5 gives ~-30 penalty at 10 dB above average.
00097  */
00098 #define CERTAINTY_DELTA0_Q8 1024
00099 #define CERTAINTY_KN_Q8 0.594f
00100 #define CERTAINTY_LOS_MAX 30
00101 #define CERTAINTY_LOS_MIN -40
00102

```

```

00103 /* ----- */
00104 /* TYPES */
00105 /* ----- */
00106
00107 typedef enum {
00108     MEAS_NONE = 0, /* No measurement received this round */
00109     MEAS_TDOA_SSTWR = 1, /* TDOA aided by SS-TWR */
00110     MEAS_SSTWR = 2, /* Single-sided two-way ranging */
00111     MEAS_DSTWR = 3, /* Double-sided two-way ranging (best) */
00112 } MeasurementType;
00113
00114 /*
00115  * @brief Timestamps captured during an active DS-TWR exchange.
00116  *
00117  * Role determines which fields are populated:
00118  *
00119  * | Role | Valid fields |
00120  * |-----|-----|
00121  * | Initiator | poll_tx, resp_rx, final_tx |
00122  * | Responder | poll_rx, resp_tx, final_rx |
00123  *
00124  * Unused fields are zero-initialised and must not be read.
00125  */
00126 typedef struct {
00127     uint64_t poll_tx;
00128     uwb_rx_meas_t poll_rx;
00129     bool poll_rx_unreliable;
00130     uint64_t resp_tx;
00131     uwb_rx_meas_t resp_rx;
00132     bool resp_rx_unreliable;
00133     uint64_t final_tx;
00134     uwb_rx_meas_t final_rx;
00135     bool final_rx_unreliable;
00136 } twr_timestamps_t;
00137
00138
00139 typedef struct {
00140     uwb_rx_meas_t poll_rx;
00141     uwb_rx_meas_t resp_rx;
00142     bool poll_rx_unreliable;
00143     bool resp_rx_unreliable;
00144 } twr_observation_t;
00145
00146
00147 typedef struct {
00148     uint64_t init_tx;
00149     uwb_rx_meas_t init_rx;
00150
00151     uint64_t answer_tx;
00152     uwb_rx_meas_t answer_rx;
00153 } ss_twr_t;
00154
00155 typedef struct {
00156     uint64_t init_tx;
00157     uwb_rx_meas_t init_rx;
00158
00159     uwb_rx_meas_t master_init_rx;
00160
00161     uint64_t answer_tx;
00162     uwb_rx_meas_t answer_rx;
00163 } tdoa_twr_t;
00164
00165 typedef struct {
00166     uint16_t initiator_id;
00167     uint16_t responder_id;
00168
00169     twr_timestamps_t twr;
00170     double result_distance_tick; /* populated after calculation */
00171 } first_order_t;
00172
00173 typedef struct {
00174     uint16_t initiator_id;
00175     uint16_t responder_id;
00176     twr_observation_t twr_observation;
00177     ss_twr_t twr;
00178     bool init_rx_unreliable;
00179     bool answer_rx_unreliable;
00180     double result_distance_tick;
00181 } second_order_t;
00182
00183 typedef struct {
00184     uint16_t initiator_id;
00185     uint16_t responder_id;
00186     twr_observation_t twr_observation_c;
00187     twr_observation_t twr_observation_d;
00188     ss_twr_t twr;
00189     bool answer_rx_unreliable;

```

```

00190     double          result_distance_tick;
00191 } third_order_t;
00192
00193 typedef struct {
00194
00195     first_order_t  first;                      // always exactly 1
00196
00197     second_order_t second[MAX_SECOND_ORDER];
00198     uint8_t        second_count;
00199
00200     third_order_t  third[MAX_THIRD_ORDER];
00201     uint8_t        third_count;
00202
00203 } timestamps_t;
00204
00205
00206 uint16_t dist_ticks_to_scale(double ticks);
00207 double dist_scale_to_ticks(uint16_t encoded);
00208 uint8_t vel_vert_to_u8(float v);
00209 float vel_vert_u8_to_ms(uint8_t u);
00210 uint8_t vel_horiz_to_u8(float v);
00211 float vel_horiz_u8_to_ms(uint8_t u);

```

8.22 App/Calculations/ekf.c File Reference

```

#include "ekf.h"
#include "main.h"
#include "../UWB_app/uwb_network.h"
#include "../Calculations/distance.h"
#include <math.h>
#include <string.h>
#include "../Generic/my_print.h"

```

Macros

- `#define EKF_INIT_RAND_M 2.0f`
Spread of the Xorshift32 random seed used for unknown tag nodes (metres).
- `#define EKF_TAG_HARDCODED_Z -0.589f`
Tag height above the floor in metres.

Functions

- `uint32_t HAL_GetTick` (void)
- `float ekf_certainty_to_sigma` (uint8_t certainty)
Convert a certainty byte to a range measurement standard deviation.
- `void ekf_init` (const `ekf_node_hint_t` *pos_hints, uint8_t pos_hints_n)
Initialise (or soft-reset) the cooperative EKF.
- `void ekf_step` (float az_self_ms, float ah_self_ms)
Advance the EKF by one step, dispatching to the current phase.
- `bool ekf_anchor_ready` (void)
Return true if Phase 1 anchor survey has completed.
- `ekf_phase_t ekf_get_phase` (void)
Return the current EKF phase.
- `const float(* ekf_get_anchor_positions` (void))[3]
Return a pointer to the frozen anchor position array.
- `const coop_ekf_t * ekf_get_state` (void)
Return a read-only pointer to the internal EKF state.
- `bool ekf_get_peer_pos` (uint16_t peer_id, float pos_out[3])
Look up the current estimated position of a peer or anchor.

Variables

- `const uint16_t EKF_ANCHOR_IDS[EKF_NUM_ANCHORS]`

8.22.1 Macro Definition Documentation

8.22.1.1 EKF_INIT_RAND_M

```
#define EKF_INIT_RAND_M 2.0f
```

Spread of the Xorshift32 random seed used for unknown tag nodes (metres).

8.22.1.2 EKF_TAG_HARDCODED_Z

```
#define EKF_TAG_HARDCODED_Z -0.589f
```

Tag height above the floor in metres.

8.22.2 Function Documentation

8.22.2.1 ekf_anchor_ready()

```
bool ekf_anchor_ready (  
    void )
```

Return true if Phase 1 anchor survey has completed.

Returns

true if the EKF is in EKF_PHASE_TAG_LOCALIZE, false otherwise.

`ekf_anchor_ready` Returns true once Phase 1 has converged and Phase 2 is active. The LED was toggled at the exact moment of transition.

8.22.2.2 ekf_certainty_to_sigma()

```
float ekf_certainty_to_sigma (
    uint8_t certainty)
```

Convert a certainty byte to a range measurement standard deviation.

Maps `certainty` linearly from [0, 255] → [EKF_SIGMA_MAX, EKF_SIGMA_MIN].

Parameters

<i>certainty</i>	Certainty value in [0, 255].
------------------	------------------------------

Returns

Corresponding measurement noise sigma in metres.

`ekf_certainty_to_sigma` Convert certainty byte [0..255] → measurement std-dev in metres. Exposed for external logging / testing.

8.22.2.3 ekf_get_anchor_positions()

```
const float (* ekf_get_anchor_positions (
    void )) [3]
```

Return a pointer to the frozen anchor position array.

Valid after Phase 1 convergence. Anchor 0 is always (0,0,0).

Returns

Pointer to `anchor_pos[EKF_NUM_ANCHORS][3]`.

`ekf_get_anchor_positions` Returns a pointer to the frozen anchor position table [EKF_NUM_ANCHORS][3]. Index 0 is always (0,0,0). Indices 1–3 are valid only after [ekf_anchor_ready\(\)](#) returns true.

8.22.2.4 ekf_get_peer_pos()

```
bool ekf_get_peer_pos (
    uint16_t peer_id,
    float pos_out[3])
```

Look up the current estimated position of a peer or anchor.

In Phase 2 anchors are not in the state vector — the frozen position from `anchor_pos[]` is returned directly. Tags (Phase 2) and anchors (Phase 1) are found by slot.

Parameters

<i>peer_id</i>	Node ID to query.
<i>pos_out</i>	Output array [x, y, z] in metres.

Returns

true if the position was found, false if the ID is unknown.

`ekf_get_peer_pos` Copy the last-known estimated position of `peer_id` into `pos_out[3]`.

Phase 1: works for estimated anchor peers. Phase 2: works for tags (estimated) AND anchors (frozen from Phase 1). Away peers retain their last position — never returns stale zeros.

Returns

true if `peer_id` is known and `pos_out` was written. false if `peer_id` is unknown.

8.22.2.5 ekf_get_phase()

```
ekf_phase_t ekf_get_phase (
    void )
```

Return the current EKF phase.

Returns

`EKF_PHASE_ANCHOR_SURVEY` or `EKF_PHASE_TAG_LOCALIZE`.

`ekf_get_phase` Returns the current phase: `EKF_PHASE_ANCHOR_SURVEY` or `EKF_PHASE_TAG_LOCALIZE`.

8.22.2.6 ekf_get_state()

```
const coop_ekf_t * ekf_get_state (
    void )
```

Return a read-only pointer to the internal EKF state.

Returns

Pointer to the `coop_ekf_t` struct.

`ekf_get_state` Read-only pointer to the internal joint filter state. Use for debug logging only — do NOT modify the returned struct.

8.22.2.7 ekf_init()

```
void ekf_init (
    const ekf_node_hint_t * pos_hints,
    uint8_t pos_hints_n)
```

Initialise (or soft-reset) the cooperative EKF.

If the EKF is already in Phase 2 (tag localization) — e.g. after a self-disconnect/reconnect — only stale motion priors are reset and P is inflated; the solved anchor map and last-known tag positions are preserved so the filter recovers quickly without repeating Phase 1.

On a fresh start (Phase 1) the full filter state is zeroed, slots are allocated for all visible anchor peers, and initial covariances are set.

Parameters

<i>pos_hints</i>	Array of node ID / position hint pairs, or NULL.
<i>pos_hints_n</i>	Length of <i>pos_hints</i> .

ekf_init Call once at startup (or after hard reset). Begins Phase 1: anchor survey.

If called again while Phase 2 is already running, it is treated as a network self-reconnect event: frozen anchors and last-known tag positions are preserved, motion priors are cleared, and covariance is inflated.

All four anchor IDs must be set in `EKF_ANCHOR_IDS[]` ([ekf.c](#)) first.

8.22.2.8 ekf_step()

```
void ekf_step (
    float az_self_ms,
    float ah_self_ms)
```

Advance the EKF by one step, dispatching to the current phase.

Computes a variable *dt* from `HAL_GetTick()`. On the first call (!*ekf.initialised*) `EKF_DT_NOM_S` is used. The step is clamped to `[0.001 s, EKF_DT_MAX_S]` to protect against missed ticks.

Parameters

<i>az_self_ms</i>	Self vertical speed from IMU (m/s).
<i>ah_self_ms</i>	Self horizontal speed from IMU (m/s).

ekf_step Call every ~200 ms when new ranging data is available.

Phase 1: *az_self_ms* / *ah_self_ms* are ignored (anchors stationary). Phase 2: used as self IMU velocity inputs (m/s, gravity removed).

Parameters

<i>az_self_ms</i>	Self vertical velocity (m/s)
-------------------	------------------------------

<code>ah_self_ms</code>	Self horizontal speed magnitude (m/s, ≥ 0)
-------------------------	--

8.22.2.9 HAL_GetTick()

```
uint32_t HAL_GetTick (
    void ) [extern]
```

8.22.3 Variable Documentation

8.22.3.1 EKF_ANCHOR_IDS

```
const uint16_t EKF_ANCHOR_IDS[EKF_NUM_ANCHORS]
```

Initial value:

```

                                = {
    0x63D8u,
    0x91EDu,
    0xC019u,
    0x28CCu,
}
```

Network IDs of the four anchors. Defined (not merely declared) in [ekf.c](#) — edit them there. Index 0 = origin anchor, always at (0, 0, 0).

8.23 App/Calculations/ekf.h File Reference

```
#include <math.h>
#include <string.h>
#include <stdint.h>
#include <stdbool.h>
#include "../Generic/UART.h"
```

Data Structures

- struct [coop_ekf_t](#)

Macros

- #define [EKF_NUM_ANCHORS](#) 4U
- #define [EKF_ANCHOR_P_CONVERGED](#) 0.02f
- #define [EKF_ANCHOR_MIN_STEPS](#) 100U
- #define [EKF_MAX_PEERS](#) 6
- #define [EKF_MAX_STATE](#) ([EKF_MAX_PEERS](#) * 3) /* 18 elements: [x,y,z] × 6 */
- #define [EKF_SIGMA_MIN](#) 0.05f /* sigma_d @ certainty=255 (m) */
- #define [EKF_SIGMA_MAX](#) 0.8f /* sigma_d @ certainty=0 (m) */
- #define [EKF_Q_Z_POS](#) 0.0f /* Process noise — Z axis (m²/step) */
- #define [EKF_Q_H_FLOOR](#) 0.002f /* Horizontal noise floor (m²/step) */

- `#define EKF_Q_H_VEL_SCALE 2.0f` /* extra variance per $(v \cdot dt)^2$ when moving */
- `#define EKF_OUTLIER_GATE_POS 2.5f` /* Reject $|\text{innov}| > \text{gate} \cdot \text{sqrtS}$ */
- `#define EKF_OUTLIER_GATE_NEG 4.0f`
- `#define EKF_INIT_P_POS 9.0f` /* Initial position variance (m^2) */
- `#define EKF_P_MAX 100.0f` /* P diagonal clamp (m^2) */
- `#define EKF_DT_NOM_S 0.2f` /* Nominal step period (s) */
- `#define EKF_DT_MAX_S 0.5f` /* dt clamp — absorbs missed steps (s) */
- `#define EKF_IMU_DEADBAND_MS 0.15f` /* Speed below this → treated as 0 */
- `#define EKF_STATIONARY_STEPS 3U` /* Consecutive deadband steps → static */
- `#define EKF_Q_H_FLOOR_STATIONARY 1e-8f` /* Horiz. noise when stationary (m^2) */
- `#define EKF_VEL_EMA_ALPHA 0.6f` /* EMA smoothing: 0=frozen 1=raw IMU */
- `#define EKF_HDG_MIN_DELTA_M 0.05f` /* Min Δpos to update heading (m) */
- `#define EKF_MAX_STEP_M 0.4f` /* Max predict displacement/step (m) */
- `#define EKF_MAX_UPDATE_M 0.5f` /* Max update displacement/step (m) */

Enumerations

- enum `ekf_phase_t` { `EKF_PHASE_ANCHOR_SURVEY = 0` , `EKF_PHASE_TAG_LOCALIZE = 1` }

Functions

- void `ekf_init` (const `ekf_node_hint_t` *pos_hints, uint8_t pos_hints_n)
Initialise (or soft-reset) the cooperative EKF.
- void `ekf_step` (float az_self_ms, float ah_self_ms)
Advance the EKF by one step, dispatching to the current phase.
- bool `ekf_anchor_ready` (void)
Return true if Phase 1 anchor survey has completed.
- `ekf_phase_t` `ekf_get_phase` (void)
Return the current EKF phase.
- const float(*) `ekf_get_anchor_positions` (void)[3]
Return a pointer to the frozen anchor position array.
- bool `ekf_get_peer_pos` (uint16_t peer_id, float pos_out[3])
Look up the current estimated position of a peer or anchor.
- float `ekf_certainty_to_sigma` (uint8_t certainty)
Convert a certainty byte to a range measurement standard deviation.
- const `coop_ekf_t` * `ekf_get_state` (void)
Return a read-only pointer to the internal EKF state.

Variables

- const uint16_t `EKF_ANCHOR_IDS` [`EKF_NUM_ANCHORS`]

8.23.1 Macro Definition Documentation

8.23.1.1 EKF_ANCHOR_MIN_STEPS

```
#define EKF_ANCHOR_MIN_STEPS 100U
```

Minimum EKF steps before convergence is declared, regardless of P. Guards against premature lock-in during the first noisy measurements.

8.23.1.2 EKF_ANCHOR_P_CONVERGED

```
#define EKF_ANCHOR_P_CONVERGED 0.02f
```

Position variance threshold (m^2) per axis. Phase 1 ends when every estimated anchor axis satisfies $P[k][k] < \text{EKF_ANCHOR_P_CONVERGED}$. $0.5 \text{ m}^2 \sim \pm 0.71 \text{ m}$ std-dev. Increase for a faster but noisier survey; decrease for a more accurate anchor map (takes longer).

8.23.1.3 EKF_DT_MAX_S

```
#define EKF_DT_MAX_S 0.5f /* dt clamp -- absorbs missed steps (s) */
```

8.23.1.4 EKF_DT_NOM_S

```
#define EKF_DT_NOM_S 0.2f /* Nominal step period (s) */
```

8.23.1.5 EKF_HDG_MIN_DELTA_M

```
#define EKF_HDG_MIN_DELTA_M 0.05f /* Min @f$\delta@f$pos to update heading (m) */
```

8.23.1.6 EKF_IMU_DEADBAND_MS

```
#define EKF_IMU_DEADBAND_MS 0.15f /* Speed below this → treated as 0 */
```

8.23.1.7 EKF_INIT_P_POS

```
#define EKF_INIT_P_POS 9.0f /* Initial position variance ( $\text{m}^2$ ) */
```

8.23.1.8 EKF_MAX_PEERS

```
#define EKF_MAX_PEERS 6
```

Maximum total EKF slots. Phase 1 uses at most three slots: anchors 1..3, including self if this device is one of them. Phase 2 uses one slot per non-anchor tag, including self when self is a tag. Keep at 6 unless EKF_MAX_STATE/↵ RAM budget is deliberately reviewed.

8.23.1.9 EKF_MAX_STATE

```
#define EKF_MAX_STATE (EKF_MAX_PEERS * 3) /* 18 elements: [x,y,z] × 6 */
```

8.23.1.10 EKF_MAX_STEP_M

```
#define EKF_MAX_STEP_M 0.4f /* Max predict displacement/step (m) */
```

8.23.1.11 EKF_MAX_UPDATE_M

```
#define EKF_MAX_UPDATE_M 0.5f /* Max update displacement/step (m) */
```

8.23.1.12 EKF_NUM_ANCHORS

```
#define EKF_NUM_ANCHORS 4U
```

Number of anchors. One is always the world origin. Change only when also updating EKF_ANCHOR_IDS[] in [ekf.c](#).

8.23.1.13 EKF_OUTLIER_GATE_NEG

```
#define EKF_OUTLIER_GATE_NEG 4.0f
```

8.23.1.14 EKF_OUTLIER_GATE_POS

```
#define EKF_OUTLIER_GATE_POS 2.5f /* Reject |innov| > gate*sqrtS */
```

8.23.1.15 EKF_P_MAX

```
#define EKF_P_MAX 100.0f /* P diagonal clamp (m2) */
```

8.23.1.16 EKF_Q_H_FLOOR

```
#define EKF_Q_H_FLOOR 0.002f /* Horizontal noise floor (m2/step) */
```

8.23.1.17 EKF_Q_H_FLOOR_STATIONARY

```
#define EKF_Q_H_FLOOR_STATIONARY 1e-8f /* Horiz. noise when stationary (m2) */
```

8.23.1.18 EKF_Q_H_VEL_SCALE

```
#define EKF_Q_H_VEL_SCALE 2.0f /* extra variance per (v*dt)2 when moving */
```

8.23.1.19 EKF_Q_Z_POS

```
#define EKF_Q_Z_POS 0.0f /* Process noise -- Z axis (m2/step) */
```

8.23.1.20 EKF_SIGMA_MAX

```
#define EKF_SIGMA_MAX 0.8f /* sigma_d @ certainty=0 (m) */
```

8.23.1.21 EKF_SIGMA_MIN

```
#define EKF_SIGMA_MIN 0.05f /* sigma_d @ certainty=255 (m) */
```

8.23.1.22 EKF_STATIONARY_STEPS

```
#define EKF_STATIONARY_STEPS 3U /* Consecutive deadband steps → static */
```

8.23.1.23 EKF_VEL_EMA_ALPHA

```
#define EKF_VEL_EMA_ALPHA 0.6f /* EMA smoothing: 0=frozen 1=raw IMU */
```

8.23.2 Enumeration Type Documentation

8.23.2.1 ekf_phase_t

```
enum ekf\_phase\_t
```

Enumerator

EKF_PHASE_ANCHOR_SURVEY	Phase 1: estimating anchor positions
EKF_PHASE_TAG_LOCALIZE	Phase 2: estimating tag positions

8.23.3 Function Documentation

8.23.3.1 ekf_anchor_ready()

```
bool ekf_anchor_ready (
    void )
```

Return true if Phase 1 anchor survey has completed.

`ekf_anchor_ready` Returns true once Phase 1 has converged and Phase 2 is active. The LED was toggled at the exact moment of transition.

Returns

true if the EKF is in `EKF_PHASE_TAG_LOCALIZE`, false otherwise.

8.23.3.2 ekf_certainty_to_sigma()

```
float ekf_certainty_to_sigma (
    uint8_t certainty)
```

Convert a certainty byte to a range measurement standard deviation.

`ekf_certainty_to_sigma` Convert certainty byte [0..255] → measurement std-dev in metres. Exposed for external logging / testing.

Maps `certainty` linearly from [0, 255] → [EKF_SIGMA_MAX, EKF_SIGMA_MIN].

Parameters

<i>certainty</i>	Certainty value in [0, 255].
------------------	------------------------------

Returns

Corresponding measurement noise sigma in metres.

8.23.3.3 ekf_get_anchor_positions()

```
const float (* ekf_get_anchor_positions (
    void )) [3]
```

Return a pointer to the frozen anchor position array.

`ekf_get_anchor_positions` Returns a pointer to the frozen anchor position table [EKF_NUM_ANCHORS][3]. Index 0 is always (0,0,0). Indices 1–3 are valid only after [ekf_anchor_ready\(\)](#) returns true.

Valid after Phase 1 convergence. Anchor 0 is always (0,0,0).

Returns

Pointer to `anchor_pos[EKF_NUM_ANCHORS][3]`.

8.23.3.4 ekf_get_peer_pos()

```
bool ekf_get_peer_pos (
    uint16_t peer_id,
    float pos_out[3])
```

Look up the current estimated position of a peer or anchor.

`ekf_get_peer_pos` Copy the last-known estimated position of `peer_id` into `pos_out[3]`.

Phase 1: works for estimated anchor peers. Phase 2: works for tags (estimated) AND anchors (frozen from Phase 1). Away peers retain their last position — never returns stale zeros.

Returns

true if `peer_id` is known and `pos_out` was written. false if `peer_id` is unknown.

In Phase 2 anchors are not in the state vector — the frozen position from `anchor_pos[]` is returned directly. Tags (Phase 2) and anchors (Phase 1) are found by slot.

Parameters

<i>peer_id</i>	Node ID to query.
<i>pos_out</i>	Output array [x, y, z] in metres.

Returns

true if the position was found, false if the ID is unknown.

8.23.3.5 ekf_get_phase()

```
ekf_phase_t ekf_get_phase (
    void )
```

Return the current EKF phase.

`ekf_get_phase` Returns the current phase: `EKF_PHASE_ANCHOR_SURVEY` or `EKF_PHASE_TAG_LOCALIZE`.

Returns

`EKF_PHASE_ANCHOR_SURVEY` or `EKF_PHASE_TAG_LOCALIZE`.

8.23.3.6 ekf_get_state()

```
const coop_ekf_t * ekf_get_state (
    void )
```

Return a read-only pointer to the internal EKF state.

`ekf_get_state` Read-only pointer to the internal joint filter state. Use for debug logging only — do NOT modify the returned struct.

Returns

Pointer to the `coop_ekf_t` struct.

8.23.3.7 ekf_init()

```
void ekf_init (
    const ekf_node_hint_t * pos_hints,
    uint8_t pos_hints_n)
```

Initialise (or soft-reset) the cooperative EKF.

ekf_init Call once at startup (or after hard reset). Begins Phase 1: anchor survey.

If called again while Phase 2 is already running, it is treated as a network self-reconnect event: frozen anchors and last-known tag positions are preserved, motion priors are cleared, and covariance is inflated.

All four anchor IDs must be set in EKF_ANCHOR_IDS[] ([ekf.c](#)) first.

If the EKF is already in Phase 2 (tag localization) — e.g. after a self-disconnect/reconnect — only stale motion priors are reset and P is inflated; the solved anchor map and last-known tag positions are preserved so the filter recovers quickly without repeating Phase 1.

On a fresh start (Phase 1) the full filter state is zeroed, slots are allocated for all visible anchor peers, and initial covariances are set.

Parameters

<i>pos_hints</i>	Array of node ID / position hint pairs, or NULL.
<i>pos_hints_n</i>	Length of <i>pos_hints</i> .

8.23.3.8 ekf_step()

```
void ekf_step (
    float az_self_ms,
    float ah_self_ms)
```

Advance the EKF by one step, dispatching to the current phase.

ekf_step Call every ~200 ms when new ranging data is available.

Phase 1: *az_self_ms* / *ah_self_ms* are ignored (anchors stationary). Phase 2: used as self IMU velocity inputs (m/s, gravity removed).

Parameters

<i>az_self_ms</i>	Self vertical velocity (m/s)
<i>ah_self_ms</i>	Self horizontal speed magnitude (m/s, ≥ 0)

Computes a variable *dt* from [HAL_GetTick\(\)](#). On the first call (!*ekf.initialised*) EKF_DT_NOM_S is used. The step is clamped to [0.001 s, EKF_DT_MAX_S] to protect against missed ticks.

Parameters

<i>az_self_ms</i>	Self vertical speed from IMU (m/s).
-------------------	-------------------------------------

<code>ah_self_ms</code>	Self horizontal speed from IMU (m/s).
-------------------------	---------------------------------------

8.23.4 Variable Documentation

8.23.4.1 EKF_ANCHOR_IDS

```
const uint16_t EKF_ANCHOR_IDS[EKF_NUM_ANCHORS] [extern]
```

Network IDs of the four anchors. Defined (not merely declared) in [ekf.c](#) — edit them there. Index 0 = origin anchor, always at (0, 0, 0).

8.24 ekf.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UWB_IMU_EKF_H
00002 #define UWB_IMU_EKF_H
00003
00004 #include <math.h>
00005 #include <string.h>
00006 #include <stdint.h>
00007 #include <stdbool.h>
00008 #include "../Generic/UART.h"
00009
00010 /* =====
00011 * Two-Phase Cooperative UWB + IMU Position Estimator
00012 * =====
00013 *
00014 * PHASE 1 -- ANCHOR SURVEY (EKF_PHASE_ANCHOR_SURVEY)
00015 * -----
00016 * Goal : Determine the 3-D positions of all EKF_NUM_ANCHORS anchors.
00017 *
00018 * Inputs : Inter-anchor UWB range measurements only.
00019 *          IMU data is IGNORED -- anchors are stationary.
00020 *
00021 * Coordinate frame:
00022 *   EKF_ANCHOR_IDS[0] is pinned at the world origin (0, 0, 0).
00023 *   EKF_ANCHOR_IDS[1..3] start with EKF_INIT_P_POS variance and
00024 *   converge via sequential EKF updates.
00025 *
00026 * Slot table (Phase 1):
00027 *   Slots hold only non-origin anchors.
00028 *   If this device is anchor 0 → no self_slot, self is pinned.
00029 *   If this device is anchor 1/2/3 → self_slot holds self estimate.
00030 *   If this device is a tag → no self_slot (tag not estimated).
00031 *   Non-anchor (tag) peers are ignored entirely.
00032 *
00033 * Convergence condition:
00034 *   1. All four anchor IDs are present in the network (not "away").
00035 *   2. Every estimated anchor axis satisfies P[k][k] < EKF_ANCHOR_P_CONVERGED.
00036 *   3. At least EKF_ANCHOR_MIN_STEPS have been executed.
00037 *
00038 * On convergence:
00039 *   Anchor positions are frozen into anchor_pos[EKF_NUM_ANCHORS][3].
00040 *   LED_R is toggled once: HAL_GPIO_TogglePin(LED_R_GPIO_Port, LED_R_Pin).
00041 *   Filter transitions to Phase 2 and is re-initialised for tag tracking.
00042 *
00043 *
00044 * PHASE 2 -- TAG LOCALIZATION (EKF_PHASE_TAG_LOCALIZE)
00045 * -----
00046 * Goal : Estimate the 3-D positions of all non-anchor (tag) devices.
00047 *
00048 * Inputs : UWB ranges -- anchor→tag AND tag→tag.
00049 *          IMU -- horizontal speed + vertical speed per tag.
00050 *
00051 * Anchor positions are known and fixed (from Phase 1 result).
00052 * Anchors are NOT in the state vector; they are fixed reference points.
00053 *
00054 * Slot table (Phase 2):
00055 *   Slots hold only non-anchor tag devices.
00056 *   If this device is a tag → self_slot allocated.
```

```

00057 *      If this device is an anchor      → no self_slot;
00058 *                                      self position published from anchor_pos[].
00059 *      New tags can join at any time      → slot allocated on first appearance.
00060 *
00061 *      Disconnect / reconnect:
00062 *          When a tag disappears: peer_away[p] = true, P reset (inflated).
00063 *                                  Last known position in x[] is PRESERVED and can
00064 *                                  still be returned by ekf_get_peer_pos().
00065 *                                  During write-back the EKF also writes this last
00066 *                                  known position if the network table still has a
00067 *                                  row for that peer ID.
00068 *          When it returns: peer_away cleared, P inflated again so fresh
00069 *                            measurements quickly reconverge.
00070 *                            peer_seeded flag and x[] position unchanged
00071 *                            -- this is the "same position as before" guarantee.
00072 *
00073 *      Joint state vector x[EKF_MAX_STATE]:
00074 *      [ x_T0, y_T0, z_T0, x_T1, y_T1, z_T1, ... ] (one 3-D block per tag slot)
00075 *
00076 *      Typical call cadence:
00077 *          ekf_init(); // once at startup
00078 *          // every ~200 ms:
00079 *          ekf_step(az_self_ms, ah_self_ms);
00080 *          if (ekf_anchor_ready()) { ... } // Phase 2 active
00081 * ===== */
00082
00083
00084 /* =====
00085 *      Anchor table
00086 *      ===== */
00087
00090 #define EKF_NUM_ANCHORS 4U
00091
00095 extern const uint16_t EKF_ANCHOR_IDS[EKF_NUM_ANCHORS];
00096
00097
00098 /* =====
00099 *      Phase 1 convergence thresholds
00100 *      ===== */
00101
00106 #define EKF_ANCHOR_P_CONVERGED 0.02f
00107
00110 #define EKF_ANCHOR_MIN_STEPS 100U
00111
00112
00113 /* =====
00114 *      Dimensions
00115 *      ===== */
00116
00122 #define EKF_MAX_PEERS 6
00123 #define EKF_MAX_STATE (EKF_MAX_PEERS * 3) /* 18 elements: [x,y,z] × 6 */
00124
00125
00126 /* =====
00127 *      Filter tuning
00128 *      ===== */
00129
00130 #define EKF_SIGMA_MIN 0.05f /* sigma_d @ certainty=255 (m) */
00131 #define EKF_SIGMA_MAX 0.8f /* sigma_d @ certainty=0 (m) */
00132 #define EKF_Q_Z_POS 0.0f /* Process noise -- Z axis (m2/step) */
00133 #define EKF_Q_H_FLOOR 0.002f /* Horizontal noise floor (m2/step) */
00134 #define EKF_Q_H_VEL_SCALE 2.0f /* extra variance per (v·dt)2 when moving */
00135 #define EKF_OUTLIER_GATE_POS 2.5f /* Reject |innov| > gate·sqrtS */
00136 #define EKF_OUTLIER_GATE_NEG 4.0f
00137 #define EKF_INIT_P_POS 9.0f /* Initial position variance (m2) */
00138 #define EKF_P_MAX 100.0f /* P diagonal clamp (m2) */
00139 #define EKF_DT_NOM_S 0.2f /* Nominal step period (s) */
00140 #define EKF_DT_MAX_S 0.5f /* dt clamp -- absorbs missed steps (s) */
00141
00142 /* IMU / motion tuning -- override via project config or ekf.h if desired */
00143 #define EKF_IMU_DEADBAND_MS 0.15f /* Speed below this → treated as 0 */
00144 #define EKF_STATIONARY_STEPS 3U /* Consecutive deadband steps → static */
00145 #define EKF_Q_H_FLOOR_STATIONARY 1e-8f /* Horiz. noise when stationary (m2) */
00146 #define EKF_VEL_EMA_ALPHA 0.6f /* EMA smoothing: 0=frozen 1=raw IMU */
00147 #define EKF_HDG_MIN_DELTA_M 0.05f /* Min @f$\\delta@f$pos to update heading (m) */
00148 #define EKF_MAX_STEP_M 0.4f /* Max predict displacement/step (m) */
00149 #define EKF_MAX_UPDATE_M 0.5f /* Max update displacement/step (m) */
00150
00151
00152 /* =====
00153 *      Phase enum
00154 *      ===== */
00155
00156 typedef enum {
00157     EKF_PHASE_ANCHOR_SURVEY = 0,
00158     EKF_PHASE_TAG_LOCALIZE = 1,
00159 } ekf_phase_t;

```

```

00160
00161
00162 /* =====
00163  * Joint filter state (single static instance inside ekf.c)
00164  * ===== */
00165
00166 typedef struct {
00167
00168     /* -- Fields used by outside code -- do NOT remove or reorder ----- */
00169     float      x[EKF_MAX_STATE];
00170     float      P[EKF_MAX_STATE][EKF_MAX_STATE];
00171     float      Q[EKF_MAX_STATE][EKF_MAX_STATE];
00172     uint8_t    n_peers;
00173     uint16_t   peer_ids[EKF_MAX_PEERS];
00174     bool       initialised;
00175
00176     /* -- Appended fields -- safe to add ----- */
00177     bool       peer_seeded[EKF_MAX_PEERS];
00178     bool       peer_away[EKF_MAX_PEERS];
00179     bool       peer_imu_valid[EKF_MAX_PEERS];
00180     uint32_t   last_tick_ms;
00181     uint16_t   stationary_count[EKF_MAX_PEERS];
00182     int8_t     self_slot;
00183 } coop_ekf_t;
00184
00185
00186
00187 /* =====
00188  * Public API
00189  * ===== */
00190
00202 void ekf_init(const ekf_node_hint_t *pos_hints, uint8_t pos_hints_n);
00203
00214 void ekf_step(float az_self_ms, float ah_self_ms);
00215
00221 bool ekf_anchor_ready(void);
00222
00227 ekf_phase_t ekf_get_phase(void);
00228
00235 const float (*ekf_get_anchor_positions(void))[3];
00236
00248 bool ekf_get_peer_pos(uint16_t peer_id, float pos_out[3]);
00249
00255 float ekf_certainty_to_sigma(uint8_t certainty);
00256
00262 const coop_ekf_t *ekf_get_state(void);
00263
00264
00265 #endif /* UWB_IMU_EKF_H */

```

8.25 App/DWM/DWM3000.c File Reference

```

#include "cmsis_os.h"
#include "queue.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include "cmsis_os2.h"
#include "dw3000_hw.h"
#include "dw3000_spi.h"
#include "dw3000_deca_regs.h"
#include "deca_device_api.h"
#include "main.h"
#include "../Generic/my_print.h"
#include "stm32l4xx_hal.h"
#include "DWM3000_setup.h"
#include "DWM3000.h"
#include "DWM3000_driver.h"
#include "../UWB_app/ufb_exchange.h"
#include "../UWB_app/ufb_network.h"
#include "../Calculations/distance.h"

```

```
#include "../Calculations/ekf.h"
#include "uart.h"
#include "power.h"
#include "sleep.h"
```

Macros

- `#define U64_HI(x)`
- `#define U64_LO(x)`
- `#define MPRINT_TS(label, val)`

Functions

- `uint32_t tx_err_watchdog (uwb_sync_result_t result_sync, uwb_etwr_result_t result_etwr, uint32_t sleep_time)`
Watchdog to catch hardware TX lockups and invalid sleep times.
- `void low_battery_check (void)`
Checks battery voltage and shuts down if critically low.
- `void StartRangingTask (void *argument)`
Main FreeRTOS ranging task for the DWM3000 UWB module.

8.25.1 Macro Definition Documentation

8.25.1.1 MPRINT_TS

```
#define MPRINT_TS(
    label,
    val)
```

Value:

```
printf("UWB_eTWR_TEST - %-20s: 0x%08lX%08lX\r\n", (label), U64_HI(val), U64_LO(val))
```

8.25.1.2 U64_HI

```
#define U64_HI(
    x)
```

Value:

```
((uint32_t)((x) >> 32))
```

8.25.1.3 U64_LO

```
#define U64_LO(
    x)
```

Value:

```
((uint32_t)((x) & 0xFFFFFFFFU))
```

8.25.2 Function Documentation

8.25.2.1 low_battery_check()

```
void low_battery_check (
    void )
```

Checks battery voltage and shuts down if critically low.

Reads the current power status and, if the battery voltage is below 3.50 V and USB is not connected, puts the DWM3000 to sleep, flashes the red LED three times, and enters MCU shutdown mode.

8.25.2.2 StartRangingTask()

```
void StartRangingTask (
    void * argument)
```

Main FreeRTOS ranging task for the DWM3000 UWB module.

Performs full system startup: battery check, DWM3000 initialization, self-test, configuration, and network setup. Then waits for a boot configuration over UART before entering the main ranging loop.

In the ranging loop, the task executes UWB sync, extended TWR, distance calculation, and EKF position update on every iteration. Between iterations, the device enters low-power sleep (STOP1 mode when USB is disconnected, or `osDelay` when USB is active). Power status is checked every 50 iterations.

If `boot_cfg.charge_only` is set, the DWM3000 is kept in sleep and the task only services UART reset requests.

Parameters

<i>argument</i>	Unused FreeRTOS task argument.
-----------------	--------------------------------

8.25.2.3 tx_err_watchdog()

```
uint32_t tx_err_watchdog (
    uwb_sync_result_t result_sync,
    uwb_etwr_result_t result_etwr,
    uint32_t sleep_time)
```

Watchdog to catch hardware TX lockups and invalid sleep times.

Tracks consecutive TX failures across sync and eTWR operations. After 10 consecutive failures, the DWM3000 is fully re-initialized. Also clamps any out-of-range sleep time to [DEEP_SLEEP](#).

Parameters

<i>result_sync</i>	Return value from uwb_sync() .
<i>result_etwr</i>	Return value from uwb_extended_twr() .
<i>sleep_time</i>	Requested sleep duration in milliseconds from uwb_share() .

Returns

Safely clamped sleep time for use with `osDelay()`.

8.26 App/DWM/DWM3000.h File Reference

Functions

- void [StartDWM](#) (void *argument)

8.26.1 Function Documentation

8.26.1.1 StartDWM()

```
void StartDWM (  
    void * argument)
```

8.27 DWM3000.h

[Go to the documentation of this file.](#)

```
00001 #pragma once  
00002  
00003 #ifdef __cplusplus  
00004 extern "C" {  
00005 #endif  
00006  
00007  
00008  
00009 /* FreeRTOS task entry point -- called by osThreadNew() in main.c */  
00010 void StartDWM(void *argument);  
00011  
00012 #ifdef __cplusplus  
00013 }  
00014 #endif
```

8.28 App/DWM/DWM3000_driver.c File Reference

```
#include <string.h>  
#include <stdint.h>  
#include <stdbool.h>  
#include "cmsis_os.h"  
#include "cmsis_os2.h"  
#include "queue.h"  
#include "stm32l4xx_hal.h"  
#include "dw3000_hw.h"  
#include "dw3000_spi.h"  
#include "dw3000_deca_regs.h"  
#include "deca_device_api.h"  
#include "main.h"  
#include "../Generic/my_print.h"  
#include "DWM3000_setup.h"  
#include "DWM3000_driver.h"
```

Functions

- `uint64_t ts_buf_to_u64 (const uint8_t buf[5])`
Convert a 5-byte little-endian DW3000 timestamp buffer to a `uint64_t`. DW3000 timestamps are 40-bit values stored LSB-first across 5 bytes.
- `bool dwm_init (void)`
Initialise the DW3000 hardware and driver. Performs GPIO/SPI init, hard reset, driver probe, IDLE_RC poll, dwt_↔ initialise, and SPI speed-up. Leaves chip in IDLE_PLL with EXTI interrupt enabled.
- `bool dwm_selftest (void)`
Verify basic DW3000 functionality after initialisation. Checks device ID, OTP crystal trim, and TX path via event counter.
- `uint16_t dwm_get_ant_delay (uint16_t addr)`
Returns the antenna delay calibration value for a given device address.
- `void dwm_restore_runtime (uint16_t addr)`
Restore runtime-only settings that are not saved to AON and are lost on sleep/reset. Must be called both at startup (via `dwm_configure`) and after every wakeup (via `dwm_wakeup`). Settings restored: RX/TX antenna delays, PAN ID, short address, interrupt mask.
- `bool dwm_init_queues (void)`
Create the three FreeRTOS queues used for TX done, RX frames, and wakeup events, and register the DW3000 ISR callbacks. Called once during `dwm_configure()`. Queues are global and used by all `dwm_tx/rx` functions.
- `bool dwm_configure (void)`
Configure the DW3000 UWB radio and TX RF parameters, derive and assign the short address, initialise queues, and apply runtime settings. The short address is derived from the MCU unique ID (XOR of UID word 0 and word 1).
- `uint16_t dwm_get_addr (void)`
Return the IEEE 802.15.4 short address assigned to this device. Valid only after `dwm_configure()` has been called successfully.
- `void cb_rx_ok (const dwt_cb_data_t *cb_data)`
DW3000 RX complete callback — fired from ISR context on successful frame reception. Reads frame data, RX timestamp, and diagnostic accumulator, then posts to `rx_queue`.
- `void cb_rx_err (const dwt_cb_data_t *cb_data)`
DW3000 RX error callback — fired from ISR context when a frame is received with errors (CRC mismatch, PHR error, frame sync loss, SFD timeout, etc.). Posts a DWM_RX_ERR frame with the raw status bitmask to `rx_queue`.
- `void cb_rx_to (const dwt_cb_data_t *cb_data)`
DW3000 RX timeout callback — fired from ISR context when the RX timeout expires without a frame being received. Posts a DWM_RX_TIMEOUT frame to `rx_queue`.
- `void cb_tx_done (const dwt_cb_data_t *cb_data)`
DW3000 TX complete callback — fired from ISR context when a frame has been fully transmitted. Reads the TX timestamp (RMARKER, adjusted by programmed antenna delay) and posts it to `tx_queue`.
- `void cb_spi_rdy (const dwt_cb_data_t *cb_data)`
DW3000 SPI ready callback — fired from ISR context when the chip asserts SPIRDY after wakeup. Posts a single byte to `wakeup_queue` to unblock `dwm_wakeup()`.
- `void dwm_rx_flush (void)`
Arm the DW3000 receiver and block until a frame is received, a timeout occurs, or an error is reported. On DWM_↔ _RX_OK, result is fully populated including timestamp, RSSI, and first-path power. On DWM_RX_ERR or DWM_↔ RX_TIMEOUT, only result->type and result->status are set.
- `void dwm_rx (dwm_rx_frame_t *result, uint32_t timeout_ms, bool keep_listening, bool first_call, int16_↔ t *clock_offset_out)`
- `dwm_tx_event_type_t dwm_tx (dwm_tx_frame_t *frame)`
Transmit a frame immediately and block until the TX done callback posts the timestamp. Forces TRX off before transmitting to ensure a clean state. On success, `frame->tx_timestamp` is updated with the RMARKER timestamp of the transmitted frame.
- `dwm_tx_event_type_t dwm_tx_delayed (dwm_tx_frame_t *frame, uint64_t final_rmarker_ts, uint32_t wait_↔ _ms)`
Transmit a frame at a programmed delayed RMARKER time and block until the TX done callback posts the actual TX timestamp. Forces TRX off before transmitting to ensure a clean state. On success, `frame->tx_timestamp` is updated with the RMARKER timestamp of the transmitted frame.

- void `dwm_sleep` (void)
Put the DW3000 into DEEPSLEEP, saving UWB config to AON. The chip will wake on assertion of the WAKEUP pin (via `dwm_wakeup`). PGF calibration will run automatically on the next wakeup.
- void `dwm_wakeup` (void)
Wake the DW3000 from DEEPSLEEP by asserting the WAKEUP pin, then restore all runtime settings. Blocks until SPIRDY is received (max 10ms), then calls `dwt_restoreconfig(1)` to reload UWB config and run PGF calibration. Restores antenna delays, PAN ID, address, and interrupts.
- int `dwm_enter_idle_rc` (void)
- int `dwm_exit_idle_rc` (void)

Variables

- const struct `dwt_probe_s` `dw3000_probe_intf`
- SPI_HandleTypeDef `hspl1`
- QueueHandle_t `rx_queue` = NULL
- QueueHandle_t `tx_queue` = NULL
- QueueHandle_t `wakeup_queue` = NULL

8.28.1 Function Documentation

8.28.1.1 `cb_rx_err()`

```
void cb_rx_err (
    const dwt_cb_data_t * cb_data)
```

DW3000 RX error callback — fired from ISR context when a frame is received with errors (CRC mismatch, PHR error, frame sync loss, SFD timeout, etc.). Posts a DWM_RX_ERR frame with the raw status bitmask to `rx_queue`.

Parameters

<code>cb_data</code>	Pointer to callback data provided by the DW3000 driver (status bitmask).
----------------------	--

Returns

None.

8.28.1.2 `cb_rx_ok()`

```
void cb_rx_ok (
    const dwt_cb_data_t * cb_data)
```

DW3000 RX complete callback — fired from ISR context on successful frame reception. Reads frame data, RX timestamp, and diagnostic accumulator, then posts to `rx_queue`.

Parameters

<code>cb_data</code>	Pointer to callback data provided by the DW3000 driver (length, status).
----------------------	--

Returns

None.

8.28.1.3 `cb_rx_to()`

```
void cb_rx_to (
    const dwt_cb_data_t * cb_data)
```

DW3000 RX timeout callback — fired from ISR context when the RX timeout expires without a frame being received. Posts a DWM_RX_TIMEOUT frame to rx_queue.

Parameters

<code>cb_data</code>	Unused — timeout carries no additional data.
----------------------	--

Returns

None.

8.28.1.4 `cb_spi_rdy()`

```
void cb_spi_rdy (
    const dwt_cb_data_t * cb_data)
```

DW3000 SPI ready callback — fired from ISR context when the chip asserts SPIRDY after wakeup. Posts a single byte to wakeup_queue to unblock [dwm_wakeup\(\)](#).

Parameters

<code>cb_data</code>	Unused.
----------------------	---------

Returns

None.

8.28.1.5 `cb_tx_done()`

```
void cb_tx_done (
    const dwt_cb_data_t * cb_data)
```

DW3000 TX complete callback — fired from ISR context when a frame has been fully transmitted. Reads the TX timestamp (RMARKER, adjusted by programmed antenna delay) and posts it to tx_queue.

Parameters

<code>cb_data</code>	Unused — TX done carries no additional data beyond the implicit timestamp.
----------------------	--

Returns

None.

8.28.1.6 `dwm_configure()`

```
bool dwm_configure (
    void )
```

Configure the DW3000 UWB radio and TX RF parameters, derive and assign the short address, initialise queues, and apply runtime settings. The short address is derived from the MCU unique ID (XOR of UID word 0 and word 1).

Returns

true on success, false if dwm_configure or queue init fails.

8.28.1.7 dwm_enter_idle_rc()

```
int dwm_enter_idle_rc (
    void )
```

Enter IDLE_RC — low-power RC-clocked idle. SPI must be slowed BEFORE the PLL goes off.

8.28.1.8 dwm_exit_idle_rc()

```
int dwm_exit_idle_rc (
    void )
```

Return to IDLE_PLL — full-speed PLL-clocked idle. `dwt_setdwstate()` handles PLL re-lock internally.

8.28.1.9 dwm_get_addr()

```
uint16_t dwm_get_addr (
    void )
```

Return the IEEE 802.15.4 short address assigned to this device. Valid only after `dwm_configure()` has been called successfully.

Returns

16-bit short address, or 0 if `dwm_configure` has not been called.

8.28.1.10 dwm_get_ant_delay()

```
uint16_t dwm_get_ant_delay (
    uint16_t addr)
```

Returns the antenna delay calibration value for a given device address.

Each device has a unique per-unit offset added to the base `DWM_ANT_DELAY` value to compensate for hardware variation. Falls back to a default offset of 58 for unknown addresses.

Parameters

<i>addr</i>	16-bit short address of the UWB device.
-------------	---

Returns

Calibrated antenna delay value.

8.28.1.11 dwm_init()

```
bool dwm_init (
    void )
```

Initialise the DW3000 hardware and driver. Performs GPIO/SPI init, hard reset, driver probe, IDLE_RC poll, `dwt_init`, and SPI speed-up. Leaves chip in IDLE_PLL with EXTI interrupt enabled. Must be called before any other `dwm_*` function.

Returns

true on success, false if any step fails (probe, IDLE_RC timeout, initialise).

8.28.1.12 dwm_init_queues()

```
bool dwm_init_queues (
    void )
```

Create the three FreeRTOS queues used for TX done, RX frames, and wakeup events, and register the DW3000 ISR callbacks. Called once during `dwm_configure()`. Queues are global and used by all `dwm_tx/rx` functions.

Returns

true on success, false if any queue allocation fails.

8.28.1.13 dwm_restore_runtime()

```
void dwm_restore_runtime (
    uint16_t addr)
```

Restore runtime-only settings that are not saved to AON and are lost on sleep/reset. Must be called both at startup (via `dwm_configure`) and after every wakeup (via `dwm_wakeup`). Settings restored: RX/TX antenna delays, PAN ID, short address, interrupt mask.

Parameters

<i>addr</i>	IEEE 802.15.4 16-bit short address to assign to the device.
-------------	---

Returns

None.

8.28.1.14 dwm_rx()

```
void dwm_rx (
    dwm_rx_frame_t * result,
    uint32_t timeout_ms,
    bool keep_listening,
    bool first_call,
    int16_t * clock_offset_out)
```

8.28.1.15 dwm_rx_flush()

```
void dwm_rx_flush (
    void )
```

Arm the DW3000 receiver and block until a frame is received, a timeout occurs, or an error is reported. On `DWM_RX_OK`, `result` is fully populated including timestamp, RSSI, and first-path power. On `DWM_RX_ERR` or `DWM_RX_TIMEOUT`, only `result->type` and `result->status` are set.

Parameters

<i>result</i>	Pointer to caller-allocated <code>dwm_rx_frame_t</code> to be populated.
<i>timeout_ms</i>	RX timeout in milliseconds. The DW3000 will always fire a timeout event, so this function always returns — it never blocks indefinitely.

Returns

None. Check `result->type` for `DWM_RX_OK` / `DWM_RX_ERR` / `DWM_RX_TIMEOUT`.

8.28.1.16 dwm_selftest()

```
bool dwm_selftest (
    void )
```

Verify basic DW3000 functionality after initialisation. Checks device ID, OTP crystal trim, and TX path via event counter. Call after `dwm_init()` and `dwm_configure()` have both succeeded.

Returns

true if all checks pass, false on any failure.

8.28.1.17 dwm_sleep()

```
void dwm_sleep (
    void )
```

Put the DW3000 into DEEPSLEEP, saving UWB config to AON. The chip will wake on assertion of the WAKEUP pin (via `dwm_wakeup`). PGF calibration will run automatically on the next wakeup.

NOTE: Must only be called when the chip is in IDLE_PLL state, i.e. after `dwm_configure()` or after a prior `dwm_wakeup()` has completed.

Returns

None.

8.28.1.18 dwm_tx()

```
dwm_tx_event_type_t dwm_tx (
    dwm_tx_frame_t * frame)
```

Transmit a frame immediately and block until the TX done callback posts the timestamp. Forces TRX off before transmitting to ensure a clean state. On success, `frame->tx_timestamp` is updated with the RMARKER timestamp of the transmitted frame.

Parameters

<i>frame</i>	Pointer to a <code>dwm_tx_frame_t</code> containing: • data: pointer to payload buffer (must remain valid until function returns) • len: payload length in bytes (excluding the 2-byte CRC added by hardware) • tx_timestamp: populated on return with the 40-bit TX RMARKER timestamp
--------------	--

Returns

DWM_TX_OK if the frame was transmitted and timestamp received successfully. DWM_TX_LATE if `dwt_starttx()` returned DWT_ERROR (delayed TX deadline missed). DWM_TX_ERROR if the TX done callback did not fire within 10ms.

8.28.1.19 dwm_tx_delayed()

```
dwm_tx_event_type_t dwm_tx_delayed (
    dwm_tx_frame_t * frame,
    uint64_t final_rmarker_ts,
    uint32_t wait_ms)
```

Transmit a frame at a programmed delayed RMARKER time and block until the TX done callback posts the actual TX timestamp. Forces TRX off before transmitting to ensure a clean state. On success, `frame->tx_timestamp` is updated with the RMARKER timestamp of the transmitted frame.

Parameters

<i>frame</i>	Pointer to a <code>dwm_tx_frame_t</code> containing: • data: pointer to payload buffer (must remain valid until function returns) • len: payload length in bytes (excluding the 2-byte CRC added by hardware) • tx_timestamp: populated on return with the 40-bit TX RMARKER timestamp
--------------	--

<i>final_rmarker</i> ↔ <i>_ts</i>	Desired final 40-bit TX RMARKER timestamp, including antenna delay.
--------------------------------------	---

Returns

DWM_TX_OK if the frame was transmitted and timestamp received successfully. DWM_TX_LATE if *dwt_↔starttx()* returned DWT_ERROR (delayed TX deadline missed). DWM_TX_ERROR if the TX done callback did not fire within 10ms.

8.28.1.20 *dwm_wakeup()*

```
void dwm_wakeup (
    void )
```

Wake the DW3000 from DEEPSLEEP by asserting the WAKEUP pin, then restore all runtime settings. Blocks until SPIRDY is received (max 10ms), then calls *dwt_restoreconfig(1)* to reload UWB config and run PGF calibration. Restores antenna delays, PAN ID, address, and interrupts.
After this function returns the chip is in IDLE_PLL and ready for TX/RX.

Returns

None. Prints a warning via *mprintf* if SPIRDY is not received within 10ms.

8.28.1.21 *ts_buf_to_u64()*

```
uint64_t ts_buf_to_u64 (
    const uint8_t buf[5])
```

Convert a 5-byte little-endian DW3000 timestamp buffer to a *uint64_t*. DW3000 timestamps are 40-bit values stored LSB-first across 5 bytes.

Parameters

<i>buf</i>	Pointer to a 5-byte buffer containing the raw timestamp.
------------	--

Returns

40-bit timestamp as *uint64_t* (upper 24 bits are always zero).

8.28.2 Variable Documentation

8.28.2.1 *dw3000_probe_intf*

```
const struct dwt_probe_s dw3000_probe_intf [extern]
```

8.28.2.2 *hspi1*

```
SPI_HandleTypeDef hspi1 [extern]
```

8.28.2.3 *rx_queue*

```
QueueHandle_t rx_queue = NULL
```

8.28.2.4 *tx_queue*

```
QueueHandle_t tx_queue = NULL
```

8.28.2.5 *wakeup_queue*

```
QueueHandle_t wakeup_queue = NULL
```

8.29 App/DWM/DWM3000_driver.h File Reference

```
#include <stdint.h>
#include "DWM3000_setup.h"
```

Data Structures

- struct [dwm_rx_raw_frame_t](#)
- struct [dwm_rx_frame_t](#)
- struct [dwm_tx_frame_t](#)

Macros

- #define [US_TO_DWT_TIME](#) 65536U
- #define [MS_TO_DWT_TIME](#) ([US_TO_DWT_TIME](#) * 1000ULL)
- #define [TX_DELAY_500US](#) (5000U * [US_TO_DWT_TIME](#))

Enumerations

- enum [dwm_rx_event_type_t](#) { [DWM_RX_OK](#), [DWM_RX_ERR](#), [DWM_RX_TIMEOUT](#) }
- enum [dwm_tx_event_type_t](#) { [DWM_TX_OK](#), [DWM_TX_LATE](#), [DWM_TX_ERROR](#) }

Functions

- uint64_t [ts_buf_to_u64](#) (const uint8_t buf[5])
Convert a 5-byte little-endian DW3000 timestamp buffer to a uint64_t. DW3000 timestamps are 40-bit values stored LSB-first across 5 bytes.
- bool [dwm_init](#) (void)
Initialise the DW3000 hardware and driver. Performs GPIO/SPI init, hard reset, driver probe, IDLE_RC poll, dwt_↔ initialise, and SPI speed-up. Leaves chip in IDLE_PLL with EXTI interrupt enabled.
- bool [dwm_selftest](#) (void)
Verify basic DW3000 functionality after initialisation. Checks device ID, OTP crystal trim, and TX path via event counter.
- bool [dwm_configure](#) (void)
Configure the DW3000 UWB radio and TX RF parameters, derive and assign the short address, initialise queues, and apply runtime settings. The short address is derived from the MCU unique ID (XOR of UID word 0 and word 1).
- uint16_t [dwm_get_addr](#) (void)
Return the IEEE 802.15.4 short address assigned to this device. Valid only after [dwm_configure\(\)](#) has been called successfully.
- void [cb_rx_ok](#) (const dwt_cb_data_t *cb_data)
DW3000 RX complete callback — fired from ISR context on successful frame reception. Reads frame data, RX timestamp, and diagnostic accumulator, then posts to rx_queue.
- void [cb_rx_err](#) (const dwt_cb_data_t *cb_data)
DW3000 RX error callback — fired from ISR context when a frame is received with errors (CRC mismatch, PHR error, frame sync loss, SFD timeout, etc.). Posts a DWM_RX_ERR frame with the raw status bitmask to rx_queue.
- void [cb_rx_to](#) (const dwt_cb_data_t *cb_data)
DW3000 RX timeout callback — fired from ISR context when the RX timeout expires without a frame being received. Posts a DWM_RX_TIMEOUT frame to rx_queue.
- void [cb_tx_done](#) (const dwt_cb_data_t *cb_data)
DW3000 TX complete callback — fired from ISR context when a frame has been fully transmitted. Reads the TX timestamp (RMARKER, adjusted by programmed antenna delay) and posts it to tx_queue.
- void [cb_spi_rdy](#) (const dwt_cb_data_t *cb_data)
DW3000 SPI ready callback — fired from ISR context when the chip asserts SPIRDY after wakeup. Posts a single byte to wakeup_queue to unblock [dwm_wakeup\(\)](#).
- void [dwm_rx](#) ([dwm_rx_frame_t](#) *result, uint32_t timeout_ms, bool keep_listening, bool first_call, int16_t ↵ *clock_offset_out)

- void `dwm_rx_flush` (void)
Arm the DW3000 receiver and block until a frame is received, a timeout occurs, or an error is reported. On DWM_RX_OK, result is fully populated including timestamp, RSSI, and first-path power. On DWM_RX_ERR or DWM_RX_TIMEOUT, only result->type and result->status are set.
- `dwm_tx_event_type_t dwm_tx` (`dwm_tx_frame_t` *frame)
Transmit a frame immediately and block until the TX done callback posts the timestamp. Forces TRX off before transmitting to ensure a clean state. On success, frame->tx_timestamp is updated with the RMARKER timestamp of the transmitted frame.
- `dwm_tx_event_type_t dwm_tx_delayed` (`dwm_tx_frame_t` *frame, uint64_t final_rmarker_ts, uint32_t wait_ms)
Transmit a frame at a programmed delayed RMARKER time and block until the TX done callback posts the actual TX timestamp. Forces TRX off before transmitting to ensure a clean state. On success, frame->tx_timestamp is updated with the RMARKER timestamp of the transmitted frame.
- void `dwm_tx_continuous` (void)
- void `dwm_tx_continuous_delayed` (void)
- void `dwm_rx_continuous_sleep` (void)
- void `dwm_wakeup` (void)
Wake the DW3000 from DEEPSLEEP by asserting the WAKEUP pin, then restore all runtime settings. Blocks until SPIRDY is received (max 10ms), then calls dwm_restoreconfig(1) to reload UWB config and run PGF calibration. Restores antenna delays, PAN ID, address, and interrupts.
- void `dwm_sleep` (void)
Put the DW3000 into DEEPSLEEP, saving UWB config to AON. The chip will wake on assertion of the WAKEUP pin (via dwm_wakeup). PGF calibration will run automatically on the next wakeup.
- int `dwm_exit_idle_rc` (void)
- int `dwm_enter_idle_rc` (void)

8.29.1 Macro Definition Documentation

8.29.1.1 MS_TO_DWT_TIME

```
#define MS_TO_DWT_TIME (US_TO_DWT_TIME * 1000ULL)
```

8.29.1.2 TX_DELAY_500US

```
#define TX_DELAY_500US (5000U * US_TO_DWT_TIME)
```

8.29.1.3 US_TO_DWT_TIME

```
#define US_TO_DWT_TIME 65536U
```

8.29.2 Enumeration Type Documentation

8.29.2.1 dwm_rx_event_type_t

```
enum dwm_rx_event_type_t
```

Enumerator

DWM_RX_OK	
DWM_RX_ERR	
DWM_RX_TIMEOUT	

8.29.2.2 dwm_tx_event_type_t

enum `dwm_tx_event_type_t`

Enumerator

DWM_TX_OK	
DWM_TX_LATE	
DWM_TX_ERROR	

8.29.3 Function Documentation

8.29.3.1 cb_rx_err()

```
void cb_rx_err (
    const dwt_cb_data_t * cb_data)
```

DW3000 RX error callback — fired from ISR context when a frame is received with errors (CRC mismatch, PHR error, frame sync loss, SFD timeout, etc.). Posts a DWM_RX_ERR frame with the raw status bitmask to rx_queue.

Parameters

<code>cb_data</code>	Pointer to callback data provided by the DW3000 driver (status bitmask).
----------------------	--

Returns

None.

8.29.3.2 cb_rx_ok()

```
void cb_rx_ok (
    const dwt_cb_data_t * cb_data)
```

DW3000 RX complete callback — fired from ISR context on successful frame reception. Reads frame data, RX timestamp, and diagnostic accumulator, then posts to rx_queue.

Parameters

<code>cb_data</code>	Pointer to callback data provided by the DW3000 driver (length, status).
----------------------	--

Returns

None.

8.29.3.3 cb_rx_to()

```
void cb_rx_to (
    const dwt_cb_data_t * cb_data)
```

DW3000 RX timeout callback — fired from ISR context when the RX timeout expires without a frame being received. Posts a DWM_RX_TIMEOUT frame to rx_queue.

Parameters

<code>cb_data</code>	Unused — timeout carries no additional data.
----------------------	--

Returns

None.

8.29.3.4 cb_spi_rdy()

```
void cb_spi_rdy (
    const dwt_cb_data_t * cb_data)
```

DW3000 SPI ready callback — fired from ISR context when the chip asserts SPIRDY after wakeup. Posts a single byte to wakeup_queue to unblock [dwm_wakeup\(\)](#).

Parameters

<i>cb_data</i>	Unused.
----------------	---------

Returns

None.

8.29.3.5 cb_tx_done()

```
void cb_tx_done (
    const dwt_cb_data_t * cb_data)
```

DW3000 TX complete callback — fired from ISR context when a frame has been fully transmitted. Reads the TX timestamp (RMARKER, adjusted by programmed antenna delay) and posts it to tx_queue.

Parameters

<i>cb_data</i>	Unused — TX done carries no additional data beyond the implicit timestamp.
----------------	--

Returns

None.

8.29.3.6 dwm_configure()

```
bool dwm_configure (
    void )
```

Configure the DW3000 UWB radio and TX RF parameters, derive and assign the short address, initialise queues, and apply runtime settings. The short address is derived from the MCU unique ID (XOR of UID word 0 and word 1).

Returns

true on success, false if dwt_configure or queue init fails.

8.29.3.7 dwm_enter_idle_rc()

```
int dwm_enter_idle_rc (
    void )
```

Enter IDLE_RC — low-power RC-clocked idle. SPI must be slowed BEFORE the PLL goes off.

8.29.3.8 dwm_exit_idle_rc()

```
int dwm_exit_idle_rc (
    void )
```

Return to IDLE_PLL — full-speed PLL-clocked idle. dwt_setdwstate() handles PLL re-lock internally.

8.29.3.9 dwm_get_addr()

```
uint16_t dwm_get_addr (
    void )
```

Return the IEEE 802.15.4 short address assigned to this device. Valid only after [dwm_configure\(\)](#) has been called successfully.

Returns

16-bit short address, or 0 if [dwm_configure](#) has not been called.

8.29.3.10 dwm_init()

```
bool dwm_init (
    void )
```

Initialise the DW3000 hardware and driver. Performs GPIO/SPI init, hard reset, driver probe, IDLE_RC poll, [dwt_init](#), and SPI speed-up. Leaves chip in IDLE_PLL with EXTI interrupt enabled. Must be called before any other [dwm_*](#) function.

Returns

true on success, false if any step fails (probe, IDLE_RC timeout, initialise).

8.29.3.11 dwm_rx()

```
void dwm_rx (
    dwm_rx_frame_t * result,
    uint32_t timeout_ms,
    bool keep_listening,
    bool first_call,
    int16_t * clock_offset_out)
```

8.29.3.12 dwm_rx_continuous_sleep()

```
void dwm_rx_continuous_sleep (
    void )
```

8.29.3.13 dwm_rx_flush()

```
void dwm_rx_flush (
    void )
```

Arm the DW3000 receiver and block until a frame is received, a timeout occurs, or an error is reported. On [DWM_RX_OK](#), result is fully populated including timestamp, RSSI, and first-path power. On [DWM_RX_ERR](#) or [DWM_RX_TIMEOUT](#), only result->type and result->status are set.

Parameters

<i>result</i>	Pointer to caller-allocated dwm_rx_frame_t to be populated.
<i>timeout_ms</i>	RX timeout in milliseconds. The DW3000 will always fire a timeout event, so this function always returns — it never blocks indefinitely.

Returns

None. Check result->type for [DWM_RX_OK](#) / [DWM_RX_ERR](#) / [DWM_RX_TIMEOUT](#).

8.29.3.14 dwm_selftest()

```
bool dwm_selftest (
    void )
```

Verify basic DW3000 functionality after initialisation. Checks device ID, OTP crystal trim, and TX path via event counter.

Call after [dwm_init\(\)](#) and [dwm_configure\(\)](#) have both succeeded.

Returns

true if all checks pass, false on any failure.

8.29.3.15 dwm_sleep()

```
void dwm_sleep (
    void )
```

Put the DW3000 into DEEPSLEEP, saving UWB config to AON. The chip will wake on assertion of the WAKEUP pin (via [dwm_wakeup](#)). PGF calibration will run automatically on the next wakeup.

NOTE: Must only be called when the chip is in IDLE_PLL state, i.e. after [dwm_configure\(\)](#) or after a prior [dwm_wakeup\(\)](#) has completed.

Returns

None.

8.29.3.16 dwm_tx()

```
dwm_tx_event_type_t dwm_tx (
    dwm_tx_frame_t * frame)
```

Transmit a frame immediately and block until the TX done callback posts the timestamp. Forces TRX off before transmitting to ensure a clean state. On success, frame->tx_timestamp is updated with the RMARKER timestamp of the transmitted frame.

Parameters

<i>frame</i>	Pointer to a dwm_tx_frame_t containing: • data: pointer to payload buffer (must remain valid until function returns) • len: payload length in bytes (excluding the 2-byte CRC added by hardware) • tx_timestamp: populated on return with the 40-bit TX RMARKER timestamp
--------------	--

Returns

DWM_TX_OK if the frame was transmitted and timestamp received successfully. DWM_TX_LATE if [dwt_starttx\(\)](#) returned DWT_ERROR (delayed TX deadline missed). DWM_TX_ERROR if the TX done callback did not fire within 10ms.

8.29.3.17 dwm_tx_continuous()

```
void dwm_tx_continuous (
    void )
```

8.29.3.18 dwm_tx_continuous_delayed()

```
void dwm_tx_continuous_delayed (
    void )
```

8.29.3.19 dwm_tx_delayed()

```
dwm_tx_event_type_t dwm_tx_delayed (
    dwm_tx_frame_t * frame,
    uint64_t final_rmarker_ts,
    uint32_t wait_ms)
```

Transmit a frame at a programmed delayed RMARKER time and block until the TX done callback posts the actual TX timestamp. Forces TRX off before transmitting to ensure a clean state. On success, frame->tx_timestamp is updated with the RMARKER timestamp of the transmitted frame.

Parameters

<i>frame</i>	Pointer to a dwm_tx_frame_t containing: • data: pointer to payload buffer (must remain valid until function returns) • len: payload length in bytes (excluding the 2-byte CRC added by hardware) • tx_timestamp: populated on return with the 40-bit TX RMARKER timestamp
<i>final_rmarker_ts</i>	Desired final 40-bit TX RMARKER timestamp, including antenna delay.

Returns

DWM_TX_OK if the frame was transmitted and timestamp received successfully. DWM_TX_LATE if `dwt_starttx()` returned DWT_ERROR (delayed TX deadline missed). DWM_TX_ERROR if the TX done callback did not fire within 10ms.

8.29.3.20 dwm_wakeup()

```
void dwm_wakeup (
    void )
```

Wake the DW3000 from DEEPSLEEP by asserting the WAKEUP pin, then restore all runtime settings. Blocks until SPIRDY is received (max 10ms), then calls `dwt_restoreconfig(1)` to reload UWB config and run PGF calibration. Restores antenna delays, PAN ID, address, and interrupts. After this function returns the chip is in IDLE_PLL and ready for TX/RX.

Returns

None. Prints a warning via `mprintf` if SPIRDY is not received within 10ms.

8.29.3.21 ts_buf_to_u64()

```
uint64_t ts_buf_to_u64 (
    const uint8_t buf[5])
```

Convert a 5-byte little-endian DW3000 timestamp buffer to a `uint64_t`. DW3000 timestamps are 40-bit values stored LSB-first across 5 bytes.

Parameters

<i>buf</i>	Pointer to a 5-byte buffer containing the raw timestamp.
------------	--

Returns

40-bit timestamp as `uint64_t` (upper 24 bits are always zero).

8.30 DWM3000_driver.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002
00003 #include <stdint.h>
00004 #ifndef __cplusplus
00005 extern "C" {
00006 #endif
00007
00008 #include "DWM3000_setup.h"
00009
00010 #define US_TO_DWT_TIME    65536U
00011 #define MS_TO_DWT_TIME    (US_TO_DWT_TIME * 1000ULL)
00012
00013 #define TX_DELAY_500US    (5000U * US_TO_DWT_TIME)
00014
00015 typedef enum {
00016     DWM_RX_OK,
00017     DWM_RX_ERR,
00018     DWM_RX_TIMEOUT,
00019 } dwm_rx_event_type_t;
00020
00021 typedef enum {
00022     DWM_TX_OK,
00023     DWM_TX_LATE,
00024     DWM_TX_ERROR,
00025 } dwm_tx_event_type_t;
00026
00027
00028 typedef struct {
00029     dwm_rx_event_type_t type;
00030     uint32_t status;
00031     uint8_t data[DWM_MAX_FRAME_LEN];
00032     uint16_t len;
00033     uint8_t ts_buf[5];
00034     dwt_cirdiags_t rx_diag;
00035 } dwm_rx_raw_frame_t;
00036
00037
00038 typedef struct {
00039     dwm_rx_event_type_t type;
00040     uint32_t status;
00041     uint8_t data[DWM_MAX_FRAME_LEN];
00042     uint16_t len;
00043     uint64_t rx_timestamp;
00044     int16_t rssi_q8;
00045     int16_t fp_q8;
00046 } dwm_rx_frame_t;
00047
00048
00049 typedef struct {
00050     uint8_t *data;
00051     uint16_t len;
00052     uint64_t tx_timestamp;
00053 } dwm_tx_frame_t;
00054
00055 uint64_t ts_buf_to_u64(const uint8_t buf[5]);
00056
00057 bool dwm_init(void);
00058 bool dwm_selftest(void);
00059 bool dwm_configure(void);
00060 uint16_t dwm_get_addr(void);
00061
00062
00063 void cb_rx_ok(const dwt_cb_data_t *cb_data);
00064 void cb_rx_err(const dwt_cb_data_t *cb_data);
00065 void cb_rx_to(const dwt_cb_data_t *cb_data);
00066 void cb_tx_done(const dwt_cb_data_t *cb_data);
00067 void cb_spi_rdy(const dwt_cb_data_t *cb_data);
00068
00069 void dwm_rx(dwm_rx_frame_t *result, uint32_t timeout_ms, bool keep_listening, bool first_call, int16_t
    *clock_offset_out);
00070 void dwm_rx_flush(void);
00071
00072 dwm_tx_event_type_t dwm_tx(dwm_tx_frame_t *frame);
00073 dwm_tx_event_type_t dwm_tx_delayed(dwm_tx_frame_t *frame, uint64_t final_rmarker_ts, uint32_t
    wait_ms);
00074 void dwm_tx_continuous(void);
00075 void dwm_tx_continuous_delayed(void);
00076 void dwm_rx_continuous_sleep(void);
00077
00078 void dwm_wakeup(void);
00079 void dwm_sleep(void);
00080 int dwm_exit_idle_rc(void);
00081 int dwm_enter_idle_rc(void);

```

```
00082 #ifdef __cplusplus
00083 }
00084 #endif
```

8.31 App/DWM/DWM3000_setup.h File Reference

```
#include "dw3000_hw.h"
#include "dw3000_spi.h"
#include "deca_device_api.h"
```

Macros

- #define [DWM_UWB_CHANNEL](#) 9
- #define [DWM_UWB_PREAMBLE_LEN](#) DWT_PLEN_128
- #define [DWM_UWB_PAC](#) DWT_PAC8
- #define [DWM_UWB_TX_CODE](#) 9
- #define [DWM_UWB_RX_CODE](#) 9
- #define [DWM_UWB_DATA_RATE](#) DWT_BR_6M8
- #define [DWM_TX_PGDLY](#) 0x34
- #define [DWM_UWB_SFD_TYPE](#) DWT_SFD_DW_8
- #define [DWM_UWB_PHR_MODE](#) DWT_PHRMODE_STD
- #define [DWM_UWB_PHR_RATE](#) DWT_PHRRATE_STD
- #define [DWM_UWB_SFD_TO](#) (128 + 1 + 8 - 8)
- #define [DWM_UWB_STS_MODE](#) DWT_STS_MODE_OFF
- #define [DWM_UWB_STS_LENGTH](#) DWT_STS_LEN_64
- #define [DWM_UWB_PDOA_MODE](#) DWT_PDOA_M0
- #define [TX_RF_PGdly](#) 0x34
- #define [TX_RF_Power](#) 0xfefefefe
- #define [TX_RF_PGcount](#) 0
- #define [DWM_IRQ_MASK](#)
- #define [DWT_DEVICE_ID](#) 0xDECA0302UL
- #define [DWM_PAN_ID](#) 0xDECA
- #define [DWM_CRC_LEN](#) 2U
- #define [DWM_RX_TIMEOUT_1S](#) 976562U
- #define [DWM_ANT_DELAY](#) (16385U - 90U)
- #define [DWM_MAX_FRAME_LEN](#) 127

8.31.1 Macro Definition Documentation

8.31.1.1 DWM_ANT_DELAY

```
#define DWM_ANT_DELAY (16385U - 90U)
```

8.31.1.2 DWM_CRC_LEN

```
#define DWM_CRC_LEN 2U
```

8.31.1.3 DWM_IRQ_MASK

```
#define DWM_IRQ_MASK
```

Value:

```
(DWT_INT_RXFCG_BIT_MASK | \
DWT_INT_RXFCE_BIT_MASK | \
DWT_INT_RXPHE_BIT_MASK | \
DWT_INT_RXFSL_BIT_MASK | \
DWT_INT_RXSTO_BIT_MASK | \
DWT_INT_RXFTO_BIT_MASK | \
DWT_INT_TXFRS_BIT_MASK | \
DWT_INT_SPIRDY_BIT_MASK)
```

8.31.1.4 DWM_MAX_FRAME_LEN

```
#define DWM_MAX_FRAME_LEN 127
```

8.31.1.5 DWM_PAN_ID

```
#define DWM_PAN_ID 0xDECA
```

8.31.1.6 DWM_RX_TIMEOUT_1S

```
#define DWM_RX_TIMEOUT_1S 976562U
```

8.31.1.7 DWM_TX_PGDLY

```
#define DWM_TX_PGDLY 0x34
```

8.31.1.8 DWM_UWB_CHANNEL

```
#define DWM_UWB_CHANNEL 9
```

8.31.1.9 DWM_UWB_DATA_RATE

```
#define DWM_UWB_DATA_RATE DWT_BR_6M8
```

8.31.1.10 DWM_UWB_PAC

```
#define DWM_UWB_PAC DWT_PAC8
```

8.31.1.11 DWM_UWB_PDOA_MODE

```
#define DWM_UWB_PDOA_MODE DWT_PDOA_M0
```

8.31.1.12 DWM_UWB_PHR_MODE

```
#define DWM_UWB_PHR_MODE DWT_PHRMODE_STD
```

8.31.1.13 DWM_UWB_PHR_RATE

```
#define DWM_UWB_PHR_RATE DWT_PHRRATE_STD
```

8.31.1.14 DWM_UWB_PREAMBLE_LEN

```
#define DWM_UWB_PREAMBLE_LEN DWT_PLEN_128
```

8.31.1.15 DWM_UWB_RX_CODE

```
#define DWM_UWB_RX_CODE 9
```

8.31.1.16 DWM_UWB_SFD_TO

```
#define DWM_UWB_SFD_TO (128 + 1 + 8 - 8)
```

8.31.1.17 DWM_UWB_SFD_TYPE

```
#define DWM_UWB_SFD_TYPE DWT_SFD_DW_8
```

8.31.1.18 DWM_UWB_STS_LENGTH

```
#define DWM_UWB_STS_LENGTH DWT_STS_LEN_64
```


8.31.1.19 DWM_UWB_STS_MODE

```
#define DWM_UWB_STS_MODE DWT_STS_MODE_OFF
```

8.31.1.20 DWM_UWB_TX_CODE

```
#define DWM_UWB_TX_CODE 9
```

8.31.1.21 DWT_DEVICE_ID

```
#define DWT_DEVICE_ID 0xDECA0302UL
```

8.31.1.22 TX_RF_PGcount

```
#define TX_RF_PGcount 0
```

8.31.1.23 TX_RF_PGdly

```
#define TX_RF_PGdly 0x34
```

8.31.1.24 TX_RF_Power

```
#define TX_RF_Power 0xfefefefe
```

8.32 DWM3000_setup.h

[Go to the documentation of this file.](#)

```
00001 #include "dw3000_hw.h"
00002 #include "dw3000_spi.h"
00003 #include "deca_device_api.h"
00004
00005 // DWM3000 configuration parameters
00006 #define DWM_UWB_CHANNEL 9
00007 #define DWM_UWB_PREAMBLE_LEN DWT_PLEN_128
00008 #define DWM_UWB_PAC DWT_PAC8
00009 #define DWM_UWB_TX_CODE 9
00010 #define DWM_UWB_RX_CODE 9
00011 #define DWM_UWB_DATA_RATE DWT_BR_6M8
00012 #define DWM_TX_PGDLY 0x34
00013 #define DWM_UWB_SFD_TYPE DWT_SFD_DW_8
00014 #define DWM_UWB_PHR_MODE DWT_PHRMODE_STD
00015 #define DWM_UWB_PHR_RATE DWT_PHRRATE_STD
00016 #define DWM_UWB_SFD_TO (128 + 1 + 8 - 8)
00017 #define DWM_UWB_STS_MODE DWT_STS_MODE_OFF
00018 #define DWM_UWB_STS_LENGTH DWT_STS_LEN_64
00019 #define DWM_UWB_PDOA_MODE DWT_PDOA_M0
00020
00021 #define TX_RF_PGdly 0x34
00022 #define TX_RF_Power 0xfefefefe
00023 #define TX_RF_PGcount 0
00024
00025 #define DWM_IRQ_MASK (DWT_INT_RXFCG_BIT_MASK | \
00026                     DWT_INT_RXFCE_BIT_MASK | \
00027                     DWT_INT_RXPHE_BIT_MASK | \
00028                     DWT_INT_RXFSL_BIT_MASK | \
00029                     DWT_INT_RXSTO_BIT_MASK | \
00030                     DWT_INT_RXFTO_BIT_MASK | \
00031                     DWT_INT_TXFRS_BIT_MASK | \
00032                     DWT_INT_SPIRDY_BIT_MASK)
00033
00034
00035 #define DWT_DEVICE_ID 0xDECA0302UL
00036 #define DWM_PAN_ID 0xDECA
00037
00038 #define DWM_CRC_LEN 2U // DW3000 auto-appends 2-byte CRC to every frame
00039 #define DWM_RX_TIMEOUT_1S 976562U // ~1s expressed in UWB symbol ticks (~1µs each)
00040 #define DWM_ANT_DELAY (16385U - 90U) // antenna delay in DWT ticks, applied to TX and RX
00041
00042 #define DWM_MAX_FRAME_LEN 127
00043
```

8.33 App/Generic/my_print.c File Reference

```
#include <stdarg.h>
#include <stdio.h>
#include <string.h>
#include <stdbool.h>
#include "main.h"
#include "my_print.h"
#include "usbd_cdc_if.h"
#include "cmsis_os2.h"
#include "task.h"
#include "FreeRTOS.h"
#include "semphr.h"
```

Data Structures

- struct [print_msg_t](#)
One queued print message — tick-stamped at enqueue time.

Macros

- #define [PRINT_QUEUE_DEPTH](#) 50
Maximum number of pending log messages in the print queue.
- #define [PRINT_BUF_SIZE](#) 128
Maximum formatted string length per message (bytes, including NUL).

Functions

- void [PrintTask_TxCpltCallback](#) (void)
USB CDC Tx-complete callback — call from the ISR context.
- void [mprintf](#) (const char *format,...)
printf-style logging function — callable from any task context.
- void [PrintTask](#) (void *arg)
Low-priority FreeRTOS print task — drains the message queue.
- bool [get_usb_ready](#) (void)
Return true if the USB device is in the configured (active) state.

Variables

- USB_D_HandleTypeDef [hUsbDeviceFS](#)

8.33.1 Macro Definition Documentation

8.33.1.1 PRINT_BUF_SIZE

```
#define PRINT_BUF_SIZE 128
```

Maximum formatted string length per message (bytes, including NUL).

8.33.1.2 PRINT_QUEUE_DEPTH

```
#define PRINT_QUEUE_DEPTH 50
```

Maximum number of pending log messages in the print queue.

8.33.2 Function Documentation

8.33.2.1 get_usb_ready()

```
bool get_usb_ready (
    void )
```

Return true if the USB device is in the configured (active) state.

Intended as a quick pre-flight check before sending large amounts of data. Does not guarantee that a subsequent [mprintf](#) will succeed — the device may disconnect between this call and the actual transmit.

Returns

true if `hUsbDeviceFS` is `USBD_STATE_CONFIGURED`, false otherwise.

8.33.2.2 mprintf()

```
void mprintf (
    const char * format,
    ...)
```

printf-style logging function — callable from any task context.

Formats the message into a [print_msg_t](#) and enqueues it non-blocking. If the queue is full the message is silently dropped; the caller is never blocked or delayed. Safe to call before [PrintTask](#) has started, in which case messages are also silently dropped.

The tick count is captured at call time so log timestamps reflect when the event occurred, not when [PrintTask](#) eventually drains the queue.

Parameters

<i>format</i>	printf-compatible format string.
...	Format arguments.

8.33.2.3 PrintTask()

```
void PrintTask (
    void * arg)
```

Low-priority FreeRTOS print task — drains the message queue.

Must be created exactly once via `osThreadNew()` before the scheduler starts. Initialises the message queue and the `s_cdc_tx_done` semaphore internally so both are guaranteed to exist before any other task calls [mprintf](#).

Output paths (selected at build time by `PRINT_MODE`):

- `STlink / Both` — transmits raw message bytes over UART1 (blocking, 100 ms timeout). Suitable for low-rate debug output.
- `USB_CDC / Both` — prepends an 8-digit tick prefix, then transmits over USB CDC with a flow-control semaphore:
 - Waits up to 50 ms for the previous transfer to complete; drops the message on timeout (USB stall or disconnect).
 - A short critical section gates the `TxState` check to prevent a concurrent disconnect IRQ from tearing down the endpoint between the check and the transmit call. The USB stack is never held across the actual DMA/PMA transfer.
 - On `USBD_OK`: [PrintTask_TxCpltCallback](#) gives the semaphore back.
 - On rejection or USB-not-ready: the semaphore token is restored immediately so the next message is not blocked.

Parameters

<i>arg</i>	Unused; required by CMSIS-RTOS2 task signature.
------------	---

8.33.2.4 PrintTask_TxCpltCallback()

```
void PrintTask_TxCpltCallback (
    void )
```

USB CDC Tx-complete callback — call from the ISR context.

Wire this into `usbd_cdc_if.c` as follows:

```
extern void PrintTask_TxCpltCallback(void);

static int8_t CDC_TransmitCplt_FS(uint8_t *Buf, uint32_t *Len, uint8_t epnum)
{
    PrintTask_TxCpltCallback();
    return USB_OK;
}
```

Releases the `s_cdc_tx_done` binary semaphore so `PrintTask` can proceed with the next queued message. Safe to call from ISR context.

8.33.3 Variable Documentation**8.33.3.1 hUsbDeviceFS**

```
USBD_HandleTypeDef hUsbDeviceFS [extern]
```

8.34 App/Generic/my_print.h File Reference

```
#include "main.h"
#include <stdbool.h>
```

Macros

- `#define STlink 1`
- `#define USB_CDC 2`
- `#define Both 3`
- `#define PRINT_MODE Both`

Functions

- void `mprintf` (const char *format,...)
printf-style logging function — callable from any task context.
- void `PrintTask` (void *arg)
Low-priority FreeRTOS print task — drains the message queue.
- bool `get_usb_ready` (void)
Return true if the USB device is in the configured (active) state.

8.34.1 Macro Definition Documentation**8.34.1.1 Both**

```
#define Both 3
```

8.34.1.2 PRINT_MODE

```
#define PRINT_MODE Both
```

8.34.1.3 STlink

```
#define STlink 1
```

8.34.1.4 USB_CDC

```
#define USB_CDC 2
```

8.34.2 Function Documentation

8.34.2.1 get_usb_ready()

```
bool get_usb_ready (
    void )
```

Return true if the USB device is in the configured (active) state.

Intended as a quick pre-flight check before sending large amounts of data. Does not guarantee that a subsequent [mprintf](#) will succeed — the device may disconnect between this call and the actual transmit.

Returns

true if `hUsbDeviceFS` is `USBD_STATE_CONFIGURED`, false otherwise.

8.34.2.2 mprintf()

```
void mprintf (
    const char * format,
    ...)
```

printf-style logging function — callable from any task context.

Formats the message into a [print_msg_t](#) and enqueues it non-blocking. If the queue is full the message is silently dropped; the caller is never blocked or delayed. Safe to call before [PrintTask](#) has started, in which case messages are also silently dropped.

The tick count is captured at call time so log timestamps reflect when the event occurred, not when [PrintTask](#) eventually drains the queue.

Parameters

<i>format</i>	printf-compatible format string.
...	Format arguments.

8.34.2.3 PrintTask()

```
void PrintTask (
    void * arg)
```

Low-priority FreeRTOS print task — drains the message queue.

Must be created exactly once via `osThreadNew()` before the scheduler starts. Initialises the message queue and the `s_cdc_tx_done` semaphore internally so both are guaranteed to exist before any other task calls [mprintf](#).

Output paths (selected at build time by `PRINT_MODE`):

- `STlink / Both` — transmits raw message bytes over UART1 (blocking, 100 ms timeout). Suitable for low-rate debug output.
- `USB_CDC / Both` — prepends an 8-digit tick prefix, then transmits over USB CDC with a flow-control semaphore:
 - Waits up to 50 ms for the previous transfer to complete; drops the message on timeout (USB stall or disconnect).
 - A short critical section gates the `TxState` check to prevent a concurrent disconnect IRQ from tearing down the endpoint between the check and the transmit call. The USB stack is never held across the actual DMA/PMA transfer.

- On USB_D_OK: [PrintTask_TxCpltCallback](#) gives the semaphore back.
- On rejection or USB-not-ready: the semaphore token is restored immediately so the next message is not blocked.

Parameters

<i>arg</i>	Unused; required by CMSIS-RTOS2 task signature.
------------	---

8.35 my_print.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MY_PRINTF_H
00002 #define MY_PRINTF_H
00003
00004 #ifdef __cplusplus
00005 extern "C" {
00006 #endif
00007
00008
00009 #include "main.h"
00010 #include <stdbool.h>
00011
00012 #define STlink 1
00013 #define USB_CDC 2
00014 #define Both 3
00015
00016 // ---- Change this to select output method ----
00017 #ifdef UWB_BOARD_V1_1
00018     #define PRINT_MODE USB_CDC
00019 #else
00020     #define PRINT_MODE Both
00021 #endif
00022
00023
00024 // Function Prototype
00025 void mprintf(const char *format, ...);
00026 void PrintTask(void *arg);
00027 bool get_usb_ready(void);
00028
00029 #ifdef __cplusplus
00030 }
00031 #endif
00032
00033 #endif

```

8.36 App/Generic/power.c File Reference

```

#include "power.h"
#include "main.h"
#include "my_print.h"

```

Macros

- `#define VDDA_MV 3300.0f`
STM32L4 VDDA supply voltage in millivolts (3.3 V).
- `#define ADC_FULL_SCALE 255.0f`
ADC full-scale count for an 8-bit result register.
- `#define DIVIDER_RATIO 1.5f`
Resistive divider ratio on the battery sense line (e.g. 2:3 → 1.5).

Functions

- void [power_read](#) (void)
Sample GPIO charge-status pins and perform one ADC battery voltage conversion.
- const [BatteryStatus_t](#) * [power_get_status](#) (void)

Return a pointer to the most recently sampled battery status.

8.36.1 Macro Definition Documentation

8.36.1.1 ADC_FULL_SCALE

```
#define ADC_FULL_SCALE 255.0f
```

ADC full-scale count for an 8-bit result register.

8.36.1.2 DIVIDER_RATIO

```
#define DIVIDER_RATIO 1.5f
```

Resistive divider ratio on the battery sense line (e.g. 2:3 → 1.5).

8.36.1.3 VDDA_MV

```
#define VDDA_MV 3300.0f
```

STM32L4 VDDA supply voltage in millivolts (3.3 V).

8.36.2 Function Documentation

8.36.2.1 power_get_status()

```
const BatteryStatus_t * power_get_status (
    void )
```

Return a pointer to the most recently sampled battery status.

The pointed-to struct is updated on each call to [power_read](#). The caller must not modify the returned struct.

Returns

Const pointer to the internal [BatteryStatus_t](#).

8.36.2.2 power_read()

```
void power_read (
    void )
```

Sample GPIO charge-status pins and perform one ADC battery voltage conversion.

Execution sequence:

1. Wakes the ADC from deep power-down and enables the internal voltage regulator.
2. Reads the three charge-status GPIO pins (VBUS sense, charge-indicator, standby).
3. Waits 1 ms for the ADC regulator to stabilise (datasheet t_{start} requirement).
4. Runs a self-calibration ($\sim 3 \mu s$ at 312 kHz ADC clock), configures the channel, and starts a single conversion.
5. Polls for completion (10 ms timeout; typical ~ 2 ms at 640.5-cycle sample time). On success, converts the raw count to volts via [VDDA_MV](#) and [DIVIDER_RATIO](#). On timeout, stores -1.0 V as a conversion-failure sentinel.
6. Stops the ADC and returns it to deep power-down ($\sim 0.3 \mu A$ quiescent).

Results are stored in the module-static `battery_status` struct and accessed via [power_get_status](#).

Note

Must not be called from ISR context — uses `osDelay` and blocking HAL calls.

8.37 App/Generic/power.h File Reference

```
#include <stdbool.h>
```

Data Structures

- struct [BatteryStatus_t](#)

Functions

- void [power_read](#) (void)
Sample GPIO charge-status pins and perform one ADC battery voltage conversion.
- const [BatteryStatus_t](#) * [power_get_status](#) (void)
Return a pointer to the most recently sampled battery status.

8.37.1 Function Documentation

8.37.1.1 [power_get_status\(\)](#)

```
const BatteryStatus\_t * power_get_status (
    void )
```

Return a pointer to the most recently sampled battery status.

The pointed-to struct is updated on each call to [power_read](#). The caller must not modify the returned struct.

Returns

Const pointer to the internal [BatteryStatus_t](#).

8.37.1.2 [power_read\(\)](#)

```
void power_read (
    void )
```

Sample GPIO charge-status pins and perform one ADC battery voltage conversion.

Execution sequence:

1. Wakes the ADC from deep power-down and enables the internal voltage regulator.
2. Reads the three charge-status GPIO pins (VBUS sense, charge-indicator, standby).
3. Waits 1 ms for the ADC regulator to stabilise (datasheet t_{start} requirement).
4. Runs a self-calibration ($\sim 3 \mu s$ at 312 kHz ADC clock), configures the channel, and starts a single conversion.
5. Polls for completion (10 ms timeout; typical ~ 2 ms at 640.5-cycle sample time). On success, converts the raw count to volts via `VDDA_MV` and `DIVIDER_RATIO`. On timeout, stores -1.0 V as a conversion-failure sentinel.
6. Stops the ADC and returns it to deep power-down ($\sim 0.3 \mu A$ quiescent).

Results are stored in the module-static `battery_status` struct and accessed via [power_get_status](#).

Note

Must not be called from ISR context — uses `osDelay` and blocking HAL calls.

8.38 [power.h](#)

[Go to the documentation of this file.](#)

```
00001 #ifndef POWER_H
00002 #define POWER_H
00003
00004 #include <stdbool.h>
00005
00006 typedef struct {
00007     float battery_voltage_V;
00008     bool  usb_connected;      // VBUS_sense PA9 HIGH = USB present
00009     bool  is_charging;        // CHRG PB0 LOW  = TP4057 charging
00010     bool  charge_complete;    // STDBY PA6 LOW  = TP4057 standby (full)
```



```

00011 } BatteryStatus_t;
00012
00013 void power_read(void);
00014 const BatteryStatus_t* power_get_status(void);
00015
00016 #endif

```

8.39 App/Generic/sleep.c File Reference

```

#include "main.h"
#include "cmsis_os2.h"
#include "FreeRTOS.h"
#include "task.h"

```

Functions

- void [HAL_LPTIM_AutoReloadMatchCallback](#) (LPTIM_HandleTypeDef *hlptim)
LPTIM1 auto-reload-match HAL callback — fires from ISR context.
- void [sleep_stop1_ms](#) (uint32_t ms)
Enter STM32L4 Stop1 mode for ms milliseconds, then resume.

Variables

- LPTIM_HandleTypeDef [hlptim1](#)

8.39.1 Function Documentation

8.39.1.1 HAL_LPTIM_AutoReloadMatchCallback()

```

void HAL_LPTIM_AutoReloadMatchCallback (
    LPTIM_HandleTypeDef * hlptim)

```

LPTIM1 auto-reload-match HAL callback — fires from ISR context.

Sets `s_lptim1_woke` to signal [sleep_stop1_ms](#) that the requested timeout has elapsed. Ignores callbacks from any other LPTIM instance.

Parameters

<i>hlptim</i>	LPTIM handle that triggered the callback.
---------------	---

8.39.1.2 sleep_stop1_ms()

```

void sleep_stop1_ms (
    uint32_t ms)

```

Enter STM32L4 Stop1 mode for ms milliseconds, then resume.

Uses LPTIM1 in one-shot timeout mode as the wakeup source. The sequence is:

1. Compute the LPTIM1 compare value from ms (`lptim1_arr_from_ms`).
2. Clear any stale LPTIM1 flags.
3. Start the one-shot timeout interrupt (`ARR = 0xFFFF`, compare = computed value).
4. Suspend the SysTick and enter Stop1 via WFI.
5. On wakeup: restore the system clock, resume SysTick, stop the LPTIM, and reinitialise the ADC (clocks are reconfigured by `SystemClock_Config`).

Note

Must not be called from ISR context or while holding any RTOS mutex. FreeRTOS tick is suspended for the full sleep duration.

Parameters

<i>ms</i>	Sleep duration in milliseconds (clamped to ~2 s by the 16-bit counter).
-----------	---

8.39.2 Variable Documentation**8.39.2.1 hlptim1**

```
LPTIM_HandleTypeDef hlptim1 [extern]
```

8.40 App/Generic/sleep.h File Reference**Functions**

- void [sleep_stop1_ms](#) (uint32_t ms)
Enter STM32L4 Stop1 mode for ms milliseconds, then resume.

8.40.1 Function Documentation**8.40.1.1 sleep_stop1_ms()**

```
void sleep_stop1_ms (
    uint32_t ms)
```

Enter STM32L4 Stop1 mode for *ms* milliseconds, then resume.

Uses LPTIM1 in one-shot timeout mode as the wakeup source. The sequence is:

1. Compute the LPTIM1 compare value from *ms* (lptim1_arr_from_ms).
2. Clear any stale LPTIM1 flags.
3. Start the one-shot timeout interrupt (ARR = 0xFFFF, compare = computed value).
4. Suspend the SysTick and enter Stop1 via WFI.
5. On wakeup: restore the system clock, resume SysTick, stop the LPTIM, and reinitialise the ADC (clocks are reconfigured by `SystemClock_Config`).

Note

Must not be called from ISR context or while holding any RTOS mutex. FreeRTOS tick is suspended for the full sleep duration.

Parameters

<i>ms</i>	Sleep duration in milliseconds (clamped to ~2 s by the 16-bit counter).
-----------	---

8.41 sleep.h

[Go to the documentation of this file.](#)

```
00001 #ifndef SLEEP_H
00002 #define SLEEP_H
00003
00004 void sleep_stop1_ms(uint32_t ms);
00005
00006 #endif /* SLEEP_H */
```

8.42 App/Generic/UART.c File Reference

```
#include "uart.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <stdio.h>
#include <inttypes.h>
#include <stdlib.h>
#include "../UWB_app/uwb_network.h"
#include "my_print.h"
#include "power.h"
```

Macros

- `#define UART_RST_BYTE 0x03U`
Single-byte sentinel that triggers a soft reset when received.
- `#define UART_RST_BUF_SIZE 8U`
RST receive buffer size (only 1 byte used; extra headroom for DMA).

Functions

- `bool uart_boot_start (void)`
Transmit this node's ID and arm the UART DMA to receive a boot config.
- `boot_config_t uart_boot_config_read (void)`
Read and decode the boot configuration received over UART.
- `void uart_rst_arm (void)`
Arm the UART DMA to listen for a single-byte RST command.
- `bool uart_periodic_poll (void)`
Transmit current node positions and check for an RST command.
- `void uart_print_power (void)`
Transmit current battery / power status over LPUART1.
- `void HAL_UARTEx_RxEventCallback (UART_HandleTypeDef *huart, uint16_t Size)`
LPUART1 Rx-event callback — fired by DMA idle-line or transfer-complete.
- `void HAL_UART_ErrorCallback (UART_HandleTypeDef *huart)`
LPUART1 error callback — records any UART error during boot receive.

8.42.1 Macro Definition Documentation

8.42.1.1 UART_RST_BUF_SIZE

```
#define UART_RST_BUF_SIZE 8U
RST receive buffer size (only 1 byte used; extra headroom for DMA).
```

8.42.1.2 UART_RST_BYTE

```
#define UART_RST_BYTE 0x03U
Single-byte sentinel that triggers a soft reset when received.
```

8.42.2 Function Documentation

8.42.2.1 HAL_UART_ErrorCallback()

```
void HAL_UART_ErrorCallback (
    UART_HandleTypeDef * huart)
```

LPUART1 error callback — records any UART error during boot receive.

Sets `uart_boot_rx_error` so [uart_boot_config_read](#) can detect and report the failure instead of trying to decode a corrupt buffer.

Parameters

<i>huart</i>	UART handle that encountered the error.
--------------	---

8.42.2.2 HAL_UARTEEx_RxEventCallback()

```
void HAL_UARTEEx_RxEventCallback (
    UART_HandleTypeDef * huart,
    uint16_t Size)
```

LPUART1 Rx-event callback — fired by DMA idle-line or transfer-complete.

Dispatches to the active receive phase:

- **RST phase** (`uart_rst_phase == true`): records `Size` and sets `uart_rst_rx_done`; clears `uart_rst_phase`.
- **Boot phase**: records `Size` and sets `uart_boot_rx_done`.

Ignores events from any UART handle other than `hlpuart1`.

Parameters

<i>huart</i>	UART handle that triggered the event.
<i>Size</i>	Number of bytes received into the DMA buffer.

8.42.2.3 uart_boot_config_read()

```
boot_config_t uart_boot_config_read (
    void )
```

Read and decode the boot configuration received over UART.

Must be called after a sufficient timeout following [uart_boot_start](#). Checks that a complete, error-free frame was received, then decodes it via `boot_config_decode`. Falls back to defaults if the receive failed or the frame could not be parsed.

Returns

Decoded [boot_config_t](#); `valid` is true on success, false if no frame was received or decoding failed (defaults still populated).

8.42.2.4 uart_boot_start()

```
bool uart_boot_start (
    void )
```

Transmit this node's ID and arm the UART DMA to receive a boot config.

Transmits the 4-hex-digit node ID followed by `\r\n` over LPUART1, then starts a `HAL_UARTEEx_ReceiveToIdle_DMA` transfer into `uart_boot_rx_buf`. The half-transfer DMA interrupt is disabled to avoid a spurious early callback.

Must be called once during the boot sequence before [uart_boot_config_read](#).

Returns

true if both the transmit and DMA arm succeeded, false otherwise.

8.42.2.5 `uart_periodic_poll()`

```
bool uart_periodic_poll (
    void )
```

Transmit current node positions and check for an RST command.

Should be called from the main application loop. On each call:

1. Re-arms the RST listener via [uart_rst_arm](#).
2. Transmits [POS] lines for `net->self` and all peers (if any peers are present).
3. If `UART_RST_BYTE` was received since the last call, acknowledges it, clears the flag, and returns true.

Returns

true if a reset was requested, false otherwise.

8.42.2.6 `uart_print_power()`

```
void uart_print_power (
    void )
```

Transmit current battery / power status over LPUART1.

Formats and sends a single [PWR] line showing voltage, USB connection state, charge-active flag, and charge-complete flag.

8.42.2.7 `uart_rst_arm()`

```
void uart_rst_arm (
    void )
```

Arm the UART DMA to listen for a single-byte RST command.

No-ops if a byte has already been received and is pending processing (`uart_rst_rx_done` is true). Aborts any active receive before re-arming. The half-transfer interrupt is disabled.

Should be called once after initialisation and then re-called by [uart_periodic_poll](#) after each processed RST event.

8.43 App/Generic/UART.h File Reference

```
#include <stdbool.h>
#include <stdint.h>
#include "main.h"
```

Data Structures

- struct [ekf_node_hint_t](#)
- struct [boot_config_t](#)

Macros

- #define [UART_BOOT_RX_LEN](#) 64

Functions

- bool [uart_boot_start](#) (void)
Transmit this node's ID and arm the UART DMA to receive a boot config.
- [boot_config_t uart_boot_config_read](#) (void)
Read and decode the boot configuration received over UART.
- void [HAL_UART_RxCpltCallback](#) (UART_HandleTypeDef *huart)
- void [HAL_UART_ErrorCallback](#) (UART_HandleTypeDef *huart)
LPUART1 error callback — records any UART error during boot receive.
- bool [uart_periodic_poll](#) (void)
Transmit current node positions and check for an RST command.
- void [uart_rst_arm](#) (void)
Arm the UART DMA to listen for a single-byte RST command.
- void [uart_print_power](#) (void)
Transmit current battery / power status over LPUART1.

8.43.1 Macro Definition Documentation

8.43.1.1 UART_BOOT_RX_LEN

```
#define UART_BOOT_RX_LEN 64
```

8.43.2 Function Documentation

8.43.2.1 HAL_UART_ErrorCallback()

```
void HAL_UART_ErrorCallback (
    UART_HandleTypeDef * huart)
```

LPUART1 error callback — records any UART error during boot receive.

Sets `uart_boot_rx_error` so [uart_boot_config_read](#) can detect and report the failure instead of trying to decode a corrupt buffer.

Parameters

<i>huart</i>	UART handle that encountered the error.
--------------	---

8.43.2.2 HAL_UART_RxCpltCallback()

```
void HAL_UART_RxCpltCallback (
    UART_HandleTypeDef * huart)
```

8.43.2.3 uart_boot_config_read()

```
boot\_config\_t uart_boot_config_read (
    void )
```

Read and decode the boot configuration received over UART.

Must be called after a sufficient timeout following [uart_boot_start](#). Checks that a complete, error-free frame was received, then decodes it via `boot_config_decode`. Falls back to defaults if the receive failed or the frame could not be parsed.

Returns

Decoded [boot_config_t](#); `valid` is true on success, false if no frame was received or decoding failed (defaults still populated).

8.43.2.4 uart_boot_start()

```
bool uart_boot_start (
    void )
```

Transmit this node's ID and arm the UART DMA to receive a boot config.

Transmits the 4-hex-digit node ID followed by `\r\n` over LPUART1, then starts a `HAL_UARTEx_ReceiveToIdle_DMA` transfer into `uart_boot_rx_buf`. The half-transfer DMA interrupt is disabled to avoid a spurious early callback.

Must be called once during the boot sequence before [uart_boot_config_read](#).

Returns

true if both the transmit and DMA arm succeeded, false otherwise.

8.43.2.5 uart_periodic_poll()

```
bool uart_periodic_poll (
    void )
```

Transmit current node positions and check for an RST command.

Should be called from the main application loop. On each call:

1. Re-arms the RST listener via [uart_rst_arm](#).
2. Transmits [POS] lines for `net->self` and all peers (if any peers are present).
3. If `UART_RST_BYTE` was received since the last call, acknowledges it, clears the flag, and returns true.

Returns

true if a reset was requested, false otherwise.

8.43.2.6 uart_print_power()

```
void uart_print_power (
    void )
```

Transmit current battery / power status over LPUART1.

Formats and sends a single [PWR] line showing voltage, USB connection state, charge-active flag, and charge-complete flag.

8.43.2.7 uart_rst_arm()

```
void uart_rst_arm (
    void )
```

Arm the UART DMA to listen for a single-byte RST command.

No-ops if a byte has already been received and is pending processing (`uart_rst_rx_done` is true). Aborts any active receive before re-arming. The half-transfer interrupt is disabled.

Should be called once after initialisation and then re-called by [uart_periodic_poll](#) after each processed RST event.

8.44 UART.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UART_BOOT_H
00002 #define UART_BOOT_H
00003
00004 #include <stdbool.h>
00005 #include <stdint.h>
00006 #include "main.h"
00007
00008 #define UART_BOOT_RX_LEN 64
00009
00010 typedef struct {
00011     uint16_t id;
00012     float x;
```

```

00013     float y;
00014     float z;
00015 } ekf_node_hint_t;
00016
00017 typedef struct {
00018     bool charge_only;
00019     ekf_node_hint_t hints[4];
00020     bool valid;
00021 } boot_config_t;
00022
00023 bool uart_boot_start(void);
00024 boot_config_t uart_boot_config_read(void);
00025
00026 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart);
00027 void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart);
00028 bool uart_periodic_poll(void);
00029 void uart_rst_arm(void);
00030 void uart_print_power(void);
00031
00032 #endif /* UART_BOOT_H */

```

8.45 App/UWB_app/messages.c File Reference

UWB TWR message encode/decode implementation.

```

#include "messages.h"
#include "DWM3000_driver.h"
#include "../Generic/my_print.h"
#include <string.h>
#include <stdint.h>
#include "cmsis_os.h"
#include "cmsis_os2.h"

```

Functions

- void `u64_to_ts_buf` (uint64_t ts, uint8_t *buf)
Serialize a 64-bit timestamp into a little-endian byte buffer.
- void `msg_decode` (const `dwm_rx_frame_t` *receive_frame, `msg_t` *msg_decoded)
Decode a raw RX frame into a typed message struct.
- `dwm_tx_frame_t` `msg_encode` (const `msg_t` *msg)
Encode a typed message struct into a TX frame.

8.45.1 Detailed Description

UWB TWR message encode/decode implementation.

Implements the public `msg_encode` / `msg_decode` API and all private per-type encoder and decoder helpers.

Each message type has a matching pair:

- `msg_decode_<type>` — extracts fields from the raw byte buffer.
- `msg_encode_<type>` — writes fields into the raw byte buffer.

The `#ifdef UWB_DEBUG` block at the bottom contains round-trip self-tests for every message type; enable by defining `UWB_DEBUG`.

8.45.2 Function Documentation

8.45.2.1 `msg_decode()`

```

void msg_decode (
    const dwm_rx_frame_t * receive_frame,
    msg_t * msg_decoded)

```

Decode a raw RX frame into a typed message struct.

Reads the message type byte and sender/receiver address fields, then dispatches to the appropriate per-type decoder. Sets `msg_decoded->type` to `MSG_TYPE_ERR` on an unknown type or an undersized frame.

Parameters

in	<i>receive_frame</i>	Raw frame from the DWM3000 RX buffer.
out	<i>msg_decoded</i>	Populated message struct on success.

Reads the message type and header fields (sender, receiver) from the frame buffer, then dispatches to the matching type-specific decoder to fill the payload union in *msg_decoded*.

On failure (frame too short or unknown type), *msg_decoded->type* is set to `MSG_TYPE_ERR` and *msg_decoded->len* is set to 0.

Parameters

in	<i>receive_frame</i>	Pointer to the raw RX frame from the DWM3000 driver.
out	<i>msg_decoded</i>	Pointer to the container struct to populate.

8.45.2.2 msg_encode()

```
dwm_tx_frame_t msg_encode (
    const msg_t * msg)
```

Encode a typed message struct into a TX frame.

Encode a typed message struct into a transmit frame.

Writes the type byte and sender/receiver address fields into a static buffer, then dispatches to the appropriate per-type encoder to fill the payload. Returns a zero-length frame if *msg* is NULL or the type is unrecognised.

Parameters

in	<i>msg</i>	Message to encode.
----	------------	--------------------

Returns

Ready-to-transmit *dwm_tx_frame_t* pointing to an internal static buffer.

Encode a typed message struct into a TX frame.

Returns a *dwm_tx_frame_t* pointing to an internal static buffer. The buffer remains valid until the next call to *msg_encode*. *tx_timestamp* is left as 0 — the caller fills it after scheduling a delayed TX.

Parameters

in	<i>msg</i>	Pointer to the populated message struct to encode.
----	------------	--

Returns

dwm_tx_frame_t with data pointer and *len* set. *len* == 0 indicates failure (null pointer or unknown type).

8.45.2.3 u64_to_ts_buf()

```
void u64_to_ts_buf (
    uint64_t ts,
    uint8_t * buf)
```

Serialize a 64-bit timestamp into a little-endian byte buffer.

Serialise a 40-bit DW3xxx timestamp into a 5-byte little-endian buffer.

Writes `MSG_TS_LEN` bytes to *buf*, LSB first.

Parameters

in	ts	64-bit DWM3000 timestamp value.
out	buf	Destination buffer of at least MSG_TS_LEN bytes.

Serialize a 64-bit timestamp into a little-endian byte buffer.
Only the low 40 bits of `ts` are written; the upper 24 bits are ignored.

Parameters

in	ts	64-bit timestamp value (only bits 0–39 are used).
out	buf	Output buffer of at least MSG_TS_LEN bytes.

8.46 App/UWB_app/messages.h File Reference

UWB TWR message definitions, offsets, and decoded data structures.

```
#include <stdbool.h>
#include <stdint.h>
#include "DWM3000_driver.h"
```

Data Structures

- struct [msg_sync_t](#)
Payload for MSG_TYPE_SYNC.
- struct [msg_poll_t](#)
Payload for MSG_TYPE_POLL.
- struct [msg_response_t](#)
Payload for MSG_TYPE_RESPONSE.
- struct [msg_final_t](#)
- struct [msg_share_t](#)
- struct [msg_passive_t](#)
- struct [msg_t](#)
Container for a fully decoded UWB message of any type.

Macros

- #define [NETWORK_MAX_PEERS](#) 7
Maximum number of peers (including self) the network can track.
- #define [MSG_START_LOCATION](#) 0
Byte offset of the message type field in the frame buffer.
- #define [MSG_OFFSET_SENDER](#) ([MSG_START_LOCATION](#) + 1)
Byte offset of the sender ID field (2 bytes, little-endian).
- #define [MSG_OFFSET_RECEIVER](#) ([MSG_START_LOCATION](#) + 3)
Byte offset of the receiver ID field (2 bytes, little-endian).
- #define [MSG_HEADER_LEN](#) ([MSG_OFFSET_RECEIVER](#) + 2)
Total length of the message header in bytes (type + sender + receiver).
- #define [MSG_TS_LEN](#) 5
Length of a DW3000 40-bit timestamp as stored on the wire (bytes).
- #define [MSG_PAYLOAD_OFFSET_SEQ](#) ([MSG_HEADER_LEN](#))
Byte offset of the sequence number — first byte after the header.
- #define [MSG_PAYLOAD_OFFSET_TS_0](#) ([MSG_HEADER_LEN](#) + 1)
Byte offset of the first timestamp field in the payload.

- `#define MSG_PAYLOAD_OFFSET_TS_1 (MSG_HEADER_LEN + 1 + MSG_TS_LEN)`
Byte offset of the second timestamp field in the payload.
- `#define MSG_PAYLOAD_OFFSET_TS_2 (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 2)`
Byte offset of the third timestamp field in the payload.
- `#define MSG_PAYLOAD_OFFSET_SHARE_SLEEP (MSG_PAYLOAD_OFFSET_SEQ + 1U)`
Byte offset of the sleep-time field inside the SHARE message.
- `#define MSG_PAYLOAD_OFFSET_RESP_RSSI (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 3)`
Byte offset of the response RSSI field inside the FINAL message.
- `#define MSG_PAYLOAD_OFFSET_RESP_FP (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 3 + sizeof(int16_t))`
Byte offset of the response first-path power field inside the FINAL message.
- `#define NETWORK_MAX_PAIRS ((NETWORK_MAX_PEERS * (NETWORK_MAX_PEERS - 1)) / 2)`

Enumerations

- `enum msg_type_t {`
`MSG_TYPE_ERR = 0x00 , MSG_TYPE_SYNC = 0x01 , MSG_TYPE_POLL = 0x02 , MSG_TYPE_RESPONSE`
`= 0x03 ,`
`MSG_TYPE_FINAL = 0x04 , MSG_TYPE_SHARE = 0x05 , MSG_TYPE_PASSIVE = 0x06 }`
Identifies the role/purpose of a UWB frame.

Functions

- `void msg_decode (const dwm_rx_frame_t *receive_frame, msg_t *msg_decoded)`
Decode a raw RX frame into a typed message struct.
- `dwm_tx_frame_t msg_encode (const msg_t *msg)`
Encode a typed message struct into a transmit frame.
- `void u64_to_ts_buf (uint64_t ts, uint8_t *buf)`
Serialise a 40-bit DW3xxx timestamp into a 5-byte little-endian buffer.

8.46.1 Detailed Description

UWB TWR message definitions, offsets, and decoded data structures.

Frame wire layout:

```
+-----+-----+-----+-----+
| [0]   | [1..2] | [3..4] | [5..] |
| type  | sender | receiver | payload |
| 1 byte | 2 bytes | 2 bytes | message-specific |
+-----+-----+-----+-----+
```

8.46.2 Macro Definition Documentation

8.46.2.1 MSG_HEADER_LEN

```
#define MSG_HEADER_LEN (MSG_OFFSET_RECEIVER + 2)
```

Total length of the message header in bytes (type + sender + receiver).

8.46.2.2 MSG_OFFSET_RECEIVER

```
#define MSG_OFFSET_RECEIVER (MSG_START_LOCATION + 3)
```

Byte offset of the receiver ID field (2 bytes, little-endian).

8.46.2.3 MSG_OFFSET_SENDER

```
#define MSG_OFFSET_SENDER (MSG_START_LOCATION + 1)
```

Byte offset of the sender ID field (2 bytes, little-endian).

8.46.2.4 MSG_PAYLOAD_OFFSET_RESP_FP

```
#define MSG_PAYLOAD_OFFSET_RESP_FP (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 3 + sizeof(int16_t))
```

Byte offset of the response first-path power field inside the FINAL message.

Carries the first-path power (Q8 fixed-point) measured by the initiator when it received the RESPONSE, transmitted back to the responder.

8.46.2.5 MSG_PAYLOAD_OFFSET_RESP_RSSI

```
#define MSG_PAYLOAD_OFFSET_RESP_RSSI (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 3)
```

Byte offset of the response RSSI field inside the FINAL message.

Carries the RSSI (Q8 fixed-point) measured by the initiator when it received the RESPONSE, transmitted back so the responder can store it.

8.46.2.6 MSG_PAYLOAD_OFFSET_SEQ

```
#define MSG_PAYLOAD_OFFSET_SEQ (MSG_HEADER_LEN)
```

Byte offset of the sequence number — first byte after the header.

8.46.2.7 MSG_PAYLOAD_OFFSET_SHARE_SLEEP

```
#define MSG_PAYLOAD_OFFSET_SHARE_SLEEP (MSG_PAYLOAD_OFFSET_SEQ + 1U)
```

Byte offset of the sleep-time field inside the SHARE message.

Immediately follows the sequence number byte.

8.46.2.8 MSG_PAYLOAD_OFFSET_TS_0

```
#define MSG_PAYLOAD_OFFSET_TS_0 (MSG_HEADER_LEN + 1)
```

Byte offset of the first timestamp field in the payload.

8.46.2.9 MSG_PAYLOAD_OFFSET_TS_1

```
#define MSG_PAYLOAD_OFFSET_TS_1 (MSG_HEADER_LEN + 1 + MSG_TS_LEN)
```

Byte offset of the second timestamp field in the payload.

8.46.2.10 MSG_PAYLOAD_OFFSET_TS_2

```
#define MSG_PAYLOAD_OFFSET_TS_2 (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 2)
```

Byte offset of the third timestamp field in the payload.

8.46.2.11 MSG_START_LOCATION

```
#define MSG_START_LOCATION 0
```

Byte offset of the message type field in the frame buffer.

8.46.2.12 MSG_TS_LEN

```
#define MSG_TS_LEN 5
```

Length of a DW3000 40-bit timestamp as stored on the wire (bytes).

8.46.2.13 NETWORK_MAX_PAIRS

```
#define NETWORK_MAX_PAIRS ((NETWORK_MAX_PEERS * (NETWORK_MAX_PEERS - 1)) / 2)
```

8.46.2.14 NETWORK_MAX_PEERS

```
#define NETWORK_MAX_PEERS 7
```

Maximum number of peers (including self) the network can track.

8.46.3 Enumeration Type Documentation

8.46.3.1 msg_type_t

enum `msg_type_t`

Identifies the role/purpose of a UWB frame.

The value is stored at [MSG_START_LOCATION](#) in the frame buffer. `MSG_TYPE_ERR` is never sent on the wire — it is used internally to signal a decode failure.

Enumerator

<code>MSG_TYPE_ERR</code>	Internal error / decode failure — not transmitted.
<code>MSG_TYPE_SYNC</code>	Sync broadcast — announces device presence.
<code>MSG_TYPE_POLL</code>	Poll — initiator starts a TWR exchange.
<code>MSG_TYPE_RESPONSE</code>	Response — responder replies to a POLL.
<code>MSG_TYPE_FINAL</code>	Final — initiator closes the DS-TWR exchange.
<code>MSG_TYPE_SHARE</code>	Share — broadcasts sleep timing after a ranging round.
<code>MSG_TYPE_PASSIVE</code>	Passive — passive observer broadcasts its TWR observations.

8.46.4 Function Documentation

8.46.4.1 msg_decode()

```
void msg_decode (
    const dwm_rx_frame_t * receive_frame,
    msg_t * msg_decoded)
```

Decode a raw RX frame into a typed message struct.

Reads the message type and header fields (sender, receiver) from the frame buffer, then dispatches to the matching type-specific decoder to fill the payload union in `msg_decoded`.

On failure (frame too short or unknown type), `msg_decoded->type` is set to `MSG_TYPE_ERR` and `msg_decoded->len` is set to 0.

Parameters

in	<i>receive_frame</i>	Pointer to the raw RX frame from the DWM3000 driver.
out	<i>msg_decoded</i>	Pointer to the container struct to populate.

Reads the message type byte and sender/receiver address fields, then dispatches to the appropriate per-type decoder. Sets `msg_decoded->type` to `MSG_TYPE_ERR` on an unknown type or an undersized frame.

Parameters

in	<i>receive_frame</i>	Raw frame from the DWM3000 RX buffer.
out	<i>msg_decoded</i>	Populated message struct on success.

8.46.4.2 msg_encode()

```
dwm_tx_frame_t msg_encode (
    const msg_t * msg)
```

Encode a typed message struct into a transmit frame.

Encode a typed message struct into a TX frame.

Returns a `dwm_tx_frame_t` pointing to an internal static buffer. The buffer remains valid until the next call to `msg_encode`. `tx_timestamp` is left as 0 — the caller fills it after scheduling a delayed TX.

Parameters

in	<i>msg</i>	Pointer to the populated message struct to encode.
----	------------	--

Returns

`dwm_tx_frame_t` with data pointer and len set. len == 0 indicates failure (null pointer or unknown type).

Encode a typed message struct into a transmit frame.

Writes the type byte and sender/receiver address fields into a static buffer, then dispatches to the appropriate per-type encoder to fill the payload. Returns a zero-length frame if *msg* is NULL or the type is unrecognised.

Parameters

in	<i>msg</i>	Message to encode.
----	------------	--------------------

Returns

Ready-to-transmit `dwm_tx_frame_t` pointing to an internal static buffer.

8.46.4.3 u64_to_ts_buf()

```
void u64_to_ts_buf (
    uint64_t ts,
    uint8_t * buf)
```

Serialise a 40-bit DW3xxx timestamp into a 5-byte little-endian buffer.

Serialize a 64-bit timestamp into a little-endian byte buffer.

Only the low 40 bits of *ts* are written; the upper 24 bits are ignored.

Parameters

in	<i>ts</i>	64-bit timestamp value (only bits 0–39 are used).
out	<i>buf</i>	Output buffer of at least <code>MSG_TS_LEN</code> bytes.

Serialise a 40-bit DW3xxx timestamp into a 5-byte little-endian buffer.

Writes `MSG_TS_LEN` bytes to *buf*, LSB first.

Parameters

in	<i>ts</i>	64-bit DWM3000 timestamp value.
out	<i>buf</i>	Destination buffer of at least <code>MSG_TS_LEN</code> bytes.

8.47 messages.h

[Go to the documentation of this file.](#)

```
00001
00014
00015 #pragma once
00016
00017 #include <stdbool.h>
00018 #include <stdint.h>
00019 // #include "uwb_network.h"
00020 #include "DWM3000_driver.h"
00021
00023 #define NETWORK_MAX_PEERS 7
00024
00025 #ifdef __cplusplus
00026 extern "C" {
00027 #endif
```

```

00028
00029 /* -- Header offsets ----- */
00030
00032 #define MSG_START_LOCATION      0
00033
00035 #define MSG_OFFSET_SENDER      (MSG_START_LOCATION + 1)
00036
00038 #define MSG_OFFSET_RECEIVER    (MSG_START_LOCATION + 3)
00039
00041 #define MSG_HEADER_LEN        (MSG_OFFSET_RECEIVER + 2)
00042
00043 /* -- Timestamp ----- */
00044
00046 #define MSG_TS_LEN      5
00047
00048 /* -- Payload offsets ----- */
00049
00051 #define MSG_PAYLOAD_OFFSET_SEQ      (MSG_HEADER_LEN)
00052
00054 #define MSG_PAYLOAD_OFFSET_TS_0    (MSG_HEADER_LEN + 1)
00055
00057 #define MSG_PAYLOAD_OFFSET_TS_1    (MSG_HEADER_LEN + 1 + MSG_TS_LEN)
00058
00060 #define MSG_PAYLOAD_OFFSET_TS_2    (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 2)
00061
00067 #define MSG_PAYLOAD_OFFSET_SHARE_SLEEP (MSG_PAYLOAD_OFFSET_SEQ + 1U)
00068
00075 #define MSG_PAYLOAD_OFFSET_RESP_RSSI \
00076     (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 3)
00077
00084 #define MSG_PAYLOAD_OFFSET_RESP_FP \
00085     (MSG_HEADER_LEN + 1 + MSG_TS_LEN * 3 + sizeof(int16_t))
00086
00087 /* -- Message type enum ----- */
00088
00096 typedef enum {
00097     MSG_TYPE_ERR      = 0x00,
00098     MSG_TYPE_SYNC     = 0x01,
00099     MSG_TYPE_POLL     = 0x02,
00100     MSG_TYPE_RESPONSE = 0x03,
00101     MSG_TYPE_FINAL    = 0x04,
00102     MSG_TYPE_SHARE    = 0x05,
00103     MSG_TYPE_PASSIVE  = 0x06,
00104 } msg_type_t;
00105
00106 /* -- Per-message payload structs ----- */
00107
00115 typedef struct {
00116     uint8_t  seq_num;
00117     uint8_t  peer_count;
00118     uint16_t peer_ids[NETWORK_MAX_PEERS];
00119 } msg_sync_t;
00120
00131 typedef struct {
00132
00133     uint8_t  seq_num;
00134
00135 } msg_poll_t;
00136
00147 typedef struct {
00148
00149     uint8_t  seq_num;
00150
00151 } msg_response_t;
00152
00153
00154 typedef struct {
00155     uint8_t  seq_num;
00156     uint64_t poll_tx_ts;
00157     uint64_t resp_rx_ts;
00158     uint64_t final_tx_ts;
00159     int16_t  resp_pwr_diff_q8;
00160     bool     resp_antenna_unreliable;
00161
00162     /* --- Preceding PASSIVE TX timestamps, in peer-list order --- */
00163     uint8_t  entry_count;
00164     uint64_t entries[NETWORK_MAX_PEERS - 2];
00165     int16_t  entry_pwr_diff_q8[NETWORK_MAX_PEERS - 2];
00166     bool     entry_antenna_unreliable[NETWORK_MAX_PEERS - 2];
00167     uint16_t entry_id[NETWORK_MAX_PEERS - 2];
00170
00171
00172     float IMU_vel_horiz;
00173     float IMU_vel_vert;
00174
00175 } msg_final_t;
00176

```

```

00177 #define NETWORK_MAX_PAIRS ((NETWORK_MAX_PEERS * (NETWORK_MAX_PEERS - 1)) / 2)
00178
00187 typedef struct {
00188     uint8_t    seq_num;
00189     uint32_t   sleep_time;
00190     uint8_t    node_count;
00191     uint16_t   node_ids[NETWORK_MAX_PEERS];
00192
00193     /* Per-node IMU (index matches node_ids[]) */
00194     uint8_t    vel_vert [NETWORK_MAX_PEERS];
00195     uint8_t    vel_horiz[NETWORK_MAX_PEERS];
00196
00197     /* Per-pair distances (upper-triangle order, no IDs stored) */
00198     uint16_t   distance_mm [NETWORK_MAX_PAIRS];
00199     uint8_t    accuracy    [NETWORK_MAX_PAIRS];
00200 } msg_share_t;
00201
00202
00203 typedef struct {
00204     uint8_t    seq_num;
00205
00206     /* --- Passive observations of the active TWR frames --- */
00207     uint64_t   poll_rx_ts;
00208     int16_t    poll_pwr_diff_q8;
00209     bool       poll_antenna_unreliable;
00210
00211     uint64_t   resp_rx_ts;
00212     int16_t    resp_pwr_diff_q8;
00213     bool       resp_antenna_unreliable;
00214
00215     /* --- This node's own PASSIVE transmission --- */
00216     uint64_t   passive_tx_ts;
00217
00218     /* --- Preceding PASSIVE TX timestamps, in peer-list order --- */
00219     uint8_t    entry_count;
00220     uint64_t   entries[NETWORK_MAX_PEERS - 2];
00221     int16_t    entry_pwr_diff_q8[NETWORK_MAX_PEERS - 2];
00222     bool       entry_antenna_unreliable[NETWORK_MAX_PEERS - 2];
00223     uint16_t   entry_ids[NETWORK_MAX_PEERS - 2];
00224
00225     float      IMU_vel_horiz;
00226     float      IMU_vel_vert;
00227 } msg_passive_t;
00228
00229
00230 /* -- Unified decoded message container ----- */
00231
00232 typedef struct {
00233     msg_type_t type;
00234     uint16_t    len;
00235     uint16_t    sender;
00236     uint16_t    receiver;
00237     union {
00238         msg_sync_t      sync;
00239         msg_poll_t      poll;
00240         msg_response_t  response;
00241         msg_final_t     final;
00242         msg_share_t     share;
00243         msg_passive_t   passive;
00244     } data;
00245 } msg_t;
00246
00247
00248 /* -- Public API ----- */
00249
00250 void msg_decode(const dwm_rx_frame_t *receive_frame, msg_t *msg_decoded);
00251
00252 dwm_tx_frame_t msg_encode(const msg_t *msg);
00253
00254 void u64_to_ts_buf(uint64_t ts, uint8_t *buf);
00255
00256 #ifdef __cplusplus
00257 }
00258 #endif

```

8.48 App/UWB_app/uwb_exchange.c File Reference

```

#include "cmsis_os.h"
#include "cmsis_os2.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>

```



```
#include <stdlib.h>
#include "main.h"
#include "../Generic/my_print.h"
#include "DWM3000_setup.h"
#include "DWM3000_driver.h"
#include "messages.h"
#include "../UWB_app/uwb_network.h"
#include "uwb_exchange.h"
#include "distance.h"
#include "lsm6dsv.h"
#include "calibrate.h"
```

Macros

- `#define RX_MEAS_FROM_FRAME(frame)`
Construct a `uwb_rx_meas_t` inline from a `dwm_rx_frame_t`.
- `#define U64_HI(x)`
Extract high 32 bits of a `uint64_t` for two-word `mprintf`.
- `#define U64_LO(x)`
Extract low 32 bits of a `uint64_t` for two-word `mprintf`.
- `#define MPRINT_TS(label, val)`
Print a 40-bit UWB timestamp label/value pair.
- `#define SYNC_PAD(t_start)`
Pad the current iteration to at least `T_SYNC_RX_ANSWER` ms from `t_start`.

Functions

- `bool uwb_unexpected_message_handler (const msg_t *rx_msg)`
Handle an unexpected message received during an exchange phase.
- `uwb_sync_result_t uwb_sync (void)`
Perform one UWB synchronisation iteration.
- `uwb_etwr_result_t uwb_extended_twr (uwb_sync_result_t sync_result)`
Execute the extended TWR ranging exchange for one round.
- `void uwb_build_share (msg_share_t *out, uint8_t seq, uint32_t sleep_time)`
Populate a `msg_share_t` from the current network state.
- `void uwb_read_share (const msg_share_t *in)`
Apply a received `msg_share_t` into the local network state.
- `uint32_t uwb_share (uwb_etwr_result_t etwr_result, uint32_t sleep_time)`
Broadcast or receive the SHARE message to close a ranging round.

8.48.1 Macro Definition Documentation

8.48.1.1 MPRINT_TS

```
#define MPRINT_TS(  
    label,  
    val)
```

Value:

```
mprintf("UWB_eTWR_TEST - %-20s: 0x%08lX%08lX\r\n", \  
    (label), U64_HI(val), U64_LO(val))
```

Print a 40-bit UWB timestamp label/value pair.

Uses two `%08lX` fields because `mprintf` does not support `l1X`.

8.48.1.2 RX_MEAS_FROM_FRAME

```
#define RX_MEAS_FROM_FRAME(  
    frame)
```

Value:

```
(uwb_rx_meas_t){ .ts = (frame).rx_timestamp, \  
    .pwr_diff_q8 = (frame).rssiq8 - (frame).fp_q8 }
```

Construct a `uwb_rx_meas_t` inline from a `dwm_rx_frame_t`.

8.48.1.3 SYNC_PAD

```
#define SYNC_PAD(  
    t_start)
```

Value:

```
do { \  
    uint32_t _e = osKernelGetTickCount() - (t_start); \  
    if (_e < T_SYNC_RX_ANSWER) osDelay(T_SYNC_RX_ANSWER - _e); \  
} while (0)
```

Pad the current iteration to at least `T_SYNC_RX_ANSWER` ms from `t_start`.

Ensures all devices in the SYNC window finish at approximately the same time regardless of how quickly they processed the SYNC reply.

8.48.1.4 U64_HI

```
#define U64_HI(  
    x)
```

Value:

```
((uint32_t)((x) >> 32))
```

Extract high 32 bits of a `uint64_t` for two-word `mprintf`.

8.48.1.5 U64_LO

```
#define U64_LO(  
    x)
```

Value:

```
((uint32_t)((x) & 0xFFFFFFFFU))
```

Extract low 32 bits of a `uint64_t` for two-word `mprintf`.

8.48.2 Function Documentation

8.48.2.1 uwb_build_share()

```
void uwb_build_share (  
    msg_share_t * out,  
    uint8_t seq,  
    uint32_t sleep_time)
```

Populate a `msg_share_t` from the current network state.

Builds the canonical node list (self first, then peers in array order), reads each node's IMU velocity, and fills all pair distances and accuracy values in upper-triangle order.

Parameters

out	<i>out</i>	Share struct to populate.
in	<i>seq</i>	Sequence number to embed.
in	<i>sleep_time</i>	Planned sleep duration in milliseconds.

8.48.2.2 uwb_extended_twr()

```
uwb_etwr_result_t uwb_extended_twr (
    uwb_sync_result_t sync_result)
```

Execute the extended TWR ranging exchange for one round.

Execute one extended two-way ranging exchange.

Dispatches on `sync_result` to run the appropriate role:

- **Master:** selects the highest-uncertainty target, sends POLL, waits for RESPONSE and PASSIVE frames, then sends FINAL.
- **Slave / Responder:** waits for a POLL addressed to self, sends RESPONSE, collects PASSIVE frames, then waits for FINAL.
- **Slave / Passive observer:** records POLL and RESPONSE timestamps and transmits a PASSIVE frame to the master.
- **Invalid sync states:** returns UWB_TWR_NOT_ENOUGH_DEVICES immediately.

IMU data is consumed at entry via `osThreadFlagsWait` and stored in the network state for inclusion in the FINAL/PASSIVE messages.

Parameters

<code>sync_result</code>	Role and peer information from the preceding <code>uwb_sync</code> call.
--------------------------	--

Returns

Exchange outcome code.

Execute the extended TWR ranging exchange for one round.

Parameters

<code>sync_result</code>	Result from the preceding <code>uwb_sync</code> call.
--------------------------	---

Returns

Success or failure mode.

8.48.2.3 uwb_read_share()

```
void uwb_read_share (
    const msg_share_t * in)
```

Apply a received `msg_share_t` into the local network state.

Updates IMU velocities for all nodes (including self), stores all pair distances and overwrites certainty counters.

Sentinel distances (0xFFFF) are skipped. Logs each pair update and per-node velocity.

Parameters

<code>in</code>	<code>in</code>	Decoded SHARE message to apply.
-----------------	-----------------	---------------------------------

8.48.2.4 uwb_share()

```
uint32_t uwb_share (
    uwb_etwr_result_t etwr_result,
    uint32_t sleep_time)
```

Broadcast or receive the SHARE message to close a ranging round.

Broadcast or receive the ranging result and compute sleep time.

The responder/passive-timeout node builds and broadcasts a SHARE frame carrying all distances, certainty values, and the next sleep duration. All other nodes wait for the SHARE, apply it via [uwb_read_share](#), and return an adjusted sleep time that accounts for early wake-up margin.

Parameters

<i>etwr_result</i>	Outcome of the preceding uwb_extended_twr call.
<i>sleep_time</i>	Desired sleep duration in milliseconds (used if broadcasting).

Returns

Adjusted sleep time in milliseconds, 0 on timeout, or 0xFFFFFFFF on TX failure.

Broadcast or receive the SHARE message to close a ranging round.

Parameters

<i>etwr_result</i>	Result from the preceding uwb_extended_twr call.
<i>sleep_time</i>	Nominal sleep duration for this cycle (ms).

Returns

Actual sleep duration (ms) adjusted for timing drift.

8.48.2.5 [uwb_sync\(\)](#)

```
uwb_sync_result_t uwb_sync (
    void )
```

Perform one UWB synchronisation iteration.

Perform one synchronisation round.

The master broadcasts a SYNC frame and listens for join replies. Slaves listen for a SYNC broadcast; if their ID is not yet in the master's peer list they reply with their own SYNC to request admission. A broadcast-storm guard demotes the master if SYNCs are sent faster than every 50 ms with no replies. Slaves promote themselves to master after `SYNC_TIMEOUT_MAX` consecutive timeouts.

Returns

Outcome code indicating role and readiness for the TWR phase.

Perform one UWB synchronisation iteration.

Returns

Outcome and role for the subsequent TWR phase.

8.48.2.6 [uwb_unexpected_message_handler\(\)](#)

```
bool uwb_unexpected_message_handler (
    const msg_t * rx_msg)
```

Handle an unexpected message received during an exchange phase.

Infers the current master from the message fields and delays for `DEEP_SLEEP` to allow the ongoing exchange to complete before this node re-enters the ranging loop. Passive and unknown message types are silently ignored.

Parameters

<code>rx_msg</code>	The unexpected message that was received.
---------------------	---

Returns

Always returns true.

8.49 App/UWB_app/uwb_exchange.h File Reference

Public API and configuration constants for the UWB ranging exchange.

```
#include <stdint.h>
```

Macros

- `#define DEEP_SLEEP 200U`
DW3xxx deep-sleep duration (ms).
- `#define T_SYNC_RX_SET 200U`
Slave RX window for the initial SYNC broadcast (ms).
- `#define T_SYNC_RX_ANSWER 10U`
Master RX window waiting for slave join replies after SYNC (ms).
- `#define T_SHARE_RX_WAIT 50U`
Slave RX window waiting for a SHARE broadcast (ms).
- `#define T_EARLY_WKUP 5U`
Early-wakeup margin before a scheduled radio event (ms).
- `#define UWB_US_TO_TICKS(us)`
Convert microseconds to DW3xxx 40-bit timestamp ticks.
- `#define T_RESPONSE_RX_WAIT 100U`
Master RX window waiting for RESPONSE after POLL (ms).
- `#define T_FINAL_RX_WAIT 100U`
Slave RX window waiting for FINAL (ms).
- `#define T_TWR_RX_WAIT 300U`
Slave RX window covering the entire TWR exchange (ms).
- `#define T_FINAL_TX_ASAP_TICKS UWB_US_TO_TICKS(800)`
Delay from RESPONSE reception to FINAL transmission (ticks).
- `#define T_PASSIVE_TX_ASAP_TICKS UWB_US_TO_TICKS(1000)`
Minimum delay before a passive observer's TX (ticks).
- `#define T_FINAL_TX_WAIT_MS 10U`
Timeout waiting for TXFRS after scheduling the delayed FINAL (ms).
- `#define T_PASSIVE_TX_WAIT_MS 10U`
Timeout waiting for TXFRS after scheduling the passive TX (ms).
- `#define SYNC_TIMEOUT_MAX 5U`
Consecutive SYNC timeouts before a slave promotes itself to master.
- `#define ALL_ID 0xFFFFU`
Broadcast destination address.

Enumerations

- enum `uwb_sync_result_t` {
`UWB_SYNC_MASTER` , `UWB_SYNC_MASTER_NO_REPLY` , `UWB_SYNC_SLAVE_ACKNOWLEDGED` ,
`UWB_SYNC_SLAVE_PENDING` ,
`UWB_SYNC_SLAVE_REPLY_FAILED` , `UWB_SYNC_NEW_MASTER` , `UWB_SYNC_NEW_SLAVE` ,
`UWB_SYNC_TX_FAILED` ,
`UWB_SYNC_UNEXPECTED_MASTER` }

Result codes returned by `uwb_sync()`.

- enum `uwb_etwr_result_t` {
`UWB_TWR_EXCHANGE_COMPLETE` , `UWB_TWR_RECEIVED` , `UWB_TWR_RECEIVED_PASSIVE` ,
`UWB_TWR_NOTHING` ,
`UWB_TWR_NOT_ENOUGH_DEVICES` , `UWB_TWR_TX_FAILED` , `UWB_TWR_TIMEOUT` , `UWB_TWR_PASSIVE_TIMEOUT`
, `UWB_TWR_UNEXPECTED_MASTER` }

Result codes returned by `uwb_extended_twr()`.

Functions

- `uwb_sync_result_t uwb_sync` (void)
Perform one synchronisation round.
- `uwb_etwr_result_t uwb_extended_twr` (`uwb_sync_result_t` sync_result)
Execute one extended two-way ranging exchange.
- `uint32_t uwb_share` (`uwb_etwr_result_t` etwr_result, `uint32_t` sleep_time)
Broadcast or receive the ranging result and compute sleep time.

8.49.1 Detailed Description

Public API and configuration constants for the UWB ranging exchange.

Provides the synchronisation (`uwb_sync`), extended two-way ranging (`uwb_extended_twr`), and result-sharing (`uwb_share`) interfaces used by the top-level network task.

Timing units

- Constants ending in `_MS` or `ms` are RTOS ticks (1 ms).
- Constants ending in `_TICKS` are DW3xxx 40-bit counter units (~15.65 ps).

8.49.2 Enumeration Type Documentation

8.49.2.1 `uwb_etwr_result_t`

enum `uwb_etwr_result_t`

Result codes returned by `uwb_extended_twr()`.

Enumerator

<code>UWB_TWR_EXCHANGE_COMPLETE</code>	Master: POLL→FINAL sent; master role passed to target.
<code>UWB_TWR_RECEIVED</code>	Responder: FINAL received; timestamps ready for ranging.
<code>UWB_TWR_RECEIVED_PASSIVE</code>	Passive observer: full exchange captured.
<code>UWB_TWR_NOTHING</code>	Exchange completed but this device was not involved.
<code>UWB_TWR_NOT_ENOUGH_DEVICES</code>	No peers available or sync state invalid.
<code>UWB_TWR_TX_FAILED</code>	TX failed.
<code>UWB_TWR_TIMEOUT</code>	RX window expired before the expected message arrived.
<code>UWB_TWR_PASSIVE_TIMEOUT</code>	

UWB_TWR_UNEXPECTED_MASTER	Frame received from an unexpected sender.
---------------------------	---

8.49.2.2 uwb_sync_result_t

enum `uwb_sync_result_t`

Result codes returned by `uwb_sync()`.

Enumerator

UWB_SYNC_MASTER	Master: SYNC sent, at least one peer acknowledged.
UWB_SYNC_MASTER_NO_REPLY	Master: SYNC sent, no replies received in window.
UWB_SYNC_SLAVE_ACKNOWLEDGED	Slave: own ID confirmed in master's peer list.
UWB_SYNC_SLAVE_PENDING	Slave: join reply sent, awaiting next SYNC.
UWB_SYNC_SLAVE_REPLY_FAILED	Slave: join reply TX failed on both attempts.
UWB_SYNC_NEW_MASTER	Slave promoted itself to master (no master heard).
UWB_SYNC_NEW_SLAVE	Master yielded role to a higher-ID master.
UWB_SYNC_TX_FAILED	Master: SYNC TX failed at the hardware level.
UWB_SYNC_UNEXPECTED_MASTER	

8.49.3 Function Documentation

8.49.3.1 uwb_extended_twr()

```
uwb_etwr_result_t uwb_extended_twr (
    uwb_sync_result_t sync_result)
```

Execute one extended two-way ranging exchange.

Execute the extended TWR ranging exchange for one round.

Parameters

<code>sync_result</code>	Result from the preceding <code>uwb_sync</code> call.
--------------------------	---

Returns

Success or failure mode.

Execute one extended two-way ranging exchange.

Dispatches on `sync_result` to run the appropriate role:

- **Master:** selects the highest-uncertainty target, sends POLL, waits for RESPONSE and PASSIVE frames, then sends FINAL.
- **Slave / Responder:** waits for a POLL addressed to self, sends RESPONSE, collects PASSIVE frames, then waits for FINAL.
- **Slave / Passive observer:** records POLL and RESPONSE timestamps and transmits a PASSIVE frame to the master.
- **Invalid sync states:** returns `UWB_TWR_NOT_ENOUGH_DEVICES` immediately.

IMU data is consumed at entry via `osThreadFlagsWait` and stored in the network state for inclusion in the FINAL/PASSIVE messages.

Parameters

<i>sync_result</i>	Role and peer information from the preceding uwb_sync call.
--------------------	---

Returns

Exchange outcome code.

8.49.3.2 uwb_share()

```
uint32_t uwb_share (
    uwb_etwr_result_t etwr_result,
    uint32_t sleep_time)
```

Broadcast or receive the ranging result and compute sleep time.
Broadcast or receive the SHARE message to close a ranging round.

Parameters

<i>etwr_result</i>	Result from the preceding uwb_extended_twr call.
<i>sleep_time</i>	Nominal sleep duration for this cycle (ms).

Returns

Actual sleep duration (ms) adjusted for timing drift.

Broadcast or receive the ranging result and compute sleep time.

The responder/passive-timeout node builds and broadcasts a SHARE frame carrying all distances, certainty values, and the next sleep duration. All other nodes wait for the SHARE, apply it via [uwb_read_share](#), and return an adjusted sleep time that accounts for early wake-up margin.

Parameters

<i>etwr_result</i>	Outcome of the preceding uwb_extended_twr call.
<i>sleep_time</i>	Desired sleep duration in milliseconds (used if broadcasting).

Returns

Adjusted sleep time in milliseconds, 0 on timeout, or 0xFFFFFFFF on TX failure.

8.49.3.3 uwb_sync()

```
uwb_sync_result_t uwb_sync (
    void )
```

Perform one synchronisation round.
Perform one UWB synchronisation iteration.

Returns

Outcome and role for the subsequent TWR phase.

Perform one synchronisation round.

The master broadcasts a SYNC frame and listens for join replies. Slaves listen for a SYNC broadcast; if their ID is not yet in the master's peer list they reply with their own SYNC to request admission. A broadcast-storm guard demotes the master if SYNCs are sent faster than every 50 ms with no replies. Slaves promote themselves to master after `SYNC_TIMEOUT_MAX` consecutive timeouts.

Returns

Outcome code indicating role and readiness for the TWR phase.

8.50 uwb_exchange.h

[Go to the documentation of this file.](#)

```

00001
00013
00014 #pragma once
00015 #include <stdint.h>
00016
00017 #ifdef __cplusplus
00018 extern "C" {
00019 #endif
00020
00021 /* -----
00022  * Timing constants
00023  * ----- */
00024
00029
00031 #define DEEP_SLEEP 200U
00032
00034 #define T_SYNC_RX_SET 200U
00035
00037 #define T_SYNC_RX_ANSWER 10U
00038
00040 #define T_SHARE_RX_WAIT 50U
00041
00043 #define T_EARLY_WKUP 5U
00044
00046 #define UWB_US_TO_TICKS(us) ((uint64_t)((us) * 63898ULL))
00047
00049 #define T_RESPONSE_RX_WAIT 100U
00050
00052 #define T_FINAL_RX_WAIT 100U
00053
00055 #define T_TWR_RX_WAIT 300U
00056
00058 #define T_FINAL_TX_ASAP_TICKS UWB_US_TO_TICKS(800)
00059
00061 #define T_PASSIVE_TX_ASAP_TICKS UWB_US_TO_TICKS(1000)
00062
00064 #define T_FINAL_TX_WAIT_MS 10U
00065
00067 #define T_PASSIVE_TX_WAIT_MS 10U
00068
00070 #define SYNC_TIMEOUT_MAX 5U
00071
00073 #define ALL_ID 0xFFFFU
00074 /* end of uwb_timing */
00076
00077 /* -----
00078  * Result enumerations
00079  * ----- */
00080
00084 typedef enum {
00085     UWB_SYNC_MASTER,
00086     UWB_SYNC_MASTER_NO_REPLY,
00087     UWB_SYNC_SLAVE_ACKNOWLEDGED,
00088     UWB_SYNC_SLAVE_PENDING,
00089     UWB_SYNC_SLAVE_REPLY_FAILED,
00090     UWB_SYNC_NEW_MASTER,
00091     UWB_SYNC_NEW_SLAVE,
00092     UWB_SYNC_TX_FAILED,
00093     UWB_SYNC_UNEXPECTED_MASTER
00094 } uwb_sync_result_t;
00095
00099 typedef enum {
00100     /* --- Success --- */
00101     UWB_TWR_EXCHANGE_COMPLETE,
00102     UWB_TWR_RECEIVED,
00103     UWB_TWR_RECEIVED_PASSIVE,
00104     UWB_TWR_NOTHING,
00105     /* --- Failure --- */
00106     UWB_TWR_NOT_ENOUGH_DEVICES,
00107     UWB_TWR_TX_FAILED,
00108     UWB_TWR_TIMEOUT,
00109     UWB_TWR_PASSIVE_TIMEOUT,
00110     UWB_TWR_UNEXPECTED_MASTER,
00111 } uwb_etwr_result_t;
00112
00113 /* -----
00114  * Public API
00115  * ----- */
00116
00121 uwb_sync_result_t uwb_sync(void);
00122
00128 uwb_etwr_result_t uwb_extended_twr(uwb_sync_result_t sync_result);
00129

```

```

00136 uint32_t  uwb_share(uwb_etwr_result_t etwr_result, uint32_t sleep_time);
00137
00138 #ifdef __cplusplus
00139 }
00140 #endif

```

8.51 App/UWB_app/uwb_network.c File Reference

Implementation of the UWB network state management API.

```

#include "cmsis_os.h"
#include "cmsis_os2.h"
#include <stdbool.h>
#include <stdint.h>
#include <string.h>
#include <stdlib.h>
#include "../Generic/my_print.h"
#include "DWM3000_setup.h"
#include "DWM3000_driver.h"
#include "../UWB_app/uwb_network.h"
#include "../Calculations/distance.h"

```

Macros

- #define [NETWORK_DISTANCE_INVALID](#) (0xFFFFu)

Functions

- [node_t](#) * [find_peer](#) (uint16_t id)
Find an active peer node by its 16-bit ID.
- void [network_init](#) (uint16_t own_id)
Initialise the network state for this device.
- void [network_add_peer](#) (uint16_t id)
Add a peer to the active peer list.
- void [network_remove_peer](#) (uint16_t id)
Remove a peer from the active peer list.
- const [node_t](#) * [network_get_peers](#) (uint8_t *out_count)
Get a read-only pointer to the active peer array.
- [network_t](#) * [network_get_network](#) (void)
Get a mutable pointer to the full network state.
- uint8_t [network_fill_peer_ids](#) (uint16_t *out_ids, uint8_t max_count)
Copy active peer IDs into a caller-supplied array.
- void [network_update_peers_from_sync](#) (uint16_t master_id, const uint16_t *peer_ids, uint8_t peer_count)
Rebuild the active peer list from a received SYNC message.
- int8_t [network_get_peer_index](#) (uint16_t id)
Look up the array index of a peer by ID.
- void [network_set_master](#) (uint16_t id)
Set the current master node ID.
- uint16_t [network_get_master](#) (void)
Get the current master node ID.
- bool [network_is_master](#) (void)
Check whether this device is currently the master.
- uint16_t [network_get_ownid](#) (void)
Get this device's own 16-bit node ID.
- uint8_t [network_get_count](#) (void)

- Get the current active peer count.*

 - void `network_set_acknowledged` (bool s)

Set the network acknowledgement flag.
- bool `network_is_acknowledged` (void)

Check whether the network has been acknowledged.
- void `network_set_self_pos` (const float pos[3])

Update the stored 3D position of this device.
- uint16_t `network_get_highest_uncertainty` (void)

Select the peer with the highest ranging uncertainty.
- void `network_reset_measurements` (void)

Reset the per-round measurement block and invalidate all distances.
- void `network_store_poll` (const `msg_poll_t` *msg, const `uwb_rx_meas_t` *rx, const bool antenna_unreliable)

Store an incoming POLL message and its RX metadata.
- void `network_store_resp_tx` (uint64_t ts)

Store the RESPONSE TX timestamp.
- void `network_store_final` (const `msg_final_t` *msg, const `uwb_rx_meas_t` *rx, const bool antenna_unreliable)

Store an incoming FINAL message and its RX metadata.
- bool `network_store_passive` (uint8_t index, const `msg_passive_t` *msg, const `uwb_rx_meas_t` *rx, const uint16_t device_id, const bool antenna_unreliable)

Store an incoming PASSIVE message at a given slot index.
- const `measurements_t` * `network_get_measurements` (void)

Get a read-only pointer to the current round's measurement block.
- uint8_t `network_get_passive_count` (void)

Get the number of PASSIVE messages received this round.
- void `network_set_expected_seq_num` (uint8_t seq_num)

Set the expected sequence number for the next exchange.
- uint8_t `network_get_expected_seq_num` (void)

Get the expected sequence number for the current exchange.
- `node_peer_state_t` * `network_get_peer_state` (uint16_t owner_id, uint16_t peer_id)

Get the ranging state between two active nodes.
- void `network_set_distance` (uint16_t a, uint16_t b, uint16_t dist_scaled)

Store a scaled distance measurement between two nodes.
- uint16_t `network_get_distance` (uint16_t a, uint16_t b)

Retrieve the stored scaled distance between two nodes.
- void `network_update_certainty` (uint16_t a, uint16_t b, uint8_t certainty)

Set the certainty counter for a node pair to a specific value.
- void `network_bump_certainty` (uint16_t a, uint16_t b)

Increment the certainty counter for a node pair by one.
- void `network_reset_certainty` (uint16_t a, uint16_t b)

Reset the certainty counter for a node pair to zero.
- uint8_t `network_get_certainty` (uint16_t a, uint16_t b)

Get the certainty counter for a node pair.
- void `network_print_certainty` (void)

Print certainty and distance state for all known node pairs.
- void `network_print_positions` (void)

Print the current 3D position of self and all active peers.

8.51.1 Detailed Description

Implementation of the UWB network state management API.

The active peer array is a per-round/network-snapshot view. Its indices are intentionally unstable: peers may be removed with swap-with-last packing or rebuilt from SYNC. Code that needs persistent identity must use the 16-bit node ID, never a saved peer index or a saved [node_t](#) pointer.

Distance state is also treated as a per-round snapshot. The UWB exchange calls [network_reset_measurements\(\)](#) at the start of each round, which clears the temporary exchange block and invalidates all stored distances. The distance-calculation stage then writes only the ranges observed in that round. SHARE and EKF run afterwards and therefore see only current-round distances.

8.51.2 Macro Definition Documentation

8.51.2.1 NETWORK_DISTANCE_INVALID

```
#define NETWORK_DISTANCE_INVALID (0xFFFFu)
```

8.51.3 Function Documentation

8.51.3.1 find_peer()

```
node_t * find_peer (
    uint16_t id)
```

Find an active peer node by its 16-bit ID.

Parameters

<i>id</i>	Node ID to search for.
-----------	------------------------

Returns

Pointer to the matching [node_t](#), or NULL if not found.

8.51.3.2 network_add_peer()

```
void network_add_peer (
    uint16_t id)
```

Add a peer to the active peer list.

Add a peer to the network peer list.

No-op if the ID is zero, already present, or the peer table is full.

Parameters

<i>id</i>	16-bit node ID to add.
-----------	------------------------

Add a peer to the active peer list.

Does nothing if the peer is already present, if *id* equals *own_id*, or if the list is full ([NETWORK_MAX_PEERS](#) reached).

Parameters

<i>id</i>	Network ID of the peer to add.
-----------	--------------------------------

8.51.3.3 network_bump_certainty()

```
void network_bump_certainty (
    uint16_t a,
```

```
uint16_t b)
```

Increment the certainty counter for a node pair by one.
Saturates at 255. Creates the peer slot if it does not yet exist.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

8.51.3.4 network_fill_peer_ids()

```
uint8_t network_fill_peer_ids (
    uint16_t * out_ids,
    uint8_t max_count)
```

Copy active peer IDs into a caller-supplied array.
Skips any entry whose ID matches self. Stops when `max_count` is reached.

Parameters

<i>out</i>	<i>out_ids</i>	Destination array.
	<i>max_count</i>	Maximum number of IDs to write.

Returns

Number of IDs written.

Skips `own_id`. Stops when `max_count` entries have been written.

Parameters

<i>out</i>	<i>out_ids</i>	Buffer to receive the peer IDs.
	<i>max_count</i>	Maximum number of IDs to write.

Returns

Number of IDs written.

8.51.3.5 network_get_certainty()

```
uint8_t network_get_certainty (
    uint16_t a,
    uint16_t b)
```

Get the certainty counter for a node pair.
Returns 0 if the slot does not exist or the distance is invalid.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

Returns

Certainty counter value, or 0 if unavailable.

8.51.3.6 network_get_count()

```
uint8_t network_get_count (
    void )
```

Get the current active peer count.
Get the current number of known peers.

Returns

Number of peers in the active peer list.

Get the current active peer count.

Returns

Peer count (0 – [NETWORK_MAX_PEERS](#)).

8.51.3.7 network_get_distance()

```
uint16_t network_get_distance (
    uint16_t a,
    uint16_t b)
```

Retrieve the stored scaled distance between two nodes.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

Returns

Stored scaled distance, or `NETWORK_DISTANCE_INVALID` if unavailable.

8.51.3.8 network_get_expected_seq_num()

```
uint8_t network_get_expected_seq_num (
    void )
```

Get the expected sequence number for the current exchange.
Get the expected sequence number for the next incoming frame.

Returns

Expected sequence number.

Get the expected sequence number for the current exchange.

Returns

Current expected sequence number.

8.51.3.9 network_get_highest_uncertainty()

```
uint16_t network_get_highest_uncertainty (
    void )
```

Select the peer with the highest ranging uncertainty.

Return the ID of the peer with the highest position uncertainty.

Node `0xA262` is treated as a priority target: any node other than `0xA262` will return it immediately when it is present in the peer list. For `0xA262` itself, or when the priority node is absent, the peer with the lowest `certainty` counter is returned. Ties are broken randomly. A peer with no certainty record is returned immediately (certainty unknown).

Returns

ID of the selected peer, or 0 if no peers are available.

Select the peer with the highest ranging uncertainty.

Used by the DS-TWR scheduler to select the next ranging target. Skips own_id.

Returns

Peer ID with highest uncertainty, or 0 if no peers are known.

8.51.3.10 network_get_master()

```
uint16_t network_get_master (  
    void )
```

Get the current master node ID.

Get the current master's network ID.

Returns

16-bit node ID of the master.

Get the current master node ID.

Returns

Master ID, or 0 if no master is set.

8.51.3.11 network_get_measurements()

```
const measurements_t * network_get_measurements (  
    void )
```

Get a read-only pointer to the current round's measurement block.

Returns

Pointer to the internal `measurements_t` struct.

8.51.3.12 network_get_network()

```
network_t * network_get_network (  
    void )
```

Get a mutable pointer to the full network state.

Get a read-write pointer to the full network state.

Returns

Pointer to the internal `network_t` struct.

Get a mutable pointer to the full network state.

Prefer the typed accessors where possible; use this only when bulk access to the internal state is unavoidable.

Returns

Pointer to the static `network_t` instance.

8.51.3.13 network_get_ownid()

```
uint16_t network_get_ownid (  
    void )
```

Get this device's own 16-bit node ID.

Get this device's own network ID.

Returns

Own node ID.

Get this device's own 16-bit node ID.

Returns

Own ID as set by [network_init](#).

8.51.3.14 network_get_passive_count()

```
uint8_t network_get_passive_count (
    void )
```

Get the number of PASSIVE messages received this round.

Returns

Count of stored passive entries.

8.51.3.15 network_get_peer_index()

```
int8_t network_get_peer_index (
    uint16_t id)
```

Look up the array index of a peer by ID.

Return the peer index for a given network ID.

Parameters

<i>id</i>	Node ID to find.
-----------	------------------

Returns

Zero-based index into the peer array, or -1 if not found.

Look up the array index of a peer by ID.

Parameters

<i>id</i>	Network ID to look up.
-----------	------------------------

Returns

Zero-based index into [network_t.peers](#), or -1 if not found.

8.51.3.16 network_get_peer_state()

```
node_peer_state_t * network_get_peer_state (
    uint16_t owner_id,
    uint16_t peer_id)
```

Get the ranging state between two active nodes.

Returns NULL if either ID is unknown, the slot does not exist, or the stored distance is invalid (i.e. not measured this round).

Parameters

<i>owner↔ _id</i>	ID of the node that owns the measurement.
<i>peer_id</i>	ID of the peer the measurement is against.

Returns

Pointer to the `node_peer_state_t` slot, or NULL.

8.51.3.17 network_get_peers()

```
const node_t * network_get_peers (
    uint8_t * out_count)
```

Get a read-only pointer to the active peer array.

Get a read-only pointer to the peer array.

Parameters

<i>out</i>	<i>out_count</i>	Filled with the current peer count. May be NULL.
------------	------------------	--

Returns

Pointer to the internal peer array.

Get a read-only pointer to the active peer array.

Parameters

<i>out</i>	<i>out_count</i>	Set to the current number of valid peers.
------------	------------------	---

Returns

Pointer to the internal `node_t` array (valid until next peer-list mutation).

8.51.3.18 network_init()

```
void network_init (
    uint16_t own_id)
```

Initialise the network state for this device.

Clears all peer and measurement data and sets the local node ID.

Parameters

<i>own↔ _id</i>	16-bit short address of this device.
---------------------	--------------------------------------

Zeroes the entire `network_t` and sets the own ID. Must be called once before any other network function.

Parameters

<i>own↔ _id</i>	This device's 16-bit network ID.
---------------------	----------------------------------

8.51.3.19 network_is_acknowledged()

```
bool network_is_acknowledged (
    void )
```

Check whether the network has been acknowledged.
Query whether this device is acknowledged by the master.

Returns

Current value of the acknowledgement flag.

Check whether the network has been acknowledged.

Returns

true if acknowledged, false otherwise.

8.51.3.20 network_is_master()

```
bool network_is_master (
    void )
```

Check whether this device is currently the master.
Check whether this device is the network master.

Returns

true if own ID matches the stored master ID.

Check whether this device is currently the master.

Returns

true if master_id == own_id, false otherwise.

8.51.3.21 network_print_certainty()

```
void network_print_certainty (
    void )
```

Print certainty and distance state for all known node pairs.
Outputs one line per peer slot for self and each active peer via `mprintf`.

8.51.3.22 network_print_positions()

```
void network_print_positions (
    void )
```

Print the current 3D position of self and all active peers.
Outputs one [POS] line per node via `mprintf`.

8.51.3.23 network_remove_peer()

```
void network_remove_peer (
    uint16_t id)
```

Remove a peer from the active peer list.
Remove a peer from the network peer list.
Uses swap-with-last packing to keep the array contiguous. Any previously saved peer index or `node_t` pointer becomes invalid after this call. Also invalidates all distance measurements that reference the removed peer.

Parameters

<i>id</i>	16-bit node ID to remove.
-----------	---------------------------

Remove a peer from the active peer list.

Uses swap-with-last to keep the array packed. Does nothing if `id` is not found.

Parameters

<i>id</i>	Network ID of the peer to remove.
-----------	-----------------------------------

8.51.3.24 `network_reset_certainty()`

```
void network_reset_certainty (
    uint16_t a,
    uint16_t b)
```

Reset the certainty counter for a node pair to zero.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

8.51.3.25 `network_reset_measurements()`

```
void network_reset_measurements (
    void )
```

Reset the per-round measurement block and invalidate all distances.

Called at the start of each ranging round. Certainty counters are preserved for target scheduling; only distances are cleared.

8.51.3.26 `network_set_acknowledged()`

```
void network_set_acknowledged (
    bool state)
```

Set the network acknowledgement flag.

Set or clear the acknowledged flag.

Parameters

<i>s</i>	New acknowledgement state.
----------	----------------------------

Set the network acknowledgement flag.

Set to `true` when `own_id` appears in a master SYNC peer list.

Parameters

<i>state</i>	New acknowledged state.
--------------	-------------------------

8.51.3.27 `network_set_distance()`

```
void network_set_distance (
    uint16_t a,
    uint16_t b,
    uint16_t dist_scaled)
```

Store a scaled distance measurement between two nodes.

Creates the peer slot if it does not yet exist.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.
<i>dist_scaled</i>	Scaled distance value to store.

8.51.3.28 network_set_expected_seq_num()

```
void network_set_expected_seq_num (
    uint8_t seq_num)
```

Set the expected sequence number for the next exchange.
Set the expected sequence number for the next incoming frame.

Parameters

<i>seq_num</i>	Expected sequence number.
----------------	---------------------------

Set the expected sequence number for the next exchange.

Parameters

<i>seq_num</i>	Expected sequence number.
----------------	---------------------------

8.51.3.29 network_set_master()

```
void network_set_master (
    uint16_t id)
```

Set the current master node ID.
Set the current network master.
If the master ID equals own ID, self is removed from the peer list to prevent self-ranging.

Parameters

<i>id</i>	16-bit node ID of the master.
-----------	-------------------------------

Set the current master node ID.
Pass `own_id` to make this device the master.

Parameters

<i>id</i>	Network ID of the new master.
-----------	-------------------------------

8.51.3.30 network_set_self_pos()

```
void network_set_self_pos (
    const float pos[3])
```

Update the stored 3D position of this device.
Update this device's own position estimate and uncertainty.

Parameters

<i>pos</i>	Array of three floats [x, y, z]. No-op if NULL.
------------	---

Update the stored 3D position of this device.

Parameters

<i>pos</i>	3-element array [x, y, z] in metres.
<i>uncertainty</i>	EKF uncertainty score to store.

8.51.3.31 network_store_final()

```
void network_store_final (
    const msg_final_t * msg,
    const uwb_rx_meas_t * rx,
    const bool antenna_unreliable)
```

Store an incoming FINAL message and its RX metadata.

Parameters

<i>msg</i>	Decoded FINAL message. No-op if NULL.
<i>rx</i>	RX measurement metadata. No-op if NULL.
<i>antenna_unreliable</i>	true if the receive antenna quality check failed.

8.51.3.32 network_store_passive()

```
bool network_store_passive (
    uint8_t index,
    const msg_passive_t * msg,
    const uwb_rx_meas_t * rx,
    const uint16_t device_id,
    const bool antenna_unreliable)
```

Store an incoming PASSIVE message at a given slot index.

Parameters

<i>index</i>	Slot index (0-based, max NETWORK_MAX_PEERS - 2).
<i>msg</i>	Decoded PASSIVE message. Returns false if NULL.
<i>rx</i>	RX measurement metadata. Returns false if NULL.
<i>device_id</i>	Node ID of the passive sender.
<i>antenna_unreliable</i>	true if the receive antenna quality check failed.

Returns

true on success, false if inputs are invalid or index is out of range.

8.51.3.33 network_store_poll()

```
void network_store_poll (
    const msg_poll_t * msg,
    const uwb_rx_meas_t * rx,
    const bool antenna_unreliable)
```

Store an incoming POLL message and its RX metadata.

Parameters

<i>msg</i>	Decoded POLL message. No-op if NULL.
<i>rx</i>	RX measurement metadata. No-op if NULL.
<i>antenna_unreliable</i>	true if the receive antenna quality check failed.

8.51.3.34 network_store_resp_tx()

```
void network_store_resp_tx (
    uint64_t ts)
```

Store the RESPONSE TX timestamp.

Parameters

<i>ts</i>	DWM3000 TX timestamp of the RESPONSE frame.
-----------	---

8.51.3.35 network_update_certainty()

```
void network_update_certainty (
    uint16_t a,
    uint16_t b,
    uint8_t certainty)
```

Set the certainty counter for a node pair to a specific value.

Creates the peer slot if it does not yet exist.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.
<i>certainty</i>	New certainty value.

8.51.3.36 network_update_peers_from_sync()

```
void network_update_peers_from_sync (
    uint16_t master_id,
    const uint16_t * peer_ids,
    uint8_t peer_count)
```

Rebuild the active peer list from a received SYNC message.

Rebuild the peer list from a received SYNC message.

Places *master_id* first, then appends the remaining *peer_ids* in order. Peers no longer present in the SYNC list have their measurements invalidated. All distances are cleared at the end to begin a fresh ranging round.

Parameters

<i>master_id</i>	Node ID of the SYNC sender (placed first in the list).
<i>peer_ids</i>	Array of additional peer IDs advertised in the SYNC.
<i>peer_count</i>	Number of entries in <i>peer_ids</i> .

Rebuild the active peer list from a received SYNC message.

Clears the existing peer list and repopulates it from the master's SYNC payload. The master is inserted as the first entry. Existing position and uncertainty data is preserved for known peers. Peers absent from the SYNC are

considered gone.

Parameters

<i>master_id</i>	ID of the master that sent the SYNC.
<i>peer_ids</i>	Array of peer IDs from the SYNC payload.
<i>peer_count</i>	Number of entries in <i>peer_ids</i> .

8.52 App/UWB_app/uwb_network.h File Reference

Network state management for the UWB ranging system.

```
#include <stdint.h>
#include <stdbool.h>
#include "messages.h"
```

Data Structures

- struct [uwb_rx_meas_t](#)
A single received UWB frame measurement.
- struct [measurements_t](#)
- struct [node_peer_state_t](#)
Identity and position state of one UWB network node.
- struct [node_t](#)
- struct [network_t](#)
Global network state for this device.

Functions

- [node_t * find_peer](#) (uint16_t id)
Find an active peer node by its 16-bit ID.
- void [network_init](#) (uint16_t own_id)
Initialise the network state for this device.
- void [network_add_peer](#) (uint16_t id)
Add a peer to the network peer list.
- void [network_remove_peer](#) (uint16_t id)
Remove a peer from the network peer list.
- const [node_t * network_get_peers](#) (uint8_t *out_count)
Get a read-only pointer to the peer array.
- [network_t * network_get_network](#) (void)
Get a read-write pointer to the full network state.
- uint8_t [network_fill_peer_ids](#) (uint16_t *out_ids, uint8_t max_count)
Copy active peer IDs into a caller-supplied array.
- void [network_update_peers_from_sync](#) (uint16_t master_id, const uint16_t *peer_ids, uint8_t peer_count)
Rebuild the peer list from a received SYNC message.
- int8_t [network_get_peer_index](#) (uint16_t id)
Return the peer index for a given network ID.
- void [network_set_master](#) (uint16_t id)
Set the current network master.
- uint16_t [network_get_master](#) (void)
Get the current master's network ID.
- bool [network_is_master](#) (void)

- Check whether this device is the network master.*

 - uint16_t [network_get_ownid](#) (void)
- Get this device's own network ID.*

 - uint8_t [network_get_count](#) (void)
- Get the current number of known peers.*

 - void [network_set_acknowledged](#) (bool state)
- Set or clear the acknowledged flag.*

 - bool [network_is_acknowledged](#) (void)
- Query whether this device is acknowledged by the master.*

 - void [network_set_self_pos](#) (const float pos[3])
- Update this device's own position estimate and uncertainty.*

 - uint16_t [network_get_highest_uncertainty](#) (void)
- Return the ID of the peer with the highest position uncertainty.*

 - void [network_reset_measurements](#) (void)
- Reset the per-round measurement block and invalidate all distances.*

 - void [network_store_poll](#) (const [msg_poll_t](#) *msg, const [uwb_rx_meas_t](#) *rx, const bool antenna_unreliable)
- Store an incoming POLL message and its RX metadata.*

 - void [network_store_resp_tx](#) (uint64_t ts)
- Store the RESPONSE TX timestamp.*

 - void [network_store_final](#) (const [msg_final_t](#) *msg, const [uwb_rx_meas_t](#) *rx, const bool antenna_unreliable)
- Store an incoming FINAL message and its RX metadata.*

 - bool [network_store_passive](#) (uint8_t index, const [msg_passive_t](#) *msg, const [uwb_rx_meas_t](#) *rx, const uint16_t device_id, const bool antenna_unreliable)
- Store an incoming PASSIVE message at a given slot index.*

 - uint8_t [network_get_passive_count](#) (void)
- Get the number of PASSIVE messages received this round.*

 - void [network_set_expected_seq_num](#) (uint8_t seq_num)
- Set the expected sequence number for the next incoming frame.*

 - uint8_t [network_get_expected_seq_num](#) (void)
- Get the expected sequence number for the next incoming frame.*

 - void [network_set_distance](#) (uint16_t a, uint16_t b, uint16_t dist_scaled)
- Store a scaled distance measurement between two nodes.*

 - uint16_t [network_get_distance](#) (uint16_t a, uint16_t b)
- Retrieve the stored scaled distance between two nodes.*

 - void [network_set_k](#) (uint16_t a, uint16_t b, double k)
- Increment the certainty counter for a node pair by one.*

 - double [network_get_k](#) (uint16_t a, uint16_t b)
- Increment the certainty counter for a node pair by one.*

 - void [network_bump_certainty](#) (uint16_t a, uint16_t b)
- Increment the certainty counter for a node pair by one.*

 - void [network_reset_certainty](#) (uint16_t a, uint16_t b)
- Reset the certainty counter for a node pair to zero.*

 - uint8_t [network_get_certainty](#) (uint16_t a, uint16_t b)
- Get the certainty counter for a node pair.*

 - void [network_update_certainty](#) (uint16_t a, uint16_t b, uint8_t certainty)
- Set the certainty counter for a node pair to a specific value.*

 - void [network_print_certainty](#) (void)
- Print certainty and distance state for all known node pairs.*

 - void [network_print_positions](#) (void)
- Print the current 3D position of self and all active peers.*

 - [node_peer_state_t](#) * [network_get_peer_state](#) (uint16_t owner_id, uint16_t peer_id)
- Get the ranging state between two active nodes.*

8.52.1 Detailed Description

Network state management for the UWB ranging system.

Defines all data types used to represent nodes, timestamps, and measurement results, and exposes the full API for managing the peer list, master election, and per-exchange measurement storage.

8.52.2 Function Documentation

8.52.2.1 find_peer()

```
node_t * find_peer (
    uint16_t id)
```

Find an active peer node by its 16-bit ID.

Parameters

<i>id</i>	Node ID to search for.
-----------	------------------------

Returns

Pointer to the matching `node_t`, or NULL if not found.

8.52.2.2 network_add_peer()

```
void network_add_peer (
    uint16_t id)
```

Add a peer to the network peer list.

Add a peer to the active peer list.

Does nothing if the peer is already present, if `id` equals `own_id`, or if the list is full (`NETWORK_MAX_PEERS` reached).

Parameters

<i>id</i>	Network ID of the peer to add.
-----------	--------------------------------

Add a peer to the network peer list.

No-op if the ID is zero, already present, or the peer table is full.

Parameters

<i>id</i>	16-bit node ID to add.
-----------	------------------------

8.52.2.3 network_bump_certainty()

```
void network_bump_certainty (
    uint16_t a,
    uint16_t b)
```

Increment the certainty counter for a node pair by one.

Saturates at 255. Creates the peer slot if it does not yet exist.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

8.52.2.4 network_fill_peer_ids()

```
uint8_t network_fill_peer_ids (
    uint16_t * out_ids,
    uint8_t max_count)
```

Copy active peer IDs into a caller-supplied array.

Skips own_id. Stops when max_count entries have been written.

Parameters

out	out_ids	Buffer to receive the peer IDs.
	max_count	Maximum number of IDs to write.

Returns

Number of IDs written.

Skips any entry whose ID matches self. Stops when max_count is reached.

Parameters

out	out_ids	Destination array.
	max_count	Maximum number of IDs to write.

Returns

Number of IDs written.

8.52.2.5 network_get_certainty()

```
uint8_t network_get_certainty (
    uint16_t a,
    uint16_t b)
```

Get the certainty counter for a node pair.

Returns 0 if the slot does not exist or the distance is invalid.

Parameters

a	ID of the measurement owner.
b	ID of the peer.

Returns

Certainty counter value, or 0 if unavailable.

8.52.2.6 network_get_count()

```
uint8_t network_get_count (
    void )
```

Get the current number of known peers.

Get the current active peer count.

Returns

Peer count (0 – [NETWORK_MAX_PEERS](#)).

Get the current number of known peers.

Returns

Number of peers in the active peer list.

8.52.2.7 network_get_distance()

```
uint16_t network_get_distance (
    uint16_t a,
    uint16_t b)
```

Retrieve the stored scaled distance between two nodes.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

Returns

Stored scaled distance, or NETWORK_DISTANCE_INVALID if unavailable.

8.52.2.8 network_get_expected_seq_num()

```
uint8_t network_get_expected_seq_num (
    void )
```

Get the expected sequence number for the next incoming frame.

Get the expected sequence number for the current exchange.

Returns

Current expected sequence number.

Get the expected sequence number for the next incoming frame.

Returns

Expected sequence number.

8.52.2.9 network_get_highest_uncertainty()

```
uint16_t network_get_highest_uncertainty (
    void )
```

Return the ID of the peer with the highest position uncertainty.

Select the peer with the highest ranging uncertainty.

Used by the DS-TWR scheduler to select the next ranging target. Skips own_id.

Returns

Peer ID with highest uncertainty, or 0 if no peers are known.

Return the ID of the peer with the highest position uncertainty.

Node 0xA262 is treated as a priority target: any node other than 0xA262 will return it immediately when it is present in the peer list. For 0xA262 itself, or when the priority node is absent, the peer with the lowest `certainty` counter is returned. Ties are broken randomly. A peer with no certainty record is returned immediately (certainty unknown).

Returns

ID of the selected peer, or 0 if no peers are available.

8.52.2.10 `network_get_k()`

```
double network_get_k (
    uint16_t a,
    uint16_t b)
```

8.52.2.11 `network_get_master()`

```
uint16_t network_get_master (
    void )
```

Get the current master's network ID.

Get the current master node ID.

Returns

Master ID, or 0 if no master is set.

Get the current master's network ID.

Returns

16-bit node ID of the master.

8.52.2.12 `network_get_network()`

```
network_t * network_get_network (
    void )
```

Get a read-write pointer to the full network state.

Get a mutable pointer to the full network state.

Prefer the typed accessors where possible; use this only when bulk access to the internal state is unavoidable.

Returns

Pointer to the static `network_t` instance.

Get a read-write pointer to the full network state.

Returns

Pointer to the internal `network_t` struct.

8.52.2.13 `network_get_ownid()`

```
uint16_t network_get_ownid (
    void )
```

Get this device's own network ID.

Get this device's own 16-bit node ID.

Returns

Own ID as set by `network_init`.

Get this device's own network ID.

Returns

Own node ID.

8.52.2.14 `network_get_passive_count()`

```
uint8_t network_get_passive_count (
    void )
```

Get the number of PASSIVE messages received this round.

Returns

Count of stored passive entries.

8.52.2.15 network_get_peer_index()

```
int8_t network_get_peer_index (
    uint16_t id)
```

Return the peer index for a given network ID.
Look up the array index of a peer by ID.

Parameters

<i>id</i>	Network ID to look up.
-----------	------------------------

Returns

Zero-based index into `network_t.peers`, or -1 if not found.

Return the peer index for a given network ID.

Parameters

<i>id</i>	Node ID to find.
-----------	------------------

Returns

Zero-based index into the peer array, or -1 if not found.

8.52.2.16 network_get_peer_state()

```
node_peer_state_t * network_get_peer_state (
    uint16_t owner_id,
    uint16_t peer_id)
```

Get the ranging state between two active nodes.

Returns NULL if either ID is unknown, the slot does not exist, or the stored distance is invalid (i.e. not measured this round).

Parameters

<i>owner_id</i>	ID of the node that owns the measurement.
<i>peer_id</i>	ID of the peer the measurement is against.

Returns

Pointer to the `node_peer_state_t` slot, or NULL.

8.52.2.17 network_get_peers()

```
const node_t * network_get_peers (
    uint8_t * out_count)
```

Get a read-only pointer to the peer array.

Get a read-only pointer to the active peer array.

Parameters

<code>out</code>	<code>out_count</code>	Set to the current number of valid peers.
------------------	------------------------	---

Returns

Pointer to the internal [node_t](#) array (valid until next peer-list mutation).

Get a read-only pointer to the peer array.

Parameters

<code>out</code>	<code>out_count</code>	Filled with the current peer count. May be NULL.
------------------	------------------------	--

Returns

Pointer to the internal peer array.

8.52.2.18 network_init()

```
void network_init (
    uint16_t own_id)
```

Initialise the network state for this device.

Zeroes the entire [network_t](#) and sets the own ID. Must be called once before any other network function.

Parameters

<code>own↔ _id</code>	This device's 16-bit network ID.
---------------------------	----------------------------------

Clears all peer and measurement data and sets the local node ID.

Parameters

<code>own↔ _id</code>	16-bit short address of this device.
---------------------------	--------------------------------------

8.52.2.19 network_is_acknowledged()

```
bool network_is_acknowledged (
    void )
```

Query whether this device is acknowledged by the master.

Check whether the network has been acknowledged.

Returns

`true` if acknowledged, `false` otherwise.

Query whether this device is acknowledged by the master.

Returns

Current value of the acknowledgement flag.

8.52.2.20 network_is_master()

```
bool network_is_master (
    void )
```

Check whether this device is the network master.
Check whether this device is currently the master.

Returns

true if master_id == own_id, false otherwise.

Check whether this device is the network master.

Returns

true if own ID matches the stored master ID.

8.52.2.21 network_print_certainty()

```
void network_print_certainty (
    void )
```

Print certainty and distance state for all known node pairs.
Outputs one line per peer slot for self and each active peer via `mprintf`.

8.52.2.22 network_print_positions()

```
void network_print_positions (
    void )
```

Print the current 3D position of self and all active peers.
Outputs one [POS] line per node via `mprintf`.

8.52.2.23 network_remove_peer()

```
void network_remove_peer (
    uint16_t id)
```

Remove a peer from the network peer list.
Remove a peer from the active peer list.
Uses swap-with-last to keep the array packed. Does nothing if `id` is not found.

Parameters

<i>id</i>	Network ID of the peer to remove.
-----------	-----------------------------------

Remove a peer from the network peer list.
Uses swap-with-last packing to keep the array contiguous. Any previously saved peer index or `node_t` pointer becomes invalid after this call. Also invalidates all distance measurements that reference the removed peer.

Parameters

<i>id</i>	16-bit node ID to remove.
-----------	---------------------------

8.52.2.24 network_reset_certainty()

```
void network_reset_certainty (
    uint16_t a,
    uint16_t b)
```

Reset the certainty counter for a node pair to zero.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.

8.52.2.25 network_reset_measurements()

```
void network_reset_measurements (
    void )
```

Reset the per-round measurement block and invalidate all distances.

Called at the start of each ranging round. Certainty counters are preserved for target scheduling; only distances are cleared.

8.52.2.26 network_set_acknowledged()

```
void network_set_acknowledged (
    bool s)
```

Set or clear the acknowledged flag.

Set the network acknowledgement flag.

Set to `true` when `own_id` appears in a master SYNC peer list.

Parameters

<i>state</i>	New acknowledged state.
--------------	-------------------------

Set or clear the acknowledged flag.

Parameters

<i>s</i>	New acknowledgement state.
----------	----------------------------

8.52.2.27 network_set_distance()

```
void network_set_distance (
    uint16_t a,
    uint16_t b,
    uint16_t dist_scaled)
```

Store a scaled distance measurement between two nodes.

Creates the peer slot if it does not yet exist.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.
<i>dist_scaled</i>	Scaled distance value to store.

8.52.2.28 network_set_expected_seq_num()

```
void network_set_expected_seq_num (
    uint8_t seq_num)
```

Set the expected sequence number for the next incoming frame.

Set the expected sequence number for the next exchange.

Parameters

<i>seq_num</i>	Expected sequence number.
----------------	---------------------------

Set the expected sequence number for the next incoming frame.

Parameters

<i>seq_num</i>	Expected sequence number.
----------------	---------------------------

8.52.2.29 network_set_k()

```
void network_set_k (
    uint16_t a,
    uint16_t b,
    double k)
```

8.52.2.30 network_set_master()

```
void network_set_master (
    uint16_t id)
```

Set the current network master.

Set the current master node ID.

Pass `own_id` to make this device the master.

Parameters

<i>id</i>	Network ID of the new master.
-----------	-------------------------------

Set the current network master.

If the master ID equals own ID, self is removed from the peer list to prevent self-ranging.

Parameters

<i>id</i>	16-bit node ID of the master.
-----------	-------------------------------

8.52.2.31 network_set_self_pos()

```
void network_set_self_pos (
    const float pos[3])
```

Update this device's own position estimate and uncertainty.

Update the stored 3D position of this device.

Parameters

<i>pos</i>	3-element array [x, y, z] in metres.
<i>uncertainty</i>	EKF uncertainty score to store.

Update this device's own position estimate and uncertainty.

Parameters

<i>pos</i>	Array of three floats [x, y, z]. No-op if NULL.
------------	---

8.52.2.32 network_store_final()

```
void network_store_final (
    const msg_final_t * msg,
    const uwb_rx_meas_t * rx,
    const bool antenna_unreliable)
```

Store an incoming FINAL message and its RX metadata.

Parameters

<i>msg</i>	Decoded FINAL message. No-op if NULL.
<i>rx</i>	RX measurement metadata. No-op if NULL.
<i>antenna_unreliable</i>	true if the receive antenna quality check failed.

8.52.2.33 network_store_passive()

```
bool network_store_passive (
    uint8_t index,
    const msg_passive_t * msg,
    const uwb_rx_meas_t * rx,
    const uint16_t device_id,
    const bool antenna_unreliable)
```

Store an incoming PASSIVE message at a given slot index.

Parameters

<i>index</i>	Slot index (0-based, max NETWORK_MAX_PEERS - 2).
<i>msg</i>	Decoded PASSIVE message. Returns false if NULL.
<i>rx</i>	RX measurement metadata. Returns false if NULL.
<i>device_id</i>	Node ID of the passive sender.
<i>antenna_unreliable</i>	true if the receive antenna quality check failed.

Returns

true on success, false if inputs are invalid or index is out of range.

8.52.2.34 network_store_poll()

```
void network_store_poll (
    const msg_poll_t * msg,
    const uwb_rx_meas_t * rx,
    const bool antenna_unreliable)
```

Store an incoming POLL message and its RX metadata.

Parameters

<i>msg</i>	Decoded POLL message. No-op if NULL.
<i>rx</i>	RX measurement metadata. No-op if NULL.
<i>antenna_unreliable</i>	true if the receive antenna quality check failed.

8.52.2.35 network_store_resp_tx()

```
void network_store_resp_tx (
    uint64_t ts)
```

Store the RESPONSE TX timestamp.

Parameters

<i>ts</i>	DWM3000 TX timestamp of the RESPONSE frame.
-----------	---

8.52.2.36 network_update_certainty()

```
void network_update_certainty (
    uint16_t a,
    uint16_t b,
    uint8_t certainty)
```

Set the certainty counter for a node pair to a specific value.
Creates the peer slot if it does not yet exist.

Parameters

<i>a</i>	ID of the measurement owner.
<i>b</i>	ID of the peer.
<i>certainty</i>	New certainty value.

8.52.2.37 network_update_peers_from_sync()

```
void network_update_peers_from_sync (
    uint16_t master_id,
    const uint16_t * peer_ids,
    uint8_t peer_count)
```

Rebuild the peer list from a received SYNC message.

Rebuild the active peer list from a received SYNC message.

Clears the existing peer list and repopulates it from the master's SYNC payload. The master is inserted as the first entry. Existing position and uncertainty data is preserved for known peers. Peers absent from the SYNC are considered gone.

Parameters

<i>master_id</i>	ID of the master that sent the SYNC.
<i>peer_ids</i>	Array of peer IDs from the SYNC payload.
<i>peer_count</i>	Number of entries in <i>peer_ids</i> .

Rebuild the peer list from a received SYNC message.

Places *master_id* first, then appends the remaining *peer_ids* in order. Peers no longer present in the SYNC list have their measurements invalidated. All distances are cleared at the end to begin a fresh ranging round.

Parameters

<i>master_id</i>	Node ID of the SYNC sender (placed first in the list).
<i>peer_ids</i>	Array of additional peer IDs advertised in the SYNC.

<code>peer_count</code>	Number of entries in <code>peer_ids</code> .
-------------------------	--

8.53 uwb_network.h

[Go to the documentation of this file.](#)

```

00001
00009
00010 #pragma once
00011 #include <stdint.h>
00012 #include <stdbool.h>
00013 #include "messages.h"
00014
00015 #ifdef __cplusplus
00016 extern "C" {
00017 #endif
00018
00019 /* -----
00020  * Positioning and ranging types
00021  * ----- */
00022
00030 typedef struct {
00031     uint64_t ts;
00032     int16_t pwr_diff_q8;
00033 } uwb_rx_meas_t;
00034
00035
00036 /* -----
00037  * Exchange measurement block
00038  * ----- */
00039
00040 typedef struct {
00041     //place to store decoded mesesches of this exchange, for master only
00042     msg_poll_t poll;
00043     uwb_rx_meas_t poll_rx;
00044     bool poll_antenna_unreliable;
00045
00046     msg_response_t resp;
00047     uint64_t resp_tx;
00048
00049     msg_final_t final;
00050     uwb_rx_meas_t final_rx;
00051     bool final_antenna_unreliable;
00052
00053     uint8_t passive_count;
00054     msg_passive_t passive[NETWORK_MAX_PEERS - 2];
00055     uwb_rx_meas_t passive_rx[NETWORK_MAX_PEERS - 2];
00056     uint16_t passive_device_id[NETWORK_MAX_PEERS - 2];
00057     bool passive_antenna_unreliable[NETWORK_MAX_PEERS - 2];
00058
00059 } measurements_t;
00060
00061 /* -----
00062  * Network peer
00063  * ----- */
00064
00072
00073 typedef struct {
00074     uint16_t peer_id;
00075     uint16_t distance_scaled;
00076     uint8_t certainty;
00077 } node_peer_state_t;
00078
00079 typedef struct {
00080     uint16_t id;
00081     float pos[3];
00082     uint8_t imu_vel_vert;
00083     uint8_t imu_vel_horiz;
00084     node_peer_state_t peers[NETWORK_MAX_PEERS];
00085
00086 } node_t;
00087
00088
00089 /* -----
00090  * Network state
00091  * ----- */
00092
00099 typedef struct {
00100     node_t peers[NETWORK_MAX_PEERS];
00101     uint8_t expected_seq_num;
00102     uint8_t count;
00103     uint16_t master_id;
00104     node_t self;
00105     bool acknowledged;
00106     measurements_t measurements;

```

```

00111 } network_t;
00112
00113
00114 node_t *find_peer(uint16_t id);
00115
00116 /* -----
00117  * Network API -- initialisation
00118  * ----- */
00119
00120 void network_init(uint16_t own_id);
00121
00122 /* -----
00123  * Network API -- peer management
00124  * ----- */
00125
00126 void network_add_peer(uint16_t id);
00127
00128 void network_remove_peer(uint16_t id);
00129
00130 const node_t *network_get_peers(uint8_t *out_count);
00131
00132 network_t *network_get_network(void);
00133
00134 uint8_t network_fill_peer_ids(uint16_t *out_ids, uint8_t max_count);
00135
00136 void network_update_peers_from_sync(uint16_t master_id,
00137                                     const uint16_t *peer_ids,
00138                                     uint8_t peer_count);
00139
00140 int8_t network_get_peer_index(uint16_t id);
00141
00142 /* -----
00143  * Network API -- master / self
00144  * ----- */
00145
00146 void network_set_master(uint16_t id);
00147
00148 uint16_t network_get_master(void);
00149
00150 bool network_is_master(void);
00151
00152 uint16_t network_get_ownid(void);
00153
00154 uint8_t network_get_count(void);
00155
00156 void network_set_acknowledged(bool state);
00157
00158 bool network_is_acknowledged(void);
00159
00160 /* -----
00161  * Network API -- position and uncertainty
00162  * ----- */
00163
00164 void network_set_self_pos(const float pos[3]);
00165
00166 uint16_t network_get_highest_uncertainty(void);
00167
00168 /* -----
00169  * Network API -- measurements
00170  * ----- */
00171
00172 /* -----
00173  * Network API -- sequence number
00174  * ----- */
00175
00176 void network_reset_measurements(void);
00177 void network_store_poll(const msg_poll_t *msg, const uwb_rx_meas_t *rx, const bool
00178 antenna_unreliable);
00179 void network_store_resp_tx(uint64_t ts);
00180 void network_store_final(const msg_final_t *msg, const uwb_rx_meas_t *rx, const bool
00181 antenna_unreliable);
00182 bool network_store_passive(uint8_t index, const msg_passive_t *msg,
00183                             const uwb_rx_meas_t *rx, const uint16_t device_id, const bool
00184 antenna_unreliable);
00185 uint8_t network_get_passive_count(void);
00186
00187 void network_set_expected_seq_num(uint8_t seq_num);
00188
00189 uint8_t network_get_expected_seq_num(void);
00190
00191 /* -----
00192  * Per-peer ranging state (distance, k, certainty)
00193  * ----- */

```

```
00323 void    network_set_distance    (uint16_t a, uint16_t b, uint16_t dist_scaled);
00324 uint16_t  network_get_distance    (uint16_t a, uint16_t b);
00325 void    network_set_k            (uint16_t a, uint16_t b, double k);
00326 double    network_get_k            (uint16_t a, uint16_t b);
00327 void    network_bump_certainty    (uint16_t a, uint16_t b);
00328 void    network_reset_certainty   (uint16_t a, uint16_t b);
00329 uint8_t   network_get_certainty    (uint16_t a, uint16_t b);
00330 void    network_update_certainty   (uint16_t a, uint16_t b, uint8_t certainty);
00331 void    network_print_certainty    (void);
00332 void    network_print_positions    (void);
00333
00334 node_peer_state_t *network_get_peer_state(uint16_t owner_id, uint16_t peer_id);
00335 #ifdef __cplusplus
00336 }
00337 #endif
```

Index

- accel_raw
 - imu_sample_t, [24](#)
- accuracy
 - msg_share_t, [33](#)
- acknowledged
 - network_t, [38](#)
- ADC_FULL_SCALE
 - power.c, [137](#)
- ALL_ID
 - Timing constants, [10](#)
- answer_rx
 - ss_twr_t, [43](#)
 - tdoa_twr_t, [44](#)
- answer_rx_unreliable
 - second_order_t, [42](#)
 - third_order_t, [45](#)
- answer_tx
 - ss_twr_t, [43](#)
 - tdoa_twr_t, [44](#)
- App Directory Reference, [13](#)
- App/Accelerometer Directory Reference, [13](#)
- App/Accelerometer/imu.c, [51](#)
- App/Accelerometer/imu.h, [54](#), [58](#)
- App/Accelerometer/imu_setup.h, [58](#), [61](#)
- App/Accelerometer/lsm6dsv.c, [63](#)
- App/Accelerometer/lsm6dsv.h, [64](#), [66](#)
- App/Accelerometer/lsm6dsv_self_test.c, [66](#)
- App/Accelerometer/lsm6dsv_ST_example.h, [68](#)
- App/Accelerometer/read_data_polling.c, [69](#)
- App/Blink Directory Reference, [13](#)
- App/Blink/blink.c, [69](#)
- App/Blink/blink.h, [70](#)
- App/Calculations Directory Reference, [14](#)
- App/Calculations/calibrate.c, [71](#)
- App/Calculations/calibrate.h, [74](#), [79](#)
- App/Calculations/distance.c, [80](#)
- App/Calculations/distance.h, [85](#), [92](#)
- App/Calculations/ekf.c, [95](#)
- App/Calculations/ekf.h, [100](#), [108](#)
- App/DWM Directory Reference, [14](#)
- App/DWM/DWM3000.c, [110](#)
- App/DWM/DWM3000.h, [113](#)
- App/DWM/DWM3000_driver.c, [113](#)
- App/DWM/DWM3000_driver.h, [121](#), [128](#)
- App/DWM/DWM3000_setup.h, [129](#), [131](#)
- App/Generic Directory Reference, [14](#)
- App/Generic/my_print.c, [132](#)
- App/Generic/my_print.h, [134](#), [136](#)
- App/Generic/power.c, [136](#)
- App/Generic/power.h, [137](#), [138](#)
- App/Generic/sleep.c, [139](#)
- App/Generic/sleep.h, [140](#)
- App/Generic/UART.c, [141](#)
- App/Generic/UART.h, [143](#), [145](#)
- App/UWB_app Directory Reference, [14](#)
- App/UWB_app/messages.c, [146](#)
- App/UWB_app/messages.h, [148](#), [152](#)
- App/UWB_app/uwb_exchange.c, [154](#)
- App/UWB_app/uwb_exchange.h, [159](#), [163](#)
- App/UWB_app/uwb_network.c, [164](#)
- App/UWB_app/uwb_network.h, [177](#), [190](#)
- battery_voltage_V
 - BatteryStatus_t, [15](#)
- BatteryStatus_t, [15](#)
- battery_voltage_V, [15](#)
- charge_complete, [15](#)
- is_charging, [15](#)
- usb_connected, [15](#)
- blink.c
 - StartBlink, [70](#)
- blink.h
 - StartBlink, [70](#)
- boot_config_t, [16](#)
- charge_only, [16](#)
- hints, [16](#)
- valid, [16](#)
- BOOT_TIME
 - lsm6dsv_self_test.c, [67](#)
 - read_data_polling.c, [69](#)
- Both
 - my_print.h, [134](#)
- CALIB_ANTENNA_MIN_DIST_TICKS
 - calibrate.h, [75](#)
- CALIB_ANTENNA_N
 - calibrate.h, [75](#)
- CALIB_ANTENNA_NULL_THRESHOLD_DEG
 - calibrate.h, [75](#)
- CALIB_ANTENNA_SIMILARITY_DEG
 - calibrate.h, [75](#)
- CALIB_ANTENNA_TILT_LIMIT_DEG
 - calibrate.h, [75](#)
- CALIB_RX_TS_A
 - calibrate.h, [76](#)
- calibrate.c
 - calibrate_get_k, [71](#)
 - calibrate_rssi_antenna, [71](#)
 - calibrate_rx_timestamp, [72](#)

- calibrate_set_clock_offset_poll, 73
- calibrate_set_clock_offset_sync, 73
- calibrate_set_pitch, 74
- calibrate_tx_timestamp, 74
- calibrate.h
 - CALIB_ANTENNA_MIN_DIST_TICKS, 75
 - CALIB_ANTENNA_N, 75
 - CALIB_ANTENNA_NULL_THRESHOLD_DEG, 75
 - CALIB_ANTENNA_SIMILARITY_DEG, 75
 - CALIB_ANTENNA_TILT_LIMIT_DEG, 75
 - CALIB_RX_TS_A, 76
 - calibrate_get_k, 76
 - calibrate_rx_timestamp, 76
 - calibrate_set_clock_offset_poll, 77
 - calibrate_set_clock_offset_sync, 78
 - calibrate_set_pitch, 78
 - calibrate_tx_timestamp, 78
 - CFO_RAW_TO_K, 76
 - CFO_RAW_TO_PPM, 76
 - PPM_TO_K, 76
- calibrate_get_k
 - calibrate.c, 71
 - calibrate.h, 76
- calibrate_rssi_antenna
 - calibrate.c, 71
- calibrate_rx_timestamp
 - calibrate.c, 72
 - calibrate.h, 76
- calibrate_set_clock_offset_poll
 - calibrate.c, 73
 - calibrate.h, 77
- calibrate_set_clock_offset_sync
 - calibrate.c, 73
 - calibrate.h, 78
- calibrate_set_pitch
 - calibrate.c, 74
 - calibrate.h, 78
- calibrate_tx_timestamp
 - calibrate.c, 74
 - calibrate.h, 78
- cb_rx_err
 - DWM3000_driver.c, 115
 - DWM3000_driver.h, 123
- cb_rx_ok
 - DWM3000_driver.c, 115
 - DWM3000_driver.h, 123
- cb_rx_to
 - DWM3000_driver.c, 115
 - DWM3000_driver.h, 123
- cb_spi_rdy
 - DWM3000_driver.c, 116
 - DWM3000_driver.h, 124
- cb_tx_done
 - DWM3000_driver.c, 116
 - DWM3000_driver.h, 124
- certainty
 - node_peer_state_t, 39
- CERTAINTY_BASE_DSTWR
 - distance.h, 87
- CERTAINTY_BASE_SSTWR
 - distance.h, 87
- CERTAINTY_BASE_TDOA
 - distance.h, 87
- CERTAINTY_DELTA0_Q8
 - distance.h, 87
- CERTAINTY_KN_Q8
 - distance.h, 87
- CERTAINTY_KT
 - distance.h, 87
- CERTAINTY_LOS_MAX
 - distance.h, 87
- CERTAINTY_LOS_MIN
 - distance.h, 87
- CERTAINTY_TIMING_MAX
 - distance.h, 87
- CERTAINTY_TIMING_MIN
 - distance.h, 87
- CERTAINTY_TREF_US
 - distance.h, 87
- CERTAINTY_UNRELIABLE_PENALTY
 - distance.h, 88
- CFO_RAW_TO_K
 - calibrate.h, 76
- CFO_RAW_TO_PPM
 - calibrate.h, 76
- charge_complete
 - BatteryStatus_t, 15
- charge_only
 - boot_config_t, 16
- CLAMP_PDIFF
 - distance.c, 81
- compute_certainty
 - distance.c, 81
- coop_ekf_t, 16
 - initialised, 17
 - last_tick_ms, 17
 - n_peers, 17
 - P, 17
 - peer_away, 17
 - peer_ids, 17
 - peer_imu_valid, 17
 - peer_seeded, 17
 - Q, 18
 - self_slot, 18
 - stationary_count, 18
 - x, 18
- count
 - imu_fifo_data_t, 23
 - network_t, 38
- data
 - dwm_rx_frame_t, 19
 - dwm_rx_raw_frame_t, 20
 - dwm_tx_frame_t, 21
 - msg_t, 36
 - print_msg_t, 41
- DEEP_SLEEP

- Timing constants, 10
- dev_ctx
 - imu.c, 54
- dist_scale_to_ticks
 - distance.c, 81
 - distance.h, 90
- DIST_SHARE_DIST_MAX_TICKS
 - distance.h, 88
- DIST_SHARE_MAX_M
 - distance.h, 88
- DIST_SHARE_TICKS_PER_LSB
 - distance.h, 88
- dist_ticks_to_scale
 - distance.c, 82
 - distance.h, 90
- distance.c
 - CLAMP_PDIF, 81
 - compute_certainty, 81
 - dist_scale_to_ticks, 81
 - dist_ticks_to_scale, 82
 - distance_calculate, 82
 - PDIF_MAX_DB, 81
 - populate_computation_structs, 83
 - timestamps, 85
 - vel_horiz_to_u8, 83
 - vel_horiz_u8_to_ms, 84
 - vel_vert_to_u8, 84
 - vel_vert_u8_to_ms, 84
- distance.h
 - CERTAINTY_BASE_DSTWR, 87
 - CERTAINTY_BASE_SSTWR, 87
 - CERTAINTY_BASE_TDOA, 87
 - CERTAINTY_DELTA0_Q8, 87
 - CERTAINTY_KN_Q8, 87
 - CERTAINTY_KT, 87
 - CERTAINTY_LOS_MAX, 87
 - CERTAINTY_LOS_MIN, 87
 - CERTAINTY_TIMING_MAX, 87
 - CERTAINTY_TIMING_MIN, 87
 - CERTAINTY_TREF_US, 87
 - CERTAINTY_UNRELIABLE_PENALTY, 88
 - dist_scale_to_ticks, 90
 - DIST_SHARE_DIST_MAX_TICKS, 88
 - DIST_SHARE_MAX_M, 88
 - DIST_SHARE_TICKS_PER_LSB, 88
 - dist_ticks_to_scale, 90
 - distance_calculate, 90
 - DWT_TICK_TO_US, 88
 - MAX_PASSIVE, 88
 - MAX_SECOND_ORDER, 88
 - MAX_THIRD_ORDER, 88
 - MEAS_DSTWR, 89
 - MEAS_NONE, 89
 - MEAS_SSTWR, 89
 - MEAS_TDOA_SSTWR, 89
 - MeasurementType, 89
 - METERS_PER_TICK, 88
 - position_calibrate_timestamp, 91
- SPEED_OF_LIGHT, 88
- UWB_40BIT_MASK, 89
- VEL_HORIZ_MAX_MS, 89
- VEL_HORIZ_MS_PER_LSB, 89
- vel_horiz_to_u8, 91
- vel_horiz_u8_to_ms, 91
- VEL_VERT_MS_PER_LSB, 89
- VEL_VERT_RANGE_MS, 89
- vel_vert_to_u8, 91
- vel_vert_u8_to_ms, 92
- distance_calculate
 - distance.c, 82
 - distance.h, 90
- distance_mm
 - msg_share_t, 33
- distance_scaled
 - node_peer_state_t, 39
- DIVIDER_RATIO
 - power.c, 137
- dw3000_probe_intf
 - DWM3000_driver.c, 120
- DWM3000.c
 - low_battery_check, 112
 - MPRINT_TS, 111
 - StartRangingTask, 112
 - tx_err_watchdog, 112
 - U64_HI, 111
 - U64_LO, 111
- DWM3000.h
 - StartDWM, 113
- DWM3000_driver.c
 - cb_rx_err, 115
 - cb_rx_ok, 115
 - cb_rx_to, 115
 - cb_spi_rdy, 116
 - cb_tx_done, 116
 - dw3000_probe_intf, 120
 - dwm_configure, 116
 - dwm_enter_idle_rc, 116
 - dwm_exit_idle_rc, 117
 - dwm_get_addr, 117
 - dwm_get_ant_delay, 117
 - dwm_init, 117
 - dwm_init_queues, 117
 - dwm_restore_runtime, 117
 - dwm_rx, 118
 - dwm_rx_flush, 118
 - dwm_selftest, 118
 - dwm_sleep, 118
 - dwm_tx, 119
 - dwm_tx_delayed, 119
 - dwm_wakeup, 120
 - hspi1, 120
 - rx_queue, 120
 - ts_buf_to_u64, 120
 - tx_queue, 120
 - wakeup_queue, 120
- DWM3000_driver.h

- cb_rx_err, [123](#)
- cb_rx_ok, [123](#)
- cb_rx_to, [123](#)
- cb_spi_rdy, [124](#)
- cb_tx_done, [124](#)
- dwm_configure, [124](#)
- dwm_enter_idle_rc, [124](#)
- dwm_exit_idle_rc, [124](#)
- dwm_get_addr, [124](#)
- dwm_init, [125](#)
- dwm_rx, [125](#)
- dwm_rx_continuous_sleep, [125](#)
- DWM_RX_ERR, [122](#)
- dwm_rx_event_type_t, [122](#)
- dwm_rx_flush, [125](#)
- DWM_RX_OK, [122](#)
- DWM_RX_TIMEOUT, [122](#)
- dwm_selftest, [125](#)
- dwm_sleep, [126](#)
- dwm_tx, [126](#)
- dwm_tx_continuous, [126](#)
- dwm_tx_continuous_delayed, [126](#)
- dwm_tx_delayed, [126](#)
- DWM_TX_ERROR, [123](#)
- dwm_tx_event_type_t, [122](#)
- DWM_TX_LATE, [123](#)
- DWM_TX_OK, [123](#)
- dwm_wakeup, [127](#)
- MS_TO_DWT_TIME, [122](#)
- ts_buf_to_u64, [127](#)
- TX_DELAY_500US, [122](#)
- US_TO_DWT_TIME, [122](#)
- DWM3000_setup.h
 - DWM_ANT_DELAY, [129](#)
 - DWM_CRC_LEN, [129](#)
 - DWM_IRQ_MASK, [129](#)
 - DWM_MAX_FRAME_LEN, [129](#)
 - DWM_PAN_ID, [130](#)
 - DWM_RX_TIMEOUT_1S, [130](#)
 - DWM_TX_PGDLY, [130](#)
 - DWM_UWB_CHANNEL, [130](#)
 - DWM_UWB_DATA_RATE, [130](#)
 - DWM_UWB_PAC, [130](#)
 - DWM_UWB_PDOA_MODE, [130](#)
 - DWM_UWB_PHR_MODE, [130](#)
 - DWM_UWB_PHR_RATE, [130](#)
 - DWM_UWB_PREAMBLE_LEN, [130](#)
 - DWM_UWB_RX_CODE, [130](#)
 - DWM_UWB_SFD_TO, [130](#)
 - DWM_UWB_SFD_TYPE, [130](#)
 - DWM_UWB_STS_LENGTH, [130](#)
 - DWM_UWB_STS_MODE, [130](#)
 - DWM_UWB_TX_CODE, [131](#)
 - DWT_DEVICE_ID, [131](#)
 - TX_RF_PGcount, [131](#)
 - TX_RF_PGdly, [131](#)
 - TX_RF_Power, [131](#)
- DWM_ANT_DELAY
 - DWM3000_setup.h, [129](#)
- dwm_configure
 - DWM3000_driver.c, [116](#)
 - DWM3000_driver.h, [124](#)
- DWM_CRC_LEN
 - DWM3000_setup.h, [129](#)
- dwm_enter_idle_rc
 - DWM3000_driver.c, [116](#)
 - DWM3000_driver.h, [124](#)
- dwm_exit_idle_rc
 - DWM3000_driver.c, [117](#)
 - DWM3000_driver.h, [124](#)
- dwm_get_addr
 - DWM3000_driver.c, [117](#)
 - DWM3000_driver.h, [124](#)
- dwm_get_ant_delay
 - DWM3000_driver.c, [117](#)
- dwm_init
 - DWM3000_driver.c, [117](#)
 - DWM3000_driver.h, [125](#)
- dwm_init_queues
 - DWM3000_driver.c, [117](#)
- DWM_IRQ_MASK
 - DWM3000_setup.h, [129](#)
- DWM_MAX_FRAME_LEN
 - DWM3000_setup.h, [129](#)
- DWM_PAN_ID
 - DWM3000_setup.h, [130](#)
- dwm_restore_runtime
 - DWM3000_driver.c, [117](#)
- dwm_rx
 - DWM3000_driver.c, [118](#)
 - DWM3000_driver.h, [125](#)
- dwm_rx_continuous_sleep
 - DWM3000_driver.h, [125](#)
- DWM_RX_ERR
 - DWM3000_driver.h, [122](#)
- dwm_rx_event_type_t
 - DWM3000_driver.h, [122](#)
- dwm_rx_flush
 - DWM3000_driver.c, [118](#)
 - DWM3000_driver.h, [125](#)
- dwm_rx_frame_t, [18](#)
 - data, [19](#)
 - fp_q8, [19](#)
 - len, [19](#)
 - rss_i_q8, [19](#)
 - rx_timestamp, [19](#)
 - status, [19](#)
 - type, [19](#)
- DWM_RX_OK
 - DWM3000_driver.h, [122](#)
- dwm_rx_raw_frame_t, [19](#)
 - data, [20](#)
 - len, [20](#)
 - rx_diag, [20](#)
 - status, [20](#)
 - ts_buf, [20](#)

- type, 20
- DWM_RX_TIMEOUT
 - DWM3000_driver.h, 122
- DWM_RX_TIMEOUT_1S
 - DWM3000_setup.h, 130
- dwm_selftest
 - DWM3000_driver.c, 118
 - DWM3000_driver.h, 125
- dwm_sleep
 - DWM3000_driver.c, 118
 - DWM3000_driver.h, 126
- dwm_tx
 - DWM3000_driver.c, 119
 - DWM3000_driver.h, 126
- dwm_tx_continuous
 - DWM3000_driver.h, 126
- dwm_tx_continuous_delayed
 - DWM3000_driver.h, 126
- dwm_tx_delayed
 - DWM3000_driver.c, 119
 - DWM3000_driver.h, 126
- DWM_TX_ERROR
 - DWM3000_driver.h, 123
- dwm_tx_event_type_t
 - DWM3000_driver.h, 122
- dwm_tx_frame_t, 20
 - data, 21
 - len, 21
 - tx_timestamp, 21
- DWM_TX_LATE
 - DWM3000_driver.h, 123
- DWM_TX_OK
 - DWM3000_driver.h, 123
- DWM_TX_PGDLTY
 - DWM3000_setup.h, 130
- DWM_UWB_CHANNEL
 - DWM3000_setup.h, 130
- DWM_UWB_DATA_RATE
 - DWM3000_setup.h, 130
- DWM_UWB_PAC
 - DWM3000_setup.h, 130
- DWM_UWB_PDOA_MODE
 - DWM3000_setup.h, 130
- DWM_UWB_PHR_MODE
 - DWM3000_setup.h, 130
- DWM_UWB_PHR_RATE
 - DWM3000_setup.h, 130
- DWM_UWB_PREAMBLE_LEN
 - DWM3000_setup.h, 130
- DWM_UWB_RX_CODE
 - DWM3000_setup.h, 130
- DWM_UWB_SFD_TO
 - DWM3000_setup.h, 130
- DWM_UWB_SFD_TYPE
 - DWM3000_setup.h, 130
- DWM_UWB_STS_LENGTH
 - DWM3000_setup.h, 130
- DWM_UWB_STS_MODE
 - DWM3000_setup.h, 130
- DWM_UWB_TX_CODE
 - DWM3000_setup.h, 131
- dwm_wakeup
 - DWM3000_driver.c, 120
 - DWM3000_driver.h, 127
- DWT_DEVICE_ID
 - DWM3000_setup.h, 131
- DWT_TICK_TO_US
 - distance.h, 88
- ekf.c
 - EKF_ANCHOR_IDS, 100
 - ekf_anchor_ready, 96
 - ekf_certainty_to_sigma, 96
 - ekf_get_anchor_positions, 97
 - ekf_get_peer_pos, 97
 - ekf_get_phase, 98
 - ekf_get_state, 98
 - ekf_init, 98
 - EKF_INIT_RANDOM, 96
 - ekf_step, 99
 - EKF_TAG_HARDCODED_Z, 96
 - HAL_GetTick, 100
- ekf.h
 - EKF_ANCHOR_IDS, 108
 - EKF_ANCHOR_MIN_STEPS, 101
 - EKF_ANCHOR_P_CONVERGED, 101
 - ekf_anchor_ready, 104
 - ekf_certainty_to_sigma, 104
 - EKF_DT_MAX_S, 102
 - EKF_DT_NOM_S, 102
 - ekf_get_anchor_positions, 105
 - ekf_get_peer_pos, 105
 - ekf_get_phase, 106
 - ekf_get_state, 106
 - EKF_HDG_MIN_DELTA_M, 102
 - EKF_IMU_DEADBAND_MS, 102
 - ekf_init, 106
 - EKF_INIT_P_POS, 102
 - EKF_MAX_PEERS, 102
 - EKF_MAX_STATE, 102
 - EKF_MAX_STEP_M, 102
 - EKF_MAX_UPDATE_M, 102
 - EKF_NUM_ANCHORS, 103
 - EKF_OUTLIER_GATE_NEG, 103
 - EKF_OUTLIER_GATE_POS, 103
 - EKF_P_MAX, 103
 - EKF_PHASE_ANCHOR_SURVEY, 104
 - ekf_phase_t, 104
 - EKF_PHASE_TAG_LOCALIZE, 104
 - EKF_Q_H_FLOOR, 103
 - EKF_Q_H_FLOOR_STATIONARY, 103
 - EKF_Q_H_VEL_SCALE, 103
 - EKF_Q_Z_POS, 103
 - EKF_SIGMA_MAX, 103
 - EKF_SIGMA_MIN, 103
 - EKF_STATIONARY_STEPS, 104
 - ekf_step, 107

- EKF_VEL_EMA_ALPHA, [104](#)
- EKF_ANCHOR_IDS
 - ekf.c, [100](#)
 - ekf.h, [108](#)
- EKF_ANCHOR_MIN_STEPS
 - ekf.h, [101](#)
- EKF_ANCHOR_P_CONVERGED
 - ekf.h, [101](#)
- ekf_anchor_ready
 - ekf.c, [96](#)
 - ekf.h, [104](#)
- ekf_certainty_to_sigma
 - ekf.c, [96](#)
 - ekf.h, [104](#)
- EKF_DT_MAX_S
 - ekf.h, [102](#)
- EKF_DT_NOM_S
 - ekf.h, [102](#)
- ekf_get_anchor_positions
 - ekf.c, [97](#)
 - ekf.h, [105](#)
- ekf_get_peer_pos
 - ekf.c, [97](#)
 - ekf.h, [105](#)
- ekf_get_phase
 - ekf.c, [98](#)
 - ekf.h, [106](#)
- ekf_get_state
 - ekf.c, [98](#)
 - ekf.h, [106](#)
- EKF_HDG_MIN_DELTA_M
 - ekf.h, [102](#)
- EKF_IMU_DEADBAND_MS
 - ekf.h, [102](#)
- ekf_init
 - ekf.c, [98](#)
 - ekf.h, [106](#)
- EKF_INIT_P_POS
 - ekf.h, [102](#)
- EKF_INIT_RAND_M
 - ekf.c, [96](#)
- EKF_MAX_PEERS
 - ekf.h, [102](#)
- EKF_MAX_STATE
 - ekf.h, [102](#)
- EKF_MAX_STEP_M
 - ekf.h, [102](#)
- EKF_MAX_UPDATE_M
 - ekf.h, [102](#)
- ekf_node_hint_t, [21](#)
 - id, [21](#)
 - x, [21](#)
 - y, [22](#)
 - z, [22](#)
- EKF_NUM_ANCHORS
 - ekf.h, [103](#)
- EKF_OUTLIER_GATE_NEG
 - ekf.h, [103](#)
- EKF_OUTLIER_GATE_POS
 - ekf.h, [103](#)
- EKF_P_MAX
 - ekf.h, [103](#)
- EKF_PHASE_ANCHOR_SURVEY
 - ekf.h, [104](#)
- ekf_phase_t
 - ekf.h, [104](#)
- EKF_PHASE_TAG_LOCALIZE
 - ekf.h, [104](#)
- EKF_Q_H_FLOOR
 - ekf.h, [103](#)
- EKF_Q_H_FLOOR_STATIONARY
 - ekf.h, [103](#)
- EKF_Q_H_VEL_SCALE
 - ekf.h, [103](#)
- EKF_Q_Z_POS
 - ekf.h, [103](#)
- EKF_SIGMA_MAX
 - ekf.h, [103](#)
- EKF_SIGMA_MIN
 - ekf.h, [103](#)
- EKF_STATIONARY_STEPS
 - ekf.h, [104](#)
- ekf_step
 - ekf.c, [99](#)
 - ekf.h, [107](#)
- EKF_TAG_HARDCODED_Z
 - ekf.c, [96](#)
- EKF_VEL_EMA_ALPHA
 - ekf.h, [104](#)
- entries
 - msg_final_t, [27](#)
 - msg_passive_t, [29](#)
- entry_antenna_unreliable
 - msg_final_t, [27](#)
 - msg_passive_t, [29](#)
- entry_count
 - msg_final_t, [27](#)
 - msg_passive_t, [29](#)
- entry_id
 - msg_final_t, [27](#)
- entry_ids
 - msg_passive_t, [29](#)
- entry_pwr_diff_q8
 - msg_final_t, [27](#)
 - msg_passive_t, [29](#)
- expected_seq_num
 - network_t, [38](#)
- final
 - measurements_t, [25](#)
 - msg_t, [36](#)
- final_antenna_unreliable
 - measurements_t, [25](#)
- final_rx
 - measurements_t, [25](#)
 - twr_timestamps_t, [47](#)
- final_rx_unreliable

- twr_timestamps_t, [47](#)
- final_tx
 - twr_timestamps_t, [48](#)
- final_tx_ts
 - msg_final_t, [27](#)
- find_peer
 - uwb_network.c, [166](#)
 - uwb_network.h, [179](#)
- first
 - timestamps_t, [46](#)
- first_order_t, [22](#)
 - initiator_id, [22](#)
 - responder_id, [22](#)
 - result_distance_tick, [22](#)
 - twr, [22](#)
- fp_q8
 - dwm_rx_frame_t, [19](#)
- get_usb_ready
 - my_print.c, [133](#)
 - my_print.h, [135](#)
- gravity_raw
 - imu_sample_t, [24](#)
- HAL_GetTick
 - ekf.c, [100](#)
- HAL_LPTIM_AutoReloadMatchCallback
 - sleep.c, [139](#)
- HAL_UART_ErrorCallback
 - UART.c, [142](#)
 - UART.h, [144](#)
- HAL_UART_RxCpltCallback
 - UART.h, [144](#)
- HAL_UARTEx_RxEventCallback
 - UART.c, [142](#)
- hints
 - boot_config_t, [16](#)
- hlptim1
 - sleep.c, [140](#)
- hspl1
 - DWM3000_driver.c, [120](#)
- hUsbDeviceFS
 - my_print.c, [134](#)
- id
 - ekf_node_hint_t, [21](#)
 - node_t, [40](#)
- imu.c
 - dev_ctx, [54](#)
 - imu_calibrate, [52](#)
 - imu_init, [52](#)
 - imu_integrate, [52](#)
 - imu_read_fifo, [53](#)
 - LSM6DSV_FIFO_DATA_OUT_TAG_ADDR, [52](#)
- imu.h
 - IMU_ACCEL_X_MS2, [55](#)
 - IMU_ACCEL_Y_MS2, [55](#)
 - IMU_ACCEL_Z_MS2, [55](#)
 - imu_calibrate, [56](#)
 - IMU_GRAV_X, [55](#)
 - IMU_GRAV_Y, [55](#)
 - IMU_GRAV_Z, [55](#)
 - imu_init, [56](#)
 - imu_integrate, [56](#)
 - imu_read_fifo, [57](#)
 - IMU_ACCEL_X_MS2
 - imu.h, [55](#)
 - IMU_ACCEL_Y_MS2
 - imu.h, [55](#)
 - IMU_ACCEL_Z_MS2
 - imu.h, [55](#)
 - IMU_BOOT_TIME_MS
 - imu_setup.h, [59](#)
 - IMU_CAL_ODR_HZ
 - imu_setup.h, [59](#)
 - IMU_CAL_SAMPLES
 - imu_setup.h, [59](#)
 - imu_calibrate
 - imu.c, [52](#)
 - imu.h, [56](#)
 - imu_fifo_data_t, [23](#)
 - count, [23](#)
 - samples, [23](#)
 - IMU_FIFO_GY_BATCH
 - imu_setup.h, [59](#)
 - IMU_FIFO_RAW_BUF_ENTRIES
 - imu_setup.h, [59](#)
 - IMU_FIFO_TS_DEC
 - imu_setup.h, [59](#)
 - IMU_FIFO_XL_BATCH
 - imu_setup.h, [59](#)
 - imu_get_results
 - lsm6dsv.c, [63](#)
 - lsm6dsv.h, [65](#)
 - IMU_GRAV_X
 - imu.h, [55](#)
 - IMU_GRAV_Y
 - imu.h, [55](#)
 - IMU_GRAV_Z
 - imu.h, [55](#)
 - IMU_GY_FULL_SCALE
 - imu_setup.h, [59](#)
 - IMU_GY_LP1_BW
 - imu_setup.h, [60](#)
 - IMU_GY_ODR
 - imu_setup.h, [60](#)
 - imu_init
 - imu.c, [52](#)
 - imu.h, [56](#)
 - imu_integrate
 - imu.c, [52](#)
 - imu.h, [56](#)
 - IMU_MAX_SAMPLES
 - imu_setup.h, [60](#)
 - imu_read_fifo
 - imu.c, [53](#)
 - imu.h, [57](#)

- imu_sample_t, 23
 - accel_raw, 24
 - gravity_raw, 24
 - timestamp_ticks, 24
- imu_setup.h
 - IMU_BOOT_TIME_MS, 59
 - IMU_CAL_ODR_HZ, 59
 - IMU_CAL_SAMPLES, 59
 - IMU_FIFO_GY_BATCH, 59
 - IMU_FIFO_RAW_BUF_ENTRIES, 59
 - IMU_FIFO_TS_DEC, 59
 - IMU_FIFO_XL_BATCH, 59
 - IMU_GY_FULL_SCALE, 59
 - IMU_GY_LP1_BW, 60
 - IMU_GY_ODR, 60
 - IMU_MAX_SAMPLES, 60
 - IMU_SFLP_BATCH_GBIAS, 60
 - IMU_SFLP_BATCH_GRAVITY, 60
 - IMU_SFLP_BATCH_QUAT, 60
 - IMU_SFLP_ODR, 60
 - IMU_XL_FULL_SCALE, 60
 - IMU_XL_LP2_BW, 60
 - IMU_XL_ODR, 60
- IMU_SFLP_BATCH_GBIAS
 - imu_setup.h, 60
- IMU_SFLP_BATCH_GRAVITY
 - imu_setup.h, 60
- IMU_SFLP_BATCH_QUAT
 - imu_setup.h, 60
- IMU_SFLP_ODR
 - imu_setup.h, 60
- IMU_vel_horiz
 - msg_final_t, 27
 - msg_passive_t, 29
- imu_vel_horiz
 - node_t, 40
- IMU_vel_vert
 - msg_final_t, 27
 - msg_passive_t, 30
- imu_vel_vert
 - node_t, 40
- IMU_XL_FULL_SCALE
 - imu_setup.h, 60
- IMU_XL_LP2_BW
 - imu_setup.h, 60
- IMU_XL_ODR
 - imu_setup.h, 60
- init_rx
 - ss_twr_t, 43
 - tdoa_twr_t, 44
- init_rx_unreliable
 - second_order_t, 42
- init_tx
 - ss_twr_t, 43
 - tdoa_twr_t, 44
- initialised
 - coop_ekf_t, 17
- initiator_id
 - first_order_t, 22
 - second_order_t, 42
 - third_order_t, 45
- is_charging
 - BatteryStatus_t, 15
- last_tick_ms
 - coop_ekf_t, 17
- len
 - dwm_rx_frame_t, 19
 - dwm_rx_raw_frame_t, 20
 - dwm_tx_frame_t, 21
 - msg_t, 36
 - print_msg_t, 41
- low_battery_check
 - DWM3000.c, 112
- lsm6dsv.c
 - imu_get_results, 63
 - StartAcc, 64
- lsm6dsv.h
 - imu_get_results, 65
 - StartAcc, 65
- LSM6DSV_FIFO_DATA_OUT_TAG_ADDR
 - imu.c, 52
- lsm6dsv_read_data_polling
 - lsm6dsv_ST_example.h, 68
 - read_data_polling.c, 69
- lsm6dsv_self_test.c
 - BOOT_TIME, 67
 - MAX_ST_LIMIT_mdps, 67
 - MAX_ST_LIMIT_mg, 67
 - MIN_ST_LIMIT_mdps, 67
 - MIN_ST_LIMIT_mg, 67
 - SelfTest, 67
 - ST_FAIL, 67
 - ST_PASS, 67
 - tx_buffer, 68
- lsm6dsv_ST_example.h
 - lsm6dsv_read_data_polling, 68
 - SelfTest, 68
- master_id
 - network_t, 38
- master_init_rx
 - tdoa_twr_t, 44
- MAX_PASSIVE
 - distance.h, 88
- MAX_SECOND_ORDER
 - distance.h, 88
- MAX_ST_LIMIT_mdps
 - lsm6dsv_self_test.c, 67
- MAX_ST_LIMIT_mg
 - lsm6dsv_self_test.c, 67
- MAX_THIRD_ORDER
 - distance.h, 88
- MEAS_DSTWR
 - distance.h, 89
- MEAS_NONE
 - distance.h, 89

- MEAS_SSTWR
 - distance.h, [89](#)
- MEAS_TDOA_SSTWR
 - distance.h, [89](#)
- measurements
 - network_t, [38](#)
- measurements_t, [24](#)
 - final, [25](#)
 - final_antenna_unreliable, [25](#)
 - final_rx, [25](#)
 - passive, [25](#)
 - passive_antenna_unreliable, [25](#)
 - passive_count, [25](#)
 - passive_device_id, [25](#)
 - passive_rx, [25](#)
 - poll, [26](#)
 - poll_antenna_unreliable, [26](#)
 - poll_rx, [26](#)
 - resp, [26](#)
 - resp_tx, [26](#)
- MeasurementType
 - distance.h, [89](#)
- messages.c
 - msg_decode, [146](#)
 - msg_encode, [147](#)
 - u64_to_ts_buf, [147](#)
- messages.h
 - msg_decode, [151](#)
 - msg_encode, [151](#)
 - MSG_HEADER_LEN, [149](#)
 - MSG_OFFSET_RECEIVER, [149](#)
 - MSG_OFFSET_SENDER, [149](#)
 - MSG_PAYLOAD_OFFSET_RESP_FP, [149](#)
 - MSG_PAYLOAD_OFFSET_RESP_RSSI, [150](#)
 - MSG_PAYLOAD_OFFSET_SEQ, [150](#)
 - MSG_PAYLOAD_OFFSET_SHARE_SLEEP, [150](#)
 - MSG_PAYLOAD_OFFSET_TS_0, [150](#)
 - MSG_PAYLOAD_OFFSET_TS_1, [150](#)
 - MSG_PAYLOAD_OFFSET_TS_2, [150](#)
 - MSG_START_LOCATION, [150](#)
 - MSG_TS_LEN, [150](#)
 - MSG_TYPE_ERR, [151](#)
 - MSG_TYPE_FINAL, [151](#)
 - MSG_TYPE_PASSIVE, [151](#)
 - MSG_TYPE_POLL, [151](#)
 - MSG_TYPE_RESPONSE, [151](#)
 - MSG_TYPE_SHARE, [151](#)
 - MSG_TYPE_SYNC, [151](#)
 - msg_type_t, [151](#)
 - NETWORK_MAX_PAIRS, [150](#)
 - NETWORK_MAX_PEERS, [150](#)
 - u64_to_ts_buf, [152](#)
- METERS_PER_TICK
 - distance.h, [88](#)
- MIN_ST_LIMIT_mdps
 - lsm6dsv_self_test.c, [67](#)
- MIN_ST_LIMIT_mg
 - lsm6dsv_self_test.c, [67](#)
- MPRINT_TS
 - DWM3000.c, [111](#)
 - uwb_exchange.c, [155](#)
- mprintf
 - my_print.c, [133](#)
 - my_print.h, [135](#)
- MS_TO_DWT_TIME
 - DWM3000_driver.h, [122](#)
- msg_decode
 - messages.c, [146](#)
 - messages.h, [151](#)
- msg_encode
 - messages.c, [147](#)
 - messages.h, [151](#)
- msg_final_t, [26](#)
 - entries, [27](#)
 - entry_antenna_unreliable, [27](#)
 - entry_count, [27](#)
 - entry_id, [27](#)
 - entry_pwr_diff_q8, [27](#)
 - final_tx_ts, [27](#)
 - IMU_vel_horiz, [27](#)
 - IMU_vel_vert, [27](#)
 - poll_tx_ts, [27](#)
 - resp_antenna_unreliable, [28](#)
 - resp_pwr_diff_q8, [28](#)
 - resp_rx_ts, [28](#)
 - seq_num, [28](#)
- MSG_HEADER_LEN
 - messages.h, [149](#)
- MSG_OFFSET_RECEIVER
 - messages.h, [149](#)
- MSG_OFFSET_SENDER
 - messages.h, [149](#)
- msg_passive_t, [28](#)
 - entries, [29](#)
 - entry_antenna_unreliable, [29](#)
 - entry_count, [29](#)
 - entry_ids, [29](#)
 - entry_pwr_diff_q8, [29](#)
 - IMU_vel_horiz, [29](#)
 - IMU_vel_vert, [30](#)
 - passive_tx_ts, [30](#)
 - poll_antenna_unreliable, [30](#)
 - poll_pwr_diff_q8, [30](#)
 - poll_rx_ts, [30](#)
 - resp_antenna_unreliable, [30](#)
 - resp_pwr_diff_q8, [30](#)
 - resp_rx_ts, [30](#)
 - seq_num, [31](#)
- MSG_PAYLOAD_OFFSET_RESP_FP
 - messages.h, [149](#)
- MSG_PAYLOAD_OFFSET_RESP_RSSI
 - messages.h, [150](#)
- MSG_PAYLOAD_OFFSET_SEQ
 - messages.h, [150](#)
- MSG_PAYLOAD_OFFSET_SHARE_SLEEP
 - messages.h, [150](#)

- MSG_PAYLOAD_OFFSET_TS_0
 - messages.h, [150](#)
- MSG_PAYLOAD_OFFSET_TS_1
 - messages.h, [150](#)
- MSG_PAYLOAD_OFFSET_TS_2
 - messages.h, [150](#)
- msg_poll_t, [31](#)
 - seq_num, [31](#)
- msg_response_t, [32](#)
 - seq_num, [32](#)
- msg_share_t, [32](#)
 - accuracy, [33](#)
 - distance_mm, [33](#)
 - node_count, [33](#)
 - node_ids, [33](#)
 - seq_num, [33](#)
 - sleep_time, [33](#)
 - vel_horiz, [33](#)
 - vel_vert, [34](#)
- MSG_START_LOCATION
 - messages.h, [150](#)
- msg_sync_t, [34](#)
 - peer_count, [34](#)
 - peer_ids, [34](#)
 - seq_num, [35](#)
- msg_t, [35](#)
 - data, [36](#)
 - final, [36](#)
 - len, [36](#)
 - passive, [36](#)
 - poll, [36](#)
 - receiver, [36](#)
 - response, [36](#)
 - sender, [36](#)
 - share, [36](#)
 - sync, [37](#)
 - type, [37](#)
- MSG_TS_LEN
 - messages.h, [150](#)
- MSG_TYPE_ERR
 - messages.h, [151](#)
- MSG_TYPE_FINAL
 - messages.h, [151](#)
- MSG_TYPE_PASSIVE
 - messages.h, [151](#)
- MSG_TYPE_POLL
 - messages.h, [151](#)
- MSG_TYPE_RESPONSE
 - messages.h, [151](#)
- MSG_TYPE_SHARE
 - messages.h, [151](#)
- MSG_TYPE_SYNC
 - messages.h, [151](#)
- msg_type_t
 - messages.h, [151](#)
- my_print.c
 - get_usb_ready, [133](#)
 - hUsbDeviceFS, [134](#)
 - mprintf, [133](#)
 - PRINT_BUF_SIZE, [132](#)
 - PRINT_QUEUE_DEPTH, [132](#)
 - PrintTask, [133](#)
 - PrintTask_TxCpltCallback, [134](#)
- my_print.h
 - Both, [134](#)
 - get_usb_ready, [135](#)
 - mprintf, [135](#)
 - PRINT_MODE, [134](#)
 - PrintTask, [135](#)
 - STlink, [134](#)
 - USB_CDC, [135](#)
- n_peers
 - coop_ekf_t, [17](#)
- network_add_peer
 - uwb_network.c, [166](#)
 - uwb_network.h, [179](#)
- network_bump_certainty
 - uwb_network.c, [166](#)
 - uwb_network.h, [179](#)
- NETWORK_DISTANCE_INVALID
 - uwb_network.c, [166](#)
- network_fill_peer_ids
 - uwb_network.c, [167](#)
 - uwb_network.h, [179](#)
- network_get_certainty
 - uwb_network.c, [167](#)
 - uwb_network.h, [180](#)
- network_get_count
 - uwb_network.c, [167](#)
 - uwb_network.h, [180](#)
- network_get_distance
 - uwb_network.c, [168](#)
 - uwb_network.h, [181](#)
- network_get_expected_seq_num
 - uwb_network.c, [168](#)
 - uwb_network.h, [181](#)
- network_get_highest_uncertainty
 - uwb_network.c, [168](#)
 - uwb_network.h, [181](#)
- network_get_k
 - uwb_network.h, [181](#)
- network_get_master
 - uwb_network.c, [169](#)
 - uwb_network.h, [182](#)
- network_get_measurements
 - uwb_network.c, [169](#)
- network_get_network
 - uwb_network.c, [169](#)
 - uwb_network.h, [182](#)
- network_get_ownid
 - uwb_network.c, [169](#)
 - uwb_network.h, [182](#)
- network_get_passive_count
 - uwb_network.c, [170](#)
 - uwb_network.h, [182](#)
- network_get_peer_index

- uwb_network.c, 170
- uwb_network.h, 182
- network_get_peer_state
 - uwb_network.c, 170
 - uwb_network.h, 183
- network_get_peers
 - uwb_network.c, 171
 - uwb_network.h, 183
- network_init
 - uwb_network.c, 171
 - uwb_network.h, 184
- network_is_acknowledged
 - uwb_network.c, 171
 - uwb_network.h, 184
- network_is_master
 - uwb_network.c, 172
 - uwb_network.h, 184
- NETWORK_MAX_PAIRS
 - messages.h, 150
- NETWORK_MAX_PEERS
 - messages.h, 150
- network_print_certainty
 - uwb_network.c, 172
 - uwb_network.h, 185
- network_print_positions
 - uwb_network.c, 172
 - uwb_network.h, 185
- network_remove_peer
 - uwb_network.c, 172
 - uwb_network.h, 185
- network_reset_certainty
 - uwb_network.c, 173
 - uwb_network.h, 185
- network_reset_measurements
 - uwb_network.c, 173
 - uwb_network.h, 186
- network_set_acknowledged
 - uwb_network.c, 173
 - uwb_network.h, 186
- network_set_distance
 - uwb_network.c, 173
 - uwb_network.h, 186
- network_set_expected_seq_num
 - uwb_network.c, 174
 - uwb_network.h, 186
- network_set_k
 - uwb_network.h, 187
- network_set_master
 - uwb_network.c, 174
 - uwb_network.h, 187
- network_set_self_pos
 - uwb_network.c, 174
 - uwb_network.h, 187
- network_store_final
 - uwb_network.c, 175
 - uwb_network.h, 187
- network_store_passive
 - uwb_network.c, 175
- uwb_network.h, 188
- network_store_poll
 - uwb_network.c, 175
 - uwb_network.h, 188
- network_store_resp_tx
 - uwb_network.c, 176
 - uwb_network.h, 188
- network_t, 37
 - acknowledged, 38
 - count, 38
 - expected_seq_num, 38
 - master_id, 38
 - measurements, 38
 - peers, 38
 - self, 38
- network_update_certainty
 - uwb_network.c, 176
 - uwb_network.h, 189
- network_update_peers_from_sync
 - uwb_network.c, 176
 - uwb_network.h, 189
- node_count
 - msg_share_t, 33
- node_ids
 - msg_share_t, 33
- node_peer_state_t, 39
 - certainty, 39
 - distance_scaled, 39
 - peer_id, 39
- node_t, 39
 - id, 40
 - imu_vel_horiz, 40
 - imu_vel_vert, 40
 - peers, 40
 - pos, 40
- P
 - coop_ekf_t, 17
- passive
 - measurements_t, 25
 - msg_t, 36
- passive_antenna_unreliable
 - measurements_t, 25
- passive_count
 - measurements_t, 25
- passive_device_id
 - measurements_t, 25
- passive_rx
 - measurements_t, 25
- passive_tx_ts
 - msg_passive_t, 30
- PDIFF_MAX_DB
 - distance.c, 81
- peer_away
 - coop_ekf_t, 17
- peer_count
 - msg_sync_t, 34
- peer_id
 - node_peer_state_t, 39

- peer_ids
 - coop_ekf_t, 17
 - msg_sync_t, 34
- peer_imu_valid
 - coop_ekf_t, 17
- peer_seeded
 - coop_ekf_t, 17
- peers
 - network_t, 38
 - node_t, 40
- poll
 - measurements_t, 26
 - msg_t, 36
- poll_antenna_unreliable
 - measurements_t, 26
 - msg_passive_t, 30
- poll_pwr_diff_q8
 - msg_passive_t, 30
- poll_rx
 - measurements_t, 26
 - twr_observation_t, 47
 - twr_timestamps_t, 48
- poll_rx_ts
 - msg_passive_t, 30
- poll_rx_unreliable
 - twr_observation_t, 47
 - twr_timestamps_t, 48
- poll_tx
 - twr_timestamps_t, 48
- poll_tx_ts
 - msg_final_t, 27
- populate_computation_structs
 - distance.c, 83
- pos
 - node_t, 40
- position_calibrate_timestamp
 - distance.h, 91
- power.c
 - ADC_FULL_SCALE, 137
 - DIVIDER_RATIO, 137
 - power_get_status, 137
 - power_read, 137
 - VDDA_MV, 137
- power.h
 - power_get_status, 138
 - power_read, 138
- power_get_status
 - power.c, 137
 - power.h, 138
- power_read
 - power.c, 137
 - power.h, 138
- PPM_TO_K
 - calibrate.h, 76
- PRINT_BUF_SIZE
 - my_print.c, 132
- PRINT_MODE
 - my_print.h, 134
- print_msg_t, 40
 - data, 41
 - len, 41
 - tick, 41
- PRINT_QUEUE_DEPTH
 - my_print.c, 132
- PrintTask
 - my_print.c, 133
 - my_print.h, 135
- PrintTask_TxCpltCallback
 - my_print.c, 134
- pwr_diff_q8
 - uwb_rx_meas_t, 49
- Q
 - coop_ekf_t, 18
- read_data_polling.c
 - BOOT_TIME, 69
 - lsm6dsv_read_data_polling, 69
- receiver
 - msg_t, 36
- resp
 - measurements_t, 26
- resp_antenna_unreliable
 - msg_final_t, 28
 - msg_passive_t, 30
- resp_pwr_diff_q8
 - msg_final_t, 28
 - msg_passive_t, 30
- resp_rx
 - twr_observation_t, 47
 - twr_timestamps_t, 48
- resp_rx_ts
 - msg_final_t, 28
 - msg_passive_t, 30
- resp_rx_unreliable
 - twr_observation_t, 47
 - twr_timestamps_t, 48
- resp_tx
 - measurements_t, 26
 - twr_timestamps_t, 48
- responder_id
 - first_order_t, 22
 - second_order_t, 42
 - third_order_t, 45
- response
 - msg_t, 36
- result_distance_tick
 - first_order_t, 22
 - second_order_t, 42
 - third_order_t, 45
- rssi_q8
 - dwm_rx_frame_t, 19
- rx_diag
 - dwm_rx_raw_frame_t, 20
- RX_MEAS_FROM_FRAME
 - uwb_exchange.c, 155
- rx_queue

- DWM3000_driver.c, 120
- rx_timestamp
 - dwm_rx_frame_t, 19
- samples
 - imu_fifo_data_t, 23
- second
 - timestamps_t, 46
- second_count
 - timestamps_t, 46
- second_order_t, 41
 - answer_rx_unreliable, 42
 - init_rx_unreliable, 42
 - initiator_id, 42
 - responder_id, 42
 - result_distance_tick, 42
 - twr, 42
 - twr_observation, 42
- self
 - network_t, 38
- self_slot
 - coop_ekf_t, 18
- SelfTest
 - lsm6dsv_self_test.c, 67
 - lsm6dsv_ST_example.h, 68
- sender
 - msg_t, 36
- seq_num
 - msg_final_t, 28
 - msg_passive_t, 31
 - msg_poll_t, 31
 - msg_response_t, 32
 - msg_share_t, 33
 - msg_sync_t, 35
- share
 - msg_t, 36
- sleep.c
 - HAL_LPTIM_AutoReloadMatchCallback, 139
 - hlptim1, 140
 - sleep_stop1_ms, 139
- sleep.h
 - sleep_stop1_ms, 140
- sleep_stop1_ms
 - sleep.c, 139
 - sleep.h, 140
- sleep_time
 - msg_share_t, 33
- SPEED_OF_LIGHT
 - distance.h, 88
- ss_twr_t, 42
 - answer_rx, 43
 - answer_tx, 43
 - init_rx, 43
 - init_tx, 43
- ST_FAIL
 - lsm6dsv_self_test.c, 67
- ST_PASS
 - lsm6dsv_self_test.c, 67
- StartAcc
 - lsm6dsv.c, 64
 - lsm6dsv.h, 65
- StartBlink
 - blink.c, 70
 - blink.h, 70
- StartDWM
 - DWM3000.h, 113
- StartRangingTask
 - DWM3000.c, 112
- stationary_count
 - coop_ekf_t, 18
- status
 - dwm_rx_frame_t, 19
 - dwm_rx_raw_frame_t, 20
- STlink
 - my_print.h, 134
- sync
 - msg_t, 37
- SYNC_PAD
 - uwb_exchange.c, 156
- SYNC_TIMEOUT_MAX
 - Timing constants, 10
- T_EARLY_WKUP
 - Timing constants, 10
- T_FINAL_RX_WAIT
 - Timing constants, 10
- T_FINAL_TX_ASAP_TICKS
 - Timing constants, 10
- T_FINAL_TX_WAIT_MS
 - Timing constants, 10
- T_PASSIVE_TX_ASAP_TICKS
 - Timing constants, 10
- T_PASSIVE_TX_WAIT_MS
 - Timing constants, 11
- T_RESPONSE_RX_WAIT
 - Timing constants, 11
- T_SHARE_RX_WAIT
 - Timing constants, 11
- T_SYNC_RX_ANSWER
 - Timing constants, 11
- T_SYNC_RX_SET
 - Timing constants, 11
- T_TWR_RX_WAIT
 - Timing constants, 11
- tdoa_twr_t, 43
 - answer_rx, 44
 - answer_tx, 44
 - init_rx, 44
 - init_tx, 44
 - master_init_rx, 44
- third
 - timestamps_t, 46
- third_count
 - timestamps_t, 46
- third_order_t, 44
 - answer_rx_unreliable, 45
 - initiator_id, 45
 - responder_id, 45

- result_distance_tick, [45](#)
 - twr, [45](#)
 - twr_observation_c, [45](#)
 - twr_observation_d, [45](#)
- tick
 - print_msg_t, [41](#)
- timestamp_ticks
 - imu_sample_t, [24](#)
- timestamps
 - distance.c, [85](#)
- timestamps_t, [45](#)
 - first, [46](#)
 - second, [46](#)
 - second_count, [46](#)
 - third, [46](#)
 - third_count, [46](#)
- Timing constants, [9](#)
 - ALL_ID, [10](#)
 - DEEP_SLEEP, [10](#)
 - SYNC_TIMEOUT_MAX, [10](#)
 - T_EARLY_WKUP, [10](#)
 - T_FINAL_RX_WAIT, [10](#)
 - T_FINAL_TX_ASAP_TICKS, [10](#)
 - T_FINAL_TX_WAIT_MS, [10](#)
 - T_PASSIVE_TX_ASAP_TICKS, [10](#)
 - T_PASSIVE_TX_WAIT_MS, [11](#)
 - T_RESPONSE_RX_WAIT, [11](#)
 - T_SHARE_RX_WAIT, [11](#)
 - T_SYNC_RX_ANSWER, [11](#)
 - T_SYNC_RX_SET, [11](#)
 - T_TWR_RX_WAIT, [11](#)
 - UWB_US_TO_TICKS, [11](#)
- ts
 - uwb_rx_meas_t, [49](#)
- ts_buf
 - dwm_rx_raw_frame_t, [20](#)
- ts_buf_to_u64
 - DWM3000_driver.c, [120](#)
 - DWM3000_driver.h, [127](#)
- twr
 - first_order_t, [22](#)
 - second_order_t, [42](#)
 - third_order_t, [45](#)
- twr_observation
 - second_order_t, [42](#)
- twr_observation_c
 - third_order_t, [45](#)
- twr_observation_d
 - third_order_t, [45](#)
- twr_observation_t, [46](#)
 - poll_rx, [47](#)
 - poll_rx_unreliable, [47](#)
 - resp_rx, [47](#)
 - resp_rx_unreliable, [47](#)
- twr_timestamps_t, [47](#)
 - final_rx, [47](#)
 - final_rx_unreliable, [47](#)
 - final_tx, [48](#)
 - poll_rx, [48](#)
 - poll_rx_unreliable, [48](#)
 - poll_tx, [48](#)
 - resp_rx, [48](#)
 - resp_rx_unreliable, [48](#)
 - resp_tx, [48](#)
- tx_buffer
 - lsm6dsv_self_test.c, [68](#)
- TX_DELAY_500US
 - DWM3000_driver.h, [122](#)
- tx_err_watchdog
 - DWM3000.c, [112](#)
- tx_queue
 - DWM3000_driver.c, [120](#)
- TX_RF_PGcount
 - DWM3000_setup.h, [131](#)
- TX_RF_PGdly
 - DWM3000_setup.h, [131](#)
- TX_RF_Power
 - DWM3000_setup.h, [131](#)
- tx_timestamp
 - dwm_tx_frame_t, [21](#)
- type
 - dwm_rx_frame_t, [19](#)
 - dwm_rx_raw_frame_t, [20](#)
 - msg_t, [37](#)
- U64_HI
 - DWM3000.c, [111](#)
 - uwb_exchange.c, [156](#)
- U64_LO
 - DWM3000.c, [111](#)
 - uwb_exchange.c, [156](#)
- u64_to_ts_buf
 - messages.c, [147](#)
 - messages.h, [152](#)
- UART.c
 - HAL_UART_ErrorCallback, [142](#)
 - HAL_UARTEEx_RxEventCallback, [142](#)
 - uart_boot_config_read, [142](#)
 - uart_boot_start, [142](#)
 - uart_periodic_poll, [143](#)
 - uart_print_power, [143](#)
 - uart_rst_arm, [143](#)
 - UART_RST_BUF_SIZE, [141](#)
 - UART_RST_BYTE, [141](#)
- UART.h
 - HAL_UART_ErrorCallback, [144](#)
 - HAL_UART_RxCpltCallback, [144](#)
 - uart_boot_config_read, [144](#)
 - UART_BOOT_RX_LEN, [144](#)
 - uart_boot_start, [144](#)
 - uart_periodic_poll, [145](#)
 - uart_print_power, [145](#)
 - uart_rst_arm, [145](#)
- uart_boot_config_read
 - UART.c, [142](#)
 - UART.h, [144](#)
- UART_BOOT_RX_LEN

- UART.h, 144
- uart_boot_start
 - UART.c, 142
 - UART.h, 144
- uart_periodic_poll
 - UART.c, 143
 - UART.h, 145
- uart_print_power
 - UART.c, 143
 - UART.h, 145
- uart_rst_arm
 - UART.c, 143
 - UART.h, 145
- UART_RST_BUF_SIZE
 - UART.c, 141
- UART_RST_BYTE
 - UART.c, 141
- US_TO_DWT_TIME
 - DWM3000_driver.h, 122
- USB_CDC
 - my_print.h, 135
- usb_connected
 - BatteryStatus_t, 15
- UWB_40BIT_MASK
 - distance.h, 89
- uwb_build_share
 - uwb_exchange.c, 156
- uwb_etwr_result_t
 - uwb_exchange.h, 160
- uwb_exchange.c
 - MPRINT_TS, 155
 - RX_MEAS_FROM_FRAME, 155
 - SYNC_PAD, 156
 - U64_HI, 156
 - U64_LO, 156
 - uwb_build_share, 156
 - uwb_extended_twr, 156
 - uwb_read_share, 157
 - uwb_share, 157
 - uwb_sync, 158
 - uwb_unexpected_message_handler, 158
- uwb_exchange.h
 - uwb_etwr_result_t, 160
 - uwb_extended_twr, 161
 - uwb_share, 162
 - uwb_sync, 162
 - UWB_SYNC_MASTER, 161
 - UWB_SYNC_MASTER_NO_REPLY, 161
 - UWB_SYNC_NEW_MASTER, 161
 - UWB_SYNC_NEW_SLAVE, 161
 - uwb_sync_result_t, 161
 - UWB_SYNC_SLAVE_ACKNOWLEDGED, 161
 - UWB_SYNC_SLAVE_PENDING, 161
 - UWB_SYNC_SLAVE_REPLY_FAILED, 161
 - UWB_SYNC_TX_FAILED, 161
 - UWB_SYNC_UNEXPECTED_MASTER, 161
 - UWB_TWR_EXCHANGE_COMPLETE, 160
 - UWB_TWR_NOT_ENOUGH_DEVICES, 160
 - UWB_TWR_NOTHING, 160
 - UWB_TWR_PASSIVE_TIMEOUT, 160
 - UWB_TWR_RECEIVED, 160
 - UWB_TWR_RECEIVED_PASSIVE, 160
 - UWB_TWR_TIMEOUT, 160
 - UWB_TWR_TX_FAILED, 160
 - UWB_TWR_UNEXPECTED_MASTER, 161
- uwb_extended_twr
 - uwb_exchange.c, 156
 - uwb_exchange.h, 161
- uwb_network.c
 - find_peer, 166
 - network_add_peer, 166
 - network_bump_certainty, 166
 - NETWORK_DISTANCE_INVALID, 166
 - network_fill_peer_ids, 167
 - network_get_certainty, 167
 - network_get_count, 167
 - network_get_distance, 168
 - network_get_expected_seq_num, 168
 - network_get_highest_uncertainty, 168
 - network_get_master, 169
 - network_get_measurements, 169
 - network_get_network, 169
 - network_get_ownid, 169
 - network_get_passive_count, 170
 - network_get_peer_index, 170
 - network_get_peer_state, 170
 - network_get_peers, 171
 - network_init, 171
 - network_is_acknowledged, 171
 - network_is_master, 172
 - network_print_certainty, 172
 - network_print_positions, 172
 - network_remove_peer, 172
 - network_reset_certainty, 173
 - network_reset_measurements, 173
 - network_set_acknowledged, 173
 - network_set_distance, 173
 - network_set_expected_seq_num, 174
 - network_set_master, 174
 - network_set_self_pos, 174
 - network_store_final, 175
 - network_store_passive, 175
 - network_store_poll, 175
 - network_store_resp_tx, 176
 - network_update_certainty, 176
 - network_update_peers_from_sync, 176
- uwb_network.h
 - find_peer, 179
 - network_add_peer, 179
 - network_bump_certainty, 179
 - network_fill_peer_ids, 179
 - network_get_certainty, 180
 - network_get_count, 180
 - network_get_distance, 181
 - network_get_expected_seq_num, 181
 - network_get_highest_uncertainty, 181

- network_get_k, 181
- network_get_master, 182
- network_get_network, 182
- network_get_ownid, 182
- network_get_passive_count, 182
- network_get_peer_index, 182
- network_get_peer_state, 183
- network_get_peers, 183
- network_init, 184
- network_is_acknowledged, 184
- network_is_master, 184
- network_print_certainty, 185
- network_print_positions, 185
- network_remove_peer, 185
- network_reset_certainty, 185
- network_reset_measurements, 186
- network_set_acknowledged, 186
- network_set_distance, 186
- network_set_expected_seq_num, 186
- network_set_k, 187
- network_set_master, 187
- network_set_self_pos, 187
- network_store_final, 187
- network_store_passive, 188
- network_store_poll, 188
- network_store_resp_tx, 188
- network_update_certainty, 189
- network_update_peers_from_sync, 189
- uwb_read_share
 - uwb_exchange.c, 157
- uwb_rx_meas_t, 48
 - pwr_diff_q8, 49
 - ts, 49
- uwb_share
 - uwb_exchange.c, 157
 - uwb_exchange.h, 162
- uwb_sync
 - uwb_exchange.c, 158
 - uwb_exchange.h, 162
- UWB_SYNC_MASTER
 - uwb_exchange.h, 161
- UWB_SYNC_MASTER_NO_REPLY
 - uwb_exchange.h, 161
- UWB_SYNC_NEW_MASTER
 - uwb_exchange.h, 161
- UWB_SYNC_NEW_SLAVE
 - uwb_exchange.h, 161
- uwb_sync_result_t
 - uwb_exchange.h, 161
- UWB_SYNC_SLAVE_ACKNOWLEDGED
 - uwb_exchange.h, 161
- UWB_SYNC_SLAVE_PENDING
 - uwb_exchange.h, 161
- UWB_SYNC_SLAVE_REPLY_FAILED
 - uwb_exchange.h, 161
- UWB_SYNC_TX_FAILED
 - uwb_exchange.h, 161
- UWB_SYNC_UNEXPECTED_MASTER
 - uwb_exchange.h, 161
- UWB_TWR_EXCHANGE_COMPLETE
 - uwb_exchange.h, 160
- UWB_TWR_NOT_ENOUGH_DEVICES
 - uwb_exchange.h, 160
- UWB_TWR_NOTHING
 - uwb_exchange.h, 160
- UWB_TWR_PASSIVE_TIMEOUT
 - uwb_exchange.h, 160
- UWB_TWR_RECEIVED
 - uwb_exchange.h, 160
- UWB_TWR_RECEIVED_PASSIVE
 - uwb_exchange.h, 160
- UWB_TWR_TIMEOUT
 - uwb_exchange.h, 160
- UWB_TWR_TX_FAILED
 - uwb_exchange.h, 160
- UWB_TWR_UNEXPECTED_MASTER
 - uwb_exchange.h, 161
- uwb_unexpected_message_handler
 - uwb_exchange.c, 158
- UWB_US_TO_TICKS
 - Timing constants, 11
- valid
 - boot_config_t, 16
- VDDA_MV
 - power.c, 137
- vel_horiz
 - msg_share_t, 33
- VEL_HORIZ_MAX_MS
 - distance.h, 89
- VEL_HORIZ_MS_PER_LSB
 - distance.h, 89
- vel_horiz_to_u8
 - distance.c, 83
 - distance.h, 91
- vel_horiz_u8_to_ms
 - distance.c, 84
 - distance.h, 91
- vel_vert
 - msg_share_t, 34
- VEL_VERT_MS_PER_LSB
 - distance.h, 89
- VEL_VERT_RANGE_MS
 - distance.h, 89
- vel_vert_to_u8
 - distance.c, 84
 - distance.h, 91
- vel_vert_u8_to_ms
 - distance.c, 84
 - distance.h, 92
- wakeup_queue
 - DWM3000_driver.c, 120
- x
 - coop_ekf_t, 18
 - ekf_node_hint_t, 21

y

ekf_node_hint_t, [22](#)

z

ekf_node_hint_t, [22](#)